

ST20 Embedded Toolset R2.0 Reference Manual



Issued by the MCDT Documentation Group on behalf of STMicroelectronics

Information furnished is believed to be accurate and reliable. However, STMicroelectronics assumes no responsibility for the consequences of use of such information nor for any infringement of patents or other rights of third parties which may result from its use. No license is granted by implication or otherwise under any patent or patent rights of STMicroelectronics. Specifications mentioned in this publication are subject to change without notice. This publication supersedes and replaces all information previously supplied. STMicroelectronics products are not authorized for use as critical components in life support devices or systems without the express written approval of STMicroelectronics.

The ST logo is a registered trademark of STMicroelectronics.

All other names are the property of their respective owners.

Linux® is a registered trademark of Linus Torvalds. Red Hat® is a registered trademark of Red Hat Software, Inc. Sun and Solaris are trademarks or registered trademarks of Sun Microsystems, Inc. in the US and other countries. UNIX® is a registered trademark of the The Open Group in the US and other countries. Microsoft®, Windows® and Windows NT® are registered trademarks of Microsoft Corporation in the United States and other countries.

The C compiler implementation was developed from the
Perihelion Software "C" Compiler and the Codemist Norcroft "C" Compiler.

The C++ language front-end was developed under a Licence Agreement between
Edison Design Group, Inc. (EDG) and STMicroelectronics Limited.

This product incorporates innovative techniques which were developed with support from the
European Commission under the ESPRIT Projects:

- P2701 PUMA (Parallel Universal Message-passing Architectures),
- P5404 GPMIMD (General Purpose Multiple Instruction Multiple Data Machines),
- P7250 TMP (Transputer Macrocell Project),
- P7267 OMI/STANDARDS,
- P6290 HAMLET (High Performance Computing for Industrial Applications),
- P606 STARLIGHT (Starlight core for Hard Disk Drive Applications).

© 2001, 2002, 2003 STMicroelectronics. All Rights Reserved.

STMicroelectronics Group of Companies

Australia - Belgium - Brazil - Canada - China - Czech Republic - Finland - France - Germany - Hong Kong - India - Israel - Italy
- Japan - Malaysia - Malta - Morocco - Singapore - Spain - Sweden - Switzerland - United Kingdom - United States.

<http://www.st.com>

Contents

Preface	xv
ST20 documentation suite	xv
Conventions used in this manual	xvi
 Part 1 Advanced facilities	
 1 Assembly-level programming	3
1.1 Inline assembly code	4
1.2 Stand-alone assembly code	10
align	14
blkb	15
blkw	16
byte	17
comment	18
debug	19
defsym	20
extern	21
global	22
language	23
section	24
size	25
sourcefile	26
toolname	27
word	28
1.3 Instruction set summary	31
1.4 Support for assembly debugging	37

2	Creating EMI configuration files	39
2.1	Introduction	40
2.2	Running the EMI configurer	42
2.3	stemi range checks	53
2.4	Using the stemi -chip option	54
2.5	Using stemi in interactive mode	55
2.6	Using the stemi -read option	57
2.7	Using the stemi -p option	57
2.8	Using the stemi -poke-change option	58
3	Using the librarian tool	59
3.1	Introduction	59
3.2	Running the librarian	60
4	Using the lister tool	65
4.1	Introduction	65
4.2	Data displays	66
4.3	Running the binary lister	67
4.4	Specify an output file: option -O	69
4.5	Display all symbol information: option -A	69
4.6	Code list: option -C	70
4.7	Exported names: option -E	72
4.8	Hexadecimal/ASCII dump: option -H	72
4.9	Module data: option -M	74
4.10	Library index data: option -N	75
4.11	Specify reference: option -R	75
4.12	Section names: option -S	76
4.13	Full listing: option -T	76
4.14	File identification: option -W	78
4.15	External reference data: option -X	79
5	Stack depth and memory maps	81
5.1	Stack depth analysis	81
5.2	Memory map file	86

6	Code and data placement	91
6.1	Placing code and data	91
6.2	Default placements	94
6.3	Restrictions on placement	95
7	Profiling	97
7.1	Introduction	97
7.2	Types of profiling	98
7.3	Using profiling data	98
7.4	How to perform profiling	99
7.5	Flat profiling	100
7.6	Idle profiling	102
7.7	Call graph profiling	104
7.8	Profiling in batch mode	108
8	Using the ST20 simulator tool	109
8.1	st20sim command line	110
8.2	Undefined values	113
8.3	Command language and start-up scripts	113
8.4	Generating trace information	114
8.5	Generating statistics	122
8.6	Manipulating memory	129
8.7	Event support	129
8.8	Pseudo-ops	131
9	ROM systems	133
9.1	ROM system overview	134
9.2	An example program and target configuration	135
9.3	Multiple programs	139
9.4	Callbacks before and after the poke loop in romload()	142
9.5	OS20 awareness	143

10	Building and running relocatable code	145
10.1	Introduction	145
10.2	Overview of RCU development	149
10.3	Support for calling functions from other programs	154
10.4	Using RCUs with OS20	156
11	Using st20run with OS20	159
11.1	Selecting the OS20 run-time system	159
11.2	Task awareness	160
11.3	RTOS aware printing	164

Part 2 ANSI C and C++

12	Preprocessing	167
12.1	The #indent preprocessor directive	167
12.2	The #pragma preprocessor directive	167
12.3	Macros	177
13	Implementation details	181
13.1	Data type representation	181
13.2	Type conversions	187
13.3	Environmental arguments to main	187
13.4	Identifiers	188
13.5	Source and execution character sets	188
13.6	Integer operations	189
13.7	Registers	189
13.8	Enumeration types	189
13.9	volatile qualifier	190
13.10	Device memory access	191
13.11	Declarators	193
13.12	Switch statement	193
13.13	Preprocessing directives	194
13.14	Static data layout	194
13.15	Calling conventions	195

13.16	Thread safety	203
13.17	C++ link failures caused by conditional symbols	205
14	C++ language features for the EDG preprocessor	207
14.1	C++ dialect accepted	207

Part 3 Libraries

15	Libraries introduction	219
15.1	ANSI functions	220
15.2	stdio file buffers	223
15.3	Concurrency support for the stdio library	223
	getstdiofiletablemutexfn	227
	getstdiomutex	228
	getstdiomutexfn	229
	setstdiofiletablemutexfn	230
	setstdiomutex	231
	setstdiomutexfn	232
15.4	Concurrency support for heap functions	233
	heap_getmutexdata	234
	heap_getmutexfn	235
	heap_setmutexfn	236
15.5	Floating point support for standard ANSI functions	237
15.6	Floating point mathematics library	238
15.7	Fatal run-time errors	238
16	Debugging support functions	241
16.1	Introduction	241
16.2	Concurrency support	242
16.3	Keyboard and screen access	243
16.4	File access	244
16.5	Controlling the debugger	245
16.6	Debugging function definitions	246
	debugbreak	246

debugclose	247
debugcommand	248
debugconnected	249
debugdisconnect	250
debugexit	251
debugfilesize	252
debugflush	253
debuggetenv	254
debuggetkey	255
debuggets	256
debuginformsenabled	257
debugmessage	258
debugopen	259
debugpollkey	260
debugread	261
debugseek	262
debugsetmutexfn	263
debugtell	264
debugtime	265
debugwrite	266
17 Device-independent I/O	267
17.1 Introduction	267
17.2 Filenames and devices	269
17.3 Open file identification	269
17.4 The default device	269
17.5 POSIX functions	270
17.6 Examples	270
17.7 Managing devices	271
17.8 Redirecting streams	275
17.9 Limitations	276
17.10 POSIX API	277
17.11 Device API	278

device_default	279
device_default_set	280
device_getmutexdata	281
device_getmutexfn	282
device_register	283
device_register_init	284
device_setmutexfn	285
device_unregister	286
device_unregister_desc	287
18 Relocatable code support functions	289
18.1 Introduction	289
18.2 Relocatable Code Unit (RCU) initialization	291
rcu_init	291
rcu_deinit	293
18.3 Import/export support	294
dl_match_imports_to_exports	294
dl_reset_imports_from_exports	296
dl_get_exports	297
dl_get_imports	298
19 ROM Restart functions	299
19.1 ROM restart function definitions	300
romrestart	300
romlaststart	302

Part 4 Command language

20 Command language programming	305
20.1 Introduction	305
20.2 Language format	306
20.3 Comments	306
20.4 Operators	307
20.5 Command language objects	308

20.6	Objects in the application program	312
20.7	Command language special variables and scope	315
20.8	Command language constructs	316
20.9	Procedures	316
21	Alphabetical list of commands	319
	#line	320
	addhelp	321
	addressof	322
	alter	324
	bootdata	325
	bootiptr	326
	break	327
	cachecontrol	330
	cd	331
	chip	332
	clerror	336
	clinfo	337
	clsymbol	338
	commandline	339
	compare	341
	connect	342
	context	343
	copy	345
	core	346
	define	348
	directory	349
	disconnect	350
	display	351
	dump	354
	emi	355
	emidelay	356
	endorder	357

entry	359
entrypoint	360
eval	361
events	362
export	364
fappend	365
fclose	366
feof	367
fgets	368
file	370
fill	371
fopen	372
forcewhen	373
forward	374
fputs	375
fread	376
fwrite	377
getenv	378
go	379
heap	380
help	382
import	383
include	384
informs	385
initstack	386
interrupts	387
jtag	388
load	389
log	391
memory	392
memset	396
mkdir	397
mknum	398

mkstr	399
modify	400
modules	401
mv	402
parse	403
peek	404
place	405
poke	407
print	408
processor	411
profile	412
program	414
pwd	416
quit	417
register	418
remove	422
reset	423
restart	424
rewind	425
rm	426
rmdir	427
romimage	428
runtime	432
searchpath	433
session	434
sizeof	435
sizeused	436
space	437
st20ccoptions	438
st20toolsetversion	439
stack	440
startorder	442
statement	444

step	445
stepout	447
stop	449
store	450
symbols	451
sys	453
target	454
task	456
timer	458
trace	459
tracebuffer	461
tracedump	464
traphandler	467
wait	468
when	469
where	470
write	472
Revision history	475
Index	477





Preface

This is the reference manual for the R2.0 release of the ST20 Embedded Toolset. The *ST20 Embedded Toolset Delivery Manual* lists the host platforms supported by the toolset in an appendix *Referenced technology*.

Note: The EMI configuration tool **stemi** is available on Windows only; see the *ST20 Embedded Toolset Delivery Manual* for licensing details.

ST20 documentation suite

The document set provided with the toolset comprises the following documents.

ST20 Embedded Toolset Delivery Manual (ADCS 7257995)

The delivery manual provides installation instructions, a summary of the release and a list of changes since the previous revision.

ST20 Embedded Toolset User Manual (ADCS 7143840)

This manual provides an overview of the toolset and a getting started guide for using the graphical user interface. It also describes how the core features of the toolset are used to build and run application programs. It includes compiling and linking, connecting to a target, loading programs and application debugging.

OS20 User Manual (ADCS 7473749)

This manual is a user guide for the OS20 real-time kernel. It provides an introduction and getting started section then continues with separate chapters for each of the main features supported, such as, kernel, memory and partitions management, tasks, semaphores, message handling queues, real-time clocks and interrupts.

ST20 Embedded Toolset Reference Manual (this document)

This manual describes the advanced facilities of the toolset such as the assembler, EMI tool, librarian, lister, and ST20 instruction set simulator. It also describes facilities such as code and data placement, the stack depth and memory map files, the use of relocatable code units and profiling and trace facilities. Reference information is provided for the C and C++ implementations, the libraries and the command language.

Conventions used in this manual

Typographical conventions

The following typographical conventions are used in this manual:

Bold	Used within text for emphasis
<i>Blue Italic</i>	Denotes hyperlink cross-references
<i>Italic</i>	Denotes nonhyperlink cross-references
UPPER CASE	Denotes special terminology, for example, register or signal/pin names
Monotype bold	Denotes command options, command line examples, code fragments, and program listings
<i>Monotype bold italic</i>	Denotes arguments or parameters in command syntax definitions
<i>Monotype italic</i>	Denotes code comments
Braces { }	Denotes a list of optional items in command syntax
Brackets []	Denotes optional items in command syntax
Ellipsis ...	In general terms, used to denote the continuation of a series. For example, in syntax definitions denotes a list of one or more items.
	In command syntax, separates two mutually exclusive alternatives
---	Denotes two underscores together
	A change bar in the left margin indicates a change from the previous version of the manual. This may indicate a change in the functionality of the toolset or merely an updated description.

Command line conventions

Example command lines and directory path names are documented using UNIX®/Linux style notation which makes use of the forward slash (/) delimiter. In most cases this should be recognized by Windows hosts; if not, substitute the forward slash with a backslash (\). For example, the directory:

release/examples/simple

is the same as:

release\examples\simple

Command line options are prefixed by a hyphen (-); this is compatible with Linux, Windows and UNIX.

Examples of the debugging tools use the following convention to distinguish command prompts:

“%” is used to indicate the operating system command prompt, for example:

% **st20run** ...

“>” is used to indicate the interactive command language prompt, for example:

> **break** ...



Part 1

Advanced facilities





Assembly-level programming

1

This chapter describes the facilities provided for assembly code programming. The chapter explains how to include inline assembly code in C and C++ source code, and how to write assembly code for processing by the assembler. Macros provided to support assembly debugging are described in [Section 1.4 on page 37](#).

Assembly code may be either:

- included inline in C source code using the `__asm` or `__optasm` statements and built using the compiler, or,
- written in a stand-alone assembly code file and built into an object file using the assembler.

The syntax summaries for inline and stand-alone assembly code can be found in [Section 1.1.6 on page 8](#) and [Section 1.2.8 on page 29](#) respectively.

The instruction sets are documented in the *ST20-C1 Core Instruction Set Reference Manual* (ADCS 7473696) and the *ST20-C2 Core Instruction Set Reference Manual* (ADCS 7301761). The instructions and mnemonics for each core are listed in [Section 1.3: Instruction set summary on page 31](#). The rest of this chapter assumes some familiarity with the ST20 instruction set.

1.1 Inline assembly code

The insertion of assembly code into C programs is performed using the `__asm` or `__optasm` statements. The compiler assembles such ST20 instruction sequences inline, and issues a warning if an instruction is used which is not supported on the target processor.

`__asm` statements can contain any number of primary or secondary ST20 instructions, optionally preceded by a `size` qualifier, or ST20 pseudo-operations. Any ST20 instruction can be prefixed with a label.

The formal syntax of the `__asm` and `__optasm` statements is given in [Section 1.1.6: Syntax for inline code](#).

The `__optasm` statement is syntactically the same as the `__asm` statement, however, it supports a subset of the instructions supported by `__asm`. Its advantage is that the compiler can apply global optimization to `__optasm` statements, whereas, global optimization is suppressed for any function containing the `__asm` statement.

The language differences between `__asm` and `__optasm` are qualified in [Section 1.1.1](#).

1.1.1 Pseudo-instructions

Pseudo-instructions are directives recognized by the compiler. They are expanded into one or more instructions, depending upon their context and parameters. Pseudo-instructions that are supported by `__asm`, `__optasm` are listed in [Table 1](#). Pseudo-instructions can only be used as inline assembly code.

The load and store pseudo-instructions (`ld`, `st`, `ldab`, `stab`, `ldabc`, `stabc`) load or store the integer registers. The compiler generates the best code it can to evaluate the expressions and load or store the appropriate registers. This code may use other registers or temporary memory locations and therefore it should not be assumed that any values held on the stack persist across a load or store pseudo operation. The expressions may include C variables defined outside the `__asm`, `__optasm` construct.

The operands to the load pseudo-operations must be small enough to fit in a register.

The operands to the store pseudo-instructions can be any valid modifiable C object, such as a variable or an array element. Labels can be any valid C label.

Pseudo-instruction	Description
<code>ld expression</code>	Load a value into the AREG.
<code>st lvalue</code>	Store the value from the AREG.
<code>ldab expression, expression</code>	Load values into the AREG and BREG. The left hand expression is placed in AREG.
<code>stab lvalue, lvalue</code>	Store values from the AREG and BREG. The leftmost element receives AREG.
<code>ldabc expression, expression, expression</code>	Load values into AREG, BREG and CREG. The leftmost expression is placed in AREG.
<code>stabc lvalue, lvalue, lvalue</code>	Store values from the AREG, BREG, and CREG. The leftmost element receives AREG.
<code>[size constant] j label</code>	Jump.

Table 1: Pseudo-instructions

Pseudo-instruction	Description
<code>[size constant] cj label</code>	Conditional jump.
<code>[size constant] call label</code>	Call ^a .
<code>[size constant] ldlabeldiff label - label</code>	Load the difference between the addresses of two labels into AREG.
<code>byte constant {, constant }</code>	Copy a stream of literal bytes into the instruction stream. Only the lower 8 bits of each constant values are generated. For example, if the constant is too large to fit in a byte, the upper bits are truncated. ^a
<code>word constant {, constant }</code>	Copy a stream of literal 32-bit signed values into the instruction stream. If any constant is too large to fit in a 32-bit signed word, the upper bits are truncated. ^a
<code>align</code>	Generate padding bytes (prefix 0) until the current code address is on a word boundary. This instruction takes no operands.

Table 1: Pseudo-instructions

a. Illegal for `__optasm`

`__optasm` supports the same instruction set as `__asm` with the following exceptions.

- Variables must only be accessed using pseudo-instructions. (The operands to the store pseudo-instructions can be any size modifiable *lvalue*).
- The following operations are **not** included in the `__optasm` statement:

`ldl constant`, for example: `ldl 5`,
`stl constant`, for example: `stl 6`,
`ajw constant`, for example: `ajw 6`,
`ldlp constant`, for example: `ldlp 5`,
`call label_name`,
`fcall label_name`,
`gcall`,
`j constant`, for example: `j 5`,
`cj constant`, for example: `cj 5`,
`word constant`, for example: `word 56`,
`byte constant`, for example: `word 35`,
`jab`.

Both the pseudo-instructions and the function as a whole may be optimized when `__optasm` is used.

1.1.2 The size option

For inline assembly code, the **size** option forces an instruction to occupy a specified number of bytes. Any instruction and certain pseudo-instructions may be preceded by **size constant**, where **constant** is the number of bytes required. If the instruction is shorter than this, it is padded out with trailing **prefix 0** instruction components. If the instruction cannot fit in the specified number of bytes, a compiler error is reported. The **size** option



allows instructions to be built with the same size and is intended to assist the creation of jump tables.

1.1.3 Labels

Labels can be placed on `__asm` and `__optasm` statements or on any line of ST20 code. Labels placed inside and outside `__asm` and `__optasm` statement are handled identically. C statements are permitted to `goto` a label set inside `__asm` and `__optasm` statements and vice versa.

1.1.4 Built-in variables

The compiler has built into it special recognition of the following variables:

`__params`

```
extern volatile const void *__params
```

`__params` is a pointer to the base of the parameter block of the current function.

Given access to the parameter block, it is possible to determine the return address of the function and the work space of the calling function, as in the following example:

```
void p(int a, int b)
{
    typedef struct paramblock
    { void *return_address;
      void *static link;
      int regparam1, regparam2;
    } paramblock;

    extern volatile const void *__params;

    paramblock *pp = __params;
    /* return address is: pp->return_address
       caller's wptr is: (void *) (pp + 1); */
}
```

The above structure is suitable for running on an ST20-C2 core; on an ST20-C1 core the following parameters should be removed:

```
int regparam1, regparam2;
```

`__LOCAL_SIZE`

The integer variable `__LOCAL_SIZE` is available for use as the argument of an `ajw` instruction within an `__asm` section. Two forms only are permitted:

```
AJW __LOCAL_SIZE /*reserves the correct number of words in the
                  workspace for the current function's locals */

AJW __LOCAL_SIZE /*returns the space used by
                  the current function's locals */
```

This variable is generally only useful within a function declared as `#pragma ST_naked_function(function)`. For further information see [Section 12.2.4: Pragma ST_naked_function on page 170](#) and [Section 13.15.4: Using the trap, interrupt and naked pragmas on page 198](#).

1.1.5 Inline code examples

Labels and jumps

```
void p(void)
{
    int a, b, c;
    /* The following code performs: if (b > c) a = b; else a = c; */
    __asm{
        ldab c,b;
        gt;
        cj  label1;
        ld  b;
        st  a;
        j   done;
label1:
        ld  c;
        st  a;
done:   ;
    }
}
```

Loading constants using literals

```
#define answer 42
const int c;
__asm {
    ldc 17;           /* decimal */
    ldc 0xff;         /* hexadecimal */
    ldc 0377;         /* octal */
    ldc answer;       /* defined by macro */
    ldc sizeof(c);    /* constant expression */
    ldc 10+7;         /* ditto */
}
```

Using align and word to return an element of a table

```
int p(int i)
{
    /* The following code returns the ith
       element of the table defined below */
    int res;
    __asm{
        ld i;
        ldlabeldiff table - here;
        ldpi;
here:
        wsub;
        ldnl 0;
        st res;
        j done;
        /* Make sure table is word aligned for ldnl to work correctly */
        align;
table:
        word 1, 1, 2, 3, 5, 8, 13, 21, 34;
    }
done:
    return res;
}
```

Jump tables

```
#include <stdio.h>
#define JUMP_SIZE 3

void p(int i)
{
    __asm{ ld i;                /* load the index */
          adc -1;               /* subtract base subscript */
          ldc JUMP_SIZE;        /* scale by size of table entry */
          prod;
          ldlabeldiff table - here; /* load pointer to start of table */
          ldpi;

here:
          bsub;                /* add the offset */
          gcall;               /* jump to ith entry */

table:
          size JUMP_SIZE j lab1;
          size JUMP_SIZE j lab2;
          size JUMP_SIZE j lab3;
          size JUMP_SIZE j lab4;
          }

lab1: printf("i = 1"); return;
lab2: printf("i = 2"); return;
lab3: printf("i = 3"); return;
lab4: printf("i = 4"); return;
}
```

Inserting raw machine code

The following code inserts the actual machine code (in hexadecimal) for the ST20-C2 **ret** instruction.

```
void ret_hex(void)
{
    __asm { byte 0x22, 0xF0; }
}
```

1.1.6 Syntax for inline code

The syntax of inline assembly language is defined in this section. In the syntactic description, terminal symbols are shown in **Courier** or described between angle brackets (< >). Non-terminal symbols are shown in **Courier *italics***. Alternatives are separated by a vertical bar (|). Optional symbols are shown in square brackets ([]) and repeated items in braces ({ }).

primary-op is a mnemonic, in lower case, of one of the primary instructions listed in [Section 1.3: Instruction set summary](#). **secondary-op** is a mnemonic, in lower case, of any secondary instruction listed in [Section 1.3](#).


```

statement ::= asm-statement
asm-statement ::= __asm { asm-directive }
                 | __optasm { asm-directive }
asm-directive ::= [ size constant ] primary-op constant ;
                 | [ size constant ] secondary-op ;
                 | pseudo-op ;
                 | identifier: asm-directive
                 ;
primary-op ::= <any primary instruction (in lower case)>
secondary-op ::= <any secondary instruction (in lower case)>
pseudo-op ::= ld expression
                 | st lvalue
                 | ldab expression, expression
                 | stab lvalue, lvalue
                 | ldabc expression, expression, expression
                 | stabc lvalue, lvalue, lvalue
                 | [ size constant ] j label
                 | [ size constant ] cj label
                 | [ size constant ] call label
                 | [ size constant ] ldlabeldiff label - label
                 | byte constant { , constant }
                 | word constant { , constant }
                 | align
lvalue ::= expression
constant ::= constant-expression
label ::= identifier
expression ::= <as defined in X3.159-1989 ANSI standard for C>
constant-expression ::= <as defined in X3.159-1989 ANSI standard for C>
identifier ::= <as defined in X3.159-1989 ANSI standard for C>

```

Note: The following exception: dollar signs "\$" are reserved for use by the compiler and must not be used in assembly symbols.

1.2 Stand-alone assembly code

The assembler is supplied as an integral part of the compiler/linker tool **st20cc** and is normally run as the final stage of compilation. The command line interface of the tool enables the user to invoke the assembler directly. This suppresses the compilation phase of the compiler and the input file is passed directly to the assembler.

Assembly files have basic debug data generated automatically and can be debugged by **st20run**. A number of macro are provided in **asmdebug.h** which may be used in assembly source files to allow backtracing and symbolic debugging. These are documented in [Chapter 16: Support for assembly debugging on page 237](#).

The assembler is a cross-assembler, which enables a source file written in assembly code to be translated into object code. The assembler accepts a single ASCII text file as input, consisting of instructions and assembler directives, and issues an error if an instruction is used which is not supported on the target processor.

If required the compiler preprocessor can be run on an assembly source file and the output file generated by the preprocessor used as input to the assembler.

The assembler generates an object file in the proprietary format (TCOFF) used by the toolset. This file may then be linked with other TCOFF object files or library modules to produce a linked unit, which may then be loaded onto a target and executed.

The formal syntax for stand-alone assembly code is given in [Section 1.2.8: Syntax for stand-alone assembly language](#).

1.2.1 Language

Assembly code is made up of lines of ASCII text. Each line can contain the following:

- a label definition,
- assembler commands separated by semi-colons,
- a comment.

None of the above may extend to more than one line.

An assembler command is one of the following:

- an instruction mnemonic (with its operand, if any),
- an assembler directive.

The instructions are listed in [Section 1.3: Instruction set summary](#). Primary instructions are always followed by an operand, which can be any constant or constant expression. If prefixing is required to encode large or negative values, then the assembler automatically generates the required prefix bytes. Secondary operations never have an operand. They are automatically converted by the assembler into the appropriate **opr** instruction component and any prefixing.

All instruction mnemonics are reserved words.

Label definitions

Label names can contain alphanumeric characters and underscore (_) characters. Labels in stand-alone code may also contain the full-stop character (.). A label is defined by terminating its name by a colon (:), for example:

```
label:
```

A label is used by giving its name without the colon, for example:

```
j label          -- jump to label
```

Labels are also referred to as code symbols.

Expressions

The assembler can recognize simple integer expressions constructed from operators, operands and brackets. An operator performs a mathematical or logical operation on the operand or operands in the expression. Permitted operators are listed in [Table 2](#).

Operator	Meaning	Precedence
-	Unary minus	1
~	Bitwise not	
!	Symbol offset	
*	Multiplication	2
/	Division	
%	Modulus	
+	Addition	3
-	Subtraction	
&	Bitwise and	4
	Bitwise or	
^	Bitwise exclusive or	
<	Logical left shift	5
>	Logical right shift	

Table 2: Integer expression operators

Evaluation is left-to-right, except for unary operations where the operator closest to the operand binds more tightly, with precedence rules as follows. The lower the number in the precedence column of the table, the higher the precedence of the operator. Brackets, (. . .), may be used to alter the order of evaluation of an expression.

All calculations are performed in two's complement arithmetic, modulo 32 bits, with the following exceptions, which result in a serious error being reported:

- division and modulus by zero,
- division of the most negative integer by -1.

[Table 3](#) indicates the different types of operand which may appear within expressions:

Operand	Value
Number	The numerical value
Defined symbol	The value assigned to the symbol using defsym . (stand-alone code only)
Code symbol	The offset in bytes from the end of the instruction containing the expression to the symbol

Table 3: Operands

When code symbols are preceded by the **!** operator, the value yielded is the offset of the symbol from the start of the section in which it is defined. This is the offset after the linker has concatenated all of the sections together, and hence this value can only be calculated by the linker.

1.2.2 Running the assembler

The assembler is invoked by using the **-A** option of the compile/link tool **st20cc**. It is invoked to assemble a source file by the following command line:

```
st20cc filename.ext -A {options}
```

where **filename.ext** is the filename and extension of the assembly source file; see [Section 1.2.3](#). By convention, the extension for assembly source files is **.s**.

options are **st20cc** command options; see [Section 1.2.4](#) and the *ST20 Embedded Toolset User Manual*, chapter *st20cc compile/link tool*.

1.2.3 Specifying the source filename

The full filename, including the extension, must always be supplied on the **st20cc** command line when the **-A** option is used. If an extension is not supplied the compiler assumes a C source file is to be compiled and searches for an input file which has the appropriate name plus the **.c** file extension.

Filenames follow the same format required for any file submitted to the compiler; see the *ST20 Embedded Toolset User Manual*, chapter *st20cc compile/link tool*.

1.2.4 Use of st20cc command line options with the assembler

Many of the **st20cc** command line options have no meaning if used with the assembler and are ignored. The only options which have meaning are:

- c1** and **-c2** specify a processor core type,
- v** display progress information as the tool runs,
- o filename** specify an output filename.

1.2.5 Using the preprocessor with the assembler

Preprocessor directives may be included in assembly source files; pragma directives, however, should not. Preprocessing of assembly files is automatic.

1.2.6 Comments

A comment in stand-alone assembly code is introduced by the characters “--”. If a comment needs to be split over more than one line, each line must start with the characters “--”. All text appearing on the same line and to the right of these characters is interpreted as a comment. For example:

```
-- A comment on a line of its own
j fred -- A comment at the end of a line
```

1.2.7 Assembler directives

This section briefly describes each assembler directive and provides an example where appropriate. Directives appear in alphabetical order. Directives may not be used with inline code. The names of assembler directives are reserved keywords. [Table 4](#) summarizes the directives available.

Directive	Description
align	Align the next generated byte to word boundary
blkb	Generate a block of bytes
blkw	Generate a block of words
byte	Generate a sequence of bytes set to specified values
comment	Cause a comment to be written to the object file
debug	Generate a debug information record
defsym	Assign a value to a symbol; used only for local symbols within an assembly file
extern	Declare a symbol to be external to the module
global	Define a symbol to be globally visible
language	Define the language of the current module
section	Define the section for the following code up to a subsequent section directive
size	Pad out an instruction so that it occupies a specified number of bytes
sourcefile	Override the default source file name with the specified name
toolname	Override the record of the tool used to create the object file with the specified tool name
word	Generate a sequence of words set to specified values

Table 4: Assembler directives

align

Align the next generated byte on a word boundary

Syntax

align

Description

The **align** directive causes the next generated byte in the code section to be aligned on the next word boundary (where its address is a multiple of four).

blkb

Insert a block of bytes

Syntax

```
blkb size {, expr_ | _string}
```

Description

The **blkb** directive inserts a block of bytes in the instruction stream. The number of bytes to be inserted is given by the first expression, the **size**. The value of the bytes is given by the list of operands which follow the first expression. If no operands are given then zero bytes are generated. If an operand is an expression and its value is not in the range 0 to 255 then an error is reported.

If the operand is a string then the characters of the string are written to consecutive bytes of memory.

If **size** is greater than the number of bytes of expressions and strings, then the remaining bytes are set to zero. It is an error to give too many bytes of expressions and strings to the **blkb** directive.

Example

```
blkb 10, "hello", 6, 7, 8, 9, 10
```

This generates the byte values "h", "e", "l", "l", "o", 6, 7, 8, 9, 10 to consecutive bytes of memory.

blkw

Insert a block of words

Syntax

```
blkw size [, expr]
```

Description

The **blkw** directive inserts a block of words in the instruction stream. The number of words to be generated is given by the first expression, the *size*. The value of the words is given by the operands which follow the *size*. Each of the operands is itself an expression.

If no operands are given, or the *size* given is zero or negative, then zero words are generated. If too few operands are given then the remaining words are set to zero. It is an error to give too many operands to the **blkw** directive.

Each word is stored in little-endian format, that is the least significant byte first, at the lowest address.

Example

```
blkw 3 + 4, 1, 2, 3, 4, 5, 6, 7
```

The above generates seven words in memory with the values 1, 2, 3, 4, 5, 6 and 7.

byte

Insert a sequence of bytes

Syntax

```
byte expr_ | _string {, expr_ | _string}
```

Description

The **byte** directive inserts in the instruction stream a sequence of bytes, with values defined by the directive operands. Any number of operands may be given, each operand being either a byte expression or a string. Each expression must generate a byte value, that is a value in the range 0 to 255, or an error is generated. If an operand is a string then the characters of the string are assigned to consecutive bytes of memory.

Example

```
byte "hello", 6, 7, 8, 9, 10
```

This generates the byte values "h", "e", "l", "l", "o", 6, 7, 8, 9, 10 to consecutive bytes of memory.

comment

Insert a comment

Syntax

```
comment string
```

Description

This directive causes the *string* to be written to the object file as a TCOFF comment. The comment is printable but cannot be copied, so it does not appear in a linked unit.

Comments can be seen by using the lister tool with the **-m** option on the object file.

Example

To write the comment "Hello" in the object file:

```
comment "Hello"
```

debug

Define a debugger information record

Syntax

debug

Description

The **debug** directive defines a debugger information record and is designed for use by compilers and other programs generating assembler language output. It is listed here for completeness.

defsym

Define symbol with numeric value

Syntax

defsym *symbol* *expr*

Description

This directive allows a numeric value to be assigned to the *symbol*. The value is given by the expression *expr*. This *symbol* can then be used in expressions as if it were the value itself.

This is only for use internally to an assembly file. These symbols cannot be made global and used in other files.

There are two types of symbols used in the assembly language:

- code symbols - labels, as defined in [Section 1.2.1: Language on page 10](#),
- defined symbols - symbols defined using the **defsym** directive.

Once a symbol has been defined as one type of symbol it cannot be redefined to another.

Code symbols may not appear in the expression; only symbols defined using **defsym** are permissible. Any symbol used in the expression must have been previously defined.

Example

To set the symbol **one** to represent the value 1:

```
defsym one 1
```

one can be used in expressions as if it were the value 1.

extern

Define symbol as external

Syntax

```
extern symbol [weak] [realname string] [origin string] [datausage]  
                [routineusage] [notypeusage]
```

Description

The **extern** directive is used to declare the *symbol* as external to the current module. An example of its use is to call an external routine.

The optional **weak** argument indicates that a weak reference is to be made to the symbol. **weak** references do not force the linker to resolve the reference. They are used for example, to resolve debug information, but do not cause the linker to pull in extra library modules simply to resolve debug information.

The name used for the symbol in the object file is the same as the name of the symbol in the assembly source file, unless **realname** is specified. In this case the name used for the symbol in the object file is the name specified by the string argument to **realname**.

origin enables an origin to be specified for the symbol in the object file. This allows the linker to ensure that only a definition of a symbol with the same name and the same origin matches this external reference. If **origin** is not specified, then the symbol in the object file is not given a specific origin. The linker can match this external reference with a definition of a symbol with the same name and any origin.

The three optional arguments **datausage**, **routineusage** and **notypeusage** generate extra symbol information in the object file, which can be incorporated by the linker into a symbol table. This may be used by tools such as a debugger or profiler, when reading symbol information.

datausage Indicates that the symbol is associated with a data object.

routineusage Indicates that the symbol is associated with a routine.

notypeusage Indicates that the symbol type (data or routine) is not specified.

global

Define symbol as global

Syntax

```
global symbol [realname string] [origin string] [datausage]
               [routineusage] [notypeusage] [commonusage]
```

Description

The **global** directive is used to cause a **symbol** to become available outside the current module. It causes the **symbol** to become globally visible so that it can be accessed by the linker. The operand to a **global** directive, that is the **symbol**, may be a code or data symbol, but it **must not** be a symbol defined by the **defsymb** directive. The **global** directive can appear before the definition of its operand.

The name used for the symbol in the object file is the same as the name of the symbol in the assembly source file, unless **realname** is specified. In this case the name used for the symbol in the object file is the name specified by the string argument to **realname**.

origin enables an origin to be specified for the symbol in the object file. This allows the linker to ensure that only a reference to a symbol with the same name and the same origin matches this symbol. If **origin** is not specified, then the symbol in the object file is not given a specific origin. The linker can match this symbol to a symbol with the same name and any origin.

The four optional arguments **datausage**, **routineusage**, **notypeusage** and **commonusage** generate extra symbol information in the object file which can be incorporated by the linker into a symbol table. This may be used by tools such as a debugger or profiler, when reading symbol information.

datausage	Indicates that the symbol is associated with a data object.
routineusage	Indicates that the symbol is associated with a routine.
notypeusage	Indicates that the symbol type (data or routine) is not specified.
commonusage	Indicates to the linker that the symbol is a tentative definition; see Section 13.1.4: Tentative variables on page 186 .

Example

To create a routine called **fred** which can be called from external modules:

```
fred:
  global fred routineusage
  -- do fred's operations
```

To create a routine called **fred** which can be called from external modules by the name "**fred's name**", only with the specific origin "**fred's origin**":

```
fred:
  global fred realname "fred's name" origin "fred's origin"
  routineusage
  -- do fred's operations
```

language

Mark module language type

Syntax

language *language_type*

Description

Used to mark a module with its original source language type. For example, the C compiler would mark assembler files that it produced as **ansi_c**. *language_type* is one of the following:

- **unknown**,
- **ansi_c**,
- **assembler**,
- **c_plusplus**.

If no **language** directive appears in the input file then the language defaults to **assembler**. The language type is written into the TCOFF **START_MODULE** record.

Example

To set the language type for the current file to be ANSI C:

```
language ansi_c
```

section

Define new section

Syntax

section *string type*

Description

Defines the section for the following code up until a subsequent **section** directive. The *type* can be any of **text**, **data**, **bss** or **const**.

Example

```
section "device_routines" text
section "global_arrays" data
section "uninitialized_data" bss
```


size

Encode next instruction in a fixed number of bytes

Syntax

size *expr instruction*

The **size** directive causes the instruction following to be encoded in exactly *expr* bytes. If the instruction can be encoded in fewer bytes, then padding is added before the instruction to increase the size of the instruction to *expr* bytes. The padding used is **prefix 0** instructions.

An error is reported if the size given is too small for the instruction.

Example

To force a jump instruction to occupy 4 bytes:

```
size 4 j label
```

sourcefile

Change source file name

Syntax

sourcefile *string*

Description

The **VERSION** TCOFF record which appears at the start of all TCOFF object files contains a string which holds the name of the source file used to create the object file. The **sourcefile** directive causes this name to be changed to the name given by its *string* operand.

If no **sourcefile** directive appears in the input then the source file name defaults to the filename given on the command line by the user.

Example

To enter the filename **fred.s** as the source file:

```
sourcefile "fred.s"
```

toolname

Change source tool name

Syntax

toolname *string*

Description

The **VERSION** TCOFF record which appears at the start of all TCOFF object files contains a string which holds the name of the tool used to create the object file. The **toolname** directive causes this name to be changed to the name given by its string operand.

If no **toolname** directive appears in the input then the tool name string defaults to **iasm**.

Example

To set the tool name to be **fred**:

```
toolname "fred"
```

word

Insert word(s)

Syntax

```
word expr [, expr]
```

Description

The **word** directive inserts into the instruction stream a sequence of words containing the values of the expressions. Each word is stored in little-endian format, that is the least significant byte is at the lowest address.

Example

```
word 3, 3 + 4
```

This stores the values 3 and 7 to the next two consecutive words in the code section.

1.2.8 Syntax for stand-alone assembly language

The syntax of the stand-alone assembly language is defined in this section. In the syntactic description, terminal symbols are shown in **Courier** or described between angle brackets (< >). Non-terminal symbols are shown in *italics*. Alternatives are separated by a vertical bar (|). Optional symbols are shown in square brackets [] and repeated items in braces ({ }).

primary-op is a mnemonic, in lower case, of one of the primary instructions listed in [Section 1.3: Instruction set summary](#). **secondary-op** is a mnemonic, in lower case, of any secondary instruction listed in [Section 1.3](#).

```

assembly-file ::= line { newline line }
line          ::= [ label-def ] [ command-list ] [ comment ]
label-def     ::= symbol :
command-list  ::= command { separator command }
separator     ::= newline
               | ;
command       ::= tp-instruction
               | directive
comment       ::= -- string
tp-instruction ::= primary-op expression
               | secondary-op
primary-op    ::= <any primary instruction (in lower case)>
secondary-op  ::= <any secondary instruction (in lower case)>
directive     ::= align
               | blkb expression { , expr-or-string }
               | blkw expression { , expression }
               | byte expr-or-string { , expr-or-string }
               | comment string
               | debug number , number { , number-or-string }
               | defsymb symbol expression
               | extern symbol [weak] [realname string]
               |               [origin string] [datausage]
               |               [routineusage] [notypeusage]
               | global symbol [realname string] [origin string]
               |               [datausage] [routineusage]
               |               [notypeusage]
               | language language-type
               | section string sec-type
               | size expression tp-instruction
               | sourcefile string
               | toolname string
               | word expression { , expression }
language-type ::= unknown
               | ansi_c
               | c_plusplus
               | assembler
expr-or-string ::= expression
               | string
number-or-string ::= number
               | string
expression     ::= number
               | symbol
               | monadic-expression
               | dyadic-expression
               | ( expression )

```

```

dyadic-expression ::= expression dyadic-operator expression
monadic-expression ::= monadic-operator expression
dyadic-operator  ::= *
                  | /
                  | %
                  | +
                  | -
                  | &
                  | |
                  | ^
                  | <
                  | >
monadic-operator ::= -
                  | ~
                  | !
number           ::= decimal-number
                  | hex-number
decimal-number   ::= <any base 10 number>
hex-number       ::= hex-intro <any base 16 number>
hex-intro        ::= #
                  | 0x
                  | 0X
symbol           ::= <any alphanumeric characters>
                  | .
                  | _ (underscore)
string           ::= " <any sequence of printable ASCII characters> "

```

Note: Dollar signs, "\$", are reserved for use by the compiler and must not be used in assembly symbols.

1.3 Instruction set summary

This section gives a full listing of all mnemonics for the primary and secondary instructions and instruction components for the supported ST20 cores.

1.3.1 ST20-C1 core

Primary instructions and instruction components

Primary instructions always have an operand, shown here as *n*.

Mnemonic	Name
adc <i>n</i>	add constant
ajw <i>n</i>	adjust work space
cj <i>n</i>	conditional jump
eqc <i>n</i>	equals constant
fcall <i>n</i>	function call
j <i>n</i>	jump
ldc <i>n</i>	load constant
ldl <i>n</i>	load local
ldlp <i>n</i>	load local pointer
ldnl <i>n</i>	load non-local
ldnlp <i>n</i>	load non-local pointer
nfix <i>n</i>	negative prefix
opr <i>n</i>	operate
pfix <i>n</i>	prefix
stl <i>n</i>	store local
stnl <i>n</i>	store non-local

Table 5: ST20-C1 core - primary instructions and instruction components

Secondary instructions

Mnemonic	Function	Mnemonic	Function
add	add	or	or
addc	add with carry	order	order
and	and	orderu	unsigned order
arot	anti-rotate stack	rev	reverse
ashr	arithmetic shift right	rmw	read modify write
biquad	biquad IIR filter step	rot	rotate stack
bitld	load bit	run	run process
bitmask	create bit mask	saturate	saturate
bitst	store bit	sbinc	store byte and increment
breakpoint	breakpoint	shl	shift left
dequeue	dequeue a process	shr	shift right
divstep	divide step	signal	signal
dup	duplicate	smacinit	initialize short multiply accumulate loop
ecall	exception call	smacloop	short multiply accumulate loop
enqueue	enqueue a process	smul	short multiply
eret	exception return	ssinc	store sixteen and increment
gajw	general adjust workspace	statusclr	clear bits in STATUS register
gt	greater than	statusset	set bits in STATUS register
gtu	greater than unsigned	statustst	test STATUS register
io	input/output	stop	stop process
jab	jump absolute	sub	subtract
lbinc	load byte and increment	subc	subtract with carry
ldpi	load pointer to instruction	swap32	byte swap 32
ldprodid	load product identity	swinc	store word and increment
ldtdesc	load task descriptor	timeslice	timeslice
lsinc	load sixteen and increment	umac	unsigned multiply accumulate
lsxinc	load sixteen sign extended and increment	unsign	unsign argument
lwinc	load word and increment	wait	wait
mac	multiply accumulate	wsub	word subscript
mul	multiply	xbword	sign extend byte to word
nop	no operation	xor	exclusive or
not	bitwise not	xsword	sign extend sixteen to word

Table 6: ST20-C1 core - secondary instructions

1.3.2 ST20-C2 core

Primary instructions and instruction components

Primary instructions always have an operand, shown here as *n*.

Mnemonic	Function
adc <i>n</i>	add constant
ajw <i>n</i>	adjust work space
call <i>n</i>	call
cj <i>n</i>	conditional jump
eqc <i>n</i>	equals constant
j <i>n</i>	jump
ldc <i>n</i>	load constant
ldl <i>n</i>	load local
ldlp <i>n</i>	load local pointer
ldnl <i>n</i>	load non-local
ldnlp <i>n</i>	load non-local pointer
nfix <i>n</i>	negative prefix
opr <i>n</i>	operate
pfix <i>n</i>	prefix
stl <i>n</i>	store local
stnl <i>n</i>	store non-local

Table 7: ST20-C2 core - primary instructions and instruction components

Secondary instructions

Mnemonic	Function	Mnemonic	Function
add	add	devss	device store sixteen
alt	alt start	devsw	device store word
altend	alt end	diff	difference
altwt	alt wait	disc	disable channel
and	and	diss	disable skip
bcnt	byte count	dist	disable timer
bitcnt	count bits set in word	div	divide
bitrevnbits	reverse bottom <i>n</i> bits in word	dup	duplicate top of stack
bitrevword	reverse bits in word	enbc	enable channel
bsub	byte subscript	enbs	enable skip
causeerror	cause error	enbt	enable timer
cb	check byte	endp	end process
cbu	check byte unsigned	fmul	fractional multiply
ccnt1	check count from 1	fpsterr	test for FPU error
cflerr	check floating point error	gajw	general adjust workspace
cir	check in range	gcall	general call
ciru	check in range unsigned	gintdis	global interrupt disable
clockdis	clock disable	gintenb	global interrupt enable
clockenb	clock enable	gt	greater than
clrhalt	clear halt-on-error	gtu	greater than unsigned
crcbyte	calculate CRC on byte	in	input message
crcword	calculate CRC on word	insertqueue	insert at front of scheduler queue
cs	check sixteen	intdis	(localized) interrupt disable
csngl	check single	intenb	(localized) interrupt enable
csu	check sixteen unsigned	iret	interrupt return
csub0	check subscript from 0	ladd	long add
cword	check word	lb	load byte
devlb	device load byte	lbx	load byte and sign extend
devls	device load sixteen	ldclock	load clock
devlw	device load word	ldiff	long diff
devmove	device move	ldinf	load infinity
devsb	device store byte	ldiv	long divide

Table 8: ST20-C2 core - secondary instructions

Mnemonic	Function	Mnemonic	Function
ldmemstartval	load value of MemStart address	reboot	reboot
ldpi	load pointer to instruction	rem	remainder
ldpri	load current priority	resetch	reset channel
ldprodid	load product identity	restart	restart
ldshadow	load shadow registers	ret	return
ldtimer	load timer	rev	reverse
ldtraph	load trap handler	roundsn	round single length floating point number
ldtrapped	load trapped process status	runp	run process
lend	loop end	satadd	saturating add
lmul	long multiply	satmul	saturating multiply
ls	load sixteen	satsub	saturating subtract
lshl	long shift left	saveh	save high priority queue registers
lshr	long shift right	savel	save low priority queue registers
lsub	long subtract	sb	store byte
lsum	long sum	seterr	set error flags
lsx	load sixteen and sign extend	sethalterr	set halt-on-error flag
mint	minimum integer	settimeslice	set timeslicing status
move	move message	shl	shift left
move2dall	2D block copy	shr	shift right
move2dinit	initialize data for 2D block move	signal	signal
move2dnonzero	2D block copy non-zero bytes	slmul	signed long multiply
move2dzero	2D block copy zero bytes	ss	store sixteen
mul	multiply	ssub	sixteen subscript
nop	no operation	startp	start process
or	or	stclock	store CLOCK register
norm	normalize	sthb	store high priority back pointer
not	not	sthf	store high priority front pointer
out	write message	stlb	store low priority back pointer
outbyte	write byte	stlf	store low priority front pointer
outword	write word	stoperr	stop on error
pop	pop processor stack	stopp	stop process
postnormsn	post-normalize correction of single length fp number	stshadow	store shadow registers
prod	product	sttimer	store timer

Table 8: ST20-C2 core - secondary instructions

Mnemonic	Function	Mnemonic	Function
sttraph	store trap handler	trapdis	trap disable
strapped	store trapped process	trapenb	trap enable
sub	subtract	tret	trap return
sulmul	signed times unsigned long multiply	unpacksn	unpack single length fp number
sum	sum	wait	wait
swapqueue	swap scheduler queue	wcnt	word count
swaptimer	swap timer queue	wsub	word subscript
talt	timer alt start	wsubdb	form double word subscript
taltwt	timer alt wait	xbword	sign extend byte to word
testerr	test error flag	xdble	extend to double
testhalterr	test halt-on-error flag	xor	exclusive or
testpranal	test processor analyzing	xsword	sign extend sixteen to word
timeslice	timeslice	xword	extend to word
tin	timer input		

Table 8: ST20-C2 core - secondary instructions

1.4 Support for assembly debugging

The assembler invoked by **st20cc** has the ability to generate debug records to allow backtracking, and symbolic debugging of assembly code. This is achieved through the use of five macros, which can be found in **asmdebug.h** and are listed in [Table 9](#).

Macro	Description
FUNC_BEG	Function beginning
FUNC_RET	Function return
FUNC_END	Function end
AJW_BEG	Adjust work space beginning
AJW_END	Adjust work space end

Table 9: Assembly debugging support macros

These macros may only be used in assembly source files; they do not work with assembly inserts using the **__asm** and **__optasm** constructs.

Three of the macros relate to function definition:

FUNC_BEG(*Fname*, *Workspace*)

FUNC_RET(*Fname*, *Workspace*)

FUNC_END(*Fname*, *Workspace*)

where *Fname* is the name of the function, and *Workspace* is the size of its local work space, in words.

FUNC_BEG() performs the following jobs:

- 1 declares the label **Fname**, and globalizes it,
- 2 performs the **ajw -Workspace** operation,
- 3 generates debug records, recording information about the function.

FUNC_RET() performs the following jobs:

- 1 adjusts the work space back to the value specified for the work space beginning,
- 2 returns to the caller.

FUNC_END() marks the limit of the function's executable code.

Example 1

```
#include <asmdebug.h>
FUNC_BEG($j, 3)
    stl    4          -- <static_link>
    stl    5          -- <Arg_0>
    ldc    5
    ld1    5          -- i
    fcall  $remainder%c
    ld1    4          -- <static_link>
    ldnlp  ($pp - $__staticlink)/4
    wsub
    ldnl  0
    FUNC_RET($j, 3)
FUNC_END($j, 3)
```

There are also two macros to permit temporary work space adjustment:

AJW_BEG (*Workspace*)

AJW_END (*Workspace*)

where *Workspace* is the size of its local work space, in words.

These macros perform the work space adjustment, and generate the required debug information.

Example 2

```
#include <asmdebug.h>
    ldc 6
    stl 3          -- <Arg_2>
    ldc 77
    ldc 66
    AJW_BEG(3)
    stl 0
    AJW_BEG(1)
    stl 0
    AJW_END(1)
    stl 0
    AJW_END(3)
    ldc 4
    ldl 5          -- <static_link>
    fcall $f
```

Adjustments may be nested, but their ranges cannot overlap. Each **AJW_BEG**() must have a matching **AJW_END**() .

Example 3

Both the following examples are illegal:

```
AJW_BEG(1)
AJW_BEG(2)
AJW_END(1)  -- nesting does not match...
AJW_BEG(2)
```

and

```
AJW_BEG(1)
AJW_BEG(1)
AJW_END(2)  -- does not match...
```

There is some rudimentary error checking to enforce these rules.



Creating EMI configuration files

2

This chapter describes the tool **stemi** which generates the configuration values for ST External Memory Interface (EMI) devices. The EMI controls external memory, and before external memory can be used, the EMI must be configured. The user supplies the EMI configuration parameters to **stemi**, which generates a file that contains the pokes to the EMI registers. In addition, **stemi** can be given a file containing poke commands to “decode”, from which it generates a list of corresponding EMI configuration parameters. **stemi** interprets the toolset command language defined in [Part 4 Command language on page 303](#). **stemi** is supported on a PC host running Windows.

Three ST EMI parts are currently supported: EMI2, EMI3 and EMI4.

- EMI2 supports DRAM, FLASH and peripheral devices.
- EMI3 is the same as EMI2 with additional support for SDRAM.
- EMI4 supports MPX, SDRAM, FLASH and peripheral devices.

For example, the chip STi5510 has an EMI2, whereas the chip STi5512 has an EMI3 device. Refer to the relevant device datasheet for details of the device's EMI characteristics.

2.1 Introduction

stemi allows configuration of the EMI for MPX, SDRAM, DRAM, FLASH and peripheral devices.

The following EMI2 registers may be configured:

- 4 banks of four 16-bit data registers,
- DRAM initialize,
- PADLOGIC (optional).

The following EMI3 registers may be configured:

- EMI2 registers (see above),
- SDRAM mode registers 0/1 (SDRAM devices only).

The following EMI4 registers may be configured:

- four 32-bit data registers for each of the four or six banks,
- LOCK,
- GENCFG,
- SDRAM mode,
- SDRAM initialize,
- SDRAM refresh interval,
- FLASH clock speed,
- SDRAM clock speed,
- MPX clock speed,
- CLOCKENABLE.

stemi may be invoked without specifying any input values; in this case the tool uses a default setup, described in [Section 2.1.2](#), and produces an output file containing the default EMI register pokes. **stemi** always produces an output file unless the user is accessing the help/verbose options.

stemi options may be specified via:

- the **stemi** command line,
- an ST toolset command language file (by convention such files have a **.cfg** extension),
- an environment variable.

The output file generated by **stemi** (see [Figure 1: stemi outputs](#)) may be:

- **a poke file**

This has the form of a command language file containing a procedure with a series of **poke** statements. The file can be used by the toolset compile/link driver and load/run tools.

If required, the name of the **poke** command can be changed in this file using the **stemi** option **-poke-change**; see [Section 2.8](#) for details. This can also be set as a default option using the **STEMIARG** environment variable; see [Section 2.2.2](#).

- **a text file**

This contains **stemi** parameter options derived from a poke file. It may be fed back into **stemi** to produce a poke file, and so offers a convenient way to decode an existing poke file.

The name of the output file is provided by the user specifying one of the options `-o`, `-io` or `-iro` depending on whether command line or interactive mode is used (see [Section 2.5: Using stemi in interactive mode](#)).

Note: *Commands for more advanced EMI configurations are ignored on chips without the required support, for example, MPX (EMI4) commands to an STi5512 may generate pokes to nonexistent registers.*

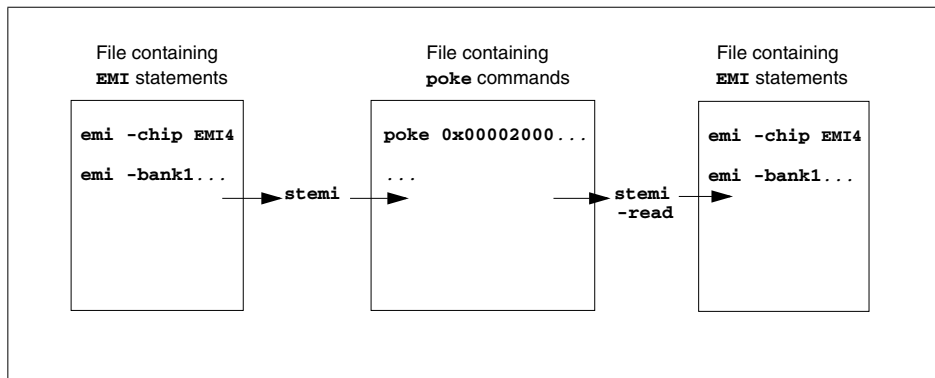


Figure 1: stemi outputs

2.1.1 Functionality

The main functions that **stemi** provides are to:

- read options from the command line,
- read options from command files,
- produce the register **poke** values from supplied configuration parameters,
- produce the configuration parameters from supplied register **poke** values,
- allow interactive usage,
- create a memory configuration file which contains the EMI initialization procedure,
- verify the range of a configuration parameter value and to check the compatibility of a set of configuration parameters.

stemi reads and saves the register/parameter values in the memory configuration file which is named by the user.

2.1.2 Default setup

stemi uses an EMI2 for the default setup. The four memory banks for EMI2 are each set to type **Peripheral** by default. [Table 16: stemi command line options listed by -help-peripheral on page 49](#) contains details of the default parameters for this type.

The default selected bank number is 0 and the default selected register number is also 0. For example, from startup the following line:

```
emi -type Peripheral -regvalue 2
```

sets **bank 0** to type "**Peripheral**" and places the hexadecimal value **2** in register number 2.

The **-bank** option is used to select a different bank and the **-regnum** option is used to select a different register; see [Section 2.2: Running the EMI configurer](#).

The only way to change the default chip type for **stemi** is to use the **-chip** option; see [Section 2.2](#).

2.2 Running the EMI configurer

To invoke the EMI configurer use the following command line:

► **stemi** *filenames* {*options*}

where **filenames** can be one or more input files separated by spaces.

Note: The input files are processed in command line order. For example:

```
stemi myfile1 myfile2 myfile3 myfile4 -chip EMI4 -o myemi.out
```

options is a list of one or more options listed below in [Table 10](#) to [Table 17](#). If an option is not specified it takes the default value specified in the appropriate table.

Options:

- may be entered in upper or lower case,
- are order-specific,
- can appear within a command file; see [Section 2.2.3: stemi command files](#),
- must be separated by spaces.

If no arguments are given on the command line, a short help page is displayed giving some of the command syntax.

Option	Description
-bank number	Specify bank number for selection. number has the range 0 to 3. Default selected bank number is 0.
-chip device	Specify the required chip device or EMI part: 5508 , 5510 , 5512 , 5514 , 5518 , EMI2 , EMI3 or EMI4 . For example, -chip 5512 selects the STi5512 device. The default value for -chip is EMI2 . See Section 2.4 on page 54 for further details.
-help	Show long help page.
-help-dram	List all DRAM options. Useful for EMI2 and EMI3 parts.
-help-emi4	List specific EMI4 options.
-help-gencfg	List all GENCFG register options. Useful for EMI4 parts.
-help-mpx	List all MPX options. Useful for EMI4 parts.
-help-padlogic	List all PADLOGIC register options. Useful for EMI2 and EMI3 parts.
-help-peripheral	List all peripheral options.
-help-sdram	List all SDRAM options. Useful for EMI3 and EMI4 parts.
-i	Enter interactive mode (use quit to exit).
-icfg	Display current EMI pokes in interactive mode.
-io filename	Write current EMI pokes from interactive mode to filename .
-iparameters	Display current parameters in interactive mode.
-ireset	Reset banks to defaults from interactive mode.
-iro filename	Write parameters from current read file from interactive mode.
-o filename	Send output to the specified filename .
-p procedure	Specify command procedure to execute.
-poke-change name	Change output poke command to specified name ; see Section 2.8 on page 58 for further details.
-proc-name name	Specify procedure name for output. Default name is configEmi .
-read filename	Read EMI poke file to obtain stemi parameters.
-regnum number	Select bank register number . number has the range 0 to 3. Default selected register number is 0.
-regvalue value	Enter value into selected bank register. value is in hexadecimal.
-type type	Specify bank type (SDRAM/DRAM/MPX/Peripheral) for selected bank number. Default for all banks is Peripheral .
-v	Show version information.

Table 10: stemi command line options listed by -help

Option	Description	Valid values	Default
-case1time value	Set CAsE1Time in phases.	1 to 4	1
-case2time value	Set CAsE2Time in phases.	0 to 3	0
-castime value	Set CAStime in cycles.	2 to 5	2
-datadrivedelay value	Set DataDriveDelay in phases.	0 to 1	0
-latchpoint value	Define Latch Point.	0 to 1	0
-multibyte value	Set Multibyte (disable/enable).	0 (disable) 1 (enable)	0
-portsize value	Set PortSize (32/16/8) in bits.	32 16 8	32
-precharge time value	Set Precharge Time in cycles.	1 to 8	1
-rasbits value	Set RASBits value in hexadecimal.		0
-rase1time value	Set RAsE1Time in phases.	1 to 2	1
-rase2time value	Set RAsE2Time in phases.	0 to 3	0
-rastime value	Set RAStime in cycles.	1 to 2	1
-refreshinterval value	Set Refresh Interval in cycles.	1 to 16	1
-refreshrasedge value	Set RefreshRASedge (1 or 2) cycles.	1 2	1
-shiftamount value	Define ShiftAmount.	7 to 14	7
-subbanks value	Define SubBanks (1/2/4).	1 2 4	1
-subbanks size value	Set SubBankSize for memory banks. DRAM (.256, 1, 4, or 16) in Mwords.	256 1 4 16	256
-type type^a	Specify bank type for selected bank number.	DRAM, dram or D	Peripheral

Table 11: stemi command line options listed by -help-dram

- a. As shown in [Table 10](#) the default for this option is always **Peripheral1**. However when using **-help-dram**, this is not a valid option, so **DRAM**, **dram** or **D** must be specified.

Option	Description	Valid values	Default
-base-address addr	Specify base address of EMI4 pokes in hexadecimal; see Section 2.4.1 for further details		0x0
-clockenable value	Clock generation for devices (disable/enable)	0 (disable) 1 (enable)	0
-flashclkssel value	Set clock ratio for FLASH	0 to 2	0
-gencfg value	Set general purpose configuration register in hexadecimal		0x0
-lock value	Enter lock register value (hexadecimal)	0x0 to 0x3f	0x0
-mpxclkse value	Set clock ratio for MPX	0 to 2	0
-numofbank value	Set number of EMI4 banks; see Section 2.4.2 for further details.	4 or 6	6
-refreshinterval value	Set refresh interval for all SDRAM in cycles	8 to 4096	8
-sdramclkssel value	Set clock ratio for SDRAM	0 to 2	0

Table 12: stemi command line options listed by -help-emi4

Option	Description	Valid values	Default
-bank0addshift value	Set Bank 0 Address Shift (disable/enable)	0 (disable) 1 (enable)	0
-bank1addshift value	Set Bank 1 Address Shift (disable/enable)	0 (disable) 1 (enable)	0
-bank2addshift value	Set Bank 2 Address Shift (disable/enable)	0 (disable) 1 (enable)	0
-bank3addshift value	Set Bank 3 Address Shift (disable/enable)	0 (disable) 1 (enable)	0
-bank4addshift value	Set Bank 4 Address Shift (disable/enable)	0 (disable) 1 (enable)	0
-bank5subbanksizes value	Set bank 5 Sub-Decoding (4/8/16/32/64/1000) in Mbits	4 8 16 32 64 1000	4
-bank5subdecode value	Set bank 5 Sub-Decoding (disable/enable)	0 (disable) 1 (enable)	0
-bootindicator value	Set Boot indicator (Boot mode/normal)	0 (boot mode) 1 (normal)	0

Table 13: stemi command line options listed by -help-gencfg

Option	Description	Valid values	Default
-csa-type value	Set CSA type (other/Bank 0 SDRAM)	0 (other) 1 (Bank 0 SDRAM)	0
-csb-type value	Set CSB type (other/Bank 0 SDRAM)	0 (other) 1 (Bank 0 SDRAM)	0
-csc-type value	Set CSC type (other/Bank 0 sub-decoded SDRAM)	0 (other) 1 (Bank 0 subdecoded SDRAM)	0
-csd-type value	Set CSD type (other/Bank 0 sub-decoded SDRAM)	0 (other) 1 (Bank 0 subdecoded SDRAM)	0
-dack0 value	Set DACK0 Control (normal/enable)	0 (normal) 1 (enable)	0
-dack1 value	Set DACK1 Control (normal/enable)	0 (normal) 1 (enable)	0
-emi-ackreq value	Set EMIAckReq (Retime Stages)	0 to 2	0
-emi-intbusreq value	Set EMI internal Bus Req Control (disable/enable)	0 (disable) 1 (enable)	0
-emi-reqGnt value	Set EMIREqGnt (Retime Stages)	0 to 2	0
-emi-wait value	Set EMIIWaitNotTready (Retime Stages)	0 to 3	0
-gencfg addr	Set general purpose configuration register in hexadecimal		0x0

Table 13: stemi command line options listed by -help-gencfg

Option	Description	Valid values	Default
-busreleasetime value	Set BusReleaseTime (1/2/3/4) in cycles	1 2 3 4	1
-clockenable value	Set frequency divider (disable/enable)	0 (disable) 1 (enable)	0
-mpxciksel value	Set clock ratio for MPX	0 to 2	0
-extendedmpx value	Use (standard/superset) opcodes	0 to 1	0
-strobeonfalling value	Do strobes on clock (rising/falling) edge	0 (rising) 1 (falling)	0
-type type^a	Specify bank type for selected bank number	MPX , mpx or M	Peripheral
-waitpolarity value	Set wait input pin polarity active (high/low)	0 (high) 1 (low)	1
-waitstatesframe value	Set number of access wait states	0 to 3	0
-waitstatesread value	Set number of read wait states	0 to 7	1
-waitstateswrite value	Set number of write wait states	0 to 7	2

Table 14: stemi command line options listed by -help-mpx

- a. As shown in [Table 10](#) the default for this option is always **Peripheral**. However when using **-help-mpx**, this is not a valid option, so **MPX**, **mpx** or **M** must be specified.

Option	Description	Valid values	Default
-bank3subdecode value	Define Bank3subdecode. Valid for chip STi5512 .	0 (disable) 1 (enable)	0
-clockenable value	Define SDRAMclockenable. Valid for chip STi5512 .	0 (disable) 1 (enable)	1
-clocktristate value	Define SDRAMclocktristate. Valid for EMI3 devices	0 (yes) 1 (no)	1
-mpeg value	Define MPEG strobos	0 (disable) 1 (enable)	0
-mshift0 value	Set MemoryshiftBank0	0 (disable) 1 (enable)	0
-mshift1 value	Set MemoryshiftBank1	0 (disable) 1 (enable)	0
-mshift2 value	Set MemoryshiftBank2	0 (disable) 1 (enable)	0
-mshift3 value	Set MemoryshiftBank3	0 (disable) 1 (enable)	0
-mwait value	Define Memwait	0 (asynch) 1 (synch)	0
-padlogic-on	Display PADLOGIC register		
-padlogic-off	Remove PADLOGIC register		
-padlogic value	Enter PADLOGIC register value in hexadecimal		
-subbanks-size-padlogic value	Set SubBankSize for PADLOGIC register in Mbits. Valid for chip STi5512 .	4 8 16 32 64 1000	4

Table 15: stemi command line options listed by -help-padlogic

Option	Description	Valid values	Default
-accesstimeread value	Set AccessTimeRead in cycles	2 to 17	10
-accesstimewrite value	Set AccessTimeWrite in cycles	2 to 17	10
-beactive value	Set BEOActive	0 (inactive) 1 (read only) 2 (write only) 3 (read and write)	0
-beelitimeread value	Set BEe1TimeRead in phases	0 to 3	0
-beelitimewrite value	Set BEe1TimeWrite in phases	0 to 3	0
-bee2timeread value	Set BEe2TimeRead in phases	0 to 3	0
-bee2timewrite value	Set BEe2TimeWrite in phases	0 to 3	0
-burstmode value	Set synchronous flash burst mode. Valid for EMI4 parts	0 to 1	0
-burstsize value	Set BurstSize (2/4/8/16/32/64/128) in bytes. Valid for EMI4 parts.	2 4 8 16 32 64 128	2
-busreleasetime value	Set BusReleaseTime in cycles	1 to 3	2
-clockenable value	Set frequency divider (disable/enable). Valid for EMI4 parts.	0 (disable) 1 (enable)	0
-csactive value	Set CSActive	0 (inactive) 1 (read only) 2 (write only) 3 (read and write)	1
-cse1timeread value	Set CSe1TimeRead in phases	0 to 3	0
-cse1timewrite value	Set CSe1TimeWrite in phases	0 to 3	0
-cse2timeread value	Set CSe2TimeRead in phases	0 to 3	0
-cse2timewrite value	Set CSe2TimeWrite in phases.	0 to 3	0
-cyclenotphaseread value	Change measure unit (phases/cycles). Valid for EMI4 parts.	0 (phases) 1 (cycles)	0
-cyclenotphasewrite value	Change measure unit (phases/cycles). Valid for EMI4 parts.	0 (phases) 1 (cycles)	0
-datadrivedelay value	Set DataDriveDelay in phases	0 to 7	5
-dataholddelay value	Set delay in burst mode (1 or 2) cycles. Valid for EMI4 parts.	1 2	1

Table 16: stemi command line options listed by -help-peripheral

Option	Description	Valid values	Default
-datalatency value	Set number of Flash clock cycles in cycles. Valid for EMI4 parts.	2 to 17	2
-devicetype value	Set DeviceType: 1 (normal) 4 (Burst FLASH). Valid for EMI4 parts.	1 (normal) 4 (Burst FLASH)	1
-flashclkssel value	Set clock ratio for FLASH. Valid for EMI4 parts.	0 to 2	0
-latchpoint value	Define Latch Point	0 to 2	0
-oeactive value	Set OEActive	0 (inactive) 1 (read only) 2 (write only) 3 (read and write)	1
-oeeltimeread value	Set OEe1TimeRead in phases	0 to 3	0
-oeel2timeread value	Set OEe2TimeRead in phases	0 to 3	0
-oeeltimewrite value	Set OEe1TimeWrite in phases	0 to 3	0
-oeel2timewrite value	Set OEe2TimeWrite in phases	0 to 3	0
-portsize value	Set PortSize in bits	32 16 8	32
-strobeonfalling value	Set clock (rising/falling) edge. Valid for EMI4 parts.	0 (rising) 1 (falling)	0
-type type	Specify bank type for selected bank number	Peripheral, peripheral or P	Peripheral
-waitpolarity value	Set wait (active high/low). Valid for EMI4 parts.	0 (active high) 1 (active low)	0
-weuseoeconfig value	Set configurable readwrite signal (low/high). Valid for EMI4 parts.	0 (low) 1 (high)	0

Table 16: stemi command line options listed by -help-peripheral

Option	Description	Valid values	Default
-activatetoread value	Define ActivateToRead in cycles	1 to 4	1
-activatetowrite value	Define ActivateToWrite in cycles	1 to 4	1
-burstlength value	Set BurstLength	0 to 7	2
-bursttype value	Set BurstType	0 to 1	0
-busreleasetime value	Set BusReleaseTime in cycles	1 to 2	1
-caslatency value	Set CASLatency (and LatencyMode) in cycles. Equivalent to -latencymode . The last value set overrides any previous value .	1 to 4 (EMI3) 1 to 8 (EMI4)	2
-clockenable value	Set frequency divider (disable/enable). Valid for EMI4 parts.	0 (disable) 1 (enable)	0
-datadrivedelay value	Set DataDriveDelay in phases. Valid for EMI3 parts.	0 to 1	0
-drambanks value	Set the number of DRAMBanks to 2 or 4.	0 (2) 1 (4)	1
-latencymode value	Set LatencyMode (and CASLatency). Equivalent to -caslatency . The last value set overrides any previous value .	1 to 4 (EMI3) 1 to 8 (EMI4)	2
-modesetdelay value	Define ModeSetDelay in cycles	3 to 4	3
-portsize value	Set PortSize in bits	32 16 8	32
-precharge time value	Set Precharge Time in cycles	1 to 8	2
-rasbits value	Set RASBits value in hexadecimal		0
-refreshinterval value	Set Refresh Interval. value * 128 cycles (EMI3) value in cycles (EMI4)	1 to 16 (EMI3) 8 to 4096 (EMI4)	6 (EMI3) 8 (EMI4)
-refreshtime value	Define RefreshTime in cycles	1 to 16	3
-read filename	Read EMI poke file to obtain stemi parameters		
-shiftamount value	Define ShiftAmount	7 to 14	8
-sdramclkssel value	Set clock ratio for SDRAM. Valid for EMI4 parts	0 to 2	0
-sdrammodereg addr	Enter SDRAMMODEREG register value in hexadecimal. See Section 2.4.3 for further details.		0x0
-strobeonfalling value	Set clock (rising/falling) edge ^a	0 (rising) 1 (falling)	0

Table 17: **stemi** command line options listed by **-help-sdram**

Option	Description	Valid values	Default
-subbanks value	Define SubBanks (1/2/4)	1 2 4	1
-subbanksizes value	Set SubBankSize for memory banks (16/32/64/128) in Mbits	16 32 64 128	16
-type type^b	Specify bank type for selected bank number	SDRAM, sdram or S	Peripheral
-writerecoverytime value	Set WriteRecoveryTime in cycles	0 to 3	1

Table 17: stemi command line options listed by -help-sdram

- If two banks are configured as SDRAM, this bit is set the same for both banks. Valid for EMI4 parts.
- As shown in [Table 10](#) the default for this option is always **Peripheral**. However when using **-help-sdram**, this is not a valid option, so **SDRAM**, **sdram** or **S** must be specified.

2.2.1 Examples

```
stemi -chip EMI3 -type SDRAM -padlogic-on -o myemi.out
```

In this example, the target chip uses an EMI3 part, bank number 0 is configured as an SDRAM type and the PADLOGIC register is set to be included in the output. The EMI pokes are then written to the file **myemi.out**.

```
stemi -chip EMI2 -read poke.cfg -o myemiparams.out
```

In this example, **stemi** is given a file containing **poke** commands for an EMI2 part. It reads the **poke** commands and generates an output file **myemiparams.out** containing the EMI parameters that **stemi** would use to generate those **poke** commands. The EMI parameters are given in the form of **emi** commands (see [Section 2.2.3](#)).

2.2.2 Environment variable

A set of default command line options can be defined for the tool using the environment variable **STEMIARG**. Options must be specified in the variable using the syntax required by the command line, for example options should be set for a specified bank. Options from an environment variable are processed before other options.

2.2.3 stemi command files

A list of commands can be submitted to **stemi** via one or more command files, which by convention are given the extension **.cfg**. **stemi** supports the toolset command language syntax and recognizes the command **emi**:

```
emi [options]
```

where **options** is a list of one or more options from [Table 10](#) to [Table 17](#), specified as described in [Section 2.2: Running the EMI configurer](#). Options beginning with the letter “i” in [Table 10](#) (those which select interactive mode) should not be used in command files as they

may cause undesired results. This point also applies to any of the help options “-help...” and the -p option which calls a procedure (see [Section 2.7](#)). To call a procedure from a command file use can be made of the command language's expression handling capabilities.

The following is an example of a command file suitable for input to **stemi**:

```
emi -chip EMI3 -bank0 -type SDRAM -regnum 3 -regvalue FF
emi -bank1 -type DRAM -rastime 2 -precharge time 1
emi -padlogic-off
emi -regnum 3
```

To specify command files on the command line enter each command filename:

```
stemi myemi.cfg -mpeg 1 -o myemi.out
```

The command language is described in [Part 4 Command language on page 303](#).

2.3 stemi range checks

In order to prevent the user from creating a device which is unworkable and unrealistic, **stemi** carries out range checks on parameters affected by user inputs. This may result from a parameter input or register/bank changes. For each entry several checks are made, examples for each bank type are:

DRAM

```
DataDriveDelay < CAStime
CASelTime < CAStime
CASe2Time < CAStime - CASelTime
```

SDRAM

```
CasLatency == LatencyMode
```

To ensure this condition is met, a change of either parameter causes the same change in the other.

Peripheral

```
DataDriveDelay < AccessTimeWrite
CSe2TimeRead < AccessTimeRead - CSe1TimeRead
OEe2TimeRead < AccessTimeRead - OEe1TimeRead
BEe2TimeRead < AccessTimeRead - BEe1TimeRead
OEe2TimeWrite < AccessTimeRead - OEe1TimeWrite
BEe2TimeWrite < AccessTimeRead - BEe1TimeWrite
```

If these conditions are not met, **stemi** produces an error describing the problem parameter and ignores the faulty input.

2.4 Using the stemi -chip option

Each ST chip has a number of attributes, one of which is the EMI part it uses. The option **-chip** is useful to design for a specific chip type, using the default attributes available for the type. Some attributes are not available for certain chips. It is recommended that **stemi** is always used with the **-chip** option. The **-chip** option may also take the values **EMI2**, **EMI3** and **EMI4**. These values specify an ST EMI part rather than a chip and may be used when the more specific chip attributes are not required. Using the **-chip** option and specifying a chip requires extra range checks to be performed by **stemi**. These checks may result in warning or error messages about invalid option selections and inputs.

Supported **-chip** options are:

- **5508** - selects STi5508 chip attributes which include an EMI3,
- **5510** - selects STi5510 chip attributes which include an EMI2,
- **5512** - selects STi5512 chip attributes which include an EMI3,
- **5514** - selects STi5514 chip attributes which include an EMI4,
- **5518** - selects STi5518 chip attributes which include an EMI3,
- **EMI2** - selects EMI2 part attributes,
- **EMI3** - selects EMI3 part attributes,
- **EMI4** - selects EMI4 part attributes.

Some EMI parameter defaults are set dependent upon the **-chip** option specified, and some EMI parameter options are only supported for certain chips. For example, the SDRAM padlogic options of **-bank3subdecode**, **-clockenable** and **-clocktristate** are only supported for **-chip 5512**.

Example

If the following command was used to create an EMI file:

```
stemi -chip 5510 -type SDRAM -o myemi.out
```

the result would be the warning:

```
stemi: Error: option -type SDRAM not supported by chip 5510
```

2.4.1 Using the stemi -base-address option

For chips which use an EMI4 part, **stemi** treats the default base address of the EMI registers as **0x0**. If the user wishes to change this then the **-base-address** option should be used. So for example, in default usage the LOCK register **poke** operation could be:

```
poke 0x00000020 0x00000007
```

Using the option **-base-address 0x20200000** would change the **poke** operation to:

```
poke 0x20200020 0x00000007
```

2.4.2 Using the stemi -numofbanks option

Chips which use an EMI4 part may have four- or six-bank configurations in their design. The **stemi** option **-numofbanks** allows the user to specify a value of **4** or **6**, thus configuring the number of banks displayed or saved to the output file. **stemi** is also able to read in poke files with either number of banks and reflect it in the parameter options produced.

2.4.3 Using the stemi -sdrammodereg option

For chips with an EMI4 part, bits 0 to 15 of the SDRAM mode register are used for one SDRAM bank or for bank 0 when two SDRAM banks are used. Bits 16 to 31 are used for SDRAM in the higher bank when two SDRAM banks are used. The correct positioning of data is handled by **stemi**, so the user needs only to enter the valid data for the currently selected bank. It is not possible to set bits 0 to 31 with one use of this option. This option should only be used when the selected bank type is “**SDRAM**”, otherwise an error will be reported.

For chips with EMI3, the **-sdrammodereg** option causes **stemi** to place the entered data in either the SDRAMMODEREG0 or SDRAMMODEREG1 locations depending on the selected bank and number of SDRAM banks selected. SDRAMMODEREG0 is used in the case of one SDRAM bank in the device and when bank0 is selected when there are two SDRAMs. SDRAMMODEREG1 is used when bank1 is selected when two SDRAM banks exist in the device.

For both devices, using this option may affect the values of the following options:

-bursttype, **-burstlength**, **-latencymode** and **-caslatency**; see [Table 17 on page 51](#).

2.5 Using stemi in interactive mode

stemi supports an interactive mode so that the user can experiment with the options.

Interactive mode options begin with the letter “i” and are shown in [Table 10 on page 43](#). Use of these options in normal command line usage will generate errors. By placing the **-i** option on the command line, **stemi** receives a continuous feed from stdin until **quit** or **q** is entered on to the command line. Once in interactive mode, the options shown in [Table 10](#) are still valid.

Example

```
stemi -i
>-icfg

#####
##
##  EMI configuration file
##
#####

proc configEmi {

    poke 0x00002000    0xb289
    poke 0x00002004    0x8000
    poke 0x00002008    0x8000
    poke 0x0000200c    0x0000

    poke 0x00002010    0xb289
    poke 0x00002014    0x8000
    poke 0x00002018    0x8000
    poke 0x0000201c    0x0000

    poke 0x00002020    0xb289
    poke 0x00002024    0x8000
    poke 0x00002028    0x8000
    poke 0x0000202c    0x0000
```

```

poke 0x00002030    0xb289
poke 0x00002034    0x8000
poke 0x00002038    0x8000
poke 0x0000203c    0x0000

poke 0x00002060    0x0000
}
>-iparameters

emi  -chip EMI2

emi  -bank0 -type Peripheral
emi  -accesstimeread 10 -accesstimewrite 10
emi  -beactive 0 -beeltimeread 0 -bee2timeread 0 -beeltimewrite 0
emi  -bee2timewrite 0 -csactive 1 -cseltimeread 0 -cse2timeread 0
emi  -cseltimewrite 0 -cse2timewrite 0 -datadrivedelay 5 -latchpoint 0
emi  -oeactive 1 -oeeltimeread 0 -oe2timeread 0 -oeeltimewrite 0
emi  -oe2timewrite 0 -portsize 1 -busreleasetime 2

emi  -bank1 -type Peripheral
emi  -accesstimeread 10 -accesstimewrite 10
emi  -beactive 0 -beeltimeread 0 -bee2timeread 0 -beeltimewrite 0
emi  -bee2timewrite 0 -csactive 1 -cseltimeread 0 -cse2timeread 0
emi  -cseltimewrite 0 -cse2timewrite 0 -datadrivedelay 5 -latchpoint 0
emi  -oeactive 1 -oeeltimeread 0 -oe2timeread 0 -oeeltimewrite 0
emi  -oe2timewrite 0 -portsize 1 -busreleasetime 2

emi  -bank2 -type Peripheral
emi  -accesstimeread 10 -accesstimewrite 10
emi  -beactive 0 -beeltimeread 0 -bee2timeread 0 -beeltimewrite 0
emi  -bee2timewrite 0 -csactive 1 -cseltimeread 0 -cse2timeread 0
emi  -cseltimewrite 0 -cse2timewrite 0 -datadrivedelay 5 -latchpoint 0
emi  -oeactive 1 -oeeltimeread 0 -oe2timeread 0 -oeeltimewrite 0
emi  -oe2timewrite 0 -portsize 1 -busreleasetime 2

emi  -bank3 -type Peripheral
emi  -accesstimeread 10 -accesstimewrite 10
emi  -beactive 0 -beeltimeread 0 -bee2timeread 0 -beeltimewrite 0
emi  -bee2timewrite 0 -csactive 1 -cseltimeread 0 -cse2timeread 0
emi  -cseltimewrite 0 -cse2timewrite 0 -datadrivedelay 5 -latchpoint 0
emi  -oeactive 1 -oeeltimeread 0 -oe2timeread 0 -oeeltimewrite 0
emi  -oe2timewrite 0 -portsize 1 -busreleasetime 2
>-io myemi.out
>-type DRAM
>-o myemi2.out
>quit

```

By using the `-io` option the user is able to make multiple file saves (as the save request is carried out immediately) rather than the one available from the use of the `-o` option (where the save occurs at the end of program execution).

2.6 Using the stemi -read option

stemi is able to read in files containing EMI pokes and from these generate the equivalent EMI statements to generate those pokes. The user supplies at most one filename via the **-read** option (see [Table 10 on page 43](#)). An optional output filename may be provided via the **-o** option.

If the file specified by the **-read** option contains more than one procedure, the **-p** option must be used to specify the procedure required otherwise an error is reported; see [Section 2.7](#).

Example

```
stemi -chip EMI3 -read mypokes.cfg -o myparameters.cfg
```

Here the EMI pokes in **mypokes.cfg** are read and the resulting parameters are written to **myparameters.cfg**. Using the **-chip** option is important here in order to let the EMI configurer know which chip type to expect and thus report valid errors and warnings.

If a **-read** option is specified and no output file is given, then the calculated parameters are written to a file with the command procedure name and a **.dat** extension.

For example, if the procedure name in the file to be read is:

```
proc ConfigureEMI_SDRAM {...
```

and no output file is given then the file written to is **ConfigureEMI_SDRAM.dat**.

2.7 Using the stemi -p option

This option allows selected procedures to be run from a command file. The command language input file may contain a number of procedures. The **-p** option specifies which statement **stemi** invokes.

Example

If **myfile.cfg** contains:

```
proc test1 {
    emi -chip EMI2 -type DRAM -o test1.out
}
proc test2 {
    emi -chip EMI3 -bank 1-type SDRAM -o test2.out
}
```

By running the command:

```
stemi myfile.cfg -p test2
```

The output file **test2.out** is generated, which contains an EMI device with bank number 1 of type SDRAM. This also works for the **-read** option, when the EMI pokes are placed in selected procedures in a command file.

2.8 Using the stemi -poke-change option

By default, **stemi** outputs **poke** statements in the following format:

<poke_command> <address> <value>

where **<poke_command>** is the string “**poke**”.

It is expected that a tool that reads in this poke file can process a statement of this format. The **<poke_command>** string may be changed via the **-poke-change** option.

For example the following command line:

stemi -poke-change diffpoke

would result in the **poke** line output, which normally reads

poke 0x0000203c 0x0000

becoming:

diffpoke 0x0000203c 0x0000

The **-poke-change** option is only valid for generating output files from **stemi**. Any files being read in via the **-read** option (see [Section 2.6](#)) need to contain the expected **poke** commands, otherwise **stemi** will generate an error.

Using the librarian tool 3

This chapter describes the librarian tool **st20libr**, which integrates a group of compiled object files into a single unit that can be referenced by a program. The chapter describes the command line syntax and the options available for generating library files.

3.1 Introduction

The librarian builds libraries from one or more object files supplied as input files. The input files may be compiled object files produced by **st20cc** or library files already generated by **st20libr** (see [Section 3.2.3](#)).

The librarian takes a list of object files and integrates them into a single library file that can be used by a program or program module. Each object file in the input list becomes a module in the library which can be selectively loaded. Input files can either be specified as a list on the command line or in a command file, specified via the command line.

The library, once built, contains an index followed by the concatenated modules. The index is generated and sorted by the librarian to facilitate rapid access of the library content by the other tools in the toolset, for example, the linker.

The operation of the librarian in terms of standard file extensions is shown below.

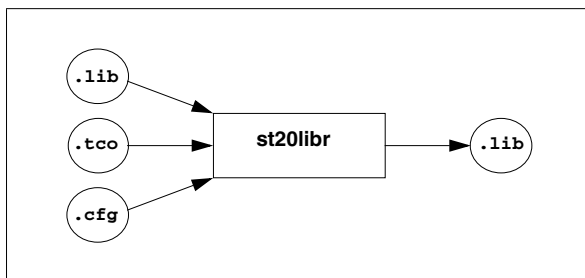


Figure 2: Librarian I/O file types

*Note: Libraries may also be built directly from **st20cc** using the **-lib** option. This may be useful when creating incremental builds using the **st20cc** build management facilities. See the ST20 Embedded Toolset User Manual, chapter *st20cc compile/link tool*.*

3.1.1 Selective loading

When **st20cc** is invoked to link in a library file, it selects modules of the library for linking via the library index on the basis of usage. Only those modules that are actually used by the program are linked into the program.

3.1.2 Building libraries

This section gives some general advice about building libraries.

- Routines that are likely to be used together in a program or procedure (such as those for accessing the file system) can be incorporated into the same library. At a lower level, routines that will **always** be used together (such as those for opening and closing files) can be placed into the same object file.
- If possible, object files should be compiled with debugging enabled. This enables the debugger to locate the library source if an error occurs inside the library.
- When building C libraries, take care to maintain code compatibility if either of the **st20cc** command line options **-fshift** or **-fsigned-char** are used during compilation.

3.2 Running the librarian

To invoke the librarian use the following command line:

► **st20libr *filenames* {*options*}**

where ***filenames*** is a list of input files separated by spaces.
options is a list of one or more options from [Table 18](#).

Options

- may be entered in upper or lower case,
- can be given in any order,
- must not appear within a command file; see [Section 3.2.2](#),
- must be separated by spaces.

The number of file names allowed on a command line is system dependent. To avoid overflow, filenames may be entered using a command file.

If no arguments are given on the command line a help page is displayed giving the command syntax.

Option	Description
EX <i>symbol</i>	Extract the object file which defines the named <i>symbol</i> from the library and store it in the file given by the -o option. The original library remains unchanged.
F <i>filename</i>	Use the specified library command file.
I	Display progress information as the library is built.
L <i>directory</i>	Add directory path to input file search path.
O <i>filename</i>	Send output to the specified file. If this option is not specified the name is taken from the first input filename with a .lib extension.
R <i>symbol</i>	Search for the named <i>symbol</i> in the library file specified on the command line. If the symbol is found, the object file which contains it is replaced by the object file specified on the command line. See Section 3.2.5 .
X	Separate the library into its constituent object files. These are saved in the current directory in individual files.

Table 18: st20libr command line options

Example

```
st20libr myfunc1.tco myfunc2.tco -o myfuncs.lib
```

In this example, the compiled object files **myfunc1.tco** and **myfunc2.tco** are used to create the library **myfuncs.lib**.

3.2.1 Default command line

A set of default command line options can be defined for the tool using the **ST20LIBRARG** environment variable. Options must be specified in the variable using the syntax required by the command line. Options from an environment variable are processed before other options.

3.2.2 Library command files

A list of filenames can be submitted to **st20libr** via a command file. **st20libr** supports the toolset command language syntax and recognizes the command **file**. By convention command files are given the extension **.cfg**.

The following is an example of a command file suitable for input to **st20libr**:

```
file usrproc1.tco      # object files
file usrproc2.tco
file usrproc3.tco
file mylib.lib         # a library file
file usrproc4.tco      # another object file
```

To specify command files on the command line, each command filename must be preceded by the **-F** option.

3.2.3 Library files as input

Library files can themselves be used as input files to **st20libr**. When a library file is used as a component of a new library, its index is discarded by **st20libr**.

3.2.4 Extraction of library modules

The **-EX *symbol*** option enables an object file to be extracted from a library and saved into a file specified using the **-O** option. The command line to perform extraction must contain the following elements:

- a library file to be searched for the required object file,
- the **-EX *symbol*** option naming the symbol which defines the object file to be extracted.

The **-O** option naming the output file in which to save the extracted object file may also be optionally specified.

Only one library file may be searched by the command. If more than one occurrence of ***symbol*** is found then the first is saved in the specified output file (if one was specified) and the remaining occurrences are saved in individual files using the naming convention:

filename~.tcx

where ***filename*** is the original name of the object file.

~ is only added if ***filename.tcx*** already exists.

x takes the value **1** if the object file is compiled for an ST20-C1 or **2** if the file is compiled for an ST20-C2.

The original library remains unchanged.

Example

```
st20libr mylib.lib -EX joe -O sub.tco
```

This example searches the library **mylib.lib** for the symbol **joe**. If **joe** is found then the first object file defining **joe** is copied to the object file **sub.tco**. Any subsequent occurrences are assigned to output files as described above.

3.2.5 Modifying library files

The **-R *symbol*** option enables one object file to be substituted in place of another in an existing library file, provided both object files were compiled for the same processor type. The command line to perform this substitution must contain the following three elements:

- a single library file which is to be modified,
- the **-R *symbol*** option naming the symbol which defines the object file to be replaced,
- the name of the object file which is to be swapped into the library.

Optionally an output file may be specified with the **-O** option.

If the symbol is found in the named library file then the object file which defines the symbol is replaced with the object file specified on the command line and a new library is created which is either:

- written to the specified output file, or,
- if an output file is not specified then the new library overwrites the original library file.

In this second case, if the original library file is corrupted during an aborted process then a backup file containing the original library is generated and named: **old.lib**.

Example

```
st201ibr mylib.lib -R fred update.tco
```

This example searches the library **mylib.lib** for the symbol **fred**. If **fred** is found then the object file defining **fred** is replaced by **update.tco**.

3.2.6 Exploding library files

The **-x** option enables a library file to be exploded, that is, the specified library file is separated into its constituent object files. The object files are saved in the current directory in individual files using the naming convention **filename~.tcx**, as described in [Section 3.2.4](#).

Example

If the following command was used to create a library:

```
st201ibr start.tco run.tco stop.tco -O mylib.lib
```

where **start.tco** **run.tco** **stop.tco** are all compiled for an ST20-C1.

The library file **mylib.lib** would be generated.

If the following command was then used:

```
st201ibr mylib.lib -x
```

it would generate the following output files in the current directory:

```
start.tc1  
run.tc1  
stop.tc1
```





Using the lister tool

4

This chapter describes the binary lister tool **st20list**, which takes a binary file and displays information about the file in a readable form. The chapter provides examples of display options and ends with a list of error messages which may be generated by **st20list**.

4.1 Introduction

The binary lister tool **st20list** reads a binary file, decodes it, and displays useful information on the screen. The output may be redirected to a file. Command line options control the type of data displayed.

The **st20list** tool can decode and display files produced by the toolset compile/link tool **st20cc** and the librarian **st20libr**. Files in editable ASCII format are listed without processing.

Also, because **st20list** uses the same method to locate files as the other tools, that is via the environment variable **ST20ROOT**, it can be used to find and display the location of header files and library files.

4.2 Data displays

There are several categories of data that can be displayed when decoding the output from **st20cc** or **st20libr**. Categories are selected by options on the command line. The main categories are:

- **symbol data**
This shows symbol names in each module, displayed in tabular form.
- **external reference data**
This shows names of external symbols used by each module, displayed in tabular form.
- **module data**
This shows data for each module including target processor, compilation mode, and module file name.
- **code listing**
This shows the code contained in each module, displayed in hexadecimal format.
- **index data**
This shows the content of library indexes.

Example displays used in this chapter

Except where indicated, the example displays used in this chapter show the output generated from the lister for a compiled (**.tco**) file generated by **st20cc**.

4.3 Running the binary lister

To invoke the binary lister, use the following command line:

```
▶ st20list {filenames} {options}
```

where **filenames** is a list of one or more files to be displayed.

options is a list of one or more of the options given in [Table 19](#). It may be entered in upper or lower case and can be given in any order.

If no arguments are given on the command line, a help page is displayed giving the command syntax.

Option	Description
-A	Display all symbol information ^a .
-C	Display the code in the specified file as hexadecimal. This option also invokes the -T option by default.
-E	Display all exported names in the specified modules ^a .
-H	Display the specified file(s) in hexadecimal format.
-I	Display lister version.
-M	Display module data.
-N	Display information from the library index.
-NT	Display with no symbol name truncation.
-NDM	Display names without demangling (only applicable for C++) ^{ab} .
-O filename	Specifies an output file. If more than one file is specified the last one specified is used.
-R reference	Display the library module(s) containing the specified reference. This option is used in conjunction with other options to display data for a specific symbol. If more than one library file is specified the last one specified is used.
-S	Display the section labels of code and data exported by the specified file or library.
-T	Display a full listing of a file in any file format.
-W	Identify a file. The filename (including the search path if applicable) is displayed followed by the file type. This is the default option.
-X	Display all external references made by the specified modules.

Table 19: st20list command line options

- a. This option also displays the full text of symbols that have been translated with the **ST_translate** pragma.
- b. C++ compilers “mangle” names, adding extra information so that, for example, several functions with the same name but with different types of argument or from within different classes/namespaces may co-exist without clashing.

Note: Options only apply to files of the appropriate file type. If the file cannot be displayed by the specified option, an error message is generated and the file is not displayed.



st20list attempts to identify the file type by its contents. If a filename is supplied without an option being specified, **st20list** assumes the default option **-w** and displays the file's identity.

Example of use

```
st20list hello.tco -t
```

Examples of **st20list** usage and the displays generated by the options are shown in subsequent sections.

4.3.1 Options to use for specific file types

Table 20 lists the available options and indicates which file types are recommended to use with each option, in order of usefulness. File types are indicated by standard extensions, where **.tco** is a compiled object file; **.lku** is a linked unit and **.lib** is a library file.

Option	Recommended usage
-H	Any file type
-I	
-O	
-T	
-NT	
-W	
-A	.lib, .lku, .tco
-C	
-E	
-M	
-NDM	
-X	
-N	.lib
-R	
-S	.lib, .tco

Table 20: Recommended usage of options

4.3.2 Output device

st20list sends its output to the standard output stream on the host, normally the terminal screen. Facilities available on the host system may include output redirection to a file or another process such as a sort program. For details of these facilities, consult the specific system documentation.

4.3.3 Default command line

A set of default command line options can be defined for the tool using the **ST20LISTARG** environment variable. Options must be specified using the syntax required by the command line. Options in environment variables are processed before other options.

4.4 Specify an output file: option -O

The **-o** option enables the user to redirect the display data to an output file. If more than one output file is specified on the command line then the last one specified is used. File extensions should be specified, because defaults are not assumed.

Display options are described in [Section 4.5](#) to [Section 4.15](#), in alphabetical order.

4.5 Display all symbol information: option -A

The **st20list** option **-A** returns a line for each section and symbol in a tcoff file (**.tco**, **.lib**, **.lku**) to summarize the section type and symbol linkage.

Each line has the following format:

```
name section_type usage index [origin] module target
```

where

```
name:: symbol name | section name
```

```
section_type:: -----
```

where each dash may be replaced, in order, by one of the following letters: **R W X D V**

R - Read section

W - Write section

X - eXecute section

D - Debug section

V - Virtual section

For example, a **section_type** of **R-X--** is a read and execute section

```
usage:: -----
```

where each dash may be replaced, in order, by one of the following letters:

```
L E I W C U P O T
```

L - Local usage

E - Export usage

I - Import usage

W - Weak usage

C - Conditional usage

U - Unindexed usage

P - Provisional usage

O - Origin usage

T - Common usage

```
index:: Integer value
```

```
optional origin value:: Currently not supported
```

```
module:: Name of link module that defines symbol/section name
```

```
target:: CPU type of symbol/section name
```

4.6 Code list: option -C

The **-c** option produces a full listing of the code in the same format as that generated by the **-t** option, but with the addition of a hexadecimal listing of the code at each **LOAD_TEXT** directive. This option **may** be accompanied by the **-t** option; if the **-t** option is not specified it is supplied automatically.

The output from this option gives an ASCII dump, in hexadecimal format, of the code for each module. It can be used on any object code.

When used to display object code, the code for each module is displayed as a contiguous block of lines, where each line has the format:

address ASCII hex ASCII characters

where **address** is the address of the first byte on the line, expressed as an offset from the start of the module.

ASCII hex is the hexadecimal representation of the code.

ASCII characters are the ASCII characters corresponding to the hexadecimal code.

In all cases code is read from left to right. If a value is not printable it is replaced by a dot “.”.

Example code listing display

Figure 3 shows the code listing display for the compiled file `hello.tco`.

```

00000000 LINKABLE
00000002 START_MODULE CORE FMUL FPSUP DUP WSUBDB MOVE2D CRC BITOPS FPTSTERR
LDDEVID LDMSTVL POP RMC_CORE1 RMC_CORE2 RMC_CORE3 SEMAPHORE DEVICE BIT32 MS<=28
ICALL X lang: ANSI_C "1st"
00000013 VERSION tool: st20icc origin: hello.c
00000025 COMMENT bytes: 20
0000003E COMMENT bytes: 20
00000057 SYMBOL EXP ROU "main" id: 0
00000061 SYMBOL IMP ROU "main" id: 1
0000006B SYMBOL IMP ROU "printf" id: 2
00000077 LOCAL_SYMBOLS number: 3 ids: 3 to 5
0000007A SECTION REA EXE EXP "<hello_text>" ad: 0x0 id: 6
0000008C SET_LOAD_POINT id: 6
0000008F COMMENT PRINT "ANSI C compiler Version 5.05.00 (19:27:41 Jun 2 1998)
(SunOS-Sun4)"
000000DA DEFINE_LABEL id: 5
000000DD DEFINE_LABEL id: 0
000000E0 LOAD_PREFIX size: 0 SV:3-SV:4 instr: ldc
000000E9 LOAD_TEXT bytes: 2
000000EC 21FB !.
000000EE DEFINE_LABEL id: 4
000000F1 LOAD_TEXT bytes: 1
000000F4 71 q
000000F5 LOAD_PREFIX size: 0 AP(SV:2-LP) instr: call
000000FE LOAD_TEXT bytes: 3
00000101 4022F0 @".
00000104 ALIGN modulo: 4
00000107 DEFINE_LABEL id: 3
0000010A LOAD_TEXT bytes: 16
0000010D 68656C6C 6F20776F 726C6420 0A002020 hello world ..
0000011D COMMENT bytes: 5
00000127 COMMENT bytes: 53
00000161 SECTION DEB EXP "debug%1" ad: 0x0 id: 7
0000016E SET_LOAD_POINT id: 7
00000171 LOAD_EXPR size: 4 (SV:0+0)-SC
0000017B LOAD_EXPR size: 4 (SV:5+0)-SC
00000185 LOAD_EXPR size: 4 (SV:2+0)-SC
0000018F END_MODULE
00000191 LINKABLE
00000193 START_MODULE CORE FMUL FPSUP DUP WSUBDB MOVE2D CRC BITOPS FPTSTERR
LDDEVID LDMSTVL POP RMC_CORE1 RMC_CORE2 RMC_CORE3 SEMAPHORE DEVICE BIT32 MS<=28
ICALL X lang: ANSI_C "Sub"
000001A4 VERSION tool: st20icc origin: hello.c
000001B6 COMMENT bytes: 20
000001CF COMMENT bytes: 20
000001E8 SECTION REA WRI EXP "<hello_data>" ad: 0x0 id: 0
000001FA SET_LOAD_POINT id: 0
000001FD END_MODULE

```

Figure 3: Example output produced by the -C option

4.7 Exported names: option -E

The output from this option is in a tabular format. It consists of a list of names exported by the modules. This option also displays any globally visible data.

The following information is given by the display:

- exported name,
- the name of the module in which the exported name is found,
- language used,
- target processor,
- error mode - this should always be “x” for C programs.

Example exported names display

Figure 4 shows the exported names display for the compiled file `hello.tco`.

<code>main-> hello.c</code>	<code>ANSI_C</code>	<code>ST20 x</code>
--------------------------------	---------------------	---------------------

Figure 4: Example output produced by the -E option

4.8 Hexadecimal/ASCII dump: option -H

This option provides a display of the specified files in hexadecimal and ASCII format. The option does not attempt to identify file types and may be used to display any files which the lister has previously identified incorrectly.

The output takes the form of a hexadecimal representation of the whole of the file content. The display has a similar appearance to that produced by the `-C` option, however, the `-C` option only functions on code found within the file.

Each file is displayed as a contiguous block of lines, where each line has the format:

address ASCII hex ASCII characters

where:

- **address** is the address of the first byte on the line, expressed as an offset from the start of the file,
- **ASCII hex** is the hexadecimal representation of the code,
- **ASCII characters** are the ASCII characters corresponding to the hexadecimal code.

In all cases code is read from left to right. If a value is not printable it is replaced by a dot “.”.

Example hexadecimal dump display

Figure 5 shows the hexadecimal dump display for the compiled file **hello.tco**.

```

00000000 0100020F FDF661E 0EFD42E0 07000403 .....f...B....
00000010 3173741B 10077374 32306963 63076865 1st...st20icc.he
00000020 6C6C6F2E 63141700 00142F76 6F622F6D llo.c...../vob/m
00000030 64747372 632D332F 73743230 63631417 dtsrc-3/st20cc..
00000040 00001473 74323069 63632068 656C6C6F ...st20icc hello
00000050 2E63202D 6332201E 08FC0201 046D6169 .c -c2 .....mai
00000060 6E1E08FC 0401046D 61696E1E 0AFC0401 n.....main.....
00000070 06707269 6E74660D 01030B10 06020C3C .printf.....<
00000080 68656C6C 6F5F7465 78743E00 04010614 hello_text>....
00000090 49000146 414E5349 20432063 6F6D7069 I..FANSI C compi
000000A0 6C657220 20566572 73696F6E 2020352E ler Version 5.
000000B0 30352E30 30202831 393A3237 3A343120 05.00 (19:27:41
000000C0 4A756E20 20322031 39393829 20202853 Jun 2 1998) (S
000000D0 756E4F53 2D53756E 34290E01 050E0100 unOS-Sun4).....
000000E0 07070007 03030304 04060302 21FB0E01 .....!....
000000F0 04060201 71070700 0D070302 02090604 ....q.....
00000100 034022F0 0A01040E 01030611 1068656C .@".....hel
00000110 6C6F2077 6F726C64 200A0020 20140800 lo world .. ...
00000120 00050D00 89940614 38000035 09010180 .....8..5....
00000130 00000004 6D61696E 03020100 010E0302 ....main.....
00000140 80000304 0800000E 05048004 0E060480 .....
00000150 04230780 08067072 696E7466 00040804 .#....printf....
00000160 090B0B08 02076465 62756725 31000401 .....debug%1...
00000170 07080804 07060300 01000E08 08040706 .....
00000180 03050100 0E080804 07060302 01000E03 .....
00000190 00010002 0FFDFF66 1E0EFD42 E0070004 .....f...B....
000001A0 03537562 1B100773 74323069 63630768 .Sub...st20icc.h
000001B0 656C6C6F 2E631417 0000142F 766F622F ello.c...../vob/
000001C0 6D647473 72632D33 2F737432 30636314 mdtsrc-3/st20cc.
000001D0 17000014 73743230 69636320 68656C6C ....st20icc hell
000001E0 6F2E6320 2D633220 0B100302 0C3C6865 o.c -c2 .....<he
000001F0 6C6C6F5F 64617461 3E000401 000300 llo_data>.....

```

Figure 5: Example output produced by the -H option

4.9 Module data: option -M

This option displays any header information which is present. This may include version control data, general comments that may have been appended to the file during use of the toolset and copyright information. The data is displayed for individual modules in the object file and includes:

- module name,
- processor type and error mode - this should always be “x” for C programs,
- language used,
- version control data,
- comments inserted by the toolset, for example copyright clauses.

Data is displayed in separate blocks for each module. Some of the data is also used by other tools; for example, the debugger tools use some comments and some tools use version information for compatibility testing.

Example module data display

Figure 6 shows the module data display for the linked unit file **hello.1ku**.

```
MODULE: LINKED          ST20 X
VERSION: st20link <various>
COMMENT: Linker Version 4.02.03 (19:44:56 May 29 1998)
```

Figure 6: Example output produced by the -M option

4.10 Library index data: option -N

This option is used to list library indexes. The data is given in a tabular format. For each entry in the index the following information is given:

- address of the module in the library,
- symbol name,
- language,
- target processor type,
- error mode - this should always be “X” for C programs.

Example library index display

Figure 7 shows part of the output produced by the **-N** option for one of the standard C library files.

0000133C	_ST_bdata	ANSI_C	ST20 X
00002FA1	_ST_bdata	ANSI_C	ST20C1 X
0000133C	_ST_mainheap	ANSI_C	ST20 X
00002FA1	_ST_mainheap	ANSI_C	ST20C1 X
0000133C	_ST_retval	ANSI_C	ST20 X
00002FA1	_ST_retval	ANSI_C	ST20C1 X
0000133C	_ST_startenv	ANSI_C	ST20 X
00002FA1	_ST_startenv	ANSI_C	ST20C1 X
0000133C	_ST_starttime	ANSI_C	ST20 X
00002FA1	_ST_starttime	ANSI_C	ST20C1 X
000001A4	cstart2%c	ANSI_C	ST20 X
00001DF2	cstart2%c	ANSI_C	ST20C1 X
0000120D	mainheap	ANSI_C	ST20 X
00002E70	mainheap	ANSI_C	ST20C1 X

Figure 7: Example output produced by the -N option

4.11 Specify reference: option -R

This option is used in conjunction with any of the other display options to locate a specific symbol within a named library. All library modules that export the symbol are displayed.

The exact format of the display depends on the main display option with which **-R** is used.

Note: Symbol names must be specified in the correct case.

4.12 Section names: option -S

The **-s** option displays all the section names into which code and data has been placed by **st20cc** for the specified file or library. For each section it lists the source file of the code or data. The only sections it does not list are those containing debug information. The following information is given:

- section name,
- file name containing the code or data placed in section name,
- language,
- target processor type.

Figure 8 shows the section data for the compiled file **hello.tco**.

<hello_text>	<- hello.c	ANSI_C	ST20C1 X
<hello_data>	<- hello.c	ANSI_C	ST20C1 X

Figure 8: Example output produced by the -S option

This option may be used with the **-R** option to display the sections used by a particular module in the library. The following command line generates the output in *Figure 9*:

st20list string.lib -s -r strcpy

<strcpy_text>	<- strcpy.c	ANSI_C	ST20 X
<strcpy_text>	<- strcpy.c	ANSI_C	ST20C1 X

Figure 9: Example output produced by the -S and -R options

4.13 Full listing: option -T

This option displays all data found in the input file. Provided that **st20list** recognizes the file type, the file is decoded in its own format. Text file are displayed as text and unrecognized file types are displayed as a hexadecimal dump.

The display formats are tailored to each file format and are intended for diagnostic support and analysis; large amounts of data are produced which may require skilled interpretation.

Example full data display

Figure 10 shows the full data display for a compiled file `hello.tco`, showing module information.

```

00000000 LINKABLE
00000002 START_MODULE CORE FMUL FPSUP DUP WSUBDB MOVE2D CRC BITOPS FPTSTERR LDDEVID
LDMSTVL POP RMC_CORE1 RMC_CORE2 RMC_CORE3 SEMAPHORE DEVICE BIT32 MS<=28 ICALL X lang:
ANSI_C "1st"
00000013 VERSION tool: st20icc origin: hello.c
00000025 COMMENT bytes: 20
0000003E COMMENT bytes: 20
00000057 SYMBOL EXP ROU "main" id: 0
00000061 SYMBOL IMP ROU "main" id: 1
0000006B SYMBOL IMP ROU "printf" id: 2
00000077 LOCAL_SYMBOLS number: 3 ids: 3 to 5
0000007A SECTION REA EXE EXP "<hello_text>" ad: 0x0 id: 6
0000008C SET_LOAD_POINT id: 6
0000008F COMMENT PRINT "ANSI C compiler Version 5.05.00 (19:27:41 Jun 2 1998) (SunOS-
Sun4)"
000000DA DEFINE_LABEL id: 5
000000DD DEFINE_LABEL id: 0
000000E0 LOAD_PREFIX size: 0 SV:3-SV:4 instr: ldc
000000E9 LOAD_TEXT bytes: 2
000000EE DEFINE_LABEL id: 4
000000F1 LOAD_TEXT bytes: 1
000000F5 LOAD_PREFIX size: 0 AP(SV:2-LP) instr: call
000000FE LOAD_TEXT bytes: 3
00000104 ALIGN modulo: 4
00000107 DEFINE_LABEL id: 3
0000010A LOAD_TEXT bytes: 16
0000011D COMMENT bytes: 5
00000127 COMMENT bytes: 53
00000161 SECTION DEB EXP "debug%1" ad: 0x0 id: 7
0000016E SET_LOAD_POINT id: 7
00000171 LOAD_EXPR size: 4 (SV:0+0)-SC
0000017B LOAD_EXPR size: 4 (SV:5+0)-SC
00000185 LOAD_EXPR size: 4 (SV:2+0)-SC
0000018F END_MODULE
00000191 LINKABLE
00000193 START_MODULE CORE FMUL FPSUP DUP WSUBDB MOVE2D CRC BITOPS FPTSTERR LDDEVID
LDMSTVL POP RMC_CORE1 RMC_CORE2 RMC_CORE3 SEMAPHORE DEVICE BIT32 MS<=28 ICALL X lang:
ANSI_C "Sub"
000001A4 VERSION tool: st20icc origin: hello.c
000001B6 COMMENT bytes: 20
000001CF COMMENT bytes: 20
000001E8 SECTION REA WRI EXP "<hello_data>" ad: 0x0 id: 0
000001FA SET_LOAD_POINT id: 0
000001FD END_MODULE

```

Figure 10: Example output produced by the -T option for a .tco file

4.14 File identification: option -W

This option causes the lister to identify the file type. **st20list** takes a heuristic approach to file identification. The filename is displayed along with the file type. The full path to the file is also displayed if the file is not in the current directory. This is the default if no other option is supplied.

Table 21 indicates how the lister classifies file types.

File format	Default extension	Listed file type
TCOFF compiled unit	.tco	TCOFF LINKABLE UNIT
TCOFF compiled library unit	.lib	TCOFF LINKABLE LIBRARY
TCOFF linked unit	.ku	TCOFF LINKED UNIT
Debug information file	.dbg	TCOFF DEBUG FILE
Empty file	-	EMPTY FILE
Text files	-	TEXT FILE
None of the above	-	UNKNOWN BINARY FORMAT

Table 21: File types recognized by st20list

Example file identification display

Figure 11 shows the file identification display for the compiled file **hello.tco** and a linker control file. This output was generated by the following command:

st20list hello.tco -W

hello.tco	COFF LINKABLE UNIT
-----------	--------------------

Figure 11: Example output produced by the -W option

4.15 External reference data: option -X

This option displays a list of all the code and data symbols imported by the modules specified to the lister, that is it lists their external references. External references are references to separately compiled units. For C programs the option also displays any external references to globally visible data.

The output from this option is in a tabular format. It consists of a list of external references and their associated modules. The following information is displayed:

- external reference, that is, the name of the separately compiled unit,
- the name of the module in which the external reference exists,
- language used,
- target processor,
- error mode - this should always be “x” for C programs.

Example external reference data display

Figure 12 shows the external reference data display for the compiled file **hello.tco**.

<code>printf</code>	<code><- hello.c</code>	<code>ANSI_C</code>	<code>ST20 X</code>
---------------------	----------------------------	---------------------	---------------------

Figure 12: Example output produced by the -X option



Stack depth and memory maps

5

When generating a linked unit, the compile/link tool **st20cc** can be directed to output an analysis of the program's stack requirement and a memory map detailing where all symbols have been placed in memory.

st20cc is described in the *ST20 Embedded Toolset User Manual*, chapter *st20cc compile/link tool*.

5.1 Stack depth analysis

st20cc attempts to calculate the stack requirement of each function in the program. In performing this analysis, **st20cc** creates a call graph, that is, a graph representing the call hierarchy of the program; see [Figure 13](#).

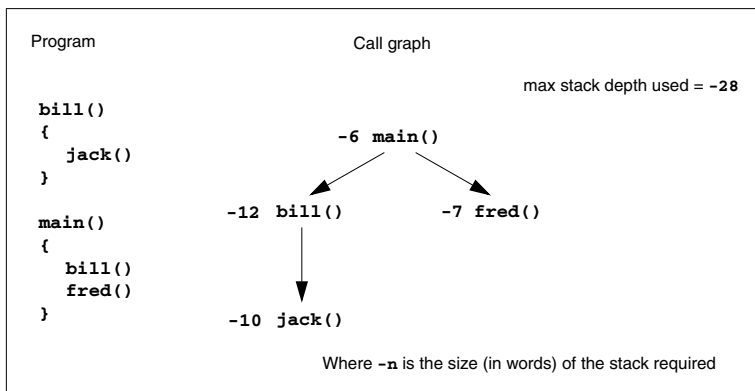


Figure 13: Call graph for stack depth analysis

Specifying the **st20cc -sk filename** option instructs **st20cc** to generate and output an analysis of its stack depth calculations. This includes an estimate of the maximum stack depth used by a program, expressed as a number of words. The format of the output is described below.

Where a function's use of the stack is only decided at run time, **st20cc** can only make an estimate of the stack requirement. This is most commonly caused by:

- **Recursion**

Recursion introduces cycles into the call graph. In this case it is impossible to know how many times a cycle is executed. **st20cc** terminates analysis of a path in the call graph if it detects a recursive call and assumes a stack depth of zero through the recursive call. The fact that a stack depth figure is affected by recursion is indicated in the file.

- **Function pointer calls**

Calls via function pointers introduce a dead end into the call graph. It is not possible to know the contribution to stack depth which is made by the code called via the function pointer. **st20cc** assumes a stack depth of zero beyond a function pointer call. The fact that a stack depth figure is affected by a function pointer call is indicated in the file. See below.

- **Inlining**

For the purpose of stack depth analysis **st20cc** assumes that no inlining takes place. This may lead to some stack depth figures being over pessimistic in the presence of inlining. A call that may be inlined is indicated in the file.

- **Internally generated calls**

For some expressions, **st20cc** inserts a call to a library function. For example, a floating point divide utilizes the library function **REAL32OPERR%c**. These internally generated calls appear in the stack depth output as an entry point, and not as a child of the caller function. Thus the stack requirements of internally generated calls are not accumulated as part of the caller function. However, the stack requirements to make the function call is added to the caller function.

For floating point operations the additional stack requirement is relatively high, in the order of 30 to 60 words.

- **Optimized calls**

It is possible for a function call to be optimized away completely, for example, if the call is unreachable. In such circumstances the stack depth analysis includes the call which has been optimized away.

The output file generated by the **-sk** option is split into two parts, *Flat display* and *Call graph display*, described in the following sections.

5.1.1 Flat display

Two "flat" displays are produced, the first being a subset of the second.

The first flat display lists all functions which are potentially entry points. Such a function has no parents, that is, it is not called directly from anywhere in the program. It may be the main entry point to the program, a function which is not used, or called via a function pointer. This list is for convenience to allow "quick" searching for entry points.

The second flat display is a list of all the functions in the program.

Figure 14 shows the format of a flat display. Against each function is given the maximum cumulative stack depth for that function, expressed as a number of words.

Index	Depth (in words)	Numcalls	Name
[1]	1	0	memcpy
[2]	6	1	debugoperation
[3]	6	1	simoperation
[4]	3	0	strlen

Figure 14: Example “flat” display from stack depth analysis

The format of this display is as follows:

- Index

A unique identifier for the function within the display.
- Depth

The maximum cumulative stack depth (expressed in words) from this function. This is the sum of the stack depth for the function plus the maximum cumulative stack depth of any functions it calls.
- Numcalls

The number of different functions that this function calls.
- Name

The name of this function. Name may be denoted by one or more of the following:

<Entry Point> : Indicates that this function has no parents, that is, it is not called directly from anywhere in the program. It may be the main entry point to the program, a function which is not used, or called via a function pointer.

For the first flat display, all the functions listed are potential entry points, so they are not individually marked as such.

<cy:[X]> : Indicates that the stack depth figure for this function is an estimate which is affected by a cycle somewhere on a call path from the function. X is the index of the function which made the recursive call. A single function may be affected by many cycles and so a number of these strings may be associated with a single function.

<fp:[X]> : Indicates that the stack depth figure for this function is an estimate which is affected by a function pointer call somewhere on a call path from the function. X is the index of the function making the function pointer call. A single function may be affected by many function pointer calls and so a number of these strings may be associated with a single function.

5.1.2 Call graph display

This display attempts to represent the call graph of the program. Each function in the program is displayed along with all the functions which call it and all functions that are called from it; see [Figure 15](#).

Index	Depth (in words)	Own (in words)	Below (in words)	Name	Parents Children	Index
	6	5	1		debugoperation	[2]
	6	5	1		simoperation	[3]
[1]	1	1	-	memcpy	[1]	

	22	16	6		debugcommand	[29]
	22	16	6		debugexit	[21]
	22	16	6		debugmessage	[6]
[2]	6	5	1	debugoperation	[2]	
	1	1	-	memcpy	[1]	

Figure 15: Example call graph display from stack depth analysis

The same information is output for the function, its callers and all the functions it calls. The format of this display is as follows:

Index	A unique identifier for the function within the display.
Depth	The maximum cumulative stack depth (expressed in words) from this function. This is the sum of the stack depth for the function plus the maximum cumulative stack depth of any functions it calls.
Own	The stack depth for this function alone (expressed in words).
Below	The maximum cumulative stack depth of the functions called by this function (expressed in words). If this function does not make any calls this field is marked as “-”.
Name	The name of this function (or the parent or child). The Depth and Below figures may have a “*” next to them. This indicates that the figure is an estimate and may be too small. The reason for this is that somewhere on a call path from the function there is either recursion or a call via a function pointer. Each line may have appended to it the strings as described for the Flat Display which indicate that recursion or function pointer calls affect the Depth figures. The function may also be denoted by one or more of the following:

<Entry Point>	Indicates a function with no parents, that is not called directly from anywhere in the program. Such a function may be the main entry point to the program, a function which is not used, or called via a function pointer.
<function pointer call>	A function pointer call is listed as a child of the function which called it. There is no way of knowing what happens beyond this call, so all the numeric quantities are listed as "?". The name of the function pointer (possibly the base name of an array or structure) is listed.
<recursive call>	A function pointer call is listed as a child of the function which makes the call. There is no way of knowing the number of times a cycle is taken, so the "Depth" and "Below" fields are listed as "?". The "Own" value for a recursive call is known and so this is listed as normal.
<possibly inlined>	If a function may be inlined then all calls to it are marked with the string "<possibly inlined>". However the stack depth analysis assumes that a function is not inlined and analyses the call as if it was a true function call which would use stack.

5.2 Memory map file

st20cc can be invoked to generate a memory map by using the **-M filename** option. By convention **filename** should be given the extension **.map**. [Section 5.2.1](#) shows the structure of an example map file produced for **hello.c**.

The mapfile is a text file which is split into different areas as follows.

- **Segment Map**

This includes an entry for each segment that has been defined using a **memory** command; see the *ST20 Embedded Toolset User Manual*, chapter *Defining a target system*. Entries take the form:

segment_name start_address size_in_bytes type

where **type** is one of **CODE**, **CONST**, **DATA**, **DEBUG**, **DEVICE**, **PERIPHERAL**, **ROM**, **RAM** or **RESERVED** (see the **memory** command in [Chapter 21: Alphabetical list of commands on page 319](#)).

- **Section/Region Map**

This includes an entry for each section or other memory region that is within the memory space of the processor. Each entry has the following form:

section/region_name start_address size_in_bytes type segment_name static_base

where **section/region_name** is either the section name, for example, **def_code** or one of the following:

- > **stack** - the main stack area,
- > **heap** - the main heap area,
- > **section-table** - ROM loader data,
- > **bootdata** - the Boot Data block,
- > **startblock** - the ROM loader start code block.

type specifies the type of the section or region and is one of the following:

- > **CODE** - the program code,
- > **INIT** - the initialized variable data,
- > **CONST** - the constant data,
- > **BSS** - the uninitialized variable data,
- > **STACK** - the main stack area,
- > **HEAP** - the main heap area,
- > **MVTORAM** - text that is moved from ROM to RAM at boot time,
- > **SECTTAB** - ROM loader data,
- > **BOOTDATA** - the Boot Data block,
- > **START** - the ROM loader start code block.

Note: **MVTORAM** sections duplicate a region stored in RAM somewhere. **MVTORAM** sections may also be marked as "**<partial>**". This indicates that the section has been compacted by the linker to take up less space while it is stored in ROM.

segment_name is the segment in which the section or region is positioned.

static_base: A marker, shown as “**” is placed against the entry containing the address of the static base (this is the base address from which the offset to any data items are calculated).

- **Onstartup Section Map**

This lists the static constructors called by **cstart** on startup. Each entry has the following form:

symbol start_address

where **symbol** is the symbol called by **cstart** and **start_address** is the symbol's start address.

- **Onshutdown Section Map**

This lists the static destructors called by **cstart** on shutdown. Each entry has the following form:

symbol start_address

where **symbol** is the symbol called by **cstart** and **start_address** is the symbol's start address.

- **Symbol Map**

This includes an entry for each symbol (function or variable name) in the linked unit. Each entry has the following form:

start_address type section_name source Mod/Lib symbol_name

where the **type** is either **G** for a global symbol or **L** for a local symbol. **source** is the name of the source file in which the symbol is defined and **Mod/Lib** is the object module or library which contains the definition of the symbol. **section_name** is the section in which the symbol is positioned.

By default the Symbol Map is sorted in ascending address order. This ordering can be changed by using the **st20cc** command line options **-MA**, **-MM**, **-MN** and **-MS**; see the *ST20 Embedded Toolset User Manual*, chapter *st20cc Compile/link tool*.

- **Symbol File Reference Map**

This lists for each symbol, each source file and associated object module/library, which makes a reference to that symbol. This includes the file in which the symbol is defined if it makes a reference to the symbol. Each entry has the following form:

file_name Mod/Lib symbol_name

where **file_name** is the name of the file which references the symbol and **Mod/Lib** is the object module or library which contains the object data generated from **file_name**.

It is possible that the following may appear instead of the file details:

<No References> This indicates that the symbol is not referenced from any file input to the link.

<Main Entry Point> This is the main entry point of the application.

Note: Static or extern variables declared in functions which are not used are displayed as being referenced from the file they are defined in if debug information is being generated. This is because a hidden reference is required in order to generate correct debug information for the symbol.

The Symbol File Reference Map is sorted in ascending address order. (See the entry for Symbol Map).

● Module Contribution Map

This shows how much code or data came from a particular source file. Each entry has the following form:

object_file module_name section_name start_address length

where:

object_file is the **.tco** file containing the specific module.

module_name is the name of the source file.

section_name is the name of the code or data section, which may be **def_bss**, **def_code**, **def_const**, **def_data**, or a user defined or compiler/linker generated section name.

start_address is the address within ***section_name***, where the code or data for the source/object file starts.

length is the size in bytes of the code or data.

5.2.1 Example map file

MAP OUTPUT FILE

LKU file : hello.lku

Segment Map

Name	Address	Size	Type
-----	-----	-----	-----
SYSTEM	0x80000000	320	RESERVED
INTERNAL	0x80000140	3776	RAM
DCU	0x3000	4096	DEVICE
INT_CONTROLLER	0x20000000	4096	PERIPHERAL
EMI	0x20002000	4096	PERIPHERAL
EXTERNAL	0x40000000	2096128	RAM
TRAPHANDLER	0x401ffc00	1024	DEBUG
ROM1	0x7ff80000	524288	ROM

Section/Region Map

Name	Address	Size	Type	Segment	SB
-----	-----	-----	-----	-----	---
def_code	0x40000000	5628	CODE	EXTERNAL	
def_data	0x400015fc	128	INIT	EXTERNAL	**
def_bss	0x4000167c	524	BSS	EXTERNAL	
<startup_section>	0x40001888	8	CONST	EXTERNAL	
<shutdown_section>	0x40001890	8	CONST	EXTERNAL	
bootdata	0x40001898	40	BOOTDATA	EXTERNAL	
stack	0x400018c0	1044896	STACK	EXTERNAL	
heap	0x40100a60	1044896	HEAP	EXTERNAL	

Onstartup Section Map

Symbol	Address
heapcinit	0x4000034c
stdio_init	0x40000068

Onshutdown Section Map

Symbol	Address
stdio_close	0x40000094
exitshutdown	0x400014b8

Symbol Map

Address	T	Section	Source	Mod/Lib	Name
0x40000000	G	def_code	hello.c	hello.tco	main
0x4000001c	G	def_code	printf.c	prtfncp.lib	printf
0x40000068	G	def_code	inopen.c	ansiio.lib	stdio_init
0x40000094	G	def_code	inopen.c	ansiio.lib	stdio_close
0x400000b2	G	def_code	inopen.c	ansiio.lib	alloc_file
0x40000108	L	def_code	fclose.c	ansiio.lib	clear_file:fclose.c
0x40000116	G	def_code	fclose.c	ansiio.lib	fclose
0x4000017c	L	def_code	fflush.c	ansiio.lib	_flush:fflush.c
0x400001c4	G	def_code	fflush.c	ansiio.lib	fflush
0x40000210	L	def_code	givbuf.c	ansiio.lib	init_buf:givbuf.c
0x4000026d	G	def_code	givbuf.c	ansiio.lib	init_file
0x4000027c	G	def_code	buf2.c	ansiio.lib	put_buf
0x400002bc	G	def_code	buf2.c	ansiio.lib	clear_write
0x400002e0	G	def_code	remove.c	ansiio.lib	remove
0x4000031c	G	def_code	heapc.c	heap.lib	calloc
0x4000032a	G	def_code	heapc.c	heap.lib	free
0x40000333	G	def_code	heapc.c	heap.lib	malloc
0x4000033d	G	def_code	heapc.c	heap.lib	realloc

Symbol map continues... followed by:

Symbol File Reference Map

Referencing Files	Name
hello.c	hello.tco main
cstart2.c	cstart2.lib
hello.c	hello.tco printf
printf.c	prtfncp.lib
inopen.c	ansiio.lib stdio_init
inopen.c	ansiio.lib stdio_close
inopen.c	ansiio.lib alloc_file
fclose.c	ansiio.lib clear_file:fclose.c
inopen.c	ansiio.lib fclose

```

fclose.c      ansiio.lib
fflush.c      ansiio.lib  _flush:fflush.c
fclose.c      ansiio.lib  fflush
fflush.c      ansiio.lib
givbuf.c      ansiio.lib  init_buf:givbuf.c
inopen.c      ansiio.lib  init_file
...

```

Symbol File Reference Map continues... followed by:

Module Contribution Map

```

-----
Object File      Module Name      Section Name      Start      Length
-----
hello.tco        hello.c          def_code          0x40000000  28
prtfncfp.lib     printf.c        def_code          0x4000001c  76
ansiio.lib       inopen.c        def_code          0x40000068  160
ansiio.lib       inopen.c        def_bss           0x4000167c  448
ansiio.lib       inopen.c        def_data          0x400015fc  8
ansiio.lib       inopen.c        <startup_section> 0x40001888  4
ansiio.lib       inopen.c        <shutdown_section> 0x40001890  4
ansiio.lib       fclose.c        def_code          0x40000108  116
ansiio.lib       fflush.c        def_code          0x4000017c  148
ansiio.lib       givbuf.c        def_code          0x40000210  108
ansiio.lib       buf2.c          def_code          0x4000027c  100
ansiio.lib       remove.c        def_code          0x400002e0  60
heap.lib         heapc.c         def_code          0x4000031c  72
heap.lib         heapc.c         def_bss           0x4000183c  4
heap.lib         heapc.c         def_data          0x40001604  32
...

```

Module Contribution Map continues to end of file...



Code and data placement

6

The ST20 toolset provides users with fine granularity code and data placement. This chapter explains how to perform code and data placement and describes the default placements which are applied by the toolset if the user makes no specific placements. Some restrictions on placement are listed in [Section 6.3](#).

A memory segment is a region of memory defined by either a **chip** or **memory** command; see [Chapter 21: Alphabetical list of commands on page 319](#). The **chip** command defines chip-specific attributes including on-chip memory. Once defined, memory segments can be referenced by other commands such as the **place** command. Code and data sections are either defined using the **ST_section** pragma within the application source code or refer to default section names provided by **st20cc** which are described in [Section 6.2](#).

6.1 Placing code and data

The default scheme, described in [Section 6.2](#), for code and data placement can be overridden by performing the following steps:

- 1 Use the **ST_section** pragma in the source code to allocate functions or data to named sections. This pragma is described in [Section 12.2.11: Pragma ST_section on page 173](#).
- 2 Define the memory map. If the **chip** command has been used then the on-chip memory is defined to be in a segment named **INTERNAL**. If the **processor** command is used the memory map is defined by specifying **memory** commands within a configuration procedure. The *ST20 Embedded Toolset User Manual*, chapter *Defining a target system* has further information about defining memory maps.
- 3 Specify **place** commands in the configuration command file to place sections within named memory segments. The **place** command is described in [Chapter 21: Alphabetical list of commands on page 319](#).

By default, sections are placed within the specified memory segment in the order they are defined in the source code; this is the order the compiler sees them. This ordering can be changed by using the `-place-exact` option; see [-place-exact st20cc option](#) below.

To find out the names of the sections to which code and data has been allocated by **st20cc**, use the **st20list** option `-s` on the object files produced by **st20cc**; see [Chapter 4: Using the linker tool on page 65](#).

The **place** command can be used to place using UNIX style regular expressions which match entire section names (see [Chapter 21: Alphabetical list of commands, place on page 405](#)). It is therefore best to avoid using section names that include regular expression characters, as this may cause confusion.

Example

To place the function `fast_calc` into a section called `fast_ram`:

```
void fast_calc(void);
#pragma ST_section(fast_calc, "fast_ram")
```

If the **processor** command is used, the following **memory** commands could be used to define the memory map:

```
K = (1024)
memory INTERNAL 0x80000000 (6 * K) RAM
memory SYSTEM 0x80000000 (0x140) RESERVED
```

If the **chip** command is used, the system and internal memory is already defined.

To place the section `fast_ram` into internal memory:

```
place fast_ram INTERNAL
```

-place-exact st20cc option

This option forces **st20cc** to place sections in memory in the order that the sections appear in the **place** commands specified in the configuration file. For example, a source file contains the code:

```
#include <stdio.h>
#include <stdlib.h>

int main(void);

void func1(void);
void func2(void);
void func3(void);
void func4(void);
void func5(void);

#pragma ST_section(func1, "section1")
#pragma ST_section(func2, "section2")
#pragma ST_section(func3, "section3")
#pragma ST_section(func4, "section4")
#pragma ST_section(func5, "section5")

void func1(void)
{
    printf("Hello from func1\n");
}
```

```
void func3(void)
{
    printf("Hello from func3\n");
}

void func2(void)
{
    printf("Hello from func2\n");
}

void func5(void)
{
    printf("Hello from func5\n");
}

void func4(void)
{
    printf("Hello from func4\n");
}

int main(void)
{
    printf("Start of main\n");
    func1();
    func2();
    func3();
    func4();
    func5();
}
```

The configuration file contains the following placements:

```
place section1 DRAM
place section2 DRAM
place section3 DRAM
place section4 DRAM
place section5 DRAM
```

When the **-place-exact** option is used the sections are placed in the order they appear in the configuration file, for example:

```
func1
func2
func3
func4
func5
```

If this option is not used they take the order that the code or data is defined within the source code, for example:

```
func1
func3
func2
func5
func4
```

The section's placement in memory can be examined by producing a map file; see [Section 5.2: Memory map file on page 86](#).

6.2 Default placements

Each compiled object file has associated with it some default sections into which any code or data, which has not been placed by use of an **ST_section** pragma, is placed. These default sections are listed below.

Note: The chevrons <> form part of the name.

- **<Filename_text>** Contains all functions which have not been explicitly placed. It also contains any anonymous literals used by the function, for example, literal strings.
- **<Filename_const>** Contains all **const** data items which have not been explicitly placed.
- **<Filename_data>** Contains all initialized non-**const** data items which have not been explicitly placed.
- **<Filename_bss>** Contains all uninitialized non-**const** data items which have not been explicitly placed. (By default **<Filename_bss>** sections are cleared to zero, but they can be marked to be left uninitialized at boot time; see below).

The use of the filename in the section names allows placement of items at higher granularity, for example, all functions in an object file or all **const** data in an object file.

The **place** command **noinit** option can be used to mark a BSS section as a **noinit** section. This is treated the same as a BSS section except it is not initialized at boot time to all zeroes. This enables the memory contents to be preserved during boot time and for example, enables the data to be mapped to a peripheral device which should not be written to at boot time.

In order to keep the size of the boot code down initialization of the BSS section is not optimized. Normally this is fast enough, however, large arrays in the source code may increase the size of the BSS section, which in turn slows down the initialization. Allocating large arrays or large C variables into named sections and placing such sections into memory segments marked with the **-noinit** option, ensures they are not initialized at boot time. If any of the variables require initialization this can be done using the **memset** function from the C source code.

All sections within the link which are not explicitly placed can be referenced collectively by predefined section names:

- **def_code** all unplaced code sections,
- **def_const** all unplaced const sections,
- **def_data** all unplaced data sections,
- **def_bss** all unplaced BSS sections.

These section names can be used in **place** commands to control the placement of all unplaced sections, for example:

```
place def_code DRAM
```

The above places all otherwise unplaced code sections into the DRAM segment.

Note: The ordering of the sections within these collective sections is not defined.

For unplaced sections, then the default placements are as follows:

- **Linked unit output**

By default sections are placed in the largest RAM segment defined in the configuration procedure. An error occurs if all the sections do not fit.

If no RAM memory segments are defined then an error is generated.

- **ROM output**

By default all sections are stored in the first ROM segment encountered in the configuration procedure. If all the sections do not fit in this ROM segment an error occurs. At run time, all sections (not specifically placed) are moved from ROM and placed in the largest RAM segment. If all the sections do not fit in this RAM segment an error occurs.

If no ROM segment is defined then an error is generated.

In the case where some sections are placed and others are left unplaced then **st20cc** ensures that all the placed sections are resolved before resorting to the above defaults for the unplaced sections.

6.3 Restrictions on placement

- When placing data in ROM, ensure that only sections containing **const** data are placed. The linker issues a warning if it detects read/write data being placed in ROM.
- Data items cannot be placed in a C file if they are declared using the **extern** storage class specifier. The only exception to this is the case where the declaration includes an initializer, in which case the **extern** specifier is ignored. In other words - only data items which reserve storage in a C file can be placed. Some examples:

➤ **int x;**

This can be placed as it reserves storage for **x**. It is a definition of **x**.

➤ **int x = 1;**

This can be placed as it reserves storage for **x**. It is a definition of **x**.

➤ **extern int x;**

This can not be placed as it does not reserve storage for **x**. The definition of **x** may be in another file.

➤ **extern int x = 1;**

This can be placed as the initializer causes the **extern** keyword to be ignored and it becomes a definition of **x**.

- Code or data items may not be explicitly placed into any of the default sections created at compile time, that is: **<Filename_text>**, **<Filename_const>**, **<Filename_data>** or **<Filename_bss>**.



7

Profiling

7.1 Introduction

This chapter describes the profiling facilities provided by the toolset for analyzing the behavior of application programs. The concepts and techniques of profiling are introduced, followed by detailed descriptions of how to use the different types of profiling provided by the toolset.

Profiling is the process of gathering statistics about which parts of an application run most often or use most processor time. Profiling helps build an understanding of how an application executes. Profiling results can indicate where code may be best optimizing and can be used to monitor whether each function is visited.

Profiling results depend on the input data and environment of the application; if the input data varies between application runs then the results will vary. When optimizing, it is important that the data used for profiling is typical of live application data.

Profiling with the toolset may be performed in batch mode or interactively. In batch mode, the profiling is controlled by options on the **st20run** command line. In interactive mode, profiling is controlled by the **profile** command, which is described in [Section 7.4](#) or via the debugging GUI, which is described in the *ST20 Embedded Toolset User Manual*, *ST Visual Develop* chapters. The **profile** command may be used interactively or in a batch command file. Batch mode profiling is described in [Section 7.8 on page 108](#). Profiling may be performed either on a simulator or on a hardware target.

7.2 Types of profiling

The types of profiling facilities provided by this toolset are:

- **Flat profiling**

This estimates the time spent in each function, the processor idle time and various other statistics. On hardware targets this is performed by sampling the instruction pointer. Flat profiling is described in [Section 7.5](#).

- **Idle profiling**

This measures both the time the CPU spends idle, that is, the time not executing code and the time that it is busy. This is performed by recording the time when the processor becomes idle and the time when it becomes active again. Idle profiling is described in [Section 7.6](#).

- **Call graph profiling**

This identifies the calling structure of the program and generates a call graph representing the entire call hierarchy of the program. Call graph profiling is only available when the ST20 simulator target is used, and is described in [Section 7.7](#).

7.3 Using profiling data

7.3.1 Using the output to optimize an application

Profiling can generate large volumes of information which require much evaluation. The following points should be considered when deciding what further optimizations to apply to the program.

- Functions that are called many times and which are small or called in few places are good candidates for making inline.
- Concentrate effort where it will have most effect. Do not waste effort on optimizations that are marginal.
- Consider different algorithms. If for example, a program is spending too much time allocating and freeing memory, try using a pool allocation scheme.

7.3.2 Limitations of profiling results

Profiling data should be interpreted with care. In particular the following points should be noted.

- Profiling reduces execution speed on the simulator.
- Sample profiling is prone to sampling errors.

When sampling is used, inaccuracies are possible, especially if the application does not run for very long. The results should always be treated as estimates. For hardware targets the profiler performs flat profiling using a sampling method. On the simulator, data for flat profiling is recorded after each instruction is executed and so it does not incur this sampling error.

- Idle profiling and flat profiling can be used concurrently, but the two facilities would have to share the same host/target connection, thereby reducing each facility's sampling frequency by at least fifty per cent.

- If input/output is being performed, the profiler may indicate many hits to the inform mechanism installed by the debugger; this is described in the *ST20 Embedded Toolset User Manual*, chapter *st20run tool*.
- The CPU to Diagnostic Controller Unit interface allows the DCU to sample the IPTR part way through the execution of an instruction. If the profiler does this part way through the execution of a **ret** instruction then the value of IPTR becomes the address of the instruction following the **ret**. If this **ret** is the exit from the bottom of a function then the IPTR may contain the address of the next function. The profiler then counts this sample as falling within the next function. This type of sampling error is most likely to occur in short functions.

One work around for this problem is to add an explicit **return** statement at the end of a function, and after the **return** add a dummy statement. For example:

```
void sub1(int val)
{
    global_var += val;
    return;
    /* dummy statement to avoid iptr sampled on function return
     * being associated with next function, sub2().
     */
    __asm{nop; nop;}
}
void sub2(int val)
{...}
```

Profiling results should be interpreted in conjunction with the program's map file (see [Chapter 5: Stack depth and memory maps on page 81](#)) and with knowledge of the program's structure.

7.4 How to perform profiling

7.4.1 Preparing programs

Programs to be profiled are developed and built in the normal way.

7.4.2 Running profiling

After building, the application may be run using **st20run** with an option on the command line (batch mode), or interactively using the command language or GUI. When using interactive mode, profiling may be turned on or off as required at different points during the run. The run generates profiling data, which is stored in memory on the host. After the application run has been completed, the data is extracted and accumulated or written as output. See the *ST20 Embedded Toolset User Manual*, chapter *st20run tool* for more information about **st20run**, and the *ST Visual Develop* chapters for a description of the graphical interface.

Profiling is controlled by the **profile** command; see [Part 4 Command language on page 303](#) of this manual for details of the command language and the **profile** command. From the graphical interface, profiling is controlled by selecting the **Trace and Profile** window.

The syntax of the **profile** command is:

```
profile [options]
```

The **profile** command without any options extracts and displays the profiling data. The options to the **profile** command are listed in [Table 22](#).

Option	Description
-call	Write call graph results. Applies to simulator target only.
-flat	Write flat profile results.
-idle	Write idle profile results.
-off	Turn off profiling data accumulation.
-on	Turn on profiling data accumulation.
-o[output] filename	Write the output to <i>filename</i> .
-p[eriod] period	Set the sample interval to <i>period</i> microseconds. The value of <i>period</i> must be not less than 1000 and not more than 10^6 . This option only applies to hardware targets. The default is 1000.
-prof mode	Set the profiling measuring <i>mode</i> . The <i>mode</i> may be i[dle] for idle profiling or s[ample] for flat profiling. The default is to perform flat and idle profiling.
-isize	Set the maximum idle profile buffer size. The default is 1000 elements.
-iwrap	Force the idle profile buffer to wrap when full and overwrite the earlier data. The default action is to discard new data when the idle profile buffer is full.

Table 22: profile command options

Profiling is initially disabled when the application starts. Profiling may be turned on using the **profile -on** command, and turned off using the **profile -off** command. Profiling may be turned on or off before running the application, or when the application is halted.

When the application completes, the data can be extracted and written using the **profile** command. If no options are given, **profile** extracts the accumulated data and writes the full results. The **-flat** or **-idle** options can be used to restrict the output. By default, the output is sent to standard output; the **-o** option can be used to direct output to a file.

7.5 Flat profiling

Flat profiling provides the estimated percentage of the total run time spent in each function. This information shows which routines are taking the most time. It also helps selection of good candidate functions for optimizing and inlining. It uses the sample profiling method.

7.5.1 How flat profiling works

Flat profiling is performed on hardware targets by testing at regular intervals which code is being executed. This technique is called sampling. The code being executed is identified as being in a function. The results give an estimate of how much time is spent in each function.

At each sample time, the instruction pointer IPTR is tested to discover which function is being executed. The testing may be performed by diagnostic controller hardware. If there is no function found to be running when the test occurs, then the idle count is incremented. The number of times each function is hit gives an estimate of the processor time taken by that function.

On the simulator, data is recorded after each instruction is executed.

7.5.2 Controlling flat profiling

Flat profiling may be selected using the **profile** command with the **-prof s** option. The sampled data can be extracted using the **profile** command with the **-flat** option.

On hardware targets, the frequency of the sampling may be varied using the **-p** option of the **profile** command. The frequency can be varied from one sample per millisecond to one per second in steps of 1 μ s. The sampling interval defaults to 1 μ s.

For example, the following sequence of commands flat profiles the application in **hello.lku** on the target called **st20** with a sampling period of 40 milliseconds.

```
% st20run -debug
> connect st20           connect to target
> load hello.lku
> go                     application stops at the beginning of user code
> profile -on -p 40000    start profiling with a period of 40 milliseconds

... later

> profile -off            stop profiling
> profile -o test.pro     output all profiling results to test.pro
```

7.5.3 Flat profiling output

This section explains the format of the output produced by the flat profiler.

The sampling technique introduces uncertainty into flat profiling results, which become less reliable for functions which are hit less often. For example, a short function may never be displayed as a hit, although it may be executed frequently. Thus a result of no hits does not mean that a function has not been executed.

The timing information in the profiling output is not exact because the sampling techniques used are subject to quantization errors. Also the sample period is not guaranteed, because the diagnostic controller may be performing other functions such as informs and so can reduce the sample frequency.

The output shown in [Figure 16](#) was generated by the following command sequence:

```
profile -prof s
profile -o test.pro -flat
```

```
-- Total          :      8734 samples    9.660 s   (100.0%)
-- pulse.dbg      :      8602 samples    9.514 s   ( 98.5%)
-- Idle           :         132 samples   145.920 ms (  1.5%)
-- Unknown        :           0 samples    0.000 s   (  0.0%)
-- Average Sample Period : 1106 us (904.16 Hz)
-----
-- Flat profile for linked unit "pulse.dbg" (8602 samples)
-----
%samples      Samples      Time      Func4 [1]
70.4          6146        6.797 s      main [2]
16.6          1452        1.606 s      Func2 [3]
4.6           405         447.930 ms    Func1 [4]
4.0           350         387.100 ms    Func3 [5]
2.4           207         228.942 ms    interrupt_init [6]
0.4           35          38.710 ms    handler_simple [7]
0.1           7           7.742 ms
```

Figure 16: Example flat profile output

The first part of the output shown in [Figure 16](#) is global information giving the total number of samples taken and the number of samples to hit each linked unit. The **Unknown** field represents the time spent in code that the profiler is unable to locate. This can happen when code is dynamically loaded or relocated.

The remainder of the output lists for each linked unit the number of times each function was hit. The functions are sorted into frequency order, with the most frequent functions first.

The columns are:

- the percentage of the total number of samples in that function,
- the total number of samples in that function,
- the total time spent in that function,
- the name of the function,
- a function identifier.

7.6 Idle profiling

Idle profiling measures the time the CPU spends idle, that is, the time not executing code. This is performed by recording the times when the processor becomes idle and the times when it becomes active again. The times generated are measured in microseconds.

The diagnostic controller on a hardware target can detect when the processor becomes idle and when it becomes active again. The simulator can provide similar signals.

Idle profiling may be selected by entering a **profile** command with the **-prof i** option. The time data can be extracted using the **profile** command with the **-idle** option. The **profile** command options **-isize** and **-iwrap** enable the size of the idle profiling buffer to be set and to specify whether, when the buffer is full, to overwrite existing data or discard new data.

For example, the following sequence of commands idle profiles the application in **hello.1ku** on the target called **st20**:

```
> connect st20           connect to target
> load hello.1ku
> go                     application stops at the beginning of user code
> profile -on -prof i    start idle profiling

... later

> profile -off           stop profiling
> profile -idle -o test.pro output only idle profiling results to test.pro
```

Idle profiling output

This section explains the format of the output produced by the idle profiler.

Figure 17 shows the beginning of the output from an idle profiling run.

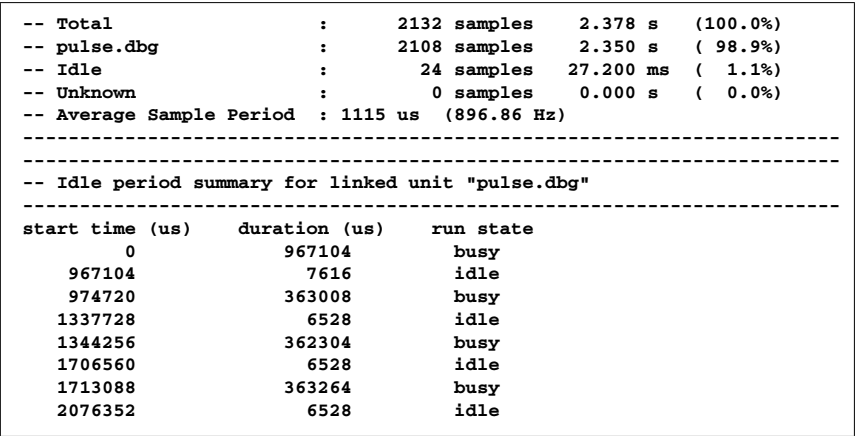


Figure 17: Example output from the idle profiler

The example in Figure 17 was generated by the following commands:

```
profile -prof i
profile -o test.pro -idle
```

The summary of the run is in the top part of the output. It shows that the total idle time was 27.2 milliseconds.

The lower part of the output lists all the idle and busy periods, their start time and duration in microseconds.



7.7 Call graph profiling

Call graph profiling is available when using the ST20 simulator as the target. While the program is running, the profiler records which functions are called by each function. Consider, for example, the following simple program **call.c**:

```
/* When using the stats input file call.cdf, nop is assigned
 * 40000000 cycles which is one second on a 40 Mhz part
 */
#define DELAY_ONE_SECOND __asm{ nop; }

void myexit(void){}

void Common(void)
{
    DELAY_ONE_SECOND
}

void Delay10(void)
{
    int i;

    DELAY_ONE_SECOND
    DELAY_ONE_SECOND
    for (i=0;i<8;i++) {
        Common();
    }
}

void Delay5(void)
{
    int i;

    DELAY_ONE_SECOND
    for (i=0;i<4;i++) {
        Common();
    }
}

int main(void)
{
    int i;

    for (i=0;i<5;i++) {
        DELAY_ONE_SECOND
    }
    Delay5();
    Delay10();

    myexit();
}
```

call.c has a **main** function which calls the function **Delay5** and **Delay10**. These two function each call a third function **Common**. The functions **Delay5**, **Delay10** and **Common** can be considered to be the descendants of **main**.

The call graph for a particular execution of this program might be represented as shown in [Figure 18](#).

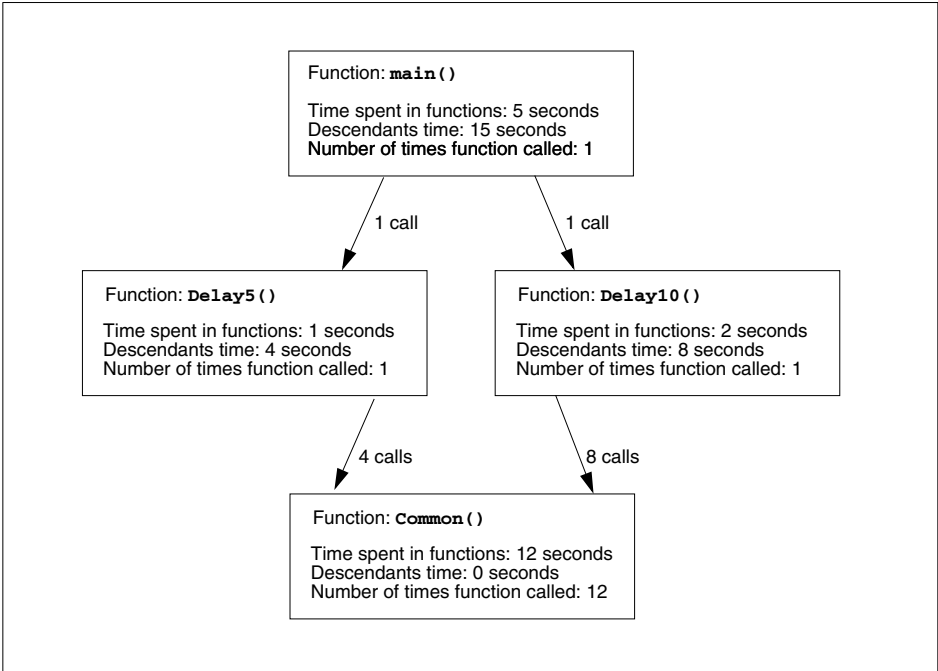


Figure 18: Example program call structure

The profiler identifies the calling structure of the program and uses this information to distribute time amongst functions. It starts at the bottom of the tree, and adds the time spent in that function to the parent function. This process is repeated up the tree, until the top is reached.

For example, **Common** has a total run time of 12 seconds. This time was split between **Delay10** and **Delay5**, so the profiler splits it up proportionally between the two. Hence **Delay10** inherits 8 seconds of the execution time of **Common**. **Delay10** spends 2 seconds executing its own code, so this gives a total execution time of 10 seconds for **Delay10**. The process is then repeated using this calculated time to inherit time for **main**. The resulting run time for **main** is 20 seconds.

Call graph profiling may be selected by entering a **profile** command with the **-call** option. For example, the following sequence of commands call graph profiles the application in **call.1ku** on the target called **mysim**:

```
> connect mysim           connect to simulator target
> load call.1ku
> go                     application stops at the beginning of user code
> profile -on -call       start call graph profiling

... later

> profile -off            stop profiling
> profile -o callgraph    write all profiling results to callgraph
```

Call graph profiling output

This section explains the format of the output produced by the call graph profiler. Each entry in the output shows a node in the call graph, and has the following format:

Index	%time	Total (Self+Desc)	Self	Desc	Calls	name
-------	-------	-------------------	------	------	-------	------

Figure 19: Call graph profiling output

The function for that entry is shown indented in the far right column. Parent functions are printed above this line, child functions are printed below. Each function is allocated an identifier (in square brackets) to make cross-referencing the entries easier.

The **%time** column represents the percentage of the total execution time spent in that function and its descendants.

The **Total** column is the sum of the **Self** and **Desc** columns.

The **Self** and **Desc** columns represent the time spent in that function and its descendant functions.

The number of times the functions are called is printed in the **Calls** column. This is shown as: (the number of calls made by that function)/(the total number of calls).

Figure 20 shows the call graph for the linked unit `call.1ku`

```

-----
-- Results for linked unit "call.dbg" (time 20.000 s )
-----

```

Index	%time	Total	Self	Desc	Caller
[1]*	100.0	20.000 s	5.000 s	15.000 s	? main [1]
		10.000 s	2.000 s	8.000 s	1/1 Delay10 [3]
		5.000 s	1.000 s	4.000 s	1/1 Delay5 [4]
		0.000 s	0.000 s	0.000 s	1/1 myexit [5]

		4.000 s	4.000 s	0.000 s	4/12 Delay5 [4]
		8.000 s	8.000 s	0.000 s	8/12 Delay10 [3]
[2]	60.0	12.000 s	12.000 s	0.000 s	12 Common [2]

		10.000 s	2.000 s	8.000 s	1/1 main [1]
[3]	50.0	10.000 s	2.000 s	8.000 s	1 Delay10 [3]
		8.000 s	8.000 s	0.000 s	8/12 Common [2]

		5.000 s	1.000 s	4.000 s	1/1 main [1]
[4]	25.0	5.000 s	1.000 s	4.000 s	1 Delay5 [4]
		4.000 s	4.000 s	0.000 s	4/12 Common [2]

		0.000 s	0.000 s	0.000 s	1/1 main [1]
[5]	0.0	0.000 s	0.000 s	0.000 s	1 myexit [5]

Figure 20: Example output from call graph profiler

Figure 21 shows the Idle and Flat profile results for this example.

```

-----
-- Total      : 20.000 s (100.0%)
-- call.dbg   : 20.000 s (100.0%)
-- Idle       : 0.000 s ( 0.0%)
-- Unknown    : 275 ns ( 0.0%)
-----

-- Flat profile for linked unit "call.dbg" (time 20.000 s )
-----

```

%time	Time	Calls	
60.0	12.000 s	12	Common [2]
25.0	5.000 s	?	main [1]
10.0	2.000 s	1	Delay10 [3]
5.0	1.000 s	1	Delay5 [4]
0.0	0.000 s	1	myexit [5]

Figure 21: Example output from Idle and Flat profiler

7.8 Profiling in batch mode

There are two methods of invoking the profiler in batch mode from the **st20run** command line:

- using the **st20run -p** command line option,
- running **st20run** in batch mode and using the **-i** option to specify a command file containing **profile** commands.

The **-p** option performs flat profiling of the complete program and writes it to the display screen once the program has terminated.

For example:

```
%st20run -t sti5516 -p hello.lku    connect to the target and
                                   profile hello.lku
```

Alternatively **profile** commands can be submitted to **st20run** in a command language file and **st20run** can be run in batch mode.

For example, the command file **prof.in** might contain the following commands:

```
connect sti5516                connect to target
load testit.lku
profile -on                    enable profiling in testit.lku
go                             run application
when (exit) {
    profile -idle -flat        once application has terminated
                               write both idle and flat profiling
    quit
}
```

and then submitted to the **st20run** command line:

```
%st20run -i prof.in -batch      run st20run in batch mode and
                                execute prof.in.
```



Using the ST20 simulator tool

8

The ST20 simulator **st20sim** provides a full register-level simulation of an ST20. It simulates the CPU, either the ST20-C1 or ST20-C2 core, and the memory system. It can generate a full trace and cycle time statistics.

The simulator has two different modes of operation.

- It can be used in exactly the same way as a hardware target, using **st20run** to load the application and provide debugging facilities. The simulator must be set up as a target, as described in the *ST20 Embedded Toolset User Manual*, chapter *Interfacing to the target*. In this case, the application code must be in the form of a linked unit (**.1ku**) file produced by **st20cc**. **st20run** loads and runs the application code on the simulator.
- It can be run as a stand-alone simulator. The application code must be in the form of a hex (**.hex**) file produced by **st20cc**. The application code is given to the simulator as a command line argument.

This chapter describes the simulator used as a stand-alone simulator. It describes how to use it to generate trace information and cycle statistics, and other facilities. Some of the facilities are also available through **st20run**. The command line options described in this chapter can be used when **st20sim** is run as a stand-alone simulator, or as the interface argument in the target definition command used by **st20run**.

8.1 st20sim command line

The **st20sim** stand-alone simulator is started with the following command line:

```
st20sim [options] [code_file]
```

where *options* is a list of zero or more options from [Table 23](#);

code_file is the file containing the application code in hexadecimal format.

When starting up the simulator, if no options or code file are given, a brief help page is displayed.

In [Table 23](#), the option arguments *addr* and *bytes* are integers. If the argument starts with **0x** or **#** then it is treated as a hexadecimal number. If it starts with **%** then it is a hexadecimal offset from the base of memory. For example, **%150** means **0x80000150**.

Option	Abbreviation	Description
-accurateclocks	-ac	This option applies only to ST20-C2 simulations and forces clocks to tick accurately by calculating cycle times of each instruction. This slows simulation down. This option is not required if using -stat .
-assert [0]	-as	Report any ST20-C1 specification assertion failure. If the argument zero is given, report the failure as a warning; otherwise reports it as an error. The default is to ignore assertion failure.
-attach	-a	Attach the debugger to the simulator process.
-c1	-c1	Simulate the ST20-C1 CPU core. This is the default.
-c2	-c2	Simulate the ST20-C2 CPU core.
-cfg <i>command_file</i>	-f	Use <i>command_file</i> as the description of the processor type, start state and memory map, overriding the -c1 -c2 -iptr -memsize -romsize and -wptr options. See Section 8.3: Command language and start-up scripts on page 113 .
-clockrate <i>freq</i>	-clk	Simulate a processor clock frequency of <i>freq</i> Mhz. The default is 40. This option applies only to ST20-C2 simulations and only has any meaning when using -stat or -ac .
-dumpstatdef <i>file</i>	-ds	Write a copy of the default cycle time definition file in <i>file</i> . This file defines the default cycle times and bus widths of memory and cycles for each instruction. See Section 8.5: Generating statistics on page 122 .
-emichk	-emi	Check that the external memory interface registers has been written to before any accesses to external RAM.
-event <i>file</i>	-ev	Read events from <i>file</i> . Each event specifies a particular action to be performed at a given instruction count number, to cause for example: interrupts, clock ticks, external I/O. See Section 8.7: Event support on page 129 .
-help	-h	Display the command line parameters.
-icntstep [<i>step</i>]	-i	Display the instruction number after every <i>step</i> instructions. The default <i>step</i> is 1000.

Table 23: st20sim options

Option	Abbreviation	Description
-iptr <i>addr</i>	-ip	Set the boot-from-ROM initial IPTR to <i>addr</i> . This may be overridden by using the bootiptr command in a command file; see option -cfg .
-l <i>dir</i>	-l	Add <i>dir</i> to the beginning of the search path for the command file.
-lnostd		No standard default search path.
-memsize <i>bytes</i>	-m	Set <i>bytes</i> as the amount of memory in bytes allocated for the application, expressed as a hexadecimal number. The default is 0x200000 . See Section 8.6: Manipulating memory on page 129 . This may be overridden by using memory commands in a command file; see option -cfg .
-noic		Disable default interrupt controller.
-nomemcheck	-n	Do not terminate if an illegal memory access is attempted.
-nomemwarn	-nw	Do not terminate or give a warning message if an illegal memory access is attempted. If this option is used then, -nomemcheck is not required. See Section 8.6 .
-p <i>procedure</i>	-p	Calls the named command <i>procedure</i> .
-perf		When the application terminates, display the elapsed time and the total number of instructions executed.
-quiet	-q	Do not display the version and copyright message; suppress some run-time diagnostic messages and the memory segment list.
-romsize <i>bytes</i>	-r	Set the ROM size to <i>bytes</i> . The default is the size of the ROM file to be executed. This may be overridden by using memory commands in a command file; see option -cfg .
-stat	-s	Generate cycle time statistics information for the whole program. This option should not be used with statistics tags in the application code. See Section 8.5: Generating statistics on page 122 .
-statdef <i>file</i>	-sd	Use <i>file</i> as the cycle time definition file. This can be used to define multiple memory bank sizes and access times. See Section 8.5 .
-trace [<i>format</i>]	-t	Display trace information at each instruction. See Section 8.4: Generating trace information on page 114 .
-tracelog <i>file</i>	-tl	Write any trace output to <i>file</i> . The default is to send the trace output to standard output. See Section 8.4 .
-undef <i>#hex_number</i>		Change the value representing an undefined word to <i>hex_number</i> . The default is 0xffffffff . See Section 8.2: Undefined values on page 113 .
-version	-v	Display the simulator version number and exit.
-wptr <i>addr</i>	-wp	Set the boot-from-ROM initial WPTR to <i>addr</i> . This may be overridden by using the startstate command in a command file; see option -cfg .

Table 23: st20sim options

8.1.1 Built-in defaults

By default **st20sim** simulates:

- a ST20-C1 core,
- 2 Mbytes of memory available in one segment starting at the base of memory (**0x80000000**),
- IPTR initially set to **0x7FFFFFFE**, WPTR initially undefined.

All these defaults can be changed by using the command language as described in [Section 8.3: Command language and start-up scripts](#). This is the **recommended** way of initializing the simulator. Alternatively, the defaults may be changed using the **-c2**, **-memsize**, **-iptr** and **-wptr** options.

For example, the hex application code file **ROM1.hex** can be loaded and run on the simulator in stand-alone mode by using the following command; see [Section 8.3](#):

```
st20sim -p c1 ROM1.hex
```

Before the application is run, **st20sim** displays its version number and a list of memory segments. These can be suppressed by the **-q** option.

If **st20sim** is simulating an ST20-C1 core booting from ROM, then an initial IPTR and memory size must be given, since there is no default. These may be given by **bootiptr** and **memory** commands in a command procedure, or the **st20sim** command line should include the following options:

- **-iptr addr** - to initialize the IPTR,
- **-m bytes** - to define the amount of memory required.

For example, **st20sim** could be run with the IPTR initially holding **0x7ffe7000** and a memory size of **0x200080** bytes (2 Mbytes plus reserved memory) by:

```
st20sim -iptr 0x7ffe7000 -m 0x200080 ROM1.hex
```

In general the use of command language is recommended in preference to using the direct options.

More details about some of the options are given in the rest of this chapter.

8.1.2 Breakpoint trap handler and inform mechanism

To support application I/O, **st20sim** installs a breakpoint trap handler and an inform mechanism into the target address space. This allows for the same I/O library to be used both on **st20sim** and on a silicon target.

st20sim installs the breakpoint trap handler and inform mechanism into a target memory segment called **TRAPHANDLER** with memory segment type **DEBUG**; see *Defining target characteristics* in the *ST20 Embedded Toolset User Manual*, chapter *Defining a target system*.

If such a memory segment is not available then **st20sim** reports a warning, the trap handler and inform mechanism are not installed and target-to-host I/O is not serviced. As a result, a call to the C run-time function **exit ()** will not terminate the simulator (an implicit call to **exit** is inserted on the return from **main**). Thus, **st20sim** simulates exactly like a boot from ROM application with no host access.

8.2 Undefined values

By default the simulator uses a special value **undefined** to represent an undefined value. This value is **0xCCCCCCC** by default, or may be defined using the **-undef** command line option. In the trace output, the simulator replaces any occurrence of this value with the string "-----".

The memory space is initialized with the value **undefined**.

8.3 Command language and start-up scripts

The simulator interprets the toolset command language defined in [Part 4 Command language on page 303](#) of this manual.

The simulator requires a description of the target core and memory layout to simulate. To achieve this it interprets the **chip** (and **processor**) **memory** and **bootiptr** command language commands. These commands may be structured as a configuration procedure and included in a start-up script.

On starting up, **st20sim** automatically executes a number of start-up command language scripts before processing any command line arguments. These scripts are listed in the *ST20 Embedded Toolset User Manual, Introduction*. This behavior may be turned off using the **-lnostd** command line option.

For example, if the following commands and configuration procedures are defined in the start-up script:

```
proc clhw {
    chip ST20C1SIM
    bootiptr 0x7ffe7000
    memory DRAM 0x80000080 (2 * 1024 * 1024) RAM
    memory ROM1 0x7ffe7000 (100 * 1024) ROM
}

proc c1 {
    clhw
    stack DRAM
    heap DRAM
}
```

then the compiler/linker driver **st20cc** and the simulator **st20sim** can call the default command procedure **c1**, to specify the target:

```
st20cc -c1 -p c1 hello.c -off hex
st20sim -p c1 ROM1.hex
```

The **-p** option (to both commands) enables a command procedure to be called from the command line.

These start-up scripts may also contain user defined default command procedures which can then be called as demonstrated in the above example. This enables the definition of a simulator target to be globally visible.

Alternatively, configuration procedures defining the target can be placed in a command file which can then be specified on the simulator command line by using the **-f** option. For example:

```
st20sim -f c1_sim.cfg ROM1.hex
```

The simulator searches for the command file in:

- 1 the current directory,
- 2 a directory specified by a **-1** option,
- 3 the standard configuration file directory, which is the **stdcfg** sub-directory of the root directory given by the environment variable **ST20ROOT**.

If a command file is given then it overrides any **-c1**, **-c2**, **-memsize**, **-iptr** or **-wptr** options given on the command line.

8.4 Generating trace information

The simulator can generate full trace information on the state of the simulated device for each instruction. The tracing can be set to different levels to show different register values and details of memory accesses. Tracing may be set:

- for the entire application using the **-trace** command line option,
- for ranges of the instruction count using the **-trace** command line option with a list of trace level and instruction count arguments, where the instruction count is a count of the number of instructions executed by the simulator so far, including bootstrap code,
- for sections of code within the application by inserting special pseudo-instructions in the code.

By default, the trace information is sent to standard output. The entire trace output can be sent to a trace file by using the **-tracelog file** option on the **st20sim** command line.

8.4.1 Trace levels

The **trace level** is a number between 1 and 6 inclusive for ST20-C1 cores and between 1 and 7 inclusive for ST20-C2 cores. Trace level 0 in both cases disables tracing. Higher trace levels give more information and lower levels less. The higher trace levels include all the information of the lower trace levels; for example, trace level 4 includes the information output by trace levels 1, 2 and 3.

The additional information given at each trace level is shown in [Table 24](#) for ST20-C1 cores and [Table 25](#) for ST20-C2 cores. The format of the output and the meaning of the trace line identifier are given in [Section 8.4.3: Trace output format, simulating ST20-C1](#). Generally, values listed before the instruction itself are values before the instruction is executed, while values listed after the instruction are values after the instruction was executed

8.4.2 Trace output format

This section describes the general trace output format. The details when simulating an ST20-C1 are described in [Section 8.4.3](#). The details for ST20-C2 are described in [Section 8.4.4](#).

Each instruction executed by the simulator can generate lines of trace output, depending on the trace level. Each line begins with a trace line identifier, which identifies the type of data on the line, as defined in [Table 24](#) or [Table 25](#).

The instruction count is given in decimal, the instruction itself is given as a mnemonic and all other values are either binary or hexadecimal. Binary values are shown grouped into fields, with the most significant bit first, and any reserved bits shown as dashes (-).

The level 1 trace information (that is, the **icnt** and **inst** lines) is shown for every instruction executed. It shows the values **before** the instruction is executed, and the instruction to be executed.

In order to reduce the volume of trace output, other register and flag lines are only shown when one of the values has changed. The values shown are those **after** the instruction has executed.

Memory access lines are only shown when an appropriate memory access occurs as part of the instruction, and the values shown are those read or written. The instruction fetch line always gives the fetches for the current instruction.

The address for a memory access is always the word address. If less than a word is read or written, then each unread or unwritten byte is shown by two dashes (--).

Trace level	Trace line identifier	Information output
0		No trace information output
1	icnt	Instruction count before the current instruction
	inst	Instruction pointer IPTR, Workspace pointer WPTR, evaluation stack AREG, BREG and CREG before the current instruction, operand, current instruction
2	stat	STATUS register after the current instruction
	tdesc	Task descriptor register TDESC after the current instruction
	ioREG	I/O register IOREG after the current instruction
	memw	Memory write address and data
3		Not used
4	memr	Memory read address and data
5		Not used
6	memf	Instruction fetch memory read address and data for the current instruction

Table 24: Trace levels for ST20-C1 core

Trace level	Trace line identifier	Information output
0		No trace information output
1	icnt	Instruction count before the current instruction
	inst	Instruction pointer IPTR, Workspace descriptor WDESC, evaluation stack AREG, BREG, CREG before the current instruction, operand, current instruction
2	enab	ENABLES register and ENABLES flags after the current instruction
	memw	Memory write (address and data) by the current instruction
	stat	STATUS register and other flags after the current instruction
3	qptr	Process queues, timer queues, timer enables after the current instruction
4	memr	Memory read (address and data) by the current instruction
5		Not used
6	memf	Instruction fetch memory read address and data for the current instruction
	memi	Memory reads that are part of an in instruction
	memo	Memory writes that are part of an out instruction
7	clks	Values of the high and low priority clock registers after the current instruction

Table 25: Trace levels for ST20-C2 core

8.4.3 Trace output format, simulating ST20-C1

On most lines, the meaning of each value is identified by a preceding letter, as defined in Table 26.

Line	Letter	Value
inst	I	Instruction pointer IPTR (the address of the current instruction)
	W	Workspace pointer WPTR at the start of the instruction
	A	Evaluation stack AREG at the start of the instruction
	B	Evaluation stack BREG at the start of the instruction
	C	Evaluation stack CREG at the start of the instruction
	N	Instruction operand for the current instruction
	O	The current instruction
stat	S	STATUS register after the instruction
memr/memw	A	Read/write word address
	D	Read/write data

Table 26: Trace value identifiers for ST20-C1

Level 1

Level 1 shows the instruction count, the IPTR, WPTR, evaluation stack, and the instruction with its operand. The following is an example of trace output at level 1:

```
icnt 78344
inst I 8000014f W 80180960 A 00000000 B 80001224 C 00000003 N 00000000 O 1d1
```

Level 2

Level 2 generates the Level 1 output plus the STATUS register, task descriptor, I/O register and any memory write. The following is an example of trace output at level 2:

```
icnt 78344
inst I 8000014f W 80180960 A 00000000 B 80001224 C 00000003 N 00000000 O 1d1
stat S 0x90000
tdesc -----
ioreg 00000000
memw A 80180964 D 00000003
```

Level 4

Level 4 generates the Level 2 output plus any memory reads. The following is an example of trace output at level 4:

```
icnt 78344
inst I 8000014f W 80180960 A 00000000 B 80001224 C 00000003 N 00000000 O 1d1
stat S 0x90000
tdesc -----
ioreg 00000000
memr A 80180960 D 00000003
```



8.4.4 Trace output format, simulating ST20-C2

The meaning of each value is identified by a preceding letter, as defined in [Table 27](#). Where high and low priority values apply, high priority values are listed first.

Line	Identifier	Value
inst	I	Instruction pointer IPTR (the address of the current instruction)
	W	Workspace descriptor WDESC at the start of the instruction
	A	Evaluation stack AREG at the start of the instruction
	B	Evaluation stack BREG at the start of the instruction
	C	Evaluation stack CREG at the start of the instruction
	N	Instruction operand for the current instruction
	O	The current instruction
enab	T	Trap enables after the instruction
	U	TimerAlarm, ExternalInterrupts, Timeslice and TakeProcessInterrupt enables flags after the instruction
stat	S	STATUS register after the instruction
	X	Analyze, Error, HaltOnError and MaxInterruptPriority flags after the instruction
qpctr	F	Task queue front pointer PROCQUEUEFPTR after the instruction
	B	Task queue back pointer PROCQUEUEBPTR after the instruction
	T	Timer queue pointer TPTRREG after the instruction
	E	Clock enable flag after the instruction
memf/memi/ memo/memr/ memw/	A	Read/write word address.
	D	Read/write data.

Table 27: Trace value identifiers for ST20-C2

Level 1

Level 1 shows the instruction count, the IPTR, WPTR or WDESC, evaluation stack, and the instruction with its operand. The following is an example of trace output at level 1:

```
icnt 78344
```

```
inst I 80000140 W 80000230 A ----- B ----- C 80000010 N 00000004 O ajw
```

Level 2

Level 2 generates the Level 1 output plus the ENABLES register, certain enable flags, any memory write, the STATUS register and certain status flags (see the *ST20-C2 Core Instruction Set Reference Manual*, section *Registers*). The ENABLES flags are the TimerAlarm enable, ExternalInterrupt enable, Timeslice enable and TakeProcessInterrupt enable, in that order. The status flags are Analyze, Error, HaltOnError and MaxInterruptPriority, in that order.

[Table 28](#) cross-references the bit names mentioned above with the formal names specified in the *ST20-C2 Instruction Set Reference manual*, ADCS 7301761.

Bit number	Formal name	Also known as
ENABLES register		
8	timer trap enable bit	TimerAlarm enable
9	timeslice trap enable bit	Timeslice enable
12	process interrupt trap enable bit	ExternallInterrupt enable
13	queue empty trap enable bit	TakeProcessInterrupt enable
STATUS register		
1	breakpoint trap status bit	HaltOnError
3	illegal opcode trap status bit	Error
16 to 17	Scheduler trap return priority status bits	MaxInterruptPriority
18 to 19	Trap group status bits	Analyze

Table 28: STATUS and ENABLES register bit nicknames

The following is an example of the extra trace output at level 2:

```
enab T - ---- - - - - - 00000000 000000 U 1111 1111
memw A 80180964 D 00000003
stat S 1 00000 - - - - - 1 00 01 0 - 00000000 000000 X 0 0 0 00
```

Level 3

Level 3 generates the Level 2 output plus the process queue pointers, timer queue pointers and clock enable. The high priority values are listed first, followed by the low priority. The following is an example of the extra trace output at level 3:

```
qptr F 80000000 B - - - - - T 80000000 E 0 F 80000000 B - - - - - T 80000000 E 0
```

Level 4

Level 4 generates the Level 3 output plus any memory reads. The following is an example of the extra trace output at level 4:

```
memr A 7fff0000 D 80000210
```

Level 5

Level 5 is the same as level 4.

Level 6

Level 6 generates the Level 4 output plus any memory accesses for instruction fetches and channel input or output instructions. The following are examples of extra trace output lines at level 6:

```
memf A 7fffffff D --62----
memi A 80000140 D -----21
memo A 800080b4 D -----08
```

Level 7

Level 7 generates the Level 6 output plus the high and low clock (timer) registers, the timeslice count and the low priority count. The timeslice count shows how many whole timeslice periods must elapse before a timeslice is due. The low priority count shows the number of high priority ticks before the low priority clock will be incremented. The following is an example of the extra trace output at level 7:

```
clks 00000280 00000009 1 1
```

8.4.5 The -trace option

The **-trace** option on the **st20sim** command line causes a trace to be generated. The option can be used to generate a trace for the complete run of the application, or to switch tracing on and off when the instruction count reaches certain values. The **-trace** option should not be used with code containing trace tags, as described in [Section 8.4.6](#).

The syntax of the **-trace** option is as follows:

```
-trace [level[.inst_count{ , level.inst_count }]]
```

where **level** is a trace level;

inst_count is the instruction count at which tracing is to start at **level**.

Any number of **level.inst_count** argument pairs can be given, the pairs being separated by commas (,). The **level** and **inst_count** in each pair are separated by a dot (.). The meaning of the trace levels is defined in [Table 24](#).

Tracing an entire application

The **-trace** option with no arguments starts trace generation at the first instruction at the default trace level 2. The simulator continues generating trace output for the entire application run, including boot, start-up and exit code. For example, to generate a trace level 2 for the application **hello.hex** using the command file **sim.cfg**:

```
st20sim -f sim.cfg -trace hello.hex
```

This generates trace output of about 100 bytes for each instruction in the application code.

A single trace level can be given as an argument to the **-trace** option, to start the trace at a level other than 2. For example, to trace at level 3:

```
st20sim -f sim.cfg -trace 3 hello.hex
```

Tracing between instruction counts

When the **-trace** option is used, the default action is to trace every instruction. This soon generates very large trace files. To avoid this, the tracing can be turned off and on using any number of **level.inst_count** pairs as the argument in the **-trace** option. For example:

```
-trace 1.20000,0.40000,1.60000
```

This example causes trace information to be written using trace level 1 and starting at instruction number 20000. When instruction number 40000 is reached, the trace level is reduced to level 0, turning off all tracing. At instruction number 60000 tracing is turned on again and proceeds at level 1 until the program terminates.

8.4.6 Tracing sections of source code

Tracing can be turned on or off for different sections of the application code. This is controlled by inserting pseudo instructions, called trace tags, in the application source code. This gives the ability to trace key sections of code. The **-trace** option, as described in [Section 8.4.4](#), should not be used with code containing trace tags, since the two methods of controlling tracing may clash.

A trace tag is a secondary instruction with a code in the range 5000 to 5004 inclusive. These codes are recognized by the simulator, but are illegal instructions on a hardware target, and are ignored provided the illegal instruction trap is disabled.

These instructions can be added to C source code by inserting:

```
__asm{opr num;}
```

where **num** is a number in the range 5000 to 5004. The meanings of the different values of num are given in [Table 29](#).

Code insertion	Meaning
<code>__asm{opr 5000;}</code>	Turn tracing off
<code>__asm{opr 5001;}</code>	Turn on tracing level 1
<code>__asm{opr 5002;}</code>	Turn on tracing level 2
<code>__asm{opr 5003;}</code>	Turn on tracing level 3
<code>__asm{opr 5004;}</code>	Turn on tracing level 4

Table 29: Trace tags

Any number of trace tags can be inserted throughout the application source. It is recommended that they are written as function calls, as in this example:

```
void TraceOnLevel12(void)
{
    __asm{opr 5002;}
}

void TraceOff(void)
{
    __asm{opr 5000;}
}
```

Any global optimization options specified at compile time (for example, using the compiler options **-O2** **-ftime**, **-fspace**) are ignored for the code in the function containing the tag. If the trace tag is not placed in a function then sections of application code will not be optimized.

8.5 Generating statistics

The simulator can accumulate statistics during the execution of an application, for part or all of the code. Statistics gathered include the number of cycles executed, the number of instructions executed, the number of jumps taken and the number of times each instruction was executed.

When the application finishes executing, the statistics are written to a statistics file. The statistics file is written to same directory as the application code file. When **st20sim** is run in stand-alone mode, the filename is derived from the application code file by changing the extension to **.sta**; for example, the statistics file for **hello.hex** is **hello.sta**. When **st20sim** is invoked as an **st20run** target, the statistics are written to the file **default.sta**. The format of the statistics file is described in [Section 8.5.1](#).

Statistics may be set:

- for the entire application using the **-stat** command line option,
- for sections of code within the application by inserting special pseudo-instructions in the code.

The number of processor cycles executed depends on how long each instruction takes to execute and how fast the memory is. This information can be supplied to the simulator in the form of a cycle time definition file, as described in [Section 8.5.4](#), or the default values can be used.

8.5.1 Output statistics file format

The statistics file consists of two parts:

- a page of general statistics,
- a list of instructions used by the application with their frequency of use.

An example of statistics output is shown in [Figure 22](#).

Cycles (excluding idle time)	=	310
Idle time	=	0
Instructions	=	105
Max address	=	#80001514
Jumps (j/cj/fcall/jab/eret)	=	100
Conditional Jumps taken	=	100
Ifetches [cycles]	=	116 [205]
Memory accesses [cycles]	=	2 [2]
Page Mode Dram hits	=	0
Workspace cache hits	=	0

Instruction	cycles	%	tot %	count	%	av
cj	298	96.13	96.13	100	95.2	2.980
ldc	7	2.26	98.39	3	2.9	2.333
stl	5	1.61	100.00	2	1.9	2.500

Figure 22: Example statistics output

The general statistics and their meanings are described in [Table 30](#).

Statistic	Meaning
Cycles (excluding idle time)	The total number of processor cycles executed (within the profiled region) excluding idle cycles
Idle time	The number of processor cycles spent in idle mode
Instructions	The total number of instructions executed
Max address (absolute)	The maximum address accessed
Jumps (j/cj/fcall/jab/eret)	The number of instruction executed involving a jump operation
Conditional jumps taken	The number of conditional jumps taken
Ifetches [cycles]	The number of instruction fetches and the number of cycles in []
Memory accesses [cycles]	The number of memory accesses and the number of cycles in []
Page Mode DRAM hits	The number of times an access is made to the current row (or page) of page mode DRAM memory
Workspace cache hits	The number of workspace cache hits (when a load from workspace loads a previously loaded value which is unchanged)

Table 30: Cycle statistics file format - part 1

The second part of the statistics file contains a list of instructions used by the application with the frequency they were used and the number of processor cycles taken. This list is sorted by the number of processor cycles taken, so that the instructions using the most processor time are at the top of the list. [Table 31](#) explains the meaning of each column in the list.

Column heading	Meaning
Instruction	The mnemonic of the instruction
cycles	The total number of processor cycles taken by the instruction
%	The processor cycles taken by the instruction, expressed as a percentage of the total cycles executed
tot %	The cumulative number of cycles taken by this instruction and the instructions above it in the list, expressed as a percentage of the total cycles executed
count	The number of times the instruction was executed
%	The number of times the instruction was executed as a percentage of the total number of instructions executed
av	The total number of processor cycles taken by the instruction divided by the number of times the instruction was executed

Table 31: Cycle statistics file format - part 2

8.5.2 Statistics for the whole application

The **st20sim** command line option **-stat** causes the simulation to generate statistics. This option causes statistics to be generated for the **whole** program, including boot, start-up and exit code. The **-stat** option should not be used with statistics tags, as described in [Section 8.5.3](#).

8.5.3 Statistics for sections of code

The generation of statistics can be turned on or off for different sections of the application code. This is controlled by inserting pseudo instructions, called statistics tags, in the application source code. This enables the generation of statistics for key sections of code, or just the user code without the bootstrap code. The `-stat` option, as described in [Section 8.5.2](#), should not be used with code containing trace tags, since the two methods of controlling tracing may clash.

A statistics tag is a secondary instruction with the code `1000`. This code is recognized by the simulator, but is an illegal instruction on a hardware target, and is ignored provided the illegal instruction trap is disabled.

The statistics tag causes the simulator to toggle statistics generation on or off. Initially the statistics generation is off, so the first statistics tag turns generation on. Any number of statistics tags may be used. The multiple tags simply define when the statistics generation is turned on and off.

This instruction can be added to C source code by inserting:

```
__asm{opr 1000;}
```

This is best written as a function call, as shown below. The statistics tag function call should be placed before and after all sections of code that require cycle timing statistics generation.

```
void ToggleStats(void)
{
    __asm{opr 1000;}
}
```

If the statistics tag instruction is not placed in a function, as shown above, any global optimization options specified at compile time (for example, using the compiler options `-O2` `-ftime`, `-fspace`) are ignored for the function containing the tag. This results in sections of unoptimized code.

8.5.4 Cycle time definition file

The simulator needs to know instruction cycle times and memory bus widths and access times in order to calculate the number of cycles each instruction takes. The instruction cycle times are the additional cycles taken by an instruction on top of any memory cycles. A set of default values are built into the simulator, which assume normal instruction times and on-chip memory. These default values can be overridden by providing a cycle time definition file.

By convention, cycle time definition files have the extension `.cdf`.

Generating the default cycle time definition file

The `st20sim` command line option `-dumpstatdef` generates a file containing the default cycle times, memory access times and bus widths. This is called a cycle time definition file. The purpose of this file is:

- to see what default values are being used,
- to act as a template for editing the file to use as a customized cycle time definition file, as described in the next section.

For example:

```
st20sim -dumpstatdef st20c1.cdf
```

This example generates the output file `st20c1.cdf` containing the cycle times and bus widths of the default memory address ranges.

Supplying a customized cycle time definition file

The default cycle time definitions may not be appropriate for generating accurate statistics. In particular, the application needs to use a different memory arrangement, with different address ranges, bus widths and speeds. These details can be supplied in a customized cycle time definition file, using the **st20sim** option **-statdef**.

In order for the **-statdef** option to have any effect, it must be used in conjunction with the **-stat** option or the application code must contain statistics tags, to enable the generation of cycle time statistics, as described in [Section 8.5.3](#).

A customized file can be generated by editing the numbers in the default file without changing the format, as described in the next section. The default file is generated using the **-dumpstatdef**, as described in the previous section.

The default cycle statistics definition file can then be edited and the “**addressCycles**” lines changed for each memory range to reflect the timing characteristics of the memory type used in this range. By default, three memory types are given: internal, external and ROM memory.

For example, a command of the following form simulates the application **app.hex** and generates the cycle time statistics file **app.sta** using the memory definition in the file **st20c1.cdf**:

```
st20sim -f c1_sim.cfg -statdef st20c1.cdf -stat app.hex
```

Editing the memory configuration

The simulation environment enables the user to model different types of memory. This can be achieved using a cycle time definition file, as defined in the previous sections.

The **-dumpstatdef** option generates a file containing the default **st20sim** memory and instruction data in the cycle time definition file format. The memory configuration part of the default cycle time definition file is shown in [Figure 23](#).

```
#####
#
#
#
#           address      type    cycles  pipelined  bus   type   type
#           address      type    cycles  cycles    width param 1 param 2
#
addressCycles #80001fff  other    1         1       32
addressCycles #7fefffff pmdram    4         4       32      128      2
addressCycles #7ffffff  other    7         7       32
#
#
#
#####
```

Figure 23: Default cycle time definition file

The address ranges, cycle times and bus widths of the blocks of memory are specified in the lines starting with the keyword **addressCycles**. The cycle time definition file also contains the non-memory cycles for each instruction.

The memory configuration is given in lines starting with the keyword **addressCycles**. This keyword is then followed by 5 standard parameters. Some memory types require a further 2

additional parameters. The meanings and examples of these parameters are given in [Table 32](#) and [Table 33](#).

Parameter number	Purpose	Examples
1	The top memory address of memory range in hexadecimal.	0x80004140 0x7fffffff
2	The memory type. Possible values are listed in Table 33 .	other
3	The number of processor cycles for one access.	4
4	The number of processor cycles for the second or subsequent accesses, provided they occur in the same instruction.	2
5	The width of the address bus in bits. Possible values are 32 , 16 or 8 .	32
6	Additional parameter 1. See Table 33 .	128
7	Additional parameter 2. See Table 33 .	2

Table 32: addressCycles parameters in the cycle statistics definition file

Memory type parameter	Type of memory	Additional parameter 1	Additional parameter 2
pm dram	Page mode DRAM	The number of words in a page	The number of cycles for a subsequent page mode access
other	Other	Not needed	Not needed

Table 33: Memory types and additional parameters

Example

Suppose we wish to configure the cycle time definition file to match the memory scheme shown in [Figure 24](#). The memory data for this example is:

- 32 Kbytes of internal memory with 2 cycle access time (1 cycle pipelined),
- 16 Mbytes of external page mode DRAM with 4 cycle access time (2 cycle pipelined) and 2 cycle page mode with 128 words (32 bit) per page,
- External peripheral at address **0x20001000** with 512 Kbytes 20 cycle (15 cycle pipelined) RAM,

- 8 Kbytes ROM ending at **0x7FFFFFFF** with 9 cycle (7 cycle pipelined) access time.

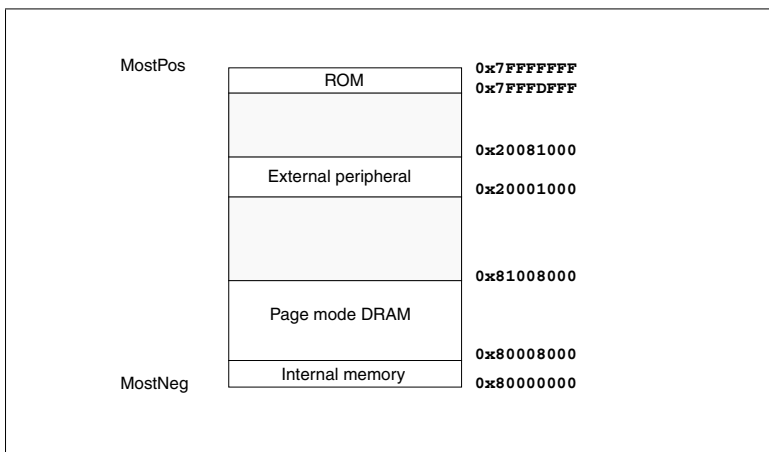


Figure 24: Example memory layout

This can be modeled by first dumping a statistics definition file so that the format can be copied, for example:

```
st20sim -f c1_sim.cfg -dumpstatdef test.cdf hello.hex
```

where **hello.hex** is the name of the hex file, and **test.cdf** is the name of the file in which to save the default cycle time definition file.

This file can now be edited and the memory configuration section at the top of the file can be changed, as shown in [Figure 8.6: Manipulating memory](#).

The first line sets up a section of memory from the lowest memory address (**0x80000000**) up to (but not including) **0x80008000** with a 2 cycle access time. This corresponds to a 32k (0x8000) internal SRAM.

The second line sets up a section of memory from **0x80008000** to **0x81008000** of type “**pmdram**” (page mode DRAM) with a 4 cycle accesses time and 2 pipelined cycles. **Param1** specifies the number of words in a page is 128 and **param2** gives the number of cycles for a page mode access.

The third line results in all accesses from **0x81008000** to **0x20081000** taking 20 cycles. This assumes that the only accesses to be made within this range will be to the external peripheral. If this is not the case, then another range from **0x81008000** up to the peripheral start address must be inserted.

Finally the last line sets up the ROM memory. Any access from **0x20081000** up to **0x7FFFFFFF** will take 9 cycles. Again if any nonROM accesses are required within this range, another addressCycles line should be added.

To simulate the new memory configuration, a command of the following form could be used:

```
st20sim -f c1_sim.cfg -statdef test.cdf -stat hello.hex
```

```
#####
#
#
#
#           address      type    cycles  pipelined  bus      type    type
#           address      type    cycles  cycles    width   param 1 param 2
#
addressCycles #80008000  other    2        1       32
addressCycles #81008000  pmdram   4        2       32      128     2
addressCycles #20081000  other   20       15       32
addressCycles #7fffffff  other    9        7       32
#
#####
```

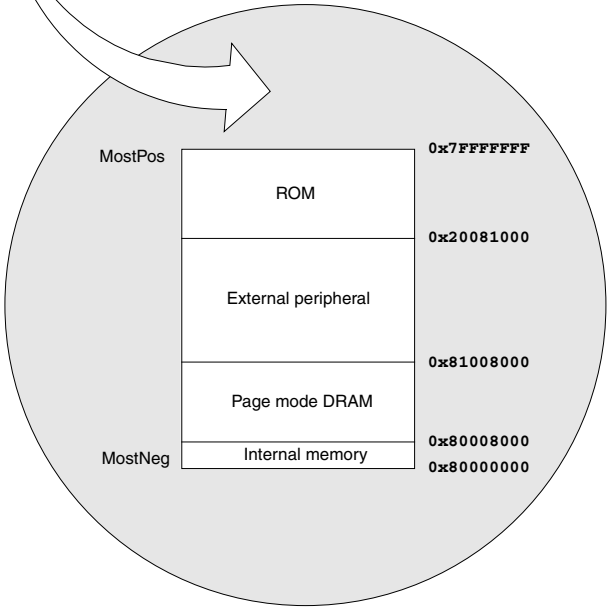


Figure 25: Example of edited statistics definition file

8.6 Manipulating memory

8.6.1 Changing the memory size

If a command file is not available, the **-memsize** option can be used to specify the total size of internal and external memory; the memory is allocated as one segment starting at the bottom of memory. The syntax of the **-memsize** option is:

```
-memsize bytes
```

where **bytes** is the number of bytes, given in decimal or hexadecimal format. The default is 2 Mbytes (2097152 or 0x200000).

For example:

```
-memsize 0x100000  
-m 4194304
```

This memory must be at least as large as the RAM size specified to the linker.

8.6.2 Illegal memory accesses

An illegal memory address is defined as an address not contained within any memory segment, or an address not aligned as required by the instruction.

When an illegal address is encountered, the default action is for the simulator to terminate with an error message.

When an illegal address is encountered with the **-nomemcheck** option set, a warning is displayed and the program is allowed to continue. If the **-nomemwarn** option is set, then the simulator is allowed to continue with no warning messages.

It is recommended that these options are used with extreme caution. If illegal memory areas are being accessed it is usually a sign that the application program is at fault.

8.7 Event support

Trap and interrupt support is provided by **st20sim** in a number of ways. The latest ST20 Embedded Toolset provides function calls that allow traps and interrupts to be set up and fired; for details see the *OS20 User Manual*, chapters *Kernel* and *Alphabetical list of Functions*. In addition, **st20sim** provides a manual method of firing interrupts at assembly language level using the **-event** option on the command line. This section describes how to use the **-event** option.

8.7.1 Triggering external events

The **-event** option allows the simulation of external events at specific points in the application. The **-event** option takes an event file as an argument. The event file is an ASCII text file, in which each line describes one event.

The format of one line of the event file is as follows:

```
event_name instruction_count [instruction_point] [event_type] [extra_info]
```

[Table 34](#) lists the possible values of parameters in the event file.

Event name	Instruction count	Instruction point	Event type	extra parameters
interrupt	any integer	0 or -1	RPC	interrupt level (0 to 7) or interrupt handler address, for example, 3 or 0x8000030c
tick				
lotick				

Table 34: Event file format

8.7.2 RPC interrupts

An interrupt can be raised at a particular instruction count by using the **RPC** event type in the event file (currently the only event type supported is **RPC**). If the *instruction_point* parameter is zero, the instruction is interrupted. If it is less than zero, the instruction is interrupted and is traced before and after the interrupt handler. This may be used to model aborted instructions. The last parameter is used to specify the interrupt level (0 to 7 for the ST20) or the address of the interrupt handler, expressed in hexadecimal.

Example

The following event file causes an interrupt level 1 to occur before instruction number 37 and an interrupt level 3, before instruction number 43.

```
interrupt 37 0 RPC 1
interrupt 43 0 RPC 3
```

8.7.3 Clock register ticks

The event file can also be used to define when the high and low priority clock registers tick, using the **tick** and **lotick** events respectively.

Example

The following lines cause the low priority clock register to be incremented immediately before instruction number 42 and then again before instruction number 47:

```
lotick 42
lotick 47
```

If this method is used to drive the clocks, **st20sim** will only increment the clocks as specified in the event file. Use trace level 7 to display the CLOCK register information.

8.8 Pseudo-ops

Certain instruction codes are interpreted by the simulator as signals to the simulator, rather than part of the application code. These instructions are known as pseudo-ops.

The values of the pseudo-ops are chosen so they do not clash with genuine ST20 instructions. The pseudo-ops are illegal ST20 instructions, but they are ignored by a hardware target provided the illegal instruction trap is disabled.

[Table 35](#) lists the pseudo-ops which are available, with their mnemonics. In the `st20sim` trace output, the mnemonic of a pseudo-op is displayed as the upper case string shown.

Pseudo-op mnemonic	Instruction
STATTAG	opr 1000;
TRACEOFF	opr 5000;
TRACEON1	opr 5001;
TRACEON2	opr 5002;
TRACEON3	opr 5003;
TRACEON4	opr 5004;
TRACEON5	opr 5005;
TRACEON6	opr 5006;
TRACEON7	opr 5007;
PRINT	opr 5012;
STOP	opr 5014;
EXIT_SERVER	opr 5015;
CYCLES	opr 5016;

Table 35: st20sim pseudo-ops

The pseudo-ops for tracing are discussed in [Section 8.4](#). The others are described below.

- PRINT

This instruction prints a copy of memory interpreted as ASCII characters, starting at the address in AREG. If BREG is positive, BREG bytes are printed. If BREG is negative then it prints up to a **NULL** terminator or for 256 bytes, whichever is encountered first. On return, AREG and BREG are unchanged and the length actually written is in CREG.
- STOP

This instruction stops the simulator with a fatal error that reports:

st20sim: FATAL ERROR st20sim halted by STOP pseudo-op
- EXIT_SERVER

The simulator makes use of a host server application which provides the interface to the host operating system. This server is used to provide an implementation of the **PRINT** pseudo-op. The server must be shut down when the program terminates; the run-time system uses the **EXIT_SERVER** pseudo-op to achieve this.

CYCLES

This instruction enables or disables statistics generation. If AREG is greater than zero, statistics collection is enabled and the elapsed cycles divided by AREG are returned in AREG. Otherwise, if AREG is less than or equal to zero, statistics collection is disabled and the elapsed cycles are returned in AREG.



ROM systems

9

This chapter describes advanced topics related to designing ROM systems using the ST20 toolset. It describes the ROM bootstrap, how ROM programs make external calls, OS20 awareness, compatibility with earlier toolsets, and has particular emphasis on debugging ROM systems. It explains some of the differences between debugging a system which is downloaded to the target from a linked unit and debugging a stand-alone ROM system; it also describes how to debug multiple programs on a ROM system.

A ROM image can be produced as a binary file (without location information), a hexadecimal file or a Motorola S-record file. The examples used in this chapter are based on a hexadecimal file, and use the command line rather than the ST Visual Develop GUI..

Details of the command language used by the toolset are given in [Part 4 Command language on page 303](#). Details of how to build a ROM image are given in the *ST20 Embedded Toolset User Manual*, chapter *st20cc compile/link tool*.

9.1 ROM system overview

When booting from a host, chip and C/C++ language initialization is performed by **st20run**. When booting from ROM, these tasks must be performed by the ROM code. When **st20cc** generates a ROM image file it adds bootstrap code to set up the processor and the C/C++ language run-time system. This is called automatically on device reset, before calling the program's **main** function.

The ROM C-startup bootstrap code performs the following tasks:

- 1 Set up a temporary workspace for itself in internal memory. If **initstack** was specified at link time, WPTR is set to this command's argument, otherwise WPTR is not modified. See [Section 9.4: Callbacks before and after the poke loop in romload\(\)](#).
- 2 Call **PrePokeLoopCallback**, as described in [Section 9.4](#).
- 3 Perform all pokes encountered during the link phase.
- 4 Call **PostPokeLoopCallback**, as described in [Section 9.4](#).
- 5 Set up the program's workspace (specified at link time with the **stack** command).
- 6 Set up the static link (the pointer to all global data; see [Chapter 13: Implementation details on page 181](#)).
- 7 Copy the program code from ROM to RAM if required.
- 8 Initialize data segments (where statics such as "**static int x=3**" are contained and zero bss segments (where statics such as "**static int y**") are contained).
- 9 Set up a **setjmp-longjmp** pair for the **romrestart** function.
- 10 Install default trap handlers for breakpoints (all cores) and errors (ST20-C2 cores only).
- 11 Call all C++ static class constructors and functions flagged with **#pragma ST_onstartup**.
- 12 Set up a **setjmp-longjmp** pair for exit.
- 13 Call the **main** function of the program.

Note: 1 Source for the bootstrap is shipped with the toolset, so it can be modified for programs which require a bespoke bootstrap procedure.

- 2 In the ST20 Embedded Toolset User Manual chapter st20run tool, the effects of various debugging commands on the diagnostic control unit are described. It may be helpful to read this section in order to understand possible debugging session states.*

9.2 An example program and target configuration

The **examples/romdebug** directory contains an example which illustrates key issues with debugging ROM systems, and uses some common set-top box software features. The program prints out a set of messages while flashing an LED, then uses the on-chip watchdog timer to reset itself.

If the target is reset (for example, by a watchdog timer) while **st20run** is connected, the target will stall. The target can only be recovered from the stall state by using the **restart** command. The example calls the function **debugcommand** to execute a command language procedure which performs a restart after a watchdog reset has occurred.

The examples below apply to an STi5500 device on an EVAL-5500 board. They may be run on an ST20TP3 device on an STB3-EVAL board by replacing instances of **STi5500MB159** with **ST20TP3MB193B** and adding a target definition to the **example.cfg** file (using the existing definition as a basis, **mk_STi5500_MB159_gen** could be replaced with **mk_ST20TP3_MB193B_gen** and **keen** with the name of the target board).

The example can be built and debugged as a linked unit or as a ROM image.

The definition of targets using the **mk_<chip_type>_<board_type>_gen** command language procedures defines targets using the default board specification procedures which can be found in the **boards** directory. These default board specification procedures are used by both **st20cc** when linking and **st20run** when loading and debugging. Examining these default procedures, we find several commands key to target definition.

- The **chip** command specifies the variant of the ST20, and defines:
 - the processor core type,
 - the system memory, declared to be **RESERVED**,
 - the internal memory,
 - cache and interrupt management,
 - a watchdog keep-alive trap handler (see the *ST20 Embedded Toolset User Manual*, chapter *st20run tool*).
- The **memory** command specifies a memory segment; for full details, see [Chapter 21: Alphabetical list of commands on page 319](#).

Stack and heap segments and their sizes are defined. When the size of heap is not specified, it is allocated the remainder of the segment it is in.

The system memory initialization procedure, **ST20C2MemoryInit**, clears various system locations but does not set locations **0x80000040** through **0x8000013c** because these are used by the debugger for trap handlers.

An External Memory Interface (EMI) configuration procedure is also called by most default procedures. This performs a series of pokes in order to allow the board to access **EXTERNAL** memory.

Instructions to build the example both as a linked unit and as a ROM system have been included so that the differences associated with building and debugging the two formats are demonstrated.

9.2.1 Building a linked unit

A linked unit is generated with the command:

```
st20cc example.c led.c watchdog.c -p STi5500MB159 -M example.map -g -O0
-DENABLE_WATCHDOG
```

The command returns as output the linked unit file **example.lku**, the debug information file **example.dbg** and the memory map file **example.map**.

The **-DENABLE_WATCHDOG** option defines the **ENABLE_WATCHDOG** symbol to **example.c**, enabling the use of the **watchdog_reset** function. This is an external command language procedure which can be found in **example.cfg**.

9.2.2 Running and debugging the LKU

The linked unit is loaded onto the target, run and debugged using **st20run**:

```
st20run -i example.cfg -t keen example.lku -d
```

The target **keen** (identified in the command line by **-t keen** and defined within **example.cfg**) has a connect procedure which resets the board (**reset**), tells the debugger about the hardware, initializes the system memory and configures the External Memory Interface (EMI).

The debugger is activated by the **st20run** command line option **-d**. On startup the debugger sets a breakpoint on **main** and executes a **go** command so that the GUI appears with the program “waiting” at the breakpoint on main.

By default, the system sets software breakpoints (that is, instructions poked into the code).

9.2.3 Building a ROM image

A ROM image file is generated with the command:

```
st20cc example.c led.c watchdog.c -p STi5500MB159 -M example_ram.map -g
-O0 -DENABLE_WATCHDOG -romimage -o example_ram.hex
```

The key difference between this command and the command to build an LKU is the **-romimage** option. By default, the **-romimage** option produces hexadecimal format output.

The **-DENABLE_WATCHDOG** option defines the **ENABLE_WATCHDOG** symbol to **example.c**, enabling the use of the **watchdog_reset** function. This is an external command language procedure which can be found in **example.cfg**.

All pokes encountered during the link phase are converted to a table in the ROM image executed by the bootstrap code, so as **STi5500MB159** performs EMI configuration, the ROM image has access to **EXTERNAL** memory when running. The example output files are generated with the name “**example_ram**” because they are executed in RAM.

By default, program code is moved to RAM by the bootstrap. Such code is listed as **MVTOGRAM** in the map file. To execute code from ROM, the following command must be included at link time:

```
place def_code FLASH
```

This can be done by adding **-T example.cfg** to the command line used to create the ROM image, and exchanging the **-p STi5500MB159** option with **-p STi5500MB159_rom**.

A debug information file, **example_ram.dbg**, is created. A ROM image file named **example_ram.hex** is created which represents the memory segment **FLASH**.

Use a flash-burn linked unit (see the **examples/flash** directory) or a programmer to burn the ROM/FLASH with the ROM image **example_ram.hex**.

9.2.4 Debugging the ROM image

ROM systems may be debugged provided there is a connection to the host from the on-chip diagnostic control unit of the ST20. ROM systems may be debugged from a reset-processor state or the debugger may be connected while the program is running.

Debugging a boot-from-ROM system (reset)

This debugging mode is started with the following command:

```
st20run -i example.cfg -t keen -d
```

The target **keen** (identified in the command line by **-t keen**) has a connect procedure which resets the board, tells the debugger about the hardware, initializes the system memory and configures the EMI.

The user must then tell the debugger about their program's debugging information. This may be done with the command:

```
program -new example_ram.dbg
```

If there is more than one C or C++ program on the ROM, a **program -new** command must be performed for each corresponding debugging information (**.dbg**) file.

The user may set initial breakpoints at this stage. If the code is in **MVTORAM**, breakpoints **must** be of hardware type; this is because software breakpoints would be overwritten when code is moved from ROM by the bootstrap.

The user may start the ROM system by invoking the **go** command.

If initial breakpoints on code are set prior to starting the target, and the code is executed from RAM, these should be of hardware type, otherwise the breakpoint will be overwritten when code is moved from ROM to RAM by the bootstrap.

If the processor is reset (for instance, by a watchdog timeout) or the program must be restarted, the command **restart** should be used. The commands **reset** followed by **go** are not adequate. The command **restart** executes the connect procedure, sets up breakpoints and executes a **go** command.

It is important that the connect procedure is executed because this defines the core type, the default register values, the cache and interrupt control procedures and perform EMI programming (which is necessary if the trap handler is not located in internal SRAM).

If the debugger is connected, the chip always stalls on reset. Bit 31 of the DCU2 (or bit 0 of the DCU3) control register is set when the CPU is stalled.

Debugging a running system

To make the example in the **romdebug** directory suitable for connecting to as a running system, it should be recompiled and programmed to ROM without the **-DENABLE_WATCHDOG** option. This turns off the calling of the external command language procedure **watchdog_reset**, which is inappropriate in this case. This is because a debugger is not always connected and the procedure uses the **restart** command. The **restart** command will not work when connected to a target whose connect procedure does not perform a **reset** and EMI initialization.

This debugging mode is started with the following command:

```
st20run -i example.cfg -t keen_nr -d
```

The target **keen_nr** (identified in the command line by **-t keen_nr**) has a connect procedure which does not perform a **reset**, **ST20C2MemoryInit**, or configure the EMI - a

reset would halt the running program and the other operations were already carried out by the ROM image's bootstrap code in order to start the program. However, the connect procedure does specify the memory segments to the debugger.

By default, the debugger's input/output does not appear after connecting to a running system. These are enabled with the debugger command **informs -enable** and may be disabled with the command **informs -disable**.

The user must provide the debugger with the program's debug information using the command:

```
program -new example_ram.dbg
```

At this stage, breakpoints may be set. If a run-time system is in use (for example OS20) the debugger should be told at this point with the **runtime** command.

If at a later time the processor is reset, the procedures for initializing the system memory and programming the EMI must be called manually before the **restart** command is issued. Otherwise, the issues discussed under the heading *Debugging a boot-from-ROM system (reset)* apply.

9.3 Multiple programs

[Section 9.2](#) describes how a simple example program is run from ROM. This section describes how multiple programs are run from ROMs.

9.3.1 Calling to another program

An example of calling from one program to another is provided in the **examples/dualboot** directory. This example may be built for both ST20-C2 and ST20-C1 cores. The **examples/dualboot-pre_1_8_1** directory contains the example in a form which may be built with earlier toolsets. This example may be built for ST20-C2 cores only. The examples illustrate the closing down of OS20 which must be done before calling another program.

Some key points that are relevant for an ST20-C2 core are illustrated and discussed below to emphasize their importance.

The following code assumes the operating system has been terminated at this point.

(For an example of closing the operating system, see [Chapter 19: ROM Restart functions on page 299](#).)

```
debugsetmutexfn(NULL, NULL);
DISABLE_CACHES ;

if (!debugconnected()) {
    __asm {
        ldab 0xffff, 0; /* disable all traps */
        trapdis;

        ldab 0xffff, 1;
        trapdis;
    }
} else {
    __asm {
        ldab 0xfffe, 0; /* disable all traps except */
        trapdis;      /* the debug trap */

        ldab 0xfffe, 1;
        trapdis;
    }
}
debugcommand( "runtime c2rtl", &res );
__asm {
    ld address;          /* the entrypoint of the main app */
    ldmstartval;         /* load 0x80000140 in AREG */
    gajw;                /* sets the WPTR */
    pop;                 /* remove the old WPTR */

    gcall ;              /* call 'address' */
}
```

The debug mutex callback function must be reset to **NULL** before calling the second program.

Caches **must** be disabled so that when code is moved from ROM to RAM in the C start-up bootstrap for the second program, it is written back to RAM not just to the data cache.

If the debugger is connected, the breakpoint-trap bit in the STATUS register must be left set; if the debugger is not connected, it must be cleared. The breakpoint-trap bit is tested by the

C-startup sequence of the second program; if clear, it installs a default breakpoint trap handler, if set, the current trap handler is left in place.

Caution: incorrectly setting or clearing the breakpoint-trap bit at this stage will cause dual-boot systems to fail.

On ST20-C1 systems, bit 23 of the STATUS register is used to replicate the functionality of the ST20-C2 core's breakpoint-trap bit.

9.3.2 Debugging multiple programs on a ROM

for each program on the ROM being debugged, specify the debug information (`.dbg`) file with the command **program -new**.

Multiple programs may share a single region of RAM for their data and for their code. This poses a problem for the debugger because it cannot determine which program is using a shared region of memory at a given point in time. As an example, if a breakpoint is hit while the target is executing, the target returns the value of the current IPTR to the debugger. The debugger searches its list of programs for source code which corresponds to this address. If more than one program has code or data at this address during the system's life span, there will be a symbolic information conflict between each program.

This is resolved by the notion of a "current" program, which is the program currently executing at a given instant in time. The current program is searched first by the debugger when performing symbolic lookups. If a match is not found, any remaining enabled programs are then searched.

The **program** command has a **-current** option to cater for specifying the current program. Programs may also be disabled from symbolic lookup and re-enabled with the **-disable** and **-enable** options. The enabled/disabled state of a program may be displayed with the **-detail** option. The current program may be displayed with the **context** command.

The program key

A 32 bit unique program key is generated by the linker every time it is invoked, and this program key is placed in a global integer in the application program with the name `_ST_ProgramIdentifier`. The C-startup sequence for a ROM image executes an **ST_onstartup** function which issues the remote debug command, where **x** is the hexadecimal value of `_ST_ProgramIdentifier`:

```
program -current -uid 0xx
```

This command notifies the debugger of the current program and ensures the debugger starts using this newly executing program for symbolic cross-referencing as early on in the program's life span as is practicably possible.

The program key may also be specified with the **program** command options **-enable** and **-disable**, for example:

```
program -enable -uid 0x1234abcd
```

Preventing program notification

The **ST_onstartup** function described above may be omitted from a ROM system by using the following option with **st20cc**:

```
-W1 p_ST_NoProgramNotify=1
```

Setting the value of the program key

The value of the program key may be explicitly specified at link time, instead of being given a unique value by the linker. This is done by providing the following option to **st20cc**, where **x** is the value to be given to the program key:

```
-Wl p_ST_ProgramIdValue=x
```

Disabling programs

It may be useful in some circumstances to disable a program when the memory it uses is reused by a program which cannot identify itself as the current program. A typical example is an arrangement with a booter program, a main program and one or more relocatable code units (RCUs) loaded by the main program. The booter program and the RCUs share the same RAM. Just before the booter program calls the main program, it issues the following remote command, where **x** is the value of **_ST_ProgramIdentifier**.

```
program -disable -uid 0xx
```

When Relocatable Code Units (RCU[s]) are loaded later by the main program, there is no symbolic conflict for the memory shared by the booter and one or more RCUs. It is recommended that booter programs always disable themselves in the above manner to aid future compatibility. If the processor is restarted, the booter program will need to be re-enabled.

Compatibility with earlier toolsets

The debugger handles ROM images built with ST20 Embedded Toolset version R1.6.2 upwards. However, it may need assistance where multiple programs exist on a ROM, of which some are built with toolsets earlier than R1.8.1. This is because these do not issue a **program -current** command as an **ST_onstartup** function. Assistance is in the form of either modifying current code or interaction with the program call mechanism. Assistance may not be necessary when the code from multiple programs is executed from ROM because all IPTR values will be unique.

If the code is modifiable, the best approach is to create an instance of the variable **int _ST_ProgramIdentifier** for each program which is built with any toolset earlier than version R1.8.1, and place a “unique” value in each. Each program should be provided with an **ST_onstartup** function which issues a remote debug command, where **x** is the hexadecimal value of **_ST_ProgramIdentifier**:

```
program -current -uid 0xx
```

Each function should have a **startorder** value of at least 100000 to be executed before any system **ST_onstartup** functions and to allow for future toolset development (see [Chapter 21: Alphabetical list of commands](#), function [startorder on page 442](#)). The booter should disable itself before calling the main program as described above under the heading [Disabling programs](#).

If the code is not modifiable, the user has to interact during the debug session to provide assistance to the debugger. The debugger should be connected to the ROM system, and all the **.dbg** files specified with the command **program -new**. Before starting the system with **go**, the following command should be entered, where **n** is the debugger id of the booter program:

```
program -current n
```

A breakpoint should be set on **romload** of the main program and the system started. When this breakpoint is hit, the following command should be entered, where **m** is the debugger id of the main program:

```
program -current m
```

The following command should be entered, where **n** is the debugger id of the booter program:

```
program -disable n
```

9.4 Callbacks before and after the poke loop in romload()

The set of **poke** commands encountered during the link phase of a ROM program is performed at run time by the function **romload**. There is no such automatic mechanism for configuration scripts which read memory or perform conditional memory operations. To perform this function, one user-supplied function may be called prior to these pokes being carried out, and another user-supplied function may be called after the pokes.

These functions must be defined as:

```
void PrePokeLoopCallback(void)
void PostPokeLoopCallback(void)
```

They should both be defined in a source module called **initfuncs.c** which is compiled to a file called **initfuncs.tco**. The header file **initfuncs.h** must be included in **initfuncs.c**. The object file **initfuncs.tco** may then be linked into the main program to override stub functions provided in the toolset libraries.

- Note: 1 The object file **initfuncs.tco** must **not** be placed in a library; this object file must be specified to **st20cc** at link time.*
- 2 If one of the callback functions is implemented as described above, the other must be implemented in the same file, even if this is merely a stub.*

The user supplied functions may not access static data because it has not been set up at this stage in the bootstrap phase. The functions may declare local variables subject to stack allocation (see below). They should not call functions which have yet to be copied from ROM to RAM.

The functions **romload**, **PrePokeLoopCallback** and **PostPokeLoopCallback** use internal memory for their workspace. If the command **initstack** was present during the link phase, the function **romload** sets the contents of the WPTR to the argument of this command as one of its first operations, and before the callback functions are called. Otherwise the WPTR is not altered; if the processor was reset it will thus contain **0x80000140** on an ST20-C2 core, or **0x80000000** on an ST20-C1 core. The function **romload** and the default callback functions use seven words above the WPTR value.

If the callback functions use any stack or call any functions, the user must allocate local workspace in internal memory. This is done by specifying the top of a negatively growing stack for user and compiler generated local variables.

For example, to allocate 32 words for the user-supplied callbacks, at the start of **romload** the WPTR must be set to:

```
Wptr at reset + 32*4
= 0x80000140 + 32*4
= 0x800001C0
```

and this may be achieved by using the following command in the link phase:

```
initstack 0x800001C0
```

Note: Seven words above this stack are used by **romload** so the WPTR must be set at least seven words below the top of the internal memory segment.

The most convenient way of determining the number of stack words required by a **PrePokeLoopCallback** or **PostPokeLoopCallback** function is to use the stack depth analysis feature in **st20cc**.

9.5 OS20 awareness

The debugger determines OS20 state by reading symbolically referenced data structures in the target. If the current program is not specified, the following message may be displayed when the remote command **runtime os20** is issued, and the target may crash.

Warning: Incompatible OS/20 interface version number found.

The remedy to the above problem is to apply one of the techniques described under the heading *Compatibility with earlier toolsets on page 141* in [Section 9.3.2](#).

Note: The current program must be correctly specified before the **runtime os20** command is issued. If the **st20cc** option **-runtime os20** is used, the **runtime os20** command will be issued remotely from an **ST_onstartup** function, which is why program notification should be carried out by a higher priority **ST_onstartup** function.



Building and running relocatable code 10

10.1 Introduction

st20cc supports the creation of relocatable code units (RCUs) which may be loaded dynamically at run time and executed. Relocatable code units may be used in the following situations:

- when field upgrades to running applications are needed in real-time,
- configuration of software at power on, for example, in set-top box applications,
- loading time-critical functions into fast memory while the application is running,
- freeing up the memory once the RCU has completed avoids having rarely-used code taking up space in fast memory.

RCUs may be loaded from an external source, for example an ST Micro Connect or satellite or copied from ROM.

This chapter describes how RCUs are built using the toolset, the mechanisms for interacting with them and special considerations when using them with OS20.

The toolset also provides support for calling functions from other programs using the **import** and **export** commands and the function prototypes in the **d1.h** header file, which are implemented in **rcu.lib**. This mechanism offers a simple way of handling RCUs and may be easier to use for new users of the toolset. An example implementation of it is provided in the **examples/d1** directory. Another RCU example, which does not use this mechanism is provided in **examples/rcu**.

The **import** and **export** commands are described in [Chapter 21: Alphabetical list of commands on page 319](#). The library functions provided to support RCUs are described in [Chapter 18: Relocatable code support functions on page 289](#). For details of **st20cc** (the Embedded Toolset's compile/link driver) see the *ST20 Embedded Toolset User Manual*.

10.1.1 What is a relocatable code unit?

An RCU is a self contained unit of code and data that may be loaded and run from within another running application. The RCU can be loaded at any memory location within the application's memory map, that is, its load point does not need to be known at link time, unlike a normal linked unit (**.lib**) or ROM image.

An RCU has a single entry point which can accept and return arbitrary arguments. An RCU is limited to one code section and one combined data and BSS section.

ANSI C, C++ or assembler source files are compiled and linked by **st20cc** to produce an RCU file. **st20cc** generates a header at the start of the RCU file which describes the structure of the remainder of the file. RCU files are given the extension **.rcu**.

Figure 26 shows the sections which form an RCU as described below.

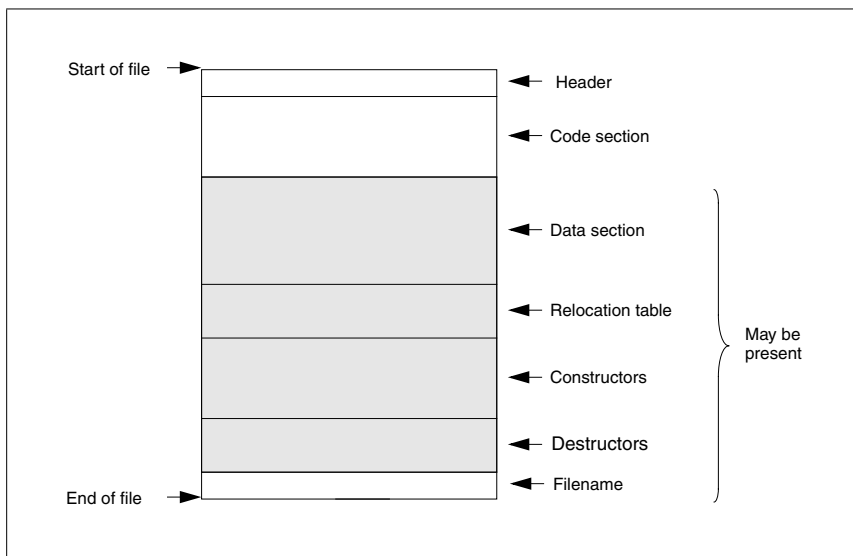


Figure 26: Constituent parts of an RCU

An RCU file generated by **st20cc** has a header, a code section a debug filename, and may contain a data section, and relocation, constructor and destructor tables if required:

Header The header describes the remainder of the RCU file which follows. All sizes are in bytes. All **int** types are 32 bits, little endian. The format of the header is:

```
typedef struct
{
int magic_number;      /* Hexadecimal number denotes file is an RCU */
int codesize;          /* Size of the code section in RCU */
int entrypointoffset;  /* Entrypoint offset into the code section */
int datasize;          /* Size of the data section */
int datalocationoffset; /* The offset in the static area where the data
                        section should be loaded */
int bsssize;           /* Size of BSS section */
int bsslocationoffset; /* The offset into the static area to place BSS */
int staticlinkoffset;  /* The offset into the static area that the static
                        link should point to */
int relsize;           /* Size of relocation table */
int conssize;          /* Size of constructor table */
int dessize;           /* Size of destructor table */
int stacksize;         /* Size of stack this RCU needs */
int heapsize;          /* Size of heap this RCU needs */
int dbgfilename;       /* Size of .dbg filename, including the null
                        terminator '\0' */
int slot0;             /* 4 words space for user defined data */
int slot1;
int slot2;
int slot3;
} rcuheader;
```

Code section Contains code for the RCU.

Data section Contains non-zero static data for the RCU. This may be empty if the RCU contains no non-zero static data.

Relocation table (Only required if relocation is being used) Describes how to patch in addresses that are unknown at link time. This is needed for any initialized data pointers; for example, an array of function pointers initialized to point to functions within the RCU. This table may not be present in the RCU if no relocations are required.

Constructors (Optional) Contains a table of relative addresses (offsets) into the RCU code that correspond to functions that must be called to initialize static and global C++ objects which have constructors (static constructors) or have been flagged using the **st20cc #pragma ST_onstartup**, that is, functions that are required to execute before the entry point of the RCU. This table may not be present in the RCU.

Destructors (Optional) Contains a table of relative addresses (offsets) into the RCU code that correspond to functions that must be called to clean up static and global C++ objects which have destructors (static destructors) or have been flagged with the **st20cc #pragma ST_onshutdown**, that is, functions that are required to execute after the body of the RCU has terminated. This table may not be present in the RCU.

Filename Contains the filename of the debug information file (**.dbg**).

10.1.2 Static area

When an RCU is loaded into memory, a static area needs to be constructed before the code can run. This contains all the global and static data for the RCU, and is composed of two sections, the data section (which contains initialized data) and the BSS section (which contains uninitialized or zero initialized data).

To allow the linker to place data more efficiently, the data and BSS sections may appear in either order, as shown in [Figure 27](#), but they must be contiguous.

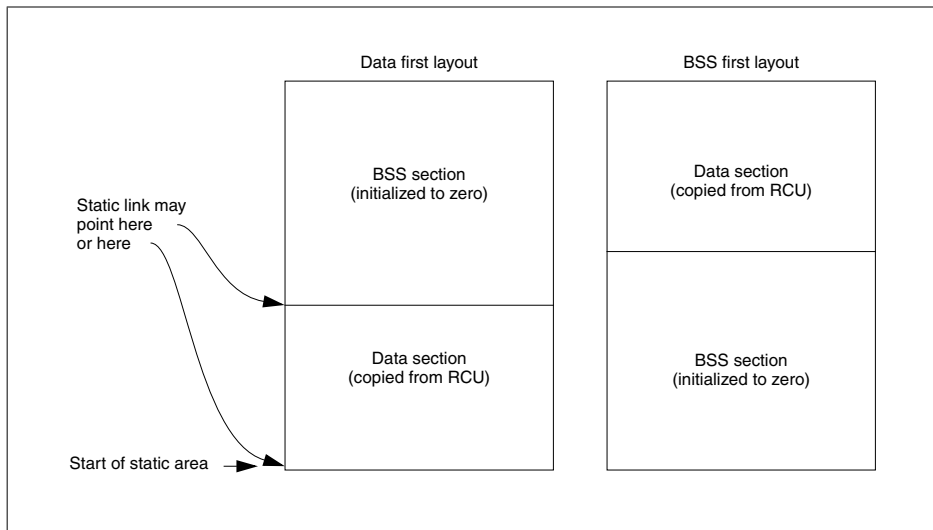


Figure 27: Alternative static area layouts

When an RCU is loaded into memory, the user is responsible for allocating the static area, using the sizes specified in the RCU header, and initializing the data section using the data section from the RCU file. The BSS section also needs to be initialized to zero, although this is usually performed by the RCU library (see [Section 10.2.3: Initializing and de-initializing an RCU](#)).

The static area must be aligned on a 32-bit boundary.

In addition, the static link (described in [Section 10.2: Overview of RCU development](#)) needs to be initialized to point into the static area. This may point at either the start of the data or the start of the BSS section, as specified by the offset in the RCU header.

10.1.3 Limitations

RCUs have some limitations that users should be aware of.

- Code and data placement using the **place** command is not supported.
- RCU files cannot be created as ROM image files and therefore the following **st20cc** options are invalid when creating an RCU:
-filled, -filledval, -off, -romimage.
- When using command files, the following commands are invalid and create warning messages. The commands are ignored by **st20cc**, and may cause later error messages or an invalid RCU:
 - > **bootdata,**
 - > **bootiptr,**
 - > **heap,**
 - > **initstack,**
 - > **memory,**
 - > **place,**
 - > **poke,**
 - > **romimage,**
 - > **stack,**
 - > **store.**

10.2 Overview of RCU development

The toolset provides a set of library functions to support systems using RCUs. **rcu.lib** contains functions to initialize and de-initialize RCUs (declared in **rcu.h**) as well as functions to support function calls between applications (declared in **dl.h**). These functions are documented in [Chapter 18: Relocatable code support functions on page 289](#).

rcu.h defines:

- a structure that corresponds to the RCU header (**rcu_header**),
- a structure that stores computed RCU values (**rcu_data**),
- a function that initializes an RCU (**rcu_init**),
- a function that de-initializes an RCU (**rcu_deinit**).

*Note: A pre-requisite to using the functions in **rcu.h** is that the RCU has been loaded into memory. (The memory for the different parts of the RCU does not have to be contiguous.)*

If **rcu.h** is used, the application should perform the following operations.

- Set up a pointer of type **rcu_header** which points to the RCU header.
- Determine the location of the code, data, relocation table, constructors, destructors and the debug filename in memory.
- Using the combined sizes of the data and BSS sections from the RCU header, allocate the static area.
- Copy the data section into the static area, at the offset specified in the RCU header.
- Allocate a structure of type **rcu_data**.

- Call the RCU initialization function `rcu_init`, passing in the address of the RCU header, the code section, the static area, the relocation, constructor, and destructor tables, the debug filename, and the address of the `rcu_data` structure. See [Section 10.2.3](#) for more details.
- Invoke the entry point, passing in the static link (both returned in the `rcu_data` structure by `rcu_init`); see below.
- If the application and RCU make function calls to each other, the functions provided in `dl.h` should also be used to import/export functions; see [Section 10.3](#).
- When the RCU is no longer required, call the RCU de-initialization function `rcu_deinit`; see [Section 10.2.3](#).

10.2.1 Static link

Each program has a static area; a pointer to the static area is normally passed as an extra parameter to every function. This parameter is called the static link and contains the address of the static area for the program. Global and static variables are accessed by adding an offset to that pointer. See [Section 13.15.2: Calling sequence on page 195](#).

When using an RCU there are in effect two static areas, one for the RCU and one for the main program, so it is important that the static link always points to the relevant area when a function is called. Normally the current static link is passed implicitly when a function is called. When calling a function in the other program, the static link needs to be passed explicitly.

This is achieved by associating the `#pragma ST_nolink` with the function pointer of the other program that is to be invoked and passing the static link value as the first parameter.

10.2.2 Entry point

An RCU provides a single entry point that should be called with the appropriate static link value. The parameters and return value that the entry point provides are not restricted in any way.

```
int (*ep)(int staticlink, int arg1, int arg2);
#pragma ST_nolink(ep)
ep = (int (*)(int,int,int))entrypoint_pointer;
res = ep(staticlink, 2, 3);
```

In this example the variable `entrypoint_pointer` is a pointer to the entry point of the RCU. The variable `staticlink` is the static link of the RCU. The entry point is invoked with the static link as the first parameter and two additional integer parameters and returns an integer result.

The entry point, pointed to by `entrypoint_pointer` would need to be defined in the RCU program as follows:

```
int rcuentrypoint(int a, int b) {
```

Note: The static link parameter is implicitly allocated by the compiler so the user does not need to define this.

10.2.3 Initializing and de-initializing an RCU

The function `rcu_init` initializes an RCU ready for the entry point to be called. This function is passed the RCU header, together with the addresses of the code section, static area, relocation, constructor, and destructor tables and the debug filename, and returns information about the RCU by writing into the `rcu_data` structure:

```
void rcu_init( rcu_header *header,
              unsigned char *codesection,
              unsigned char *staticarea,
              int *relocations,
              int *constructors,
              int *deconstructors,
              char *dbgfilename,
              rcu_data *data );
```

`rcu_init` initializes the BSS section of the static data area, patch the data section using the relocation table, invoke the constructor functions and inform the debugger of the new program. `rcu_init` then computes the entry point and static link values of the RCU and stores these values in the structure `rcu_data`. `rcu_data` contains the following fields, all of which are initialized by `rcu_init`:

```
typedef struct {
    unsigned char *codesection; /* Address of code section */
    int *destructors; /* Address of destructor section */
    long dbgprogramid; /* RCU program identity to pass to debugger */
    void (*entrypoint)(); /* Address of RCU entry point */
    int staticlink; /* Value of RCU static link */
} rcu_data;
```

The entry point can then be invoked and the static link value passed as the first parameter by associating the `#pragma ST_nolink` with the function pointer.

For example:

```
rcu_header *rh = header of RCU file
unsigned char *codesection = address of loaded code section
unsigned char *staticarea = address of the static area
int *relocations = address of relocation table
int *constructors = address of constructor table
int *destructors = address of destructor table
char *dbgfilename = address of debug filename
rcu_data rd;
int res;
int (*ep)(int staticlink, int arg1, int arg2);
#pragma ST_nolink(ep)

/* Call rcu_init function */
rcu_init(rh, codesection, staticarea, relocations, constructors,
        destructors, dbgfilename, &rd);

/* Call entry point function */
ep = (int (*)(int,int,int))rd.entrypoint; // entry point is found in rcu_data
res = ep(rd.staticlink, 5, 3); // static link is found in rcu_data
printf("res = %d\n",res);

/* Call rcu_deinit function */
rcu_deinit(rh, &rd);
```

When the RCU is no longer required, call the `rcu_deinit` function:

```
void rcu_deinit(rcu_header *header, rcu_data *data);
```

`rcu_deinit` invokes the destructor functions and inform the debugger that the program can be removed. A full RCU example is given in the directory `examples/rcu/loader.c`.

10.2.4 Using the instruction and data caches

If either of the ST20 instruction or data caches are enabled, extra care must be taken when handling relocatable code units. If the area of memory that the RCU resides in is cached then it is important to maintain cache coherency.

This is especially important when replacing one RCU with another. In this situation the following technique should be used. After calling `rcu_deinit()` and there are no further accesses to the RCU's memory planned, the instruction cache should be invalidated and the data cache should be flushed. This purges all data or instructions belonging to the old RCU from the cache. The new RCU can then be loaded safely. See the *OS20 User Manual*, chapter *Caches* for further details.

10.2.5 Creating an RCU

An RCU may be generated by `st20cc` using the following input file types:

```
.c           C
.cxx         C++
.s           Assembly language
```

An RCU can be built with a single entry point, in which case the following `st20cc` command line options must be used:

```
-rcu         to specify that an RCU file is being created,
-e          to specify the main entry point of the RCU.
```

A target processor type must also be specified. The output from `st20cc` is a binary file with the extension `.rcu`.

For example:

```
st20cc myrcu.c -rcu -e rcu_main -c2
```

Alternatively an RCU may be built with one or more exported/imported entry points, in which case the `-dl` option is used. The `-dl` option is mutually exclusive to the options `-e` and `-rcu`.

```
-dl          Build the RCU with an entry point that returns the import and export
              parameters; see Section 10.3.
```

For example:

```
st20cc myrcu.c -dl -c2
```

The following option may be used with either of the above cases:

```
-rculoc <slot><value>  Enter user defined data in the RCU's header.
                        4 words are available.
```

For example:

```
st20cc myrcu.c -rcu -e rcu_main -rculoc 0-123 -c2
```


10.2.6 Preparing an RCU for debugging

The use of the debugger and the debugger graphical interface are described in the *ST20 Embedded Toolset User Manual* in the *st20run tool* and *ST Visual Develop* chapters respectively. However, for completeness, this section is included here.

RCU files may be debugged symbolically. In order to do this, the debugger must be told where the RCU has been loaded. If the debugger is connected and `rcu_init` is used, the debugger gets this information by calling `rcu_init`. If either of these conditions is not met the following command should be used:

```
program -new dbg_filename -code address -data address -bss address
```

where:

dbg_filename specifies the name of the debug information file created by **st20cc** when it created the RCU file.

-code address -data address -bss address specify the addresses at which the code, data and BSS sections have been loaded.

For example:

```
program -new app.dbg -code 0x40001000 -data 0x80000140 -bss 0x80000400
```

or:

```
program -new simp.dbg -code 0x40001000
```

for an RCU that has no data section.

This command can be submitted to the debugger via the command console or alternatively the application can do this by using the `debugcommand` function:

```
debugcommand("program -new simp.dbg -code 0x40001000", &pid);
```

Debug operations (for example, breakpoint, variable examination) can now be performed on the RCU.

*Note: The correct program context must be used so that the debugger knows which program is being referred to. This can be done using the **context** command or the program chooser in the source display of the GUI. When the program hits a breakpoint the context is automatically updated to the correct program.*

When the RCU is ready to be unloaded the debugger must be informed. The recommended way of doing this is by using `rcu_deinit`. The command: **program -delete** can also be used. This command can be issued by the (un)loader code:

```
sprintf(buf, "program -delete %d", pid);  
debugcommad(buf);
```

where **pid** is assigned by a previous **program -new** command.

10.3 Support for calling functions from other programs

10.3.1 Import and export commands

The toolset provides support for programs calling functions from other programs using the **import** and **export** commands and the function prototypes in the **dl.h** header file, which are implemented in **rcu.lib**. An example of a program that exports/imports symbols is an RCU.

The **import** command specifies the name of a function that is to be used but is not defined within a program. The **export** command specifies the name of a function that is to be exported from a program.

A program that imports functions can call the function **dl_get_imports**, which returns the set of function names that are used by the program but are not defined:

```
dl_sym_list *dl_get_imports( void );
```

A program that exports functions can call the function **dl_get_exports**, which returns the set of function names that are exported by the program:

```
dl_sym_list *dl_get_exports( void );
```

To enable an imported symbol to be called, it must be updated with a corresponding exported symbol from another program. This process is achieved using the function

dl_match_imports_to_exports:

```
void dl_match_imports_to_exports( dl_sym_list *imports,
                                dl_sym_list *exports );
```

Example

If, for example, the program **lib1** requires the use of the symbol **fred** which is not defined, this is achieved using the command:

```
import fred
```

as input to **st20cc** when building **lib1**.

The loader program can export the symbol **fred** with the command:

```
export fred
```

When the program **lib1** is loaded, it requires its imported symbol list to be updated with the exported symbols from the loader program. This can be done by calling the **dl_match_imports_to_exports** function with the **lib1** import list and the loader export list:

```
dl_match_imports_to_export( lib1ImportList, loaderExportList );
```

10.3.2 -dl option

st20cc provides support for importing and exporting functions with the **-dl** option. With this option, an RCU is built with an entry point that returns the program's import and export list when called. These lists can then be updated by the calling program. Thus using the **-dl** option, **st20cc** generates an RCU with an entry point which is a means to access all the RCU's imported and exported symbols. Unlike the **-rcu** option, the entrypoint cannot be used to call a user specified function, which must be done through an **export** command.

Example

If the program `lib1` is built with the `-dl` option then it may be invoked as follows (where `rd` is an `rcu_data` type; see [Section 10.2](#)):

```
dl_sym_list imports, exports; // import and exports of the RCU
dl_entry_point_fn dlep = (dl_entry_point_fn)rd->entrpoint;
dlep(rd->staticlink, &imports, &exports);
```

If the `lib1` program imports functions from the loader program then an additional call is required to match up these imports with the loader exports.

```
dl_match_imports_to_exports(&imports, dl_get_exports());
```

If the `lib1` program also exports functions to the loader program then an additional call is required to match up these exports with the loader imports.

```
dl_match_imports_to_exports(dl_get_imports(), &exports);
```

10.3.3 Import and export lists

This process can be carried out for several programs. For each program, its import list should be matched with the export lists of all other programs and the import list of all other programs should be matched with the its export list.

When a program that has exported functions is no longer in use and has been unloaded it is important that the import and export lists are cleared of any redundant entries. This enables any further calls to the program that is no longer loaded to return correctly. Thus the `dl_reset_imports_from_exports` function should be used whenever an RCU program is unloaded:

```
dl_reset_imports_from_exports(dl_get_imports(), &exports);
```

Large import/export lists can be maintained using command language arrays.

Example

```
numlibs = 34
libs[0] = "libfn0"
libs[1] = "libfn1"
...
libs[numlibs-1] = "libfn33"
```

defines an array of 34 functions which can be exported using:

```
for (i=0;i<numlibs;i++) {
    export (libs[i])
}
```

and can be imported using:

```
for (i=0;i<numlibs;i++) {
    import (libs[i])
}
```

10.3.4 Importing C runtime functions

For RCUs built with the `st20cc` option `-dl`, virtually all but the simplest side-effect free C runtime functions need to be imported. These include both visible functions such as `printf`, and also hidden functions such as `_ST_errno`.

10.4 Using RCUs with OS20

This section provides some guidelines on using OS20 with Relocatable Code Units (RCUs), and makes several recommendations in order to avoid some potential problems.

OS20 is described in the *OS20 User Manual*. Function descriptions are given in the chapter *Alphabetical list of functions*.

10.4.1 OS20 kernel

Ensure that only one copy of the kernel is linked in. Many of the standard C run-time libraries can be linked into both the main application code and the RCU with no problems. For simple functions, for example `strlen`, this may even be desirable, to reduce the function call overhead at the expense of code size.

However, for OS20 it is vital that there is only one copy of the kernel running on the processor. The easiest way to do this is to ensure that the application code (which is run when the processor is booted and loads any RCUs) also starts OS20. Any RCUs should only ever **import** OS20 functions from the application.

Note: The `st20cc` option `-runtime os20` can still be used with `-rcu` or `-dl`. The linker scripts are written so that when used with an RCU, run-time libraries are never linked in.

10.4.2 Passing function pointers

Problems can occur when passing function pointers between RCUs and the application. This is because while the pointer is valid in all cases, it needs to be called with the correct static link. In particular this is a problem with some OS20 functions which take a function pointer as an argument. To overcome this a number of additional functions have been provided, which take a static link as well as function pointer:

`interrupt_install_sl`, `task_create_sl`, `task_init_sl`, `task_onexit_set_sl`

These functions are described in [Section 10.4.5](#), [Section 10.4.6](#) and in the *OS20 User Manual*, chapter *Alphabetical list of functions*.

10.4.3 OS20 functions

All OS20 functions are declared in header files with a `#pragma ST_translate`¹. Normally this is transparent to the user, however, this does become visible when using **import** and **export** commands in a linker configuration file, as the name seen by the linker has a “`os`” suffix.

To make functions of this type visible to other code units, the `-translate` option should be used on the **import** and **export** commands. For example:

```
import task_priority -translate task_priority%os
```

1. This is to ensure that even when the OS20 library is linked in, it is still possible to define functions with the same name in the user's code, by not including the OS20 header files.

10.4.4 Hidden functions

As well as all the “user visible” OS20 functions, there is a hidden function which is used by the **TIMEOUT_IMMEDIATE** and **TIMEOUT_INFINITY** macros. This is **time_pointers()**, which is used to ensure that the same values are used for these pointers in all code units. This function must also be imported by any RCUs which use these macros:

```
import time_pointers -translate time_pointers%os
```

10.4.5 Creating an OS20 task for an RCU

Two alternative functions are provided for creating a task in a relocatable code unit:

task_create_sl and **task_init_sl**. These functions are very similar to **task_create** and **task_init** but allow a static link to be specified. See the *OS20 User Manual*, chapters *Alphabetical list of functions* for function definitions and *Tasks* for OS20 tasks.

A pointer to the static area is normally passed as an extra parameter to every function. This parameter is called the static link and contains the address of the static area for the program; see [Section 10.1.2](#).

10.4.6 Attaching an interrupt handler

The function **interrupt_install_sl** enables an interrupt to be installed and the static link specified. It is intended for use with relocatable code units.

interrupt_install_sl is defined in the *OS20 User Manual*, chapter *Alphabetical list of functions*, and, for the OS20 interrupt handling mechanism, chapter *Interrupts*.

10.4.7 Code incompatibility

It is sometimes useful to write code that is compiled to run in both the main application and the loaded RCU, calling different functions depending on where it is run. This can occur when using OS20 because the application can call functions which only take a function pointer. If the same functionality is needed from the RCU, a static link needs to be specified as well. Thus it is difficult to write code which can be used in both cases.

One solution to this is to use the **-translate** option of the **export** command. In this way the name of the function called by the RCU code does not have to have the same name in the main application. For example, if the RCU uses the command:

```
import application_fn
```

the RCU is built with a stub function called **application_fn()**, which calls through a function pointer, and which is initialized by the dynamic link library **d1.lib** to point to the function, which is then exported under the name “**application_fn**”.

However, the application can export this function using the command:

```
export application_fn -translate application_fn_rcu
```

which exports the function **application_fn_rcu()**, but with the name “**application_fn**”.

Thus, when the RCU is loaded, the name “**application_fn**” exported by the application is matched with the name “**application_fn**” imported by the RCU. When the RCU calls the stub **application_fn()**, the function pointer points to the function **application_fn_rcu()** in the application, and it is this function which is called. This function can either be a complete replacement for the normal **application_fn()**, or it is possible to simply modify the parameters before calling the original function.



Using st20run with OS20

11

st20run has a number of features which make it easier to debug programs which use an RTOS. These are generally divided into “task awareness” features, which give **st20run** visibility of the tasks running on the target, and “report” features which provide the ability to print RTOS data structures in an easily readable format.

When using **st20run** with OS20, both of these features are provided. This chapter describes how to enable these features, and how this changes what is seen by the user.

11.1 Selecting the OS20 run-time system

In order to tell **st20run** which RTOS is being used on the target, the **runtime** command is used (see [runtime on page 432](#)). To select the OS20 run-time system, use one of the following commands:

runtime c2os20 *Selects the run-time system for an ST20-C2 core*

or

runtime c1os20 *Selects the run-time system for an ST20-C1 core*

Normally these commands are issued automatically when OS20 starts running, and so it is not necessary to type them in manually¹. If the program is linked using the **st20cc** command line option **-runtime os20** then OS20 is also started automatically.

However, there are some occasions when it may be necessary to select the run-time system manually, in particular when connecting to a running system (see [Section 9.2.4: Debugging the ROM image on page 137](#)).

1. The RTOS is able to do this using the function call **debugcommand()**, which allows the target to pass a command to **st20run**. OS20 calls this function from **kernel_start()**, which is one of the functions called automatically at start up when the program has been linked using the **st20cc** option **-runtime os20**.

The **runtime** command takes an optional **-v[erbose]** flag, which enables verbose messages from the task awareness system. These are informational messages which give some additional information about what the task awareness code is doing when operating normally. Errors are still reported without using this flag.

11.2 Task awareness

The most important features of task awareness in **st20run** revolve around the ability to extract the list of active tasks from the running target, and determine which task was running when the target stops.

The task awareness features of **st20run** are visible in a number of ways.

- ST Visual Develop provides a complete GUI development environment. For full details, see the *ST20 Embedded Toolset User Manual*, particularly the chapter *Debugging with ST Visual Develop*.
- Most obviously, the **Tasks** window and the **task** command show a list of all tasks in the system, and details about those tasks.
- The **Editor** window can be switched to display the last code which the chosen task executed when it stopped executing, whatever the reason. This then affects any local variables displayed, where the variable from the selected task's stack frame is printed.
- Similarly, in the **Callstack** window and the **where** command, any task can be selected, not just the one which was executing when the target stopped.
- The **Callstack** window shows the call hierarchy for the task selected in the **Tasks** window.
- Break events can be task specific (that is they are only triggered if the specified task is the one which is currently running). This is particularly useful when a function is executed by a number of tasks.
- A task can be explicitly selected through the selection box in the **Command Console** window, or using the **context** command, which sets the default task to be used in future operations and commands.

11.2.1 Interrupts and traps

The OS20 task awareness extends the concept of a task to encompass interrupts and trap handlers, and these are treated in the same way as tasks. This can be particularly useful in the context of task specific events, for example, to only trigger a breakpoint when a particular interrupt is running.

Interrupt handlers are given the fixed name:

Interrupt [n]

where **n** is the interrupt level.

Trap handler names are different between ST20-C1 and ST20-C2 cores. On the ST20-C2 they are given the name:

Trap[priority] [name]

where **priority** is either "**High**" or "**Low**" for high and low scheduling priority trap handlers, and **name** is the trap group name.

On the ST20-C1 the name is simply:

Trap[*name*]

where *name* is the name of the exception handler.

11.2.2 Editor window

When debugging a multi-tasking program, normally the **Editor** window automatically switches between tasks to show the task which was running when the target was stopped (for example by hitting a breakpoint).

However, in some cases it is very useful to have multiple **Editor** windows open, each displaying the state of a particular task. In this case, simply select the required task from the **Tasks** window, and select “Follow This Task” mode from the Options menu. Thus, when the target stops, instead of the **Editor** window switching to the task which was executing when the target stopped, it locates to the last code which the selected task executed prior to the target being stopped.

11.2.3 Task states

When **st20run** prints a task’s state, it is in one of two forms:

```
task 0 "job(0)" stopped at <main.c 33 1> by breakpoint 0
```

or:

```
task 3 "root" active and scheduled last stopped at <hello.c 53 0> in space 0
```

This line is made up from several components:

“**task n**” gives the task’s ID. This is the ID which **st20run** uses, and has no connection with task identifiers on the target. This is the value which should be specified in **st20run** commands which take a *taskid* parameter, for example **break**.

The name in quotes is the name of the task, specified when it was created.

In the first, “**stopped at**” form, this shows that this was the task which was executing when the target was stopped. It is followed by the source reference for the statement the task was executing when it was stopped. If **st20run** is unable to determine the source reference (which occurs when there is no detailed debug information for that location) then the value of the IPTR is displayed. Finally the reason why the task was stopped is shown (breakpoint, stepout or step).

The second “**last stopped at**” form is more complex, and means that this task was not the one which was executing when the target was stopped.

“**active**” is the current OS20 task state, and can be one of:

- **created**: the task has been created, but has not started to run yet,
- **active**: the task has started running,
- **terminating**: the task has returned from its entry point (or been killed) but is still executing its onexit handler,
- **terminated**: the task has finished running, and could be deleted using **task_delete()**.

“**scheduled**” indicates what, if anything, is stopping the task from running. For a task this can be:

- **channel waiting**: the task is blocked waiting for a channel (ST20-C2 only),
- **semaphore waiting**: the task is blocked waiting for a semaphore or message queue,

- **task waiting**: the task is blocked in a `task_wait()`,
- **timer waiting**: the task is blocked waiting for a time (for example, `task_delay()`),
- **interrupted**: the task was running when interrupted by an interrupt handler,
- **preempted**: the task was running when preempted by a higher priority task,
- **scheduled**: the task is waiting to run (normally caused by the task being timesliced or the task waiting to run having been recently unblocked).

Note: Some of the waiting conditions can be combined, for example a task blocked on a timeout semaphore is displayed as both semaphore and timer waiting.

For a trap or interrupt handler the state can be one of:

- **inactive**: not currently running,
- **interrupted**: the task or interrupt was running when interrupted by a trap or higher priority interrupt handler.

Finally the source reference shows the last statement which the task executed before being stopped, and which address space it is in.

11.2.4 Task details

In addition to a task's state and where the task was last executing, **st20run** can display some additional details about the task. This is done by selecting the task's name in the **Tasks** window or specifying the taskid to the **task** command.

For all tasks, interrupts, traps and high priority processes, the stack range (that is, the top and bottom addresses of the allocated stack) are shown.

In addition for a task, the details displayed include the task's current priority, and the address of its `task_t` and `tdesc_t`.

Finally the registers of task are displayed. The number of registers displayed vary depending on how the task stopped executing. In many cases OS20, or the ST20 itself, knows that only certain registers can be valid, and so does not save them. In these cases they are not displayed by **st20run** either.

11.2.5 Making task awareness available

In order to determine what was happening on the target, the OS20 task awareness code in **st20run** needs to read some of the OS20 data structures from the target memory, and some additional hardware registers.

To make this possible three conditions must be met.

- **st20run** must have the correct symbol table available for the program which is being debugged. This means that the debug data files (`.dbg` files) for the application must be present in the directory from which **st20run** was started, or specified later using the **program** command.
- All memory which holds OS20 data structures must be readable by **st20run**. In particular, any internal memory segments which hold key data structures, and also a memory segment which describes the interrupt controller's registers, must exist. Normally these are set up automatically if the **chip** command has been used, otherwise the segments must be specified in the user's `.cfg` files, for example:

```
memory INTERNAL 0x80000000 0x1000 RAM
memory INT_CONTROLLER 0x20000000 0x1000 PERIPHERAL
```

- The programmer must be careful to avoid corrupting OS20 data structures. In particular when an object (for example a `semaphore_t`) is no longer required, it should be deleted, and not simply be allowed to go out of scope. While this would not normally affect the running of the program, it can cause the debugger to display incorrect information, which could be misleading when trying to track down a problem (see [Section 11.2.6](#)).

11.2.6 Unknown tasks

One problem which is frequently seen when debugging programs is that of an “Unknown task”. This occurs because of the way the OS20 task awareness code in **st20run** scans the data structures on the target. It does this in two passes.

- First it extracts the names of all the tasks in the system.
- Secondly it examines all the data structures which a task could be blocked on, to update the task's state.

If a task is found in the second pass, which was not found in the first, then the code cannot know the task's name, and invents a task name of the form:

Unknown [n]

where **n** is a unique task identifier used by the debugger.

Normally unknown tasks are caused by an error in the target code, the most common causes being a corrupted data structure, a stack overflow, or **st20run** being unable to read target memory or registers (for example for the interrupt controller).

11.2.7 Deleted tasks

When an OS20 task has finished running, and been deleted using `task_delete()`, it disappears from the data structures which OS20 maintains on the target. The next time **st20run** reads these data structures, it finds that one of the tasks it previously knew about is no longer present on the target. At this point it assumes that the task has been deleted, and so changes the task's state to “**DELETED**”.

At this point it may be desirable to remove the task completely from **st20run**'s task list. This can be done using the **task -delete** command.

11.3 RTOS aware printing

When an OS20 object is printed, **st20run** can either display the data structure as a normal C structure, or attempt to interpret the data and display it in a more meaningful way. By default, the graphical user interface interprets the data structure, although this behavior can be suppressed if required. When using the **print** command, data structure interpretation can be enabled by using the **-rtos** option.

Data structures of the following types are interpreted:

```
task_t
semaphore_t
message_queue_t
chan_t
partition_t
```

In particular, the “class” of any object is displayed (for example whether a semaphore is FIFO or priority based semaphore). Also if tasks can be blocked waiting for a data structure to be made ready, the list of tasks waiting is displayed, using the same format as used in the task command; see [Section 11.2.3: Task states](#).

For example:

```
OS/20 fifo semaphore: count 0
tasks waiting
task 2 "sem_task0" semaphore waiting last stopped at 0x40000c22 in space 0
task 3 "sem_task1" semaphore waiting last stopped at 0x40000c22 in space 0
```



Part 2

ANSI C and C++





Preprocessing

12

This chapter provides a reference for the preprocessing directives and macros supported by **st20cc**, the compile/link tool described in the *ST20 Embedded Toolset User Manual*.

Note: When a C++ template file is compiled, **st20cc** creates a temporary **.ti** (template information) file, containing object names. This is used internally to link with libraries having the same instantiated objects. **.ti** files should be ignored, and may safely be deleted.

12.1 The #indent preprocessor directive

Syntax

```
#indent "string"
```

Description

The **#indent** directive enables a comment to be inserted in the application to be compiled in TCOFF format. This may then be viewed using **ST20list**, and is intended to be used, for example, for version information.

12.2 The #pragma preprocessor directive

Syntax

```
#pragma pragma (params)
```

Description

The **#pragma** directive activates and deactivates various compilation options in sections of C and C++ code. It may be used to set (or override) options specified on the command line. Most pragmas also take parameters or numerical arguments. [Table 36](#) lists the compilation pragmas supported by **st20cc**. [Section 12.2.14 on page 175](#) lists some additional rules for using pragmas with C++.

Pragma	Description
ST_device	Marks a variable as accessible by device instructions
ST_inline	Marks a function for inline function expansion
ST_interrupt_handler	Specifies that the function is an interrupt handler
ST_naked_function	Specifies that the function is to be compiled with no prolog or epilog code
ST_nolink	Compiles the specified function without a static link
ST_nosideeffects	Marks the specified function as being side-effect free
ST_off	Disables specific compiler actions
ST_on	Enables specific compiler actions
ST_onshutdown	Specifies that the function is to be executed after main has terminated
ST_onstartup	Specifies that the function is to be executed before main has started
ST_place_at_workspace_offset	Allocates a local variable at a specific offset in the local workspace
ST_section	Allocates a named function or data object to a named section in memory
ST_translate	The compiler replaces all references to a specified name with a new name
ST_trap_handler	Specifies that the function is a trap handler

Table 36: st20cc compilation pragmas

12.2.1 Pragma ST_device

This pragma marks a variable (which can be a pointer) which is accessible via the device instructions available on the ST20-C2 processor. The variable may be a **float**, **char**, **int**, **short** or **long**; it may also be a **typedef**, **struct** or **union**. It may not be a **double** or contain a **double**.

Note: Device memory accesses behave differently to normal accesses, and it is important to use the right one, especially when accessing peripherals or using a cache. See [Section 13.10: Device memory access on page 191](#) for a detailed explanation.

Syntax

```
#pragma ST_device(variable-name)
```

where **variable-name** may be a variable or pointer declared after the pragma.

Example

```
#pragma ST_device(wibble)
int wibble;           /* Accesses at &wibble using the variable name will
                        be device accesses. */
```

OR:

```
#pragma ST_device(wibble)
int *wibble;          /* Accesses at the address pointed to by wibble using the
                        variable name will be device accesses. */
```


If a variable or variable pointer is declared in a header file and is defined in a separate .c source file, then only one **#pragma ST_device** is required, for example:

In the header file:

```
#pragma ST_device(device)
extern int *device;
```

In the .c file:

```
#include "header.h"
int *device=(int*)0x20002000;
```

Note: If the pragma is put on a pointer to an int, only accesses to the pointed-to int will be device accesses. It is always the totally dereferenced location which will be accessed using the device instructions.

12.2.2 Pragma ST_inline

This pragma indicates to **st20cc** that the named function is to be considered as a candidate for inline function expansion. The function will not be inlined if it is used before being defined. The facility for inline function expansion is described in the *ST20 Embedded Toolset User Manual*, chapter *st20cc compile/link tool*.

Syntax

```
#pragma ST_inline (function-name)
```

The named function must have been declared before the pragma is encountered. A warning is generated and the pragma is ignored if:

- the named function has been defined before the pragma is encountered,
- its parameter is not a function identifier,
- it is given no parameters or more than one parameter.

See [Table 38 on page 175](#) for use with C++.

12.2.3 Pragma ST_interrupt_handler

This pragma indicates to **st20cc** that the named function is to be compiled as an interrupt handler.

Syntax:

```
#pragma ST_interrupt_handler(function-name)
```

This pragma changes the compiler's normal calling conventions; readers are advised to read [Section 13.15.4: Using the trap, interrupt and naked pragmas on page 198](#).

The effect of the pragmas **ST_interrupt_handler** and **ST_trap_handler** is processor-dependent. On an ST20-C1 core, interrupt and trap handling has been unified and so it is strongly recommended that interrupts should be declared as trap handlers; see [Section 12.2.13: Pragma ST_trap_handler on page 174](#).

For details of how to specify and enable a trap handler; see the appropriate processor datasheet. Trap handling examples are given in [Section 13.15.4: Using the trap, interrupt and naked pragmas on page 198](#).

Trap and interrupt handlers on the ST20-C2 core generate different code, so it is important to specify the correct pragma.

On an ST20-C2 core, the interrupt handler should take the form:

```
#pragma ST_interrupt_handler(int_hand)
void int_hand(void)
{
}
```

12.2.4 Pragma ST_naked_function

The pragma **ST_naked_function** compiles a specified function without any code to enter the function (prolog code) or code to exit the function (epilog code). Such code may be supplied by the user by means of `__asm` inserts; see [Chapter 1: Assembly-level programming on page 3](#). If no epilog code is explicitly provided, the function exits to the following function.

Note: This pragma alters the normal calling convention used by the compiler and should be used with care. Users are advised to first read [Section 13.15.4: Using the trap, interrupt and naked pragmas on page 198](#) which describes this pragma.

When writing `__asm` inserts for naked functions, the **ajw** instruction may make use of the language extension `__LOCAL_SIZE`; see [Section 1.1.4: Built-in variables on page 6](#).

For example:

```
ajw - __LOCAL_SIZE
```

reserves sufficient space in the local workspace for the variables defined by a function. Conversely:

```
ajw __LOCAL_SIZE
```

returns the local workspace used by the current function. The `__LOCAL_SIZE` term may not be combined with other terms, that is, `__LOCAL_SIZE+2` is illegal.

12.2.5 Pragma ST_nolink

The pragma **ST_nolink** compiles a specified function without a static link. The function must already have been declared and must not have been defined or called.

A function that does not require to access data via the static link (that is, which does not refer to global data) may be identified, using this pragma, as a “nolink” routine. Doing this saves up to one byte of code in the function, and up to two bytes in the caller. It may also ease interfacing with routines written in assembler language. However, it restricts the function, specified by the **ST_nolink** pragma as it cannot then call other functions unless they also have been declared to be nolink, nor can it access global (or local static) data.

Note: Functions which have had the **ST_nolink** pragma applied may not be called through a pointer unless that pointer also has the **ST_nolink** pragma applied to it. In this case the function pointer could be used to pass the static link to the function. This is useful for relocatable code units; see [Chapter 10: Building and running relocatable code on page 145](#).

Syntax:

```
#pragma ST_nolink (function-name)
```

The following code allows the C function **max** to be called without a static link:

```
int max(const int x, const int y);
#pragma ST_nolink (max)
int max(const int x, const int y)
{ return x > y ? x : y; }
```

12.2.6 Pragma ST_nosideeffects

This pragma enables **st20cc** to be provided with more information about functions, whose behavior could otherwise be ambiguous, and therefore helps to maximize the degree of optimization performed.

Syntax:

```
#pragma ST_nosideeffects (function-name)
```

where **function-name** must have been declared but not defined or instantiated before the pragma is encountered. A warning is generated and the pragma ignored if:

- the named function has been defined, part defined or instantiated before the pragma is encountered,
- its parameter is not a function identifier,
- it is given more than one parameter or none at all.

Functions marked as side-effect free enable **st20cc** to avoid making worst-case assumptions when compiling variables which are modified when the function is called. A function is side-effect free when, within the body of a function:

- there are no assignments to variables which are defined outside the function, including writing through a pointer where the pointer points to a variable defined outside the function, for example I/O or “passing by reference” are not allowed,
- there are no assignments to static variables (although initialization of static variables within definitions are allowed),
- there are no reads from or assignments to volatile variables,
- there are no calls to functions which may have side-effects.

Ensure that a function meets the above criteria before marking it with the pragma **ST_nosideeffects**. If a function is marked as side-effect free and it does not meet the above criteria, **st20cc** reports an error. Assembler inserts, that is, **__asm** statements and pointers to functions, are a particular risk area because unconsciously, a function may be introduced which breaks the above criteria. For this reason, **st20cc** generates a warning each time assembler code or a pointer to a function is encountered.

*Note: These warning messages may be disabled by using the **-ws** command line option.*

st20cc may mark a function as side-effect free, even though the pragma **ST_nosideeffects** has not been applied. Use of the **-fcheck-referenced** command line option causes an information message to be generated if this occurs.

st20cc only marks those functions that it detects as unambiguously side-effect free and which are not already marked. It is possible for there to be functions which are side-effect free that **st20cc** does not detect; for such functions, only the application of the pragma **ST_nosideeffects** causes these functions to be marked as side-effect free.

If a function that is side-effect free is externally visible, then it may be useful to put the pragma **ST_nosideeffects** into the header file following the prototype of the function. This enables **st20cc** to generate better code for calls to that function from other files.

12.2.7 Pragma ST_on and ST_off

ST_on enables specific compiler actions; **ST_off** disables the same actions. Both pragmas are listed in [Table 37](#), and take a comma-separated list of parameters which specify the actions to be enabled or disabled.

Syntax:

```
#pragma ST_off (params)
#pragma ST_on (params)
```

Parameter	Short form	Description
stack_checking	sc	Checks for stack overflow at the start of each function. Default is off.
warn_deprecated	wd	Warns of parameterless function declarations. Default is on.
warn_implicit	wi	Warns of undeclared functions. Default is on.

Table 37: Parameters to ST_on and ST_off

12.2.8 Pragma ST_onshutdown

ST_onshutdown specifies that a function is to be executed after **main** has terminated. The function must be void and have no arguments. If the function returns any values, they are ignored.

Syntax:

```
#pragma ST_onshutdown (function-name)
```

where **function** must have been declared before the pragma is encountered.

12.2.9 Pragma ST_onstartup

ST_onstartup specifies that a function is to be executed before **main** has started. The function must be void and have no arguments. If the function returns any values, they are ignored.

Syntax:

```
#pragma ST_onstartup (function-name)
```

where **function** must have been declared before the pragma is encountered.

12.2.10 Pragma ST_place_at_workspace_offset

This pragma directs **st20cc** to allocate a local (automatic) variable at **offset** words from the base of the current function's local workspace. **st20cc** ensures that at least **offset** words are allocated, and that no other variables in scope at the same time as **local.variable** are allocated in the same place in workspace.

Syntax:

```
#pragma ST_place_at_workspace_offset (local.variable, offset)
```

This pragma is useful when using instructions which require that values are placed at fixed workspace offsets, or which may overwrite values in workspace.

For example, the ST20-C2 **disc** instruction writes to workspace location “0”, so to ensure that no other local variables will be allocated in this location, a dummy variable can be declared and placed at this location, as in the following example:

```
{
    int dummy;
    #pragma ST_place_at_workspace_offset (dummy, 0)

    __asm {
        ... set up the alt
        ... disabling code
        disc;
        ... more disabling code
    }
}
```

The named local variable must have been defined before the pragma is encountered.

A warning is generated and the pragma is ignored if:

- the named variable has not been defined before the pragma is encountered,
- the named variable is not local (automatic),
- the workspace offset supplied is negative.

12.2.11 Pragma ST_section

Syntax:

```
#pragma ST_section(function_name, "section_name")
        | (data_name, "section_name")
```

This pragma causes a named function or data item to be placed in a named section.

For a function or data item to be placed into a section, the function or data item is first declared. The **ST_section** pragma may then be used.

For example, to place the function **fast_calc** into a section called **fast_ram**:

```
void fast_calc(void);
#pragma ST_section(fast_calc, "fast_ram")
```

User defined section names should not be prefixed with “__” or “<”, as section names of this form are system reserved.

st20cc also uses a number of default sections into which any code or data which has not been placed by use of an **ST_section** pragma is placed. At link time, **st20cc** allocates the named and default sections space in memory according to **place** commands, which are specified in a command file. **place** commands enable sections to be placed in a named memory region or at an absolute address. If **place** commands are not specified then **st20cc** uses a default order for placing the sections in memory; see [Chapter 6: Code and data placement on page 91](#) for further details.

12.2.12 Pragma ST_translate

st20cc replaces all references to **name** (for example, an external routine) by **newname**.

Syntax:

```
#pragma ST_translate (name, "newname")
```

newname is a C string which can contain alphanumeric characters, the underscore (_), percent (%), or full stop (.) characters.

12.2.13 Pragma ST_trap_handler

This pragma indicates to **st20cc** that the named function is to be compiled as a trap handler.

Syntax:

```
#pragma ST_trap_handler (function-name)
```

This pragma changes the compiler's normal calling conventions; readers are advised to read [Section 13.15.4: Using the trap, interrupt and naked pragmas on page 198](#), where some examples are provided.

The effects of the pragmas **ST_interrupt_handler** and **ST_trap_handler** are processor-dependent. On an ST20-C1 core, interrupt and trap handling has been unified and so it is strongly recommended that interrupts should be declared as trap handlers. For an ST20-C1 core, the user should define the interrupt or trap handler as:

```
#pragma ST_trap_handler(trap_hand)
void trap_hand(unsigned int Areg, /* A Register at interrupt point */
               unsigned int Breg, /* B Register at interrupt point */
               unsigned int Creg, /* C Register at interrupt point */
               unsigned int Wptr, /* Workspace pointer at interrupt point */
               unsigned int Iptr, /* Instruction pointer at interrupt point */
               unsigned int Tdesc, /* Task Descriptor at interrupt point */
               unsigned int Status, /* Status Reg at interrupt point */
               void (*handler)(void)) /* Pointer to trap handler function */
{
}
```

Note: The parameters are used to exit from the trap or interrupt handler. Altering them changes the expected behavior of the handler.

On the ST20-C2 core, **ST_trap_handler** and **ST_interrupt_handler** generate different code, so it is important to specify the correct pragma.

For ST20-C2 cores, a trap handler should take the form:

```
#pragma ST_trap_handler(trap_hand)
void trap_hand(unsigned int Areg, /* A Register at interrupt point */
               unsigned int Breg, /* B Register at interrupt point */
               unsigned int Creg, /* C Register at interrupt point */
               {
}
```

For details of how to specify and enable a trap handler; see the appropriate processor datasheet. Trap handling examples are given in [Section 13.15.4](#).

12.2.14 Pragas in C++

General pragma rules

- A pragma declaration associated with a symbol should be in the same scope as the declaration of that symbol. For example, if the symbol `an_int` is a class member, the pragma must be within the class declaration.
- For pragmas applied to functions, the parameter types may be specified as part of the pragma argument. If the parameter list is not specified, all non-template overloaded versions of the function will be candidates for the pragma. See examples in [Table 38](#).

Declaration	Pragma
<code>int func1(int a)</code>	<code>#pragma ST_foo(func1(int))</code>
<code>int func1(double a)</code>	<code>#pragma ST_foo(func1(double))</code>
<code>int func1(char a)</code>	<code>#pragma ST_foo(func1(char))</code>
<code>int func1(void)</code>	<code>#pragma ST_foo(func1(void))</code> or <code>#pragma ST_foo(func1())</code>
<code>class A { int getR() const }</code>	<code>#pragma ST_foo(getR() const)</code>
Apply to all <code>func1</code> 's	<code>#pragma ST_foo(func1)</code>

Table 38: Specifying pragma parameter types

Note: For const methods, the trailing `const` must also be put in the pragma (the same applies to volatile and const volatile methods). Warnings will be given if the parameter list is missed off and there are overloaded versions of the function. Also, where parameter types are used, they must be base types and not typedefs – the pragma cannot recognize typedefs.

Special pragma rules for pragmas applied to symbols within templates

- For class templates, if a pragma is to apply to the entity within all instantiations of the class, no qualification should be put in the pragma (including no function arguments to distinguish between overloaded functions; where overloading exists, the pragma will be applied to all of the overloaded functions without warnings).

```
template<class A> class Thing
{
    // Apply to all instantiations
    #pragma ST_device(the_thing)
public:
    A the_thing;
    int get_an_int(A not_needed)
    {
        return 5;
    }
    int get_an_int()
    {
        return 5;
    }
    // Apply to all instantiations and overloaded versions
    #pragma ST_nolink(get_an_int)
};
```

- To apply a pragma within a particular instantiation of the class, full qualification including all namespace and class names (with template arguments) should be put in the pragma.

```
namespace Contain
{
    template<class A> class Thing
    {
    public:
        A the_thing;

        A get_the_thing(void)
        {
            return the_thing;
        }
        // Apply only within int instantiation
        #pragma ST_nolink(Contain::Thing<int>::get_the_thing())
        // Apply only within char instantiation
        #pragma ST_nolink(Contain::Thing<char>::get_the_thing())
        int get_an_int(A not_needed)
        {
            return 5;
        }
        // Apply only within int instantiation
        #pragma ST_nosideeffects(Contain::Thing<int>::get_an_int(int))
    };
};
```


- For template functions, all pragmas should be put between the name and opening brace in the definition of the template function, for example:

```
template <class A> void do_nothing(A arg)
// Apply to all
#pragma ST_nolink(do_nothing)
// Apply to int version
#pragma ST_nosideeffects(void do_nothing<int>(int))
{
    return;
}
```

- Pragmas on locals within template functions, or template class member functions may only be applied to all instantiations. Specific instantiations cannot be specified.

```
template<class A> A max(A obj1, A obj2)
{
    // Applies to thing in every instantiation of max
    #pragma ST_device(thing)
    // thing is a local within this template function
    A thing;
    thing = obj1;
    if (thing > obj2)
        return obj1;
    return obj2;
};
```

12.3 Macros

12.3.1 Defining a macro on the command line

The `-D` option enables a macro to be defined on the **st20cc** command line, which is passed through to the compiler:

```
-D name[=value]
```

If **value** is specified then **name** is defined and assigned that value.

When **value** is a string, the host operating system removes quotes from the command line. In order for the string to be passed correctly through to the source code, quotation marks must be preceded by a backslash (`\`) character. `"foo"` would therefore be expressed as `\ "foo\"`.

The `-U` option disables a name definition and the `-EDU` option forces processing of any `-D` defines prior to `-U` undefines. By default interpretation of the `-D` and `-U` options is performed in command line order.

Example

In order for **st20cc** to pass the string `"FRED BILL"` to the source code `myprog.c` then the `-D` option could be expressed as follows:

```
st20cc myprog.c -DSTRING="\ "FRED BILL\" -c1 -T myprog.cfg
```

12.3.2 Predefinitions

Certain macros which identify global information, and some function names, are automatically recognized by **st20cc**. Generally, these items can be referenced directly in C programs and do not need to be declared.

Macro names

All predefined macro names defined by the ISO (ANSI) standard are present; they are:

Macro name	Description
<code>__DATE__</code>	The date of compilation of the source file
<code>__FILE__</code>	The name of the current source file
<code>__LINE__</code>	The line number of the current line of source
<code>__STDC__</code>	The decimal constant "1" showing the implementation conforms to ISO (ANSI) C
<code>__TIME__</code>	The time of compilation of the source file

Table 39: ISO (ANSI) predefined macros

The following STMicroelectronics macro names are also defined:

Macro constant	Description
<code>__ICC</code>	STMicroelectronics ISO (ANSI) C compiler. Set to decimal constant 1.
<code>__CC_NORCROFT</code>	Derived from the Norcroft C compiler. Set to decimal constant 1.
<code>__CORE__</code>	Indicates the target processor. Takes the following values: -1 No meaning, dummy value 1 ST20-C1 processor core 4 ST20-C2 processor core
<code>__FULL_DEBUG__</code>	A decimal constant indicating full debugging data, defined if the <code>-g</code> command line option is used. When defined it takes the value 1.
<code>__ICC_VERSION_NUMBER__</code>	A decimal constant representing the compiler version number. The version number is calculated as follows: $10000.v+100.r+b$ Where v is the release number, r is the revision number and b is the build number.
<code>__ICC_VERSION_STRING__</code>	A string literal indicating the compiler version string. This is the same string which is displayed when the <code>-help</code> or the <code>-v</code> command line options are used.
<code>__SIGNED_CHAR__</code>	A decimal constant indicating the signedness of the plain <code>char</code> type. It is only defined if the command line option <code>-fsigned-char</code> is used. When defined it takes the value 1.

Table 40: st20cc STMicroelectronics predefined macros

C++ predefined macros

The C++ EDG front end defines a number of preprocessing macros. Many of them are only defined under certain circumstances. The table below defines the macros that are provided and the circumstances under which they are defined.

Macro name	Definition
<code>__STDC__</code>	Defined in ANSI C mode and in C++ mode. In C++ mode the value may be redefined.
<code>__cplusplus</code>	Defined in C++ mode.
<code>cplusplus</code>	Defined in default C++ mode, but not in strict mode.
<code>__SIGNED_CHARS__</code> and <code>__SIGNED_CHAR__</code>	Defined when plain char is signed.
<code>_WCHAR_T</code>	Defined in C++ mode when <code>wchar_t</code> is a keyword.
<code>_BOOL</code>	Defined in C++ mode when <code>bool</code> is a keyword.
<code>__ARRAY_OPERATORS</code>	Defined in C++ mode when array new and delete are enabled.
<code>__EXCEPTIONS</code>	Defined in C++ mode when exception handling is enabled.
<code>__RTTI</code>	Defined in C++ mode when run-time type information is enabled.
<code>__PLACEMENT_DELETE</code>	Defined in C++ mode when placement delete is enabled.
<code>__EDG__</code>	Always defined.
<code>__EDG_VERSION__</code>	Defined to an integral value that represents the version number of the front end. For example, version 2.30 is represented as 230.
<code>_ICC</code>	As for C code; see Table 40 .
<code>__CORE__</code>	As for C code; see Table 40 .
<code>__FULL__DEBUG__</code>	As for C code; see Table 40 .

Table 41: st20cc EDG predefined macros



Implementation details

13

This chapter describes the implementation of the language in areas where the ANSI standard is flexible or allows alternative solutions.

13.1 Data type representation

All variables, whether of scalar, array, structure or union type, are allocated on a word boundary. Array elements and structure fields may be allocated at a less strict alignment; see [Section 13.1.2](#) and [Section 13.1.3](#) below.

*Note: When the toolset compiler (invoked by **st20cc**) is handling pointers to variables, it assumes that the variable pointed to is correctly aligned. Thus it may not be valid to cast a pointer to a less strictly aligned type to be a pointer to a more strictly aligned type. For example:*

```
char *pc;  
int *pi;  
pi = (int *)pc;
```

*is not valid unless the **char** pointed to by **pc** has been allocated at a word boundary.*

13.1.1 Scalar types

C scalar type representations are described in the following table.

C Type	Description
unsigned char	Represented in a word in which the lower eight bits are significant, the upper bits are zero.
signed char	Represented in a word in which the lower eight bits are significant, bit 7 is the sign-bit, the upper bits are zero.
char	The representation of char differs depending on whether the st20cc option -fsigned-char is used to make plain chars signed. When -fsigned-char is used, char has the same representation as signed char ; without -fsigned-char the representation is the same as unsigned char .
unsigned short	Represented in a word in which the lower 16 bits are significant, the upper bits are zero.
signed short	Represented in a word in which the lower 16 bits are significant, bit 15 is the sign bit, the upper bits are zero.
unsigned int	Represented in a word in which all 32 bits are significant.
signed int	Represented in a word in which all 32 bits are significant, bit 31 is the sign bit.
unsigned long	Represented in a word in which all 32 bits are significant.
signed long	Represented in a word in which all 32 bits are significant, bit 31 is the sign bit.
float	Represented in a word, in IEEE single-precision format.
double	Represented in two words, in IEEE double-precision format.
enumeration	Represented in a word in which all 32 bits are significant.

Table 42: Scalar types

All signed integer types are represented in two's-complement form and all unsigned integer types in binary form.

*Note: The **st20cc** command line option **-fsigned-char** makes plain chars and plain bit-fields signed.*

All floating point types are represented in a form defined by the ANSI/IEEE standard 754-1985 and conform to the operation of that standard with the following exceptions.

- The treatment of non numbers (NaNs and Infinities) in comparisons and arithmetic can be non-standard. For example, `NaN==NaN` should always return false, but depending on the error which generated the NaN, may not.
- Only “round to nearest” arithmetic is provided whereas IEEE also requires “round to zero”, “round to -infinity” and “round to infinity”.
- There are no error flag registers for handling exceptions nor routines to manipulate or trap the errors.

13.1.2 Arrays

Each element of an array of **char** occupies 8 bits and each element of an array of **short** occupies 16 bits. This means that an element of an array of **char** may be aligned on any boundary and an element of an array of **short** is aligned on an even boundary.

Elements of arrays of any other type are represented as the element would be represented if it was not in an array. An array is padded at the high-end address to the next word boundary: the padding has no defined value.

13.1.3 Structures and unions

Layout of structures is very much dependent upon the particular compiler and the architecture of the target processor. The rules described here are very specific to the STMicroelectronics ISO (ANSI) C compiler, used by **st20cc** and the ST20 family of processors; they should not be treated as general rules for laying out structures.

Each field of a union is represented as it would be if it was not in a union.

The compiler uses the following rules when laying out the fields within a structure.

- C requires that structure fields are laid out in memory in the same order that they are in the source code.
- chars (represented in one byte) may have any alignment.
- shorts (represented in two bytes) are aligned on an even boundary.
- Adjacent bit-fields are packed into the same word if possible: the first bit-field is placed in the least significant bits of the word. If there is not enough room left after a previous bit-field, a bit-field is placed in the least significant bits of the next word.
- Word-sized or larger objects are aligned on a word boundary.
- Structures and unions are aligned on a word boundary.
- Arrays are aligned in accordance with the alignment rules of the elements of the array.

char and **short** fields are packed into the same word, where possible without breaking any of the above rules.

Example 1 (default layout)

struct d {	
char hid[8];	The first byte of hid is on a word boundary (as the first byte of structure is on a word boundary); it occupies 8 bytes (2 whole words).
unsigned short inuse;	This occupies the lower two bytes of the following word.
char flags1;	This is packed into byte 2 of the same word as inuse .
char flags2;	This is packed into the upper byte of the same word as inuse and flags1 .
unsigned long tkey;	This occupies the following word.
unsigned short tfil;	This occupies the lower two bytes of the following word.
long npos;	This has to be allocated on the next word boundary, so two bytes are left unused.
unsigned short kmod;	This occupies the lower two bytes of the following word.
unsigned short kbhz;	This is packed into the upper two bytes of the same word as kmod ; 16-bit objects are placed at even addresses (rule 2), not word-addresses.
unsigned short rmod;	This occupies the lower two bytes of the following word.
} structure;	Two bytes are left unused.

This can be represented diagrammatically as follows:

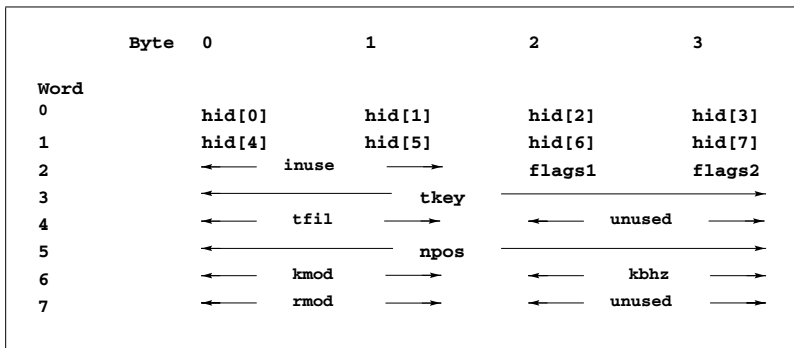


Figure 28: Scalar types structures and unions - example 1

Example 2 (default layout)

If the structure fields are reordered, by moving `tfil` so that it is no longer word aligned, then a more efficient packing could be obtained:

```
struct d {
    char hid[8];
    unsigned short inuse;
    char flags1;
    char flags2;
    unsigned long tkey;
    long npos;
    unsigned short kmod;
    unsigned short kbh;
    unsigned short rmod;
    unsigned short tfil;
} structure;
```

This would give the following:

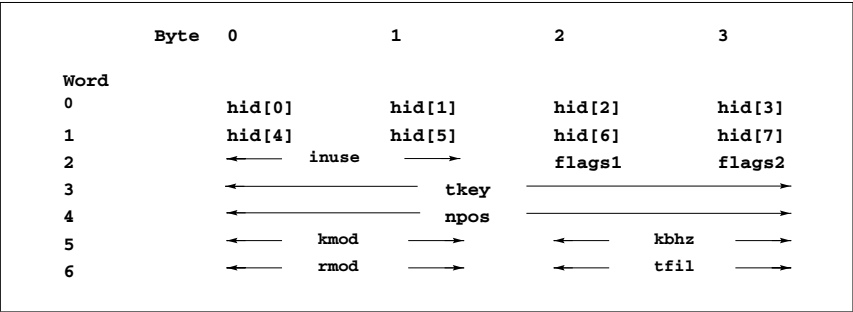


Figure 29: Scalar types structures and unions - example 2

*Note: The compiler generates more efficient code to load a **short** if it is word-aligned, so this new packing means that more code is needed to access `tfil`, as it is no longer word-aligned (again, this is very dependent upon the way the toolset compiler handles structures).*

A general rule for obtaining the smallest structure size possible is to order the fields in increasing order of size.

Changing the alignment of structures and unions

By default, structures and unions are aligned so that they begin on word boundaries. The `st20cc` option `-falignn` enables the user to specify that nested structures and unions are aligned to an address which is a multiple of `n`, provided that the alignment requirements of all the fields of the structure and union permit this. The `-falign` option operates on structures and unions and not on their members.

`-falignn` is applicable to types `char` and `short` on 32-bit machines. `-falign1` aligns to a one-byte boundary, `-falign2` to a two-byte boundary and `-falign4` to a four-byte boundary.



Example

```

struct strc {
    char cfield;
}

struct strA {
    struct strc a;
    struct strc b;
    struct strc c;
};

```

In this example, the alignment of field **a** is always word aligned because it is the first item in the non-nested structure **strA**. By default, fields **b** and **c** are word aligned. If the **st20cc** option **-falign1** is used, **b** and **c** are byte aligned.

*Note: Code compiled with the **-falign** option (whose interface contains structures whose alignment is changed by the **-falign** option) should not be mixed with code compiled without it, or with a different value for the **-falign** option.*

13.1.4 Tentative variables

Where variables are defined tentatively in file scope the linker “commons” them up, that is treats them as a single variable. For example, the linker treats variables in the following manner:

File 1	File 2	Linker action
<code>int i;</code>	<code>int i;</code>	Treats as the same variable
<code>int i;</code>	<code>char i;</code>	
<code>double i;</code>	<code>int i;</code>	
<code>int i;</code>	<code>int i=1;</code>	
<code>int i=1;</code>	<code>int i=1;</code>	Gives an error - multiply defined

Table 43: Linker variables treatment

The linker uses the size of the first variable encountered and so the user should make sure such variables are the same size.

Where a variable has been defined in two places to have a value, even if this is the same value, the linker generates a **“multiply defined”** error.

13.2 Type conversions

13.2.1 Integers

The result of converting an unsigned integer, u , to a signed integer, s , of equal length, if the value cannot be represented, is calculated as follows:

If max.s is the largest number that can be represented in the signed type then:

$$\text{result} = u - 2(\text{max.s} + 1)$$

This is equivalent to doing an bitwise copy from u to s .

Converting an integer i_1 , to a shorter integer i_2 is equivalent to copying the least significant n bits of i_1 , to i_2 , where n is the number of bits in i_2 .

An integer is converted to a shorter signed integer, by first converting it to an unsigned integer of the same length as the shorter signed integer (by taking the non-negative remainder on division by the number one greater than the largest unsigned number that can be represented in the type with smaller size) and then converting to the corresponding signed integer, as described above.

13.2.2 Floating point

When converting an integral number to a floating point number that cannot exactly represent the original value, the IEEE 754 “Round to Nearest” rounding mode is used.

When converting a floating point number to a narrower floating-point number, the IEEE 754 “Round to Nearest” rounding mode is used.

When converting a floating point number to an integral number, the IEEE 754 “Round to Zero” rounding mode is used; this is mandated by the ANSI standard.

13.3 Environmental arguments to main

The interface to main is as follows:

```
int main( int argc,
          char *argv[] )
```

For ROM booted programs

int argc the number of arguments passed to the program is set to one.

char *argv[] is an array of pointers to the passed arguments. This points to an array of two elements: **argv[0]** is a pointer to the null string (""); and **argv[1]** is **NULL**.

For a linked unit loaded by st20run

int argc the value corresponds to the number of arguments that have been passed via the load command plus two (one for the linked unit and one for **NULL**).

char *argv[] is an array of pointers to the passed arguments. This points to an array containing the arguments that have been passed to the load command with an additional **NULL** argument at the end. **argv[0]** contains the linked unit name.

13.4 Identifiers

The first 256 characters in an identifier are significant.

13.5 Source and execution character sets

The source character set comprises those characters explicitly specified in the Standard, together with all other printable ASCII characters and those characters with values 128 to 254 inclusive. The execution character set comprises all 255 values 0 to 254. Values 0 to 127 represent the ASCII character set. The ASCII value 255 is used as the end-of-file indicator; if this value is required, use the string “\xff” instead.

Note: When the **st20cc** command line option **-fsigned-char** is used, the execution character set comprises 128 values in the range 0 to 127. (See the ST20 Embedded Toolset User Manual, chapter *st20cc compile/link tool*).

Characters of the execution character set use 8 bits.

Each member of the source character set is a member of the ASCII character set and maps to the same member of the ASCII character set in the execution character set.

All characters and wide characters are represented in the basic execution characters set. The escape sequences not represented in the basic execution character set are the octal integer and hexadecimal integer escape sequences, whose values are defined by the Standard.

13.5.1 Shift states for encoding multibyte characters

There is only one shift state, which is the initial shift state as specified in the Standard. Multibyte characters do not alter the shift state.

13.5.2 Integer character constants

The value of an integer character constant that contains more than one character is given by:

$$\sum_{i=0}^n (\text{value of } i^{\text{th}} \text{ character} \ll (8 \times i))$$

where n is the number of characters in the integer character constant.

The value of a wide character constant which contains more than one multibyte character is the value of the first multibyte character. Any multibyte characters after the first are ignored in the value calculation.

13.5.3 Locale used to convert multibyte characters

The only locale supported to convert multibyte characters into corresponding wide characters (codes) for a wide character constant is the “C” locale.

13.5.4 Plain chars

By default a “plain” char has the same range of values as unsigned char. However, if the **st20cc** command line option **-fsigned-char** is used, a “plain” char has the same range of values as a signed char. (See the *ST20 Embedded Toolset User Manual*, chapter *st20cc compile/link tool*).

13.6 Integer operations

13.6.1 Bitwise operations on signed integers

Signed integers are represented in two's complement form. The bitwise operations operate on this two's complement representation.

13.6.2 Sign of the remainder on integer division

The remainder on integer division takes the same sign as the dividend.

13.6.3 Right shifts on negative-valued signed integral types

Signed integers are represented in two's complement form. The **default** behavior of the compiler is as follows:

The right-shift operates on the two's complement form; zero bits are shifted in at the left-hand side; thus a negative-valued signed integer, if right-shifted more than zero places, becomes positive.

It is possible, using the **st20cc** command line option **-fshift**, to **change** this behavior to the following:

The right-shift operates on the two's complement form; the sign-bit is duplicated at the left-hand side; thus a negative-valued signed integer, if right-shifted more than zero places, remains negative.

13.7 Registers

st20cc attempts to place register variables as close to the workspace pointer as possible.

13.8 Enumeration types

The values of enumeration types are represented as **ints**.

Bit fields

By default, a “plain” bit-field is treated as an **unsigned int** bit-field. However, if the **st20cc** command line option **-fsigned-char** is used, a plain bit-field is treated as a **signed int** bit-field.

Bit-fields are allocated low-order to high-order within an integer (that is, the first field textually is placed in lower bits in the integer).

A bit-field cannot straddle a word boundary.

13.9 volatile qualifier

An access to an object that has volatile-qualified type is a “read” from the memory location containing the object (if the object’s value is required) or a “write” to the memory location containing the object (if the object is assigned to).

If the volatile object is an array, then the access is only to the appropriate element of the array.

If the volatile object is a structure and only a field of the structure is required, then the access is only to the appropriate field.

If the access is an indirect access through a pointer, then the access is equivalent to an access of an array element of that type.

If the object is not an array element or structure field and the access is not indirect, then the object occupies a whole number of words, and all the words are accessed. Otherwise, if the array element or structure field is shorter than a word, then only the appropriate bytes are accessed.

If the object is a bit-field, then in the case of read access, the entire word containing the bit-field is read; and in the case of write access, the entire word containing the bit-field is first read, and then written.

When the volatile qualifier is used, the compiler reduces the scope of its optimization. A typical use of volatile is where a variable is accessed from two separate threads of control, and compiler optimizations may otherwise prevent the correct functioning of the code.

The following example shows the use of an interrupt handler which updates a shared synchronization variable, and the main program busy waits for the variable to be updated. Without the use of **volatile** in this example, the compiler would be free to optimize away reading the **sync** variable, because it knows that nothing in the **wait_for_isr()** function can write to it:

```
void isr(unsigned int Areg, unsigned int Breg, unsigned int Creg);
#pragma ST_interrupt_handler(isr)

volatile int sync=0;

void isr(unsigned int Areg, unsigned int Breg, unsigned int Creg)
{
    sync = 1;
}

void wait_for_isr(void)
{
    while (sync == 0)
        ;
}
```

Note: 1 For ST20-C2 targets, if the object is an array element or structure field or if the object is larger than two words, then the block move instruction is used for the access. If a block move instruction is interrupted, when it resumes it may reread the same word of memory which was read immediately before the interrupt. This may cause problems with some peripheral devices.

*2 On ST20-C2 targets, if an object is declared as volatile and the **-fdisable-device** option is used, the compiler uses device instructions to access it. This option is intended for backward compatibility; all new development should use the **ST_device** pragma for device access, see [Section 13.10](#) below.*

13.10 Device memory access

In the ST20-C2 instruction set there are two classes of memory access instructions, normal accesses and “device” accesses. Device memory accesses behave slightly differently to normal accesses, and it is important to use the right one, especially when accessing peripherals or using a cache.

Device accesses are performed using a number of special instructions, collectively referred to as device instructions:

- **dev1b** and **devsb** - read and write a byte of memory,
- **dev1s** and **devss** - read and write a 16-bit quantity of memory,
- **dev1w** and **devsw** - read and write a 32-bit word of memory,
- **devmove** - move a block of memory (both source and destination addresses use device accesses).

All other instructions which read or write to memory generate “normal” memory accesses.

A device access has a number of special attributes.

- When accessing the internal peripheral space, device instructions use the peripheral bus rather than the memory bus. All on-chip peripherals within the internal peripheral space are connected to the peripheral bus, so device instructions must be used to access them.
- When accessing cacheable memory, device instructions always bypass the cache, and access memory directly. The data accessed is not loaded into the cache. Even if the memory location being accessed has been cached, a device access to that location accesses memory directly, and the data in the cache is not updated.
- Device instructions prevent the CPU performing any optimizations which may be unsafe when applied to peripheral registers. In particular, any device writes must complete before the next instruction is executed, while a normal memory write may be buffered.

In general, there are two cases when a device instruction must be used:

- when accessing on-chip peripherals in the internal peripheral space,
The address of the internal peripheral space varies between devices, but is usually between **0x00000000** and **0x3fffffff** or **0x20000000** to **0x3fffffff**.
- when accessing cacheable memory which should not be loaded into the cache.

The most common case where this is useful is when memory is written to by a peripheral using DMA. DMA memory accesses are always direct to memory, ignoring the cache. So when the CPU accesses the memory location, it is important it does not read a value from the cache rather than memory. This is because the cache would not be updated when the DMA access was performed and so could contain “stale” or old data.

A beneficial side-effect of not caching data in this way is that frequently data written into memory using DMA is read once, immediately written to another location (possibly an output peripheral) and never read again. Thus loading the data into cache would be of no benefit, and would prevent useful data being cached.

When using memory in this way it is necessary to be aware of the cache alignment of the data. When a word is read from cacheable memory using a normal memory access instruction, not only the word accessed is brought into the cache, but also the surrounding words, to fill the whole cache line. Later when the cache line is discarded, if any word within the cache line has been written to, the whole cache line is written out into memory. If one of the adjacent words had been written to using a device instruction (or by a peripheral performing DMA) the cache write back would overwrite this word with old data.

The easiest way to avoid this problem is to ensure that all buffers accessed using device instructions are aligned to the size of a cache line, and are extended so their size is a multiple of the cache line size. The alignment can be achieved by placing the buffer in a named section, and then use the **-alignment** option to the **place** command.

(An alternative to using device instructions to bypass the cache is to use the “cacheability control” registers of the cache to reset the device hardware. See the appropriate device datasheet for details. This allows entire blocks of memory to be marked as “not cacheable” and access is then performed using normal memory access).

A memory location should always be accessed using only device or non-device instructions, never a mixture of the two. Failure to follow this rule can lead to some very hard-to-diagnose problems.

13.10.1 Generating device instructions from the compiler

The easiest way to generate device instructions is to use the C compiler. A pointer or an integer variable can be flagged using the **#pragma ST_device** (see [Section 12.2: The #pragma preprocessor directive on page 167](#)) so that when the pointer is de-referenced or the variable is accessed, a device instruction is generated rather than a normal memory access instruction. The variable may be a **char**, **int**, **short** or **long**; it may also be a type definition, structure or union. For example:

```
int main( void )
{
    int normal;
    #pragma ST_device( device_val )
    int device_val;
    normal = 1;
    device_val = 2;
}
```

generates:

```
ldc 1
stl 0 -- normal
--
ldc 2
ldlp 1 -- device_val
devsw
```

It is also possible to apply the **ST_device** pragma to a type:

```
#pragma ST_device(device_ptr)
typedef int* device_ptr;
device_ptr device;
```

This is convenient when placing device pointers in structures or passing them as parameters.

The C compiler treats **ST_device** as another qualifier, like **const**, and so checks the types of function arguments and assignments:

```
#pragma ST_device(device_param)
void setup(int** device_param);

void test(void)
{
    #pragma ST_device(device)
    int* device;
    int* normal;

    setup(&device);    // OK
    setup(&normal);    // Error
}
```

When the **ST_device** pragma is used, it is normal to also mark the variable as volatile. A peripheral register, or memory which is being written to by a DMA engine (when **ST_device** would normally be required) are classic examples of when the volatile qualifier is needed. Device access may also be optimized away, this is prevented by marking the variable as volatile.

Although it is not mandatory to mark a device access as volatile, there are very few cases where marking the variable as volatile is not required. An on-chip peripheral register may be marked as device rather than volatile and optimized as normal only if all the following conditions are met:

- it can only be modified by the CPU,
- accessing it has no side-effects,
- it is not accessed by any other threads or interrupt handlers.

However, when normal data (not peripheral register data) is shared between tasks, or tasks and interrupt handlers, it should be marked as volatile, but not device. Normal memory accesses can be used, because all accesses are from the CPU, and are cached or not in exactly the same way.

Erroneously marking data shared in this way as “device” can cause problems. Data of this type is unlikely to be cache line aligned and may share a cache line with data not accessed using device instructions. This is exactly the scenario required to exhibit the cache line write back problem described in [Section 13.10](#).

Another way to generate device instructions is the **st20cc** command line option **-fdisable-device**. This can be used to automatically generate device accesses for all variables which are marked as volatile. This option is provided for backward compatibility with old compilers only. As mentioned above, there are problems when using device instructions with volatile, therefore this option should not be used for new projects.

13.11 Declarators

There is no restriction upon the number of declarators that may modify an arithmetic, structure, or union type.

13.12 Switch statement

There is no restriction upon the number of case values in a switch statement.



13.13 Preprocessing directives

Preprocessing directives are described in [Chapter 12: Preprocessing on page 167](#).

Constants controlling conditional inclusion

A character constant within a preprocessing directive has the same numeric value as it has within any other expression. Character constants in this context may have negative values only if a “plain” char type is a signed char, see [Section 13.5: Source and execution character sets on page 188](#).

Date and time defaults

When date of translation is not available, `__DATE__` expands to “**Jan 1 1900**”.

When time of translation is not available, `__TIME__` expands to “**00:00:00**”.

13.14 Static data layout

By default, static data that has not been explicitly placed using a `place` command is placed in one of three sections:

- `<Filename__const>` - contains all `const` data items,
- `<Filename__data>` - contains all initialized non-`const` data items,
- `<Filename__bss>` - contains all uninitialized non-`const` data items.

Placement within each of these sections is arbitrary. The three sections from all input object files are coalesced by the linker into default const, data and BSS sections. The order of placement of these default sections in the memory map is arbitrary and determined by a combination of default behavior and user placement commands. The base address of the default section positioned at the lowest address in memory is called the static link. To access static data `st20cc` adds an offset to the static link, this gives a pointer to the static area.

The map file generated by the `st20cc` option `-M` records the static data layout. It is described in [Chapter 5: Stack depth and memory maps on page 81](#).

13.15 Calling conventions

13.15.1 Parameter passing

There are two methods of parameter passing, depending upon whether or not the function involved has a type which includes a prototype.

For a function call, if the function has a type that includes a prototype, then each actual parameter is converted to the type of the corresponding formal parameter, otherwise the default argument promotions are performed on each actual parameter.

Default promotions

- An argument of type `char`, `short int`, `int` bit-field, or enumeration type is converted to type `int` (signed `int` if this correctly represents the argument, unsigned `int` otherwise).
- An argument of type `float` is converted to type `double`.
- Arguments of all other types are unmodified.

For a function definition, if the function type does not include a prototype, then callee narrowing is performed upon each formal parameter: this converts it from its promoted type (as obtained by the default argument promotions) to its declared type. If the function type does include a prototype, then no type conversion is performed.

The default argument promotions are performed upon arguments forming part of a variable parameter list.

13.15.2 Calling sequence

A pointer to the static area is normally passed as an extra parameter to every function. This parameter is called the static link, and contains the address of the static area for the program.

The pragma `ST_nolink(f)` directs `st20cc` to compile the function `f` without a static link parameter. Any direct calls to `f` within the scope of this pragma do not have a static link included in the argument list. If the function is defined within the scope of the pragma, then it is compiled without a static link formal parameter (the compiler flags a serious error if the function definition requires a static link, for example, if it accesses static data).

A function's declared parameters are found in order immediately after the static link. The type of each parameter is determined using the rules described in [Section 13.15.1](#) above. Each parameter occupies an integral number of words. Parameters are represented in memory exactly the same as if they had been declared as automatic.

On ST20-C2 cores, the first three words of parameters are loaded onto the integer register stack (AREG contains the static link) and are written into memory by the **call** instruction. See [Figure 30](#) below.

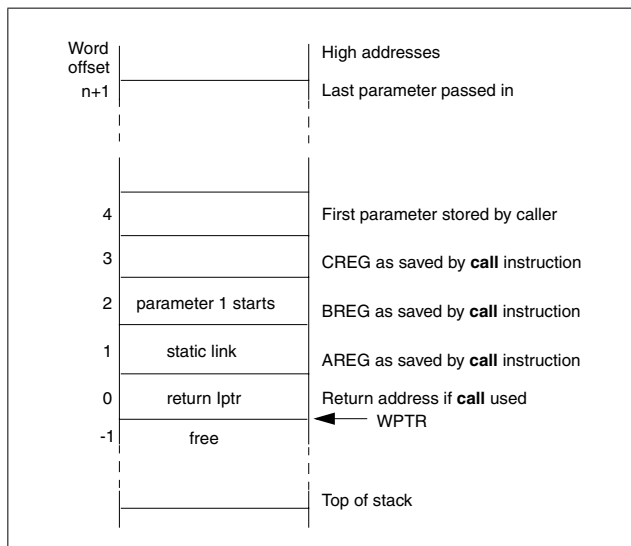


Figure 30: Calling sequence on ST20-C2 cores

On ST20-C1 cores, the first two words of parameters are passed in the AREG and BREG and are loaded into workspace by the **calllee** function. The caller must reserve enough space to allow for this. See [Figure 31](#).

Parameters may be modified by the called routine. Thus after the call, they cannot be guaranteed to contain the same value as was passed in.

On entry to a function the contents of the CPU evaluation stack is undefined and the workspace pointer addresses the workspace containing the return address and parameters.

The return value from a scalar-valued function is returned in the AREG provided it fits. If the result occupies less than a word, the value returned in AREG is the value of the scalar sign-extended or zero-extended (depending upon whether the value is signed or unsigned) to the number of bits per word.

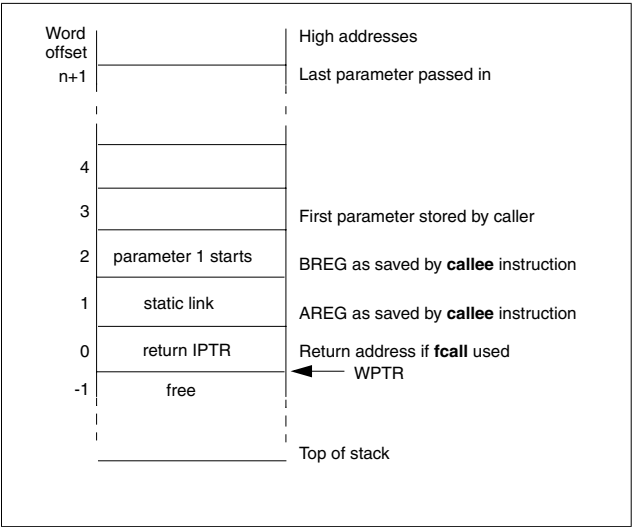


Figure 31: Calling sequence on ST20-C1 cores

Structures and unions are returned in the AREG providing they satisfy the following conditions:

- The structure/union occupies at most one word.
- Each field of the structure is one of the following:
 - a word-sized integer,
 - a pointer,
 - an enumeration,
 - a bitfield,
 - a structure/union which recursively satisfies the conditions for being returned in AREG.

If the return value is not returned in AREG, the caller supplies another parameter as the second actual parameter (when the user's parameters begin in position three). This is a pointer to an area large enough to receive the return value.

13.15.3 Rules for aliasing between formal parameters

The following rules cover assumptions made by the STMicroelectronics C compiler with regard to aliasing between function parameters.

- The compiler may not assume that there are no aliases between formal parameters.
- Where a function result is returned by assignment through a result pointer in the function body, the compiler may not assume that there are no aliases of the object referred to by the result pointer parameter.

Hence the compiler must ensure that all accesses to variables which could potentially be aliased by the result pointer have already occurred before the assignment through the result pointer.

13.15.4 Using the trap, interrupt and naked pragmas

Three pragmas provide a way to alter the normal C language calling conventions:

- **ST_trap_handler**,
- **ST_interrupt_handler**,
- **ST_naked_function**.

Two of these are special purpose, and allow trap handlers and interrupt handlers to be written in C. The third removes all code at the start and end of a function, allowing the user to handle function entry and exit. Their syntax and use is described in [Section 12.2: The #pragma preprocessor directive on page 167](#).

The effect of these pragmas is to change:

- at which workspace offset the generated code expects to find the static link,
- at which workspace offset parameters start to be stored,
- how the function returns,
- how local variables are allocated.

Changing the offsets at which the static link and parameters are stored is required because the ST20-C1 and ST20-C2 cores use workspace to store the CPU registers immediately prior to taking the interrupt or trap. This either allows the handler access to the previous state in the form of parameters, or additional information can be passed into the handler by the user explicitly placing this into the workspace. For example, by using a pointer to the workspace as shown in [ST_trap_handler pragma ST20-C1 example](#) and [ST_trap_handler pragma ST20-C2 example](#) later in this section.

[Table 44](#) shows the effect of the pragmas for ST20-C2 and ST20-C1 devices:

Pragma	Static link position	1st Parameter position	Return Instruction	Comment
C1 devices				
Default state	1	2	ldl 0, jab	
ST_trap_handler	8	0	eret	See note 1
ST_interrupt_handler	9	10	eret	See note 1
ST_naked_function	1	2	none	See note 1
C2 devices				
Default state	1	2	ret	
ST_trap_handler	1	2	tret	See note 2
ST_interrupt_handler	9	10	iret	
ST_naked_function	1	2	none	

Table 44: Effect of trap, interrupt and naked pragmas on ST20 -C1 and -C2 devices

Note: 1 The normal calling conventions on the ST20-C1 specify that the static link is passed in the AREG and the first parameter in the BREG. The caller has reserved workspace offsets 1 and 2 for the called function to store AREG and BREG respectively if it needs to. However, all three pragmas suppress the generation of code to store these registers, and assume that they will be explicitly set by the user.

2 **ST_trap_handler** causes three instructions to be added at the start the function:

```
stl 2
stl 3
stl 4
```

These instructions are executed even before the **ajw** to reserve workspace for local variables and function calls. The user is responsible for restoring them before the function returns.

Another effect of the **ST_trap_handler** and **ST_interrupt_handler** pragmas is to suppress the allocation of local variables to unused call slots.

By default the compiler allocates local variables to unused call slots. This optimization allows the compiler to allocate local variables in the workspace offsets used for parameters and the static link, when they are no longer required. This is extended on the ST20-C2 by using the knowledge that functions are normally executed using the call instruction, and so at least three workspace slots are available, even if there are not enough parameters to use them all.

ST_trap_handler pragma ST20-C1 example

This example sets trap level 0 and is invoked by a call to exception level 0.

```
#include <stdio.h>

extern volatile const void *_params;
#define el_breakpoint_trap -1
#define el_illegal_instr_trap -2
#define el_idle_trap -3
#define el_schedule_exception_trap -4
#define el_run_trap -5
#define el_stop_trap -6
#define el_timeslice_trap -7

#define TRAPHANDLER(foo) void foo(unsigned int Areg,\
                                   unsigned int Breg,\
                                   unsigned int Creg,\
                                   unsigned int Wptr,\
                                   unsigned int Iptr,\
                                   unsigned int Tdesc,\
                                   unsigned int Status,\
                                   void (*handler)(void))

typedef TRAPHANDLER(traphandler);

TRAPHANDLER(th);
#pragma ST_trap_handler(th)
extern void *malloc(int n);
static int xx = 42;

typedef struct
{
    unsigned int stack[1];
    unsigned int InterptdAreg;
    unsigned int InterptdBreg;
    unsigned int InterptdCreg;
    unsigned int InterptdWptr;
    unsigned int InterptdIptr;
    unsigned int InterptdTdesc;
    unsigned int InterptdStatus;
```

```

    traphandler *HandlerIpPtr;
    void *StaticLink;
    void *start;
} exception_control_block;

typedef struct paramblock {
    void *return_address;
    void *static_link;
    int regparam1, regparam2;
} paramblock;

TRAPHANDLER(th)
{
    xx++;
}

void set_handler(int level, traphandler *handler, int stacksize)
{
    paramblock* pp = (paramblock *)_params;
    exception_control_block *ecb=
        (exception_control_block *)malloc(sizeof(exception_control_block)+
        (stacksize-1)*sizeof(unsigned int));

    ecb=(exception_control_block *)&ecb->stack[stacksize-1];
    ecb->start=(void *)&ecb->stack[1-stacksize]; /* So we can return it */
    ecb->HandlerIpPtr=handler;
    ecb->StaticLink=pp->static_link;
    ((exception_control_block **)0x80000040)[level]=
        (exception_control_block *)(((int)&ecb->InterptdAreg)|1);
}

int main(void)
{
    set_handler(0,(traphandler *)th,1024);

    printf("Before ecall, xx=%d\n", xx);
    __asm
    {
        ldc 0;
        ecall; /* call exception level 0 */
    }
    printf("After ecall, xx=%d\n", xx);
}

```

ST_trap_handler pragma ST20-C2 example

This example sets a trap for an arithmetic error and is invoked by a divide by 0 operation.

```

#include <stdlib.h>
#include <stdio.h>
#include <errno.h>

typedef struct paramblock {
    void *return_address;
    void *static_link;
    int regparam1, regparam2;
} paramblock;

```



```

typedef unsigned int      enables_word_t;
typedef unsigned int      status_register_t;
typedef int*              workspace_t;

typedef enum trap_group_e
{
    trap_group_breakpoint,
    trap_group_error,
    trap_group_system,
    trap_group_scheduler
} trap_group_t;

typedef struct trap_handler_s
{
    enables_word_t      trap_handler_enables_word;
    status_register_t    trap_handler_status_reg;
    workspace_t          trap_handler_wptr;
    void                (*trap_handler_handler)(int Areg, int Breg, int Creg);
} trap_handler_t;

extern volatile const void *_params;

#pragma ST_off(stack_checking)

void mytrap_install(trap_group_t Group,
                   void (*Handler)(int Areg, int Breg, int Creg),
                   void* Stack,
                   size_t StackSize,
                   enables_word_t EnablesWord,
                   status_register_t StatusReg,
                   int priority,
                   enables_word_t EnableTrapGroup)
{
    trap_handler_t TrapHandler;
    paramblock* pp = (paramblock *)_params;

    TrapHandler.trap_handler_enables_word = EnablesWord;
    TrapHandler.trap_handler_status_reg = StatusReg;
    TrapHandler.trap_handler_wptr = (workspace_t)((int*)Stack +
    (StackSize - 5));
    TrapHandler.trap_handler_handler = Handler;
    TrapHandler.trap_handler_wptr[1] = ((int *)_params)[1];
    __asm
    {
        ldabc    Group, &TrapHandler, priority;
        ldtraph;
        ldab     EnableTrapGroup, priority;
        trapenb;
    }
}

static int xx = 42;

void th(int Areg, int Breg, int Creg);
#pragma ST_trap_handler(th)
#define STACK 1024

```

```

void th(int Areg, int Breg, int Creg)
{
    xx++;
    __asm
    {
        ldabc    Areg, Breg, Creg;
    }
}

int main(void)
{
    int y = 1;
    void *stack=calloc(STACK*sizeof(int),1);

    mytrap_install(trap_group_error,
                   th,
                   stack,
                   STACK,
                   0x00000001,
                   0x80000000,
                   0, /* high pri */
                   0x0006);

    printf("Before divide by 0, xx=%d\n", xx);
    y = y/0;
    printf("After divide by 0, xx=%d, y=0x%x\n", xx, y);

    return y;
}

```

ST_naked_function pragma

Note: This pragma should be used very carefully. It enables performance of some operations in C which would otherwise have to be written in assembler. However, if it is used, it may be worth checking the assembler code generated by the compiler.

The **ST_naked_function** pragma has two effects.

- It prevents the generation of **ajw** instructions at the beginning and end of the function.
- It suppresses the generation of the return instruction at the end of the function.

Although it omits the **ajw** (if required) at the start and end of the function, it still generates code assuming it is there. Code is generated in the assumption that the user will allocate the workspace for the function. This allocation may be made before the function is called, using assembler code.

An alternative method of making this allocation is to insert the following instruction at the start of the function:

```
ajw - __LOCAL_SIZE
```

Note: This instruction can cause problems if the compiler needs to generate code prior to the start of the user's code. In particular any initialization of local variables because the user has explicitly initialized them, or local temporary variables introduced by the compiler as optimizations, will cause problems, as there is no opportunity to allocate workspace before this code is executed. In this case the workspace may be corrupted.

In addition, the compiler may allocate variables to "unused" parameter slots if it thinks that they are available. As with any other variable, they are accessed at an offset which assumes that the correct local workspace allocation has been performed.

Normally if the workspace pointer is altered through an **ajw** instruction, it affects the offsets used to access local variables, parameters or the static link, so that they are still valid. However, to allow the initial and final **ajw** instructions to be inserted, this tracking is suppressed for the specific case of:

```
ajw - __LOCAL_SIZE
```

and:

```
ajw __LOCAL_SIZE
```

The other effect of this pragma is to omit the **ret** instruction at the end of the function. If the function returns a value, it will be left in AREG (or copied into the callers workspace using an additional parameter if it will not fit into 32 bits, as normal). This could be used to “fall through” to the next function, which may need to be a naked function for this to be useful. It is probably worth checking the assembly code generated to make sure that the compiler has not generated a literal immediately following the function.

13.16 Thread safety

The Standard Template Library is not inherently thread-safe. Data entities must be protected against concurrent accesses.

I/O streams (for example **cout** and **cin**) are thread-safe when using OS20; see [Section 13.16.3](#).

C++ exception handling is thread-safe when using OS20; see [Section 13.16.1](#).

Various aspects of the C++ run-time have been protected by a mutex known as the **generic_cpp** mutex. These include the **_new_handler** global variable (changed by **set_new_handler()**) and the thread-safeness of function local static constructors. See [Section 13.16.2](#).

Some ANSI functions are non-reentrant. This means that in a multi-threaded environment they must be used carefully. See [Section 15.1.1: Concurrency support for libraries on page 222](#), [Section 15.3: Concurrency support for the stdio library on page 223](#) and [Section 15.4: Concurrency support for heap functions on page 233](#).

Concurrency with debug functions is discussed in [Section 16.2: Concurrency support on page 242](#), and for individual functions in [Section 16.6: Debugging function definitions on page 246](#).

Managing devices with possible concurrent access is discussed in [Section 17.7.10: Concurrency support on page 275](#).

The POSIX I/O library and concurrency are discussed in [Section 17.9: Limitations on page 276](#).

13.16.1 C++ exception handling thread protection callback function

To change exception handling thread protection for another operating system, the following function may be used:

```
void ehsetmutex (ehmutexfn mfn, void* data);
```

where *mfn* is a function returning a **long int** (which is always ignored) and taking an **_ST_mutex op** (to specify locking or unlocking) and a **void*** (ignored) as parameters. The *mfn* should lock and unlock to protect exceptions. An appropriate semaphore-like construct should be used by *mfn*. The initialization used by OS20 can be found in **src/os20/clibprot/ehprot.c**, and may be used as a guide of what to provide for other operating systems.

13.16.2 generic_cpp thread protection callback functions

To change the implementation of the protection, the following function may be used:

```
void* _ST_generic_cpp_setmutexfn(
    _ST_generic_mutex_handler_fn fn, void* data );
```

where *fn* is a function which takes a **_ST_mutexop** (**_ST_mutexop_lock** or **_ST_mutexop_unlock**) and a **void*** to the semaphore which is used for protection of various aspects of the C++ runtime. In the above definition, *data* is used as the semaphore.

Pointers to the function and semaphore currently in use are retrieved by the following functions. These return **NULL** if there is no protection installed.

```
void* _ST_generic_cpp_getmutexfn( void );
void* _ST_generic_cpp_getmutexdata( void );
```

13.16.3 I/O streams thread protection

The I/O streams thread protection makes use of OS20's mutex functions. If a different implementation is to be used, the interface should be the same as that of the OS20 mutex functions. The function to set the protection is:

```
void _ST_dinkum_setmutexfns(void* (*mutex_create_fifo_f)(void),
                           void (*mutex_delete_f)(void*),
                           void (*mutex_lock_f)(void*),
                           void (*mutex_release_f)(void*));
```

The pointers passed refer to a function which uses **malloc()** and **mutex_init_fifo()**, **mutex_delete()**, **mutex_lock()**, and **mutex_release()** respectively. (**mutex_create_fifo** is not used as it relies on the **system_partition** memory partition existing, which may not always be true).

13.17 C++ link failures caused by conditional symbols

Many C++ symbols may need to be commoned up at link time if created in multiple object files (for example **extern inline** symbols and template statics). The EDG front end records these symbols (known as “conditional” symbols) and places them in unique sections (based upon their C++ mangled names).

It is possible to override the **ST_section** pragma that EDG applies and put symbols in different sections. There are risks involved with doing this.

- If a user puts a conditional symbol named **fred** in the same section as a non-conditional symbol named **elmo**, **elmo** may be lost because the linker throws away the whole of the section if another occurrence of **fred** from another object file is encountered. In this situation, the linker will pick up on the error and report:

Fatal-st20link- threw away ST_conditional module containing "fred", which contained non-conditional and referenced symbols.

To correct this problem, ensure that all occurrences of the symbol **fred** in the C++ source either do not have an **ST_section** pragma applied in the C++ source, or that **fred** is not in the same section as any other symbol.

- If two conditional symbols are put into the same section, this may result in missing symbols at link time. If **file1.cpp** has symbol **cond1**, and **file2.cpp** has symbols **cond1** and **cond2** (all of which are conditional) and both symbols are put in the same section called **CONDITIONAL_SYMBOLS**, then at link time the second occurrence of **cond1** will cause the section from **file2.cpp** to be discarded, which will result in **cond2** being also discarded, and the link will fail.

The error message would look something like:

Error-st20link-file2.tco- symbol cond2 not found

To fix this, remove the **ST_section** pragma from all occurrences of the symbol **cond2**, or ensure that no other symbols share the same section.



C++ language features for the EDG preprocessor

14

The material in this chapter is derived from material which is © 1992-2002 Edison Design Group, Inc. (EDG). This material is reproduced under a License Agreement between EDG and STMicroelectronics.

Note: This toolset uses a preprocessor supplied by a third party which may change in subsequent releases, therefore STMicroelectronics cannot guarantee C++ object and library compatibility. Users may have to re-compile C++ source code when migrating to subsequent releases of the toolset.

14.1 C++ dialect accepted

The toolset's implementation of the EDG 3.0 preprocessor accepts the C++ language as defined by the ISO/IEC 14882:1998 standard, with the exceptions listed in [Section 14.1.2](#).

The anachronism of being able to assign to the `this` pointer is no longer supported.

14.1.1 New language features accepted

The following features not in the traditional C++ language of *The Annotated C++ Reference Manual (ARM)* by Ellis and Stroustrup but in the standard are implemented.

- The dependent statement of an **if**, **while**, **do-while**, or **for** is considered to be a scope, and the restriction on having such a dependent statement be a declaration is removed.
- The expression tested in an **if**, **while**, **do-while**, or **for**, as the first operand of a “?” operator, or as an operand of the “&&”, “||”, or “!” operators may have a pointer-to-member type or a class type that can be converted to a pointer-to-member type in addition to the scalar cases permitted by the ARM.

- Qualified names are allowed in elaborated type specifiers.
- A global-scope qualifier is allowed in member references of the form **x.::A::B** and **p->::A::B**.
- The precedence of the third operand of the “?” operator is changed.
- If control reaches the end of the **main()** routine, and **main()** has an integral return type, it is treated as if a **return 0;** statement were executed.
- Pointers to arrays with unknown bounds as parameter types are diagnosed as errors.
- A functional-notation cast of the form **A()** can be used even if **A** is a class without a (nontrivial) constructor. The temporary created gets the same default initialization to zero as a static object of the class type.
- A cast can be used to select one out of a set of overloaded functions when taking the address of a function.
- Template friend declarations and definitions are permitted in class definitions and class template definitions.
- Type template parameters are permitted to have default arguments.
- Function templates may have nontype template parameters.
- A reference to **const volatile** cannot be bound to an rvalue.
- Qualification conversions such as conversion from **T**** to **T const * const *** are allowed.
- Digraphs are recognized.
- Operator keywords (for example **and**, **bitand**) are recognized.
- Static data member declarations can be used to declare member constants.
- **wchar_t** is recognized as a keyword and a distinct type.
- **bool** is recognized.
- RTTI (runtime type identification), including **dynamic_cast** and the **typeid** operator, is implemented.
- Declarations in tested conditions (in **if**, **switch**, **for**, and **while** statements) are supported.
- Array **new** and **delete** are implemented.
- New-style casts (**static_cast**, **reinterpret_cast**, and **const_cast**) are implemented.
- Definition of a nested class outside its enclosing class is allowed.
- **mutable** is accepted on nonstatic data member declarations.
- Namespaces are implemented, including **using** declarations and directives. Access declarations are broadened to match the corresponding **using** declarations.
- Explicit instantiation of templates is implemented.
- The **typename** keyword is recognized.
- **explicit** is accepted to declare non-converting constructors.
- The scope of a variable declared in the **for-init-statement** of a **for** loop is the scope of the loop (not the surrounding scope).
- Member templates are implemented.
- The new specialization syntax (using “**template <>**”) is implemented.
- Cv-qualifiers are retained on rvalues (in particular, on function return values).

- The distinction between trivial and nontrivial constructors has been implemented, as has the distinction between PODs and non-PODs with trivial constructors.
- The linkage specification is treated as part of the function type (affecting function overloading and implicit conversions).
- **extern inline** functions are supported, and the default linkage for inline functions is external.
- A typedef name may be used in an explicit destructor call.
- Placement delete is implemented.
- An array allocated via a placement new can be deallocated via delete.
- Covariant return types on overriding virtual functions are supported.
- **enum** types are considered to be non-integral types.
- Partial specialization of class templates is implemented.
- Partial ordering of function templates is implemented.
- Function declarations that match a function template are regarded as independent functions, not as “guiding declarations” that are instances of the template.
- It is possible to overload operators using functions that take enum types and no class types.
- Explicit specification of function template arguments is supported.
- Unnamed template parameters are supported.
- The new lookup rules for member references of the form **x.A:B** and **p->A:B** are supported.
- The notation **:: template** (also **->template** and so on) is supported.
- In a reference of the form **f()->g()**, with **g** a static member function, **f()** is evaluated, and likewise for a similar reference to a static data member. The ARM specifies that the left operand is not evaluated in such cases.
- **enum** types can contain values larger than can be contained in an **int**.
- Default arguments of function templates and member functions of class templates are instantiated only when the default argument is used in a call.
- String literals and wide string literals have **const** type.
- Class name injection is implemented.
- Argument-dependent (Koenig) lookup of function names is implemented.
- Class and function names declared only in unqualified friend declarations are not visible except for functions found by argument-dependent lookup.
- A **void** expression can be specified on a return statement in a **void** function.
- Function-try-blocks, that is, try-blocks that are the top-level statements of functions, constructors, or destructors, are implemented.
- Universal character set escapes (for example **\uabcd**) are implemented.
- On a call in which the expression to the left of the opening parenthesis has class type, overload resolution looks for conversion functions that can convert the class object to pointer-to-function types, and each such pointed-to “surrogate function” type is evaluated alongside any other candidate functions.
- Dependent name lookup in templates is implemented. Nondependent names are looked up only in the context of the template definition. Dependent names are also looked up in the instantiation context, via argument-dependent lookup.

- Value-initialization is implemented. This form of initialization (added in TC1) is indicated by an initializer of “()” and causes zeroing of certain POD-typed members, where the usual default-initialization would leave them uninitialized.
- A partial specialization of a class member template cannot be added outside of the class definition.
- Qualification conversions may be performed as part of the template argument deduction process.
- The **export** keyword for templates is implemented.

14.1.2 Extensions accepted in normal C++ mode

The following extensions are accepted in all modes (except when strict ANSI violations are diagnosed as errors):

- A **friend** declaration for a class may omit the **class** keyword:

```
class B;
class A {
    friend B; // Should be "friend class B"
};
```

- Constants of scalar type may be defined within classes (this is an old form; the modern form uses an initialized static data member):

```
class A {
    const int size = 10;
    int a[size];
};
```

- In the declaration of a class member, a qualified name may be used:

```
struct A {
    int A::f(); // Should be int f();
};
```

- The preprocessing symbol **c_plusplus** is defined in addition to the standard **_cplusplus**.
- Implicit type conversion between a pointer to an **extern "C"** function and a pointer to an **extern "C++"** function is permitted.

Here is an example:

```
extern "C" void f(); // f's type has extern "C" linkage
void (*pf)() // pf points to an extern "C++" function
= &f; // error unless implicit conversion is allowed
```

This extension is allowed in environments where C and C++ functions share the same calling conventions.

- A “?” operator whose second and third operands are string literals or wide string literals can be implicitly converted to “**char ***” or “**wchar_t ***”. (Recall that in C++ string literals are **const**. There is a deprecated implicit conversion that allows conversion of a string literal to “**char ***”, dropping the **const**. That conversion, however, applies only to simple string literals. Allowing it for the result of a “?” operation is an extension.)

```
char *p = x ? "abc" : "def";
```

- Default arguments may be specified for function parameters other than those of a top-level function declaration (for example, they are accepted on **typedef** declarations and on pointer-to-function and pointer-to-member-function declarations).
- Nonstatic local variables of an enclosing function can be referenced in a non-evaluated expression (for example a **sizeof** expression) inside a local class. A warning is issued.

14.1.3 Template instantiation modes

Normally, when a C++ file is compiled, only the template entities used are instantiated. The overall instantiation mode can, however, be changed by a command line option.

-tnone	Do not automatically create instantiations of any template entities.
-tused	Instantiate those template entities that were used in the compilation. This is the default, and includes all static data members for which there are template definitions.
-tall	Instantiate all template entities declared or referenced in the compilation unit. For each fully instantiated template class, all of its member functions and static data members are instantiated whether or not they were used. Nonmember template functions are instantiated, even if the only reference was a declaration.

14.1.4 Template instantiation #pragma directives

Instantiation pragmas can be used to control the instantiation of specific template entities or sets of template entities. There are two instantiation pragmas.

- The **instantiate** pragma causes a specified entity to be instantiated.
- The **do_not_instantiate** pragma suppresses the instantiation of a specified entity. It is typically used to suppress the instantiation of an entity for which a specific definition will be supplied.

The argument to the instantiation pragma may be:

- a template class name - **A<int>**,
- a template class declaration - **class A<int>**,
- a member function name - **A<int>::f**,
- a static data member name - **A<int>::i**,
- a static data declaration - **int A<int>::i**,
- a member function declaration - **void A<int>::f(int, char)**,
- a template function declaration - **char* f(int, float)**.

A pragma in which the argument is a template class name (for example, **A<int>** or **class A<int>**) is equivalent to repeating the pragma for each member function and static data member declared in the class. When instantiating an entire class, a given member function or static data member may be excluded using the **do_not_instantiate** pragma.

For example,

```
#pragma instantiate A<int>
#pragma do_not_instantiate A<int>::f
```

The template definition of a template entity must be present in the compilation for an instantiation to occur. If an instantiation is explicitly requested by use of the `instantiate` pragma and no template definition is available or specific definition is provided, the compiler issues an error.

```
template <class T> void f1(T); // No body provided
template <class T> void g1(T); // No body provided
void f1(int) {} // Specific definition
void main()
{
    int    i;
    double d;
    f1(i);
    f1(d);
    g1(i);
    g1(d);
}
#pragma instantiate void f1(int) // error-specific definition
#pragma instantiate void g1(int) // error-no body provided
```

`f1(double)` and `g1(double)` will not be instantiated (because no bodies were supplied) but no errors will be produced during the compilation (if no bodies are supplied at link time, a linker error will be produced).

A member function name (for example, `A<int>::f`) can only be used as a pragma argument if it refers to a single user defined member function (that is, not an overloaded function). Preprocessor generated functions are not considered, so a name may refer to a user defined constructor even if a preprocessor-generated copy constructor of the same name exists. Overloaded member functions can be instantiated by providing the complete member function declaration, as in:

```
#pragma instantiate char* A<int>::f(int, char*)
```

The argument to an instantiation pragma may not be a preprocessor-generated function, an inline function, or a pure virtual function.

14.1.5 Implicit inclusion

When implicit inclusion is enabled (using the `st20cc` option `-implicit-include`) the front end is given permission to assume that if it needs a definition to instantiate a template entity declared in a header file, it can implicitly include the corresponding “`.cxx`” file to get the source code for the definition. For example, if a template entity `ABC::f` is declared in file `xyz.h`, and an instantiation of `ABC::f` is required but no definition of `ABC::f` appears in the source code being processed, the preprocessor will look to see if a file `xyz.cxx` exists, and if so it will process it as if it were included at the end of the main source file.

To find the template definition file for a given template entity, the front end needs to know the full path name of the file in which the template was declared and whether the file was included using the system include syntax (for example, `#include <file.h>`). This information is not available for preprocessed source containing `#line` directives. Consequently, the front end will not attempt implicit inclusion for source code containing `#line` directives.

To find the implicit file the preprocessor works through the list of files with the same stem as the header file and with suffices “`.c`”, “`.C`”, “`.cpp`”, “`.CPF`”, “`.cxx`”, “`.CXX`”, and “`.cc`”.

14.1.6 Exported templates

Exported templates are templates declared with the keyword **export**. Exporting a class template is equivalent to exporting each of its static data members and each of its non-inline member functions. An exported template is special because its definition does not need to be present in a translation unit that uses that template. In other words, the definition of an exported (non-class) template does not need to be explicitly or implicitly included in a translation unit that instantiates that template. For example, the following is a valid C++ program consisting of two separate translation units:

// File 1:

```
#include <stdio.h>
static void trace() { printf("File 1\n"); }
export template<class T> T const& min(T const&, T const&);
int main() {
    trace();
    return min(2, 3);
}
```

// File 2:

```
#include <stdio.h>
static void trace() { printf("File 2\n"); }
export template<class T> T const& min(T const &a, T const &b) {
    trace();
    return a<b? a: b;
}
```

Note that these two files are separate translation units: one is not included in the other. That allows the two functions `trace()` to coexist (with internal linkage).

Support for exported templates can be enabled using the **-export** command-line option (it is enabled by default in strict ANSI mode; **-strict**). For example, the program above could be built as follows:

```
st20cc -p c2 -export -c file_1.cpp
st20cc -p c2 -export -c file_2.cpp
st20cc -p c2 file_1.cpp file_2.cpp
```

or simply,

```
st20cc -p c2 -export file_2.cpp file_1.cpp
```

*Note: Note that as **file_2.cpp** exports the template, it must come before **file_1.cpp**. If templates were being exported in both directions, this single command would not work.*

14.1.7 Finding the exported template definition

An instantiation of an exported template involves at least two translation units: one which requires the instantiation, and one which contains the template definition.

When a file containing definitions of exported templates is compiled, a file with a **.et** suffix is created in the same directory as the object file. The **.et** files are used later by the front end to find the translation unit that defines a given exported template.

When a file that potentially makes use of exported templates is compiled, the compiler must be told where to look for **.et** files for exported templates used by a given translation unit. By default, the compiler looks in the current directory. Other directories may be specified with the **-export-dir** option. The **.et** files are needed when it comes time to generate an instantiation.

The “.et” files only inform the front end about the location of exported template definitions; they do not actually contain those definitions. The sources containing the exported template definitions must therefore be made available at the time of instantiation. In particular, the export facility is **not** a mechanism for avoiding the publication of template definitions in source form.

14.1.8 Secondary translation units

An instantiation of an exported template can be triggered by an explicit instantiation directive, or by the `-tused` command-line option. In each case, the translation unit that contains the initial point of instantiation will be processed as the *primary translation unit*. Based on information it finds in the “.et” files, the front end will then load and parse the translation unit containing the definition of the template to instantiate. This is a *secondary translation unit*. The simultaneous processing of the primary and secondary translation units enables the front end to create instantiations of the exported templates (which can include entities from both translation units). This process may reveal the need for additional instantiations of exported templates, which in turn can cause additional secondary translation units to be loaded.

When secondary translation units are processed, the declarations they contain are checked for consistency. This process may report errors that would otherwise not be caught. Many of these errors are so-called “ODR violations” (ODR stands for “one-definition rule”). For example:

// File 1:

```
struct X {
    int x;
};
int main() {
    return min(2, 3);
}
```

// File 2:

```
struct X {
    unsigned x; // Error: X::x declared differently in File 1
};
export template<class T> T const& min(T const &a, T const &b) {
    return a<b? a: b;
}
```

If there are no errors, the instantiations are generated in the output associated with the primary translation unit (or in separate associated files in one-instantiation-per-object mode). This may also require that entities with internal linkage in secondary translation units be “externalized” so they can be accessed from the instantiations in the primary translation unit.

14.1.9 Libraries with exported templates

Typically a (non-export) library consists of an **include** directory and a **lib** directory. The **include** directory contains the header files required by users of the library and the **lib** directory contains the object code libraries that client programs must use when linking programs.

With exported templates, users of the library must also have access to the source code of the exported templates and the information contained in the associated **.et** files. This information should be placed in a directory that is distributed along with the **include** and **lib** directories: This is the **export** directory. It must be specified using the **-export-dir** option when compiling client programs.

The recommended procedure to build the export directory is as follows:

- 1 For each **.et** file in the original source directory, copy the associated source file to the export directory.
- 2 Concatenate all of the **.et** files into a single **.et** file (for example **mylib.et**) in the export directory. The individual **.et** files could be copied to the export directory, but having all of the **.et** information in one file will make use of the library more efficient.
- 3 Create an **export_info** file in the export directory. The **export_info** file specifies the include search paths to be used when recompiling files in the export directory. If no **export_info** file is provided, the include search path used when compiling the client program that uses the library will also be used to recompile the library exported template files.

The **export_info** file consists of a series of lines of the form

```
include=x
```

or

```
sys_include=x
```

where **x** is a path name to be placed on the include search path. The directories are searched in the order in which they are encountered in the **export_info** file. The file can also contain comments—which begin with a **#**—and blank lines. Spaces are ignored but tabs are not currently permitted. For example:

```
# The include directories to be used for the xyz library
include = /disk1/xyz/include
sys_include = /disk2/abc/include
include = /disk3/jkl/include
```

The include search path specified for a client program is ignored by the front end when it processes the source in the export library, except when no **export_info** file is provided. Command-line macro definitions specified for a client program are also ignored by the front end when processing a source file from the export library; the command line macros specified when the corresponding **.et** file was produced do apply. All other compilation options (other than the include search path and command-line macro definitions) used when recompiling the exported templates will be used to compile the client program.

When a library is installed on a new system, it is likely that the **export_info** file will need to be adapted to reflect the location of the required headers on that system.



Part 3

Libraries





Libraries introduction

15

This part of the manual describes the libraries provided with this toolset. These libraries support the standard ANSI C run-time libraries plus the extra libraries for debugging, device-independent input and output, float length mathematics, ROM restart and standard input/output concurrency support.

The header files for these additional functions are given in [Table 45](#).

Header	Description
debug.h	Debugging support functions
dev.h	Device-independent input and output
mathf.h	Float length mathematics
restart.h	ROM restart support functions
stdiomutex.h	Concurrency support for stdio library
st_mutex.h	Concurrency support for heap functions

Table 45: Non-ANSI library header files

The functions provided by **stdiomutex.h** are described in [Section 15.1](#), because they support the ANSI **stdio** library.

Dynamic libraries should only use C interfaces (declared with '**extern "C"'** ...') even in a C++ environment.

The OS20 real-time kernel functions provided with the toolset are defined in *Part 4 OS20 functions*.

The C++ libraries including the Standard Template Library are shipped with the toolset, on-line documentation is provided on the CD. Both an embedded and a full version of the

libraries are provided. They are introduced in the *Release notes* chapter of the *ST20 Embedded Toolset Delivery Manual*.

15.1 ANSI functions

Table 46 lists the ANSI header files supplied. The functions defined in these header files are listed in *Table 47*. Full descriptions of the functions can be found in any ANSI C reference book.

Header	Description	Header	Description
<code>assert.h</code>	Assertions	<code>signal.h</code>	Signal handling
<code>ctype.h</code>	Character testing	<code>stdarg.h</code>	Variable argument lists
<code>errno.h</code>	Error numbers	<code>stddef.h</code>	Standard definitions
<code>float.h</code>	Floating constants	<code>stdio.h</code>	Standard I/O
<code>limits.h</code>	Integer constants	<code>stdlib.h</code>	Miscellaneous functions
<code>locale.h</code>	Locale support	<code>string.h</code>	String handling
<code>math.h</code>	Double mathematics	<code>time.h</code>	Time support
<code>setjmp.h</code>	Non-local jumps		

Table 46: ANSI library header files

ANSI Header File	Functions			
<code>assert.h</code>	<code>assert</code>			
<code>ctype.h</code>	<code>isalnum</code>	<code>isalpha</code>	<code>iscntrl</code>	<code>isdigit</code>
	<code>isgraph</code>	<code>islower</code>	<code>isprint</code>	<code>ispunct</code>
	<code>isspace</code>	<code>isupper</code>	<code>isxdigit</code>	<code>tolower</code>
	<code>toupper</code>			
<code>errno.h</code>	<code>errno</code>			
<code>math.h</code>	<code>acos</code>	<code>asin</code>	<code>atan</code>	<code>atan2</code>
	<code>ceil</code>	<code>cos</code>	<code>cosh</code>	<code>exp</code>
	<code>fabs</code>	<code>floor</code>	<code>fmod</code>	<code>frexp</code>
	<code>ldexp</code>	<code>log</code>	<code>log10</code>	<code>modf</code>
	<code>pow</code>	<code>sin</code>	<code>sinh</code>	<code>sqrt</code>
	<code>tan</code>	<code>tanh</code>		
<code>setjmp.h</code>	<code>longjmp</code>	<code>setjmp</code>		
<code>signal.h</code>	<code>raise</code>	<code>signal</code>		
<code>stdarg.h</code>	<code>va_arg</code>	<code>va_end</code>	<code>va_start</code>	

Table 47: ANSI functions supplied with the toolset

ANSI Header File	Functions			
stdio.h	clearerr	fclose	feof	ferror
	fflush	fgetc	fgetpos	fgets
	fopen	fprintf	fputc	fputs
	fread	freopen	fscanf	fseek
	fsetpos	ftell	fwrite	getc
	getchar	gets	perror	printf
	putc	putchar	puts	remove
	rename	rewind	scanf	setbuf
	setvbuf	sprintf	sscanf	tmpfile
	tmpnam	ungetc	vfprintf	vprintf
	vsprintf			
stdlib.h	abort	abs	atexit	atof
	atoi	atol	bsearch	calloc
	div	exit	free	getenv
	labs	ldiv	malloc	qsort
	rand	realloc	srand	strtod
	strtol	strtoul	system	
string.h	memchr	memcmp	memcpy	memmove
	memset	strcat	strchr	strcmp
	strcpy	strcspn	strerror	strlen
	strncat	strncmp	strncpy	strpbrk
	strrchr	strspn	strstr	strtok
time.h	asctime	clock ^a	ctime	difftime
	gmtime ^b	localtime ^c	mktime ^c	strftime
	time ^c			

Table 47: ANSI functions supplied with the toolset

- a. The **clock** function is only supported for high priority ST20-C2 tasks.
- b. The **gmtime()** function always returns **NULL**.
- c. The functions **time()**, **localtime()** and **mktime()** require a connection to the host and will fail in a standalone ROM system.



15.1.1 Concurrency support for libraries

Some ANSI functions are non-reentrant. This means that in a multi-threaded environment they must be used carefully. The following functions are non-reentrant:

atexit, asctime, ctime, localtime, rand, signal, srand, strtok.

Use of OS20 (either with or without the **st20cc** option **-runtime os20**) provides support for the following:

- the heap functions: **calloc, free, malloc, realloc**,
- all **stdio** functions including **printf**,
- all device independent I/O functions described in [Chapter 17: Device-independent I/O](#),
- all debug support functions described in [Chapter 16: Debugging support functions](#),
- C++ exception handling (when required),
- the C++ **set_new_handler** functionality.

To provide alternative mutual exclusion, or if using an operating system other than OS20, concurrency support may be set up manually. However, be aware of the following.

- If setting up concurrency support for **stdio**, concurrency support must be included for the device independent I/O functions and debug support functions.
- If setting up concurrency support for device independent I/O functions, concurrency support must be included for the debug support functions.
- Concurrency support may be implemented for the debug support functions without affecting other functions.

Setting up concurrency support is described in the sections listed below:

- for stdio functions, [Section 15.3: Concurrency support for the stdio library](#),
- for device independent I/O functions, [Section 17.7.10: Concurrency support on page 275](#),
- for debug support functions, [Section 16.2: Concurrency support on page 242](#),
- for the heap functions, [Section 15.4: Concurrency support for heap functions](#),
- for C++ exceptions and other C++ protection, [Section 13.16: Thread safety on page 203](#).

15.2 stdio file buffers

When a **stdio** function is used in an application, a 4 Kbytes buffer is provided for **stdin** and **stdout**. The size of the buffer is defined in **stdio.h** (**BUFSIZ**), but can be changed, if required, to free up the amount of memory available.

The following example can be used to release the **stdin** buffer and set the **stdout** buffer to 256 bytes. This reduces the total buffer size from 8 Kbytes to 256 bytes.

```
#include <stdio.h>

#if !defined(STDIO_BUFSIZ)
#define STDIO_BUFSIZ 256
#endif

int main(void)
{
    setvbuf(stdin, NULL, _IONBF, 0);
    setvbuf(stderr, NULL, _IONBF, 0);
    setvbuf(stdout, NULL, _IOLBF, STDIO_BUFSIZ);

    printf("stdout buffering reset buffer size from %d to %d\n", BUFSIZ,
STDIO_BUFSIZ);
}
```

15.3 Concurrency support for the stdio library

Before reading this section please read [Section 15.1.1](#). This describes the knock-on effects of manually setting up concurrency support for the **stdio** library.

Support is provided in the **stdio** (standard input/output) library to enable multiple threads to access files concurrently without risking data corruption. This allows several threads to open and close files, or operate on the same file, concurrently. When using an OS other than OS20, the user must supply a mutual exclusion mechanism, which will be automatically used for critical regions of code.

Two types of objects have to be protected:

- the file table which is the list of all **FILE** structures, and is accessed whenever a file is opened or closed,
- each individual **FILE** structure, which is accessed whenever an open file is read or written.

The user-provided mutual exclusion mechanism must include:

- a mutual exclusion object for the file table,
- a mutual exclusion object to be associated with each **FILE**,
- mutual exclusion claim/release operations to be called when the file table is accessed,
- mutual exclusion claim/release operations to be called when the file is accessed.

The device-independent and debug layers are required to provide mutual exclusion because they are used for file input/output. See [Section 16.2](#) and [Section 17.7.10](#).

15.3.1 Interface

The **stdio** concurrency support is defined in the header file **stdiomutex.h**. The following functions are provided:

```
void* getstdiofiletablemutexfn(void);
```

This function gets the mutual exclusion function for the file table.

```
void setstdiofiletablemutexfn(stdiofiletablemutexfn sftm);
```

This function sets up the mutual exclusion function for the file table.

```
void* getstdiomutexfn(void);
```

This function gets the mutual exclusion function for individual files.

```
void setstdiomutexfn(stdiomutexfn mfn);
```

This function sets up the mutual exclusion function for individual files.

```
void* getstdiomutex(FILE *f);
```

This function returns the mutual exclusion object for an individual file.

```
void setstdiomutex(FILE *f, void *v);
```

This function assigns the mutual exclusion object for an individual file.

The functions use the following type definitions:

```
typedef void (*stdiomutexfn)(FILE *f, stdiomutexop op)
```

```
typedef void (*stdiofiletablemutexfn)(stdiofiletablemutexop op)
```

```
typedef enum {
    stdiofiletablemutexop_lock, /* called at beginning of critical region */
    stdiofiletablemutexop_unlock /* called at the end of critical region */
} stdiofiletablemutexop;
```

```
typedef enum {
    stdiomutexop_init,
    stdiomutexop_lock, /* called at beginning of critical region */
    stdiomutexop_unlock, /* called at the end of critical region */
    stdiomutexop_deinit
} stdiomutexop;
```

15.3.2 How to use the concurrency support

The concurrency support in the **stdio** library requires the user to supply two functions, one of which is called for operations on the file table, and the other is called for operations on individual files.

The protection function for the **stdio** file table is installed using the library function **setstdiofiletablemutexfn()**. The function which this installs is called by the **stdio** library before any operations on the file table, with a parameter of **stdiofiletablemutexop_lock**. Once the operation has been completed, the function is called again with a parameter of **stdiofiletablemutexop_unlock**.

Typically this is used to claim and release a semaphore, to ensure that only one thread is manipulating the file table at any given time.

The protection facility for individual files is a little more complex. As well as a function which is called by the **stdio** library before and after any operations on the file, it is also possible to

associate a mutex with each **FILE**. The mutex is represented within the **stdio** library as a **void***, which the user can then cast to a pointer to whatever mutex is required.

The mutex function for file operations is called with the argument:

- **stdiomutexop_init** when a **FILE*** should have a new mutex associated with it,
- **stdiomutexop_deinit** when a **FILE*** should have its mutex disassociated with it and possibly deleted.

The mutex to be used with each **FILE** is set using the **setstdiomutex()** library function, and can be retrieved using **getstdiomutex()**.

The mutex function for file operations is installed using the **setstdiomutexfn()** library function. This is called by the **stdio** library before a file operation with an **op** parameter of **stdiomutexop_lock**. Once the operation has been completed, the function is called a second time with a parameter of **stdiomutexop_unlock**.

The file mutex function simply claims the mutex when called with an **op** parameter of **stdiomutexop_lock**, and releases the mutex when called with a parameter of **stdiomutexop_unlock**.

The per **FILE*** mutex callback may be called for each open **FILE*** with the operation **stdiomutexop_init** by calling the function **_ST_ApplyStdioFileMutex**.

Example

This example shows how **stdio** concurrency support with OS20 would be initialized if this were not automatically done. If applied to another OS, replace the OS20-specific semaphore type and functions with equivalent constructs for the OS being used.

Note: Protection for the device-independent and debug layers is also required.

```
/*
 * std_mutex.c
 *
 * Copyright (C) ST Microelectronics Ltd. 2000
 *
 * mutex for stdio.
 */
#include <stdio.h>
#include <stdiomutex.h>
#include <stdlib.h>
#include <semaphor.h>

static semaphore_t *filetablesemp; /* File table semaphore */

/*****
 * File table semaphore operations function.
 *****/

static void mysfiletablemutexfn(stdiofiletablemutexop op)
{
    switch (op) {
        case stdiofiletablemutexop_lock:
            semaphore_wait(filetablesemp);
            break;
        case stdiofiletablemutexop_unlock:
            semaphore_signal(filetablesemp);
            break;
    }
}
```

```

/*****
 * FILE semaphore creation and operations function.
 *****/

static void myfilemutexfn(FILE *f, stdiomutexop op)
{
    semaphore_t *s = (semaphore_t *)getstdiomutex(f);

    switch (op) {
        case stdiomutexop_init:
            s = malloc(sizeof(semaphore_t));
            semaphore_init_fifo(s, 1); /*mutual exclusion object */
            setstdiomutex(f, s); /* assign mutual exclusion object to FILE*/
            break;
        case stdiomutexop_lock:
            if (s)
                semaphore_wait(s);
            break;
        case stdiomutexop_unlock:
            if (s)
                semaphore_signal(s);
            break;
        case stdiomutexop_deinit:
            if (s) {
                semaphore_delete(s);
                free(s);
            }
            break;
    }
}

/*****
 * Function to create file table semaphore and assign mutual exclusion
 * object operations.
 *****/

void init_stdio_mutex(void)
{
    static semaphore_t filetablesem;
    filetablesem = &filetablesem;

    semaphore_init_fifo(&filetablesem, 1); /* mutual exclusion object */

    /* assign File table semaphore function. */
    setstdiofiletablemutexfn(mysfiletablemutexfn);

    /* assign FILE semaphore function. */
    setstdiomutexfn(myfilemutexfn);

    /* initialise mutex object on stdout*/
    myfilemutexfn(stdout, stdiomutexop_init);
}

```

15.3.3 Function definitions

getstdiofiletablemutexfn

Get file table mutual exclusion callback function

Syntax

```
#include <stdiomutex.h>

void* getstdiofiletablemutexfn( void );
```

Arguments

None

Results

Returns a pointer to the file table mutual exclusion callback function.

Errors

None

Description

Returns a pointer to the file table mutual exclusion callback function installed for handling file table mutex callbacks.

getstdiomutex

Get mutual exclusion object for a file

Syntax

```
#include <stdiomutex.h>

void* getstdiomutex( FILE *f );
```

Arguments

FILE *f File structure pointer

Results

Returns a pointer to the mutual exclusion object assigned to **FILE *f**.

Errors

None

Description

Returns a pointer to the mutual exclusion object for file *f*.

getstdiomutexfn

Get per-file mutual exclusion callback function

Syntax

```
#include <stdiomutex.h>

void* getstdiomutexfn( void );
```

Arguments

None

Results

Returns a pointer to the mutual exclusion callback function.

Errors

None

Description

Returns a pointer to the mutual exclusion callback function installed for handling per-file mutex callbacks.

setstdiofiletablemutexfn

Set mutual exclusion function for the file table

Syntax

```
#include <stdiomutex.h>

void setstdiofiletablemutexfn( stdiofiletablemutexfn sftm );
```

Arguments

stdiofiletablemutexfn *sftm* Pointer to file table mutual exclusion function

Results

None

Errors

None

Description

Sets the file table mutual exclusion function to be *sftm*. *sftm* is a pointer to a function which must have the prototype:

```
void mutexfn(stdiofiletablemutexop op);
```

This is called with **op** set to **stdiofiletablemutexop_lock** before any operations on the file table, and **stdiofiletablemutexop_unlock** after the operation has been completed.

setstdiomutex

Set mutual exclusion object for file

Syntax

```
#include <stdiomutex.h>

void setstdiomutex( FILE *f,
                   void *v );
```

Arguments

FILE *f	File structure pointer
void *v	Pointer to mutual exclusion object

Results

None

Errors

None

Description

Sets the mutual exclusion object for **FILE *f** to **v**, unless it has already been set, in which case this call has no effect.

setstdiomutexfn

Set up mutual exclusion function for files

Syntax

```
#include <stdiomutex.h>

void setstdiomutexfn( stdiomutexfn mfn );
```

Arguments

stdiomutexfn mfn Pointer to file mutual exclusion function

Results

None

Errors

None

Description

Sets the file mutual exclusion function to be **mfn**. **mfn** is a pointer to a function which must have the prototype:

```
void mutexfn(FILE *f, stdiomutexop op);
```

This is called with **op** set to **stdiomutexop_lock** before any operations on the file, and **stdiomutexop_unlock** after the operation has been completed.

15.4 Concurrency support for heap functions

The heap may be protected against concurrent accesses from multiple threads by using the following mutual exclusion callback functions:

```
void heap_setmutexfn(void*, void*);  
void* heap_getmutexfn(void);  
void* heap_setmutexdata(void*);
```

15.4.1 Function definitions

heap_getmutexdata

Get mutual exclusion user data for the heap

Syntax

```
#include <st_mutex.h>

void* heap_getmutexdata( void );
```

Arguments

None

Results

A **void*** representing the user data.

Errors

None

Description

Returns a **void*** representing the user data.

heap_getmutexfn

Get mutual exclusion callback for the heap

Syntax

```
#include <st_mutex.h>
void* heap_getmutexfn( void );
```

Arguments

None

Results

A pointer to the callback function.

Errors

None

Description

Returns a pointer to the callback function used to implement mutual exclusion for the heap.

heap_setmutexfn

Set mutual exclusion callback for the heap

Syntax

```
#include <st_mutex.h>

void heap_setmutexfn( void* fn,
                      void* data );
```

Arguments

void* fn	Callback function pointer
void* data	Data passed to the callback function

Results

None

Errors

None

Description

Sets the mutual exclusion callback for the heap. This callback is called when the heap needs protecting from concurrent accesses by multiple tasks. The callback function is called with the arguments (**_ST_mutexop op, void* data**). The callback function is called prior to a task accessing the heap with the **_ST_mutexop** argument set to **_ST_mutexop_lock**.

The callback function is called after the task has accessed the heap with the **_ST_mutexop** argument set to **_ST_mutexop_unlock**. The data argument is an arbitrary **void*** available to the user. This normally points to the semaphore used to implement the locking mechanism.

15.5 Floating point support for standard ANSI functions

Floating point support is expensive in terms of code size. For this reason two versions of the library are provided, one which supports floating point and one which does not. By default the standard run-time libraries linked with a user application do not support floating point.

- If an attempt is made to format (using **sprintf** or similar) a floating point number then the string “[**fp**]” is written into the string instead of the number.
- If an attempt is made to read a floating point number (using **sscanf**) then **sscanf** stops conversion at that point and terminates immediately returning the number of inputs which have been successfully converted up to that point. No value is written into the associated float or double object.

*Note: The functions **strtod** and **atof** return 0 values if an attempt is made to use them when linking with the default libraries.*

The full list of functions affected is:

```

atof
sprintf
vsprintf
sscanf
printf
strtod
snprintf (non-ANSI function)
vsnprintf (non-ANSI function)

```

To enable floating point support for these functions, use **st20cc** with the **-fp** option.

The degree of compliancy to the IEEE standard for floating point types is stated in [Section 13.1.1](#).

snprintf - write formatted data to a string

The function **snprintf** is identical to **sprintf**, except a buffer length may be specified so no buffer overflow will occur. If the number of bytes required to store the data exceeds **count**, then **count** bytes of data are stored in **s** (including the terminating NULL character). The total length of the string formatted is returned (which excludes the terminating NULL character).

```
int snprintf(char *s, size_t count, const char *format, ...)
```

vsnprintf - write formatted data to a string

The function **vsnprintf** is identical to **vsprintf**, except a buffer length may be specified so no buffer overflow will occur. If the number of bytes required to store the data exceeds **count**, then **count** bytes of data are stored in **s** (including the terminating NULL character). The total length of the string formatted is returned (which excludes the terminating NULL character).

```
int vsnprintf(char *s, size_t count, const char *format, va_list a)
```

15.6 Floating point mathematics library

The floating point mathematics functions provide the same functionality as the ANSI standard double length mathematics functions. The difference is that they work on float values, so the associated code is smaller and faster but has less precision. The names of the float mathematics functions are the same as for their double length counterparts except that an **f** is appended to the end of the function name and the arguments and returned values are of type **float**. For example, the float equivalent of **cos** is **cosf**, with prototype:

```
float cosf (float x);
```

15.7 Fatal run-time errors

The C run-time system is able to detect various run-time errors and report these to the user. The behavior on detecting a fatal error is as follows:

- 1 Set the run-time library variable **errno** to an error-specific value.
- 2 Display an error message. The message can only be displayed if the debugger is in use or if the application is run on the simulator in stand-alone mode.
- 3 Trigger a breakpoint. If a debugger is present then it detects the breakpoint and behaves as if a user breakpoint has been hit. This causes the debugger to backtrace to the offending function call or code sequence.

If no debugger is present then the breakpoint is ignored.

- 4 Raise the **SIGTERM** signal. The default signal handler for **SIGTERM** causes the program to exit with the value **EXIT_FAILURE**. This behavior can be modified by installing a user handler for the **SIGTERM** signal. A user handler can determine which error has occurred by checking the value of **errno**. Installation of a user signal handler is achieved using the **signal** function:

```
void myhandler(int sig)
{
    /* body of handler */
}

...

signal(SIGTERM, myhandler);
```

The errors which are detected and their associated messages and **errno** values are given below:

Error in free(), bad pointer or heap corrupted

errno value: **EFREE**

An invalid pointer has been passed to the **free** function. This error is only issued if the **free** function is used. It can occur for two reasons.

- A pointer, not previously returned from **malloc**, **calloc** or **realloc**, is passed to **free**.
- The heap has become corrupted in some way, usually by the application overwriting its data structures.

Error in realloc(), bad pointer or heap corrupted

errno value: **ERELLOC**

An invalid pointer has been passed to the **realloc** function. This error is only issued if the **realloc** function is used. It can occur for two reasons:

- A pointer, not previously returned from **malloc**, **calloc** or **realloc**, is passed to **realloc**.
- The heap has become corrupted in some way, usually by the application overwriting its data structures.

Stack overflow

errno value: **ESTACK**

The stack has overflowed. This error only occurs if stack checking has been enabled at compile time. It signifies that the current function call would extend the stack beyond the available stack space.

Bad workspace pointer

errno value: **WMPTR**

Workspace pointer invalid. This error only occurs if stack checking has been enabled at compile time. It signifies that during stack checking a workspace pointer (WMPTR) value has been detected which is not within the bounds of any recognized stack.

Read through null pointer

errno value: **ENULLRD**

A read through a NULL pointer has been detected. This error only occurs if pointer checking has been enabled at compile time.

Read through incorrectly aligned pointer (even alignment expected)

errno value: **EALIGNRDE**

A read through an unaligned pointer has been detected when an even alignment was expected. This error only occurs if pointer checking has been enabled at compile time.

Read through incorrectly aligned pointer (word alignment expected)

errno value: **EALIGNRDW**

A read through an unaligned pointer has been detected when word alignment was expected. This error only occurs if pointer checking has been enabled at compile time.

Write through null pointer

errno value: **ENULLWR**

A write through a NULL pointer has been detected. This error only occurs if pointer checking has been enabled at compile time.

Write through incorrectly aligned pointer (even alignment expected)

errno value: **EALIGNWRE**

A write through an unaligned pointer has been detected when an even alignment was expected. This error only occurs if pointer checking has been enabled at compile time. The expected alignment was on an even address boundary.

Write through incorrectly aligned pointer (word alignment expected)

errno value: **EALIGNWRW**

A write through an unaligned pointer has been detected when word alignment was expected. This error only occurs if pointer checking has been enabled at compile time. The expected alignment was on a word boundary.



Debugging support functions 16

16.1 Introduction

The debugging support library allows an application to access host files and the host screen and keyboard when the debugger is present. Strings can be displayed, and characters can be read from the keyboard. Files can be opened, closed, read from and written to. Functions are also provided to execute a debugger command and to close down the debugger.

The debugger is always present when the application is being run by **st20run** on a hardware target. It is also present when the application is run on a hardware target from ROM and the host is present and **st20run** is running and connected to the target.

On hardware targets the functions signal that a host I/O operation is required via the diagnostic controller unit of the device. The debugger acts on such signal events when it detects them. A defined interface between the functions and the debugger allows the debugger to service the I/O request, for example, to open a file.

The simulator can also service some of these functions in the event that the debugger is not present. In this case the functions issue a special pseudo-instruction which informs the simulator of a debugging event. The event is serviced before the function returns, at which point normal simulation can continue. Those debug functions which are designed to control the debugger are not serviced and instead cause a warning message to be displayed by the simulator.

The debugger is not present when the application is being run on a hardware target as a stand-alone system from ROM. If neither the debugger nor the simulator is in use then there is no way for the debugging events to be serviced. When the debugger is connected, it is also possible for debug input/output to be turned off by the **informs** command. In both cases the debug functions return the error code **DEBUG_NOT_CONNECTED**.

The header file for the debugging functions is **debug.h**; see [Table 48](#). The debugging functions are defined in [Section 16.6](#).

16.2 Concurrency support

The debugger can only service one request at a time and so only one debug function can be executing at any time. To prevent multiple debug functions executing simultaneously, for example from different concurrent threads of execution, each debug function immediately returns with the value **DEBUG_FN_REENTERED** if it detects that another debug function is already running.

To use the debug functions in a concurrent environment, a function pointer can be registered with the debug library that ensures exclusion. This is done using the function **debugsetmutexfn** which is described in [Section 16.6](#). When OS20 is used, the application does not have to manage the debug mutex; this is automatically set up by the OS20 function **kernel_initialize()**. However, the application may replace this default mutex after **kernel_initialize()** has been called.

Alternatively, a retry strategy may be implemented by checking for the **DEBUG_FN_REENTERED** result. Another strategy is to confine any I/O to a single thread of execution.

Function	Description
debugbreak	Cause a breakpoint
debugclose	Close a file on the host
debugcommand	Invoke a debugger command
debugconnected	Determine whether a debugger is connected
debugdisconnect	Disconnects the debugger
debugexit	Cause the debugger to exit
debugfilesize	Return size of file in bytes
debugflush	Flush outstanding file operations
debuggetenv	Get value of host environment variable
debuggetkey	Read a character from the keyboard
debuggets	Read a string from the keyboard
debuginformsenabled	Determine whether informs are enabled
debugmessage	Write a string to the screen
debugopen	Open a file
debugpollkey	Get a character from the keyboard
debugread	Read bytes from a file
debugseek	Reposition file-pointer of a file
debugsetmutexfn	Set mutual exclusion function of debug library
debugtell	Return position of file-pointer of a file
debugtime	Return the number of seconds elapsed since preset date and time
debugwrite	Write a number of bytes to a file

Table 48: Functions defined in debug.h

16.3 Keyboard and screen access

The application can send a message to the host screen using **debugmessage**.

```
debugmessage ("Hello World\n");
```

The message is displayed in the window where the **st20run** command line was executed.

A string can be read into a string buffer from the host keyboard using **debuggets**. The string is terminated from the keyboard with a newline character, but the newline is replaced by a null character in the application string buffer. The function takes the maximum string size as a parameter, and the input is truncated to this length if necessary. The function returns the actual number of characters read, including the terminating null character.

```
#define MAX_STRING_SIZE 100

char buffer[MAX_STRING_SIZE];
long int num_bytes;
num_bytes = debuggets(buffer, MAX_STRING_SIZE);
```

Two functions enable a character to be read from the keyboard:

```
long int debuggetkey(long int *buf);
long int debugpollkey(long int *buf);
```

debuggetkey returns the ASCII value of the character typed at the keyboard. The routine waits indefinitely for the next keystroke and only returns when a key is available.

debugpollkey polls the keyboard and returns the ASCII value of the key pressed.

Due to an operating system limitation, the **ESC** key cannot be polled on Windows® NT®, 2000 or XP; however, the control left square brace key combination (^ t) causes **debugpollkey** to register **ESC**.

16.4 File access

The application can read and write host files. A file is opened using **debugopen**, which takes the standard ANSI file mode string, as used by **fopen**, to define the type of file. The function returns the integer file descriptor, which must be used for subsequent reads, writes and to close the file. If an error occurs then **-1** is returned. The following code opens an output file named **testout.dat** in binary mode:

```
#define ERROR -1

long int outfile;

outfile = debugopen("testout.dat", "wb");
if (outfile == ERROR) debugmessage ("Can't open testout.dat");
```

Similarly, the following code opens an input file named **testin.dat**:

```
#define ERROR -1

long int infile;

infile = debugopen("testin.dat", "r");
if (infile == ERROR) debugmessage ("Can't open testin.dat");
```

An output file can be written to using **debugwrite**. This takes the number of bytes to be written as a parameter and returns the actual number of bytes written, or **0** if there is an error.

```
char *test_string = "Hello World";
long int num_bytes_written;
num_bytes_written =
    debugwrite(outfile, teststring, strlen(teststring));
if (num_bytes_written == 0)
    debugmessage ("Error writing to testout.dat");
```

An input file can be read into a buffer using **debugread**. **debugread** takes the number of bytes to be read as a parameter, though fewer bytes may be read if an EOF is found. It returns the actual number of bytes read, or **0** if there are no bytes left to read or if there is an error.

```
#define BUFFER_SIZE 100

long int num_bytes_read, end_of_file;
char buffer[BUFFER_SIZE];

num_bytes_read = debugread(infile, buffer, BUFFER_SIZE);
if (num_bytes_read < BUFFER_SIZE) end_of_file = TRUE;
```

Files can be randomly accessed using **debugtell**, and **debugseek**, for example:

```
long int fpos = debugtell(infile);
debugseek(infile, dist, 0);
debugread(infile, buffer, BUFFER_SIZE);
debugseek(infile, fpos, 0);
```

Outstanding file operations can be flushed using **debugflush**, and the size of a file can be returned using **debugfilesize**.

Files are closed using **debugclose**:

```
debugclose(outfile);
debugclose(infile);
```

16.5 Controlling the debugger

The application can instruct the debugger to execute a command or a command procedure by using **debugcommand**. The result of the command, if any, is returned as a parameter, and the function returns whether there was an error.

```
long int result, error;  
error = debugcommand("display -r all", &result);  
if (error) debugmessage ("Could not display\n");
```

The debugger can be halted by **debugexit**.

If the application is loaded by the debugger in batch mode then **debugexit** causes the debugger to exit back to the host environment with the return value given by **status**.

If the application is loaded by the debugger in interactive mode then the function displays a warning message stating that **debugexit** is not supported in interactive mode. Execution of the application then continues as if **debugexit** had returned.

If the application is running on a simulator which has no debugger connected then **debugexit** causes the simulator to exit back to the host environment with the return value given by **status**.

If the application is running on a target processor which is not connected to a debugger then **debugexit** performs no action and simply returns, allowing the application to continue execution.

16.6 Debugging function definitions

debugbreak

Cause a breakpoint

Syntax

```
#include <debug.h>

void debugbreak( void );
```

Arguments

None

Results

None

Errors

None

Description

Cause a user breakpoint to occur. The behavior is as if a breakpoint had been set in the body of the **debugbreak** function. The debugger graphical interface backtraces and locates from here if it can; see the *ST20 Embedded Toolset User Manual*, chapter *Debugging with ST Visual Develop*.

When an application is being run in batch mode, executing **debugbreak** switches **st20run** to command mode. If the debugger is not connected to the target this function has no effect.

debugclose

Close a file on the host

Syntax

```
#include <debug.h>

long int debugclose( const long int fd );
```

Arguments

const long int fd The file descriptor of the file to be closed

Results

Returns 0 if the close succeeded; other return results are errors.

Errors

The function will fail to execute if another debug function is already executing (for example from another concurrent thread of execution) and a mutual exclusion function pointer has not been registered with the debug library. In this case the function returns **DEBUG_FN_REENTERED**. If the debugger is not connected to the target, the function will fail and return **DEBUG_NOT_CONNECTED**. The value -1 is returned for any other error.

Description

The file given by the file descriptor **fd** is closed on the host. **fd** should have been returned by a previous call to **debugopen**.

Example

```
char *teststring = "Hello World";
long int fd, numwritten;
fd = debugopen("test.dat", "w");
numwritten = debugwrite(fd, teststring, strlen(teststring));
debugclose(fd);
```

debugcommand

Invoke a debugger command

Syntax

```
#include <debug.h>

long int debugcommand( const char *cmd,
                      long int *result );
```

Arguments

const char *cmd	A string containing the debugger command to be invoked
long int *result	A pointer to a location where the result yielded by the command is stored

Results

Returns **0** if the command succeeded; other values are errors.

Errors

The function will fail to execute if another debug function is already executing (for example from another concurrent thread of execution) and a mutual exclusion function pointer has not been registered with the debug library. In this case the function returns

DEBUG_FN_REENTERED. If the debugger is not connected to the target, the function will fail and return **DEBUG_NOT_CONNECTED**.

If the command could not be executed for any other reason then the function returns **1** and the contents of the location pointed to by **result** are undefined.

This function makes use of memory on the host, via the debugger, if there is no free memory available then this function may cause a fatal error in the debugger.

Description

The null-terminated string pointed to by **cmd** is interpreted by the debugger as a command string. The command string is echoed to the screen and then executed. If the command yields a value then this value is stored in the location pointed to by **result**, otherwise the contents of this location are undefined.

Example

```
long int result;
if (debugcommand("display -r all", &result))
    /* error */;
```


debugconnected

Determine debugger status

Syntax

```
#include <debug.h>

int debugconnected( void );
```

Arguments

None

Results

Returns 1 if the debugger is connected, 0 otherwise.

Errors

None

Description

The function returns an integer indicating whether the debugger is currently connected.

Example

```
if (debugconnected()) {
    /* do something */
}
```

debugdisconnect

Disconnect debugger

Syntax

```
#include <debug.h>

long int debugdisconnect( void );
```

Arguments

None

Results

Return result is for errors.

Errors

The function will fail to execute if another debug function is already executing (for example from another concurrent thread of execution) and a mutual exclusion function pointer has not been registered with the debug library. In this case the function returns **DEBUG_FN_REENTERED**. If the debugger is not connected to the target, the function will fail and return **DEBUG_NOT_CONNECTED**.

Description

The function disconnects a debugger if one is attached, otherwise it does nothing.

Example

```
debugdisconnect();
```

debugexit

Cause the debugger to exit

Syntax

```
#include <debug.h>

long int debugexit( const long int status );
```

Arguments

const long int status A value to be returned to the host environment when the debugger or simulator terminates

Results

Return result is for errors.

Errors

The function will fail to execute if another debug function is already executing (for example from another concurrent thread of execution) and a mutual exclusion function pointer has not been registered with the debug library. In this case the function returns **DEBUG_FN_REENTERED**. If the debugger is not connected to the target, the function will fail and return **DEBUG_NOT_CONNECTED**.

Description

The behavior of this function depends on the manner in which the application which calls it is being executed.

If the application is loaded by the debugger in batch mode then **debugexit** causes the debugger to exit back to the host environment with the return value given by **status**.

If the application is loaded by the debugger in interactive mode then the function displays a warning message stating that **debugexit** is not supported in interactive mode. Execution of the application then continues as if **debugexit** had returned.

If the application is running on a simulator which has no debugger connected then **debugexit** causes the simulator to exit back to the host environment with the return value given by **status**.

Example

```
debugexit(42);
```

debugfilesize

Return size of file in bytes

Syntax

```
#include <debug.h>

long int debugfilesize( const long int fd );
```

Arguments

const long int fd The file descriptor of the file to be sized

Results

Returns the size of the file. Negative values are errors.

Errors

The function will fail to execute if another debug function is already executing (for example from another concurrent thread of execution) and a mutual exclusion function pointer has not been registered with the debug library. In this case the function returns **DEBUG_FN_REENTERED**. If the debugger is not connected to the target, the function will fail and return **DEBUG_NOT_CONNECTED**. The value returned for any other error is **-1**.

Description

The size of the file given by the file descriptor **fd** is returned. **fd** should have been returned by a previous call to **debugopen**.

debugflush

Flush outstanding file operations

Syntax

```
#include <debug.h>

long int debugflush( const long int fd );
```

Arguments

const long int fd The file descriptor of the file to be flushed

Results

Returns **0** on success. Other values are errors.

Errors

The function will fail to execute if another debug function is already executing (for example from another concurrent thread of execution) and a mutual exclusion function pointer has not been registered with the debug library. In this case the function returns **DEBUG_FN_REENTERED**. If the debugger is not connected to the target, the function will fail and return **DEBUG_NOT_CONNECTED**. The value returned for any other error is **-1**.

Description

The file given by the file descriptor **fd** is flushed on the host. **fd** should have been returned by a previous call to **debugopen**.

debuggetenv

Get value of host environment variable

Synopsis

```
#include <debug.h>

long int debuggetenv( const char *name,
                     void *buf,
                     const size_t bufsize );
```

Arguments

const char *name	The name of the environment variable
void *buf	A pointer to a buffer for the variable value
const size_t bufsize	The size of buffer

Results

Returns the length of the string (including null) on success. Other values are errors.

Errors

The function will fail to execute if another debug function is already executing (for example from another concurrent thread of execution) and a mutual exclusion function pointer has not been registered with the debug library. In this case the function returns **DEBUG_FN_REENTERED**. If the debugger is not connected to the target, the function will fail and return **DEBUG_NOT_CONNECTED**. The value returned for any other error is **-1**.

Description

The value of the given environment variable is written into the buffer as a null terminated string. The value is truncated to the size of the buffer.

debuggetkey

Read a character from the keyboard

Synopsis

```
#include <debug.h>

long int debuggetkey( long int *buf );
```

Arguments

long int *buf A pointer to a location where the input key will be stored

Results

Returns **1** on success. Other values are errors.

Errors

The function will fail to execute if another debug function is already executing (for example from another concurrent thread of execution) and a mutual exclusion function pointer has not been registered with the debug library. In this case the function returns **DEBUG_FN_REENTERED**. If the debugger is not connected to the target, the function will fail and return **DEBUG_NOT_CONNECTED**. The value returned for any other error is **-1**.

Description

debuggetkey stores the ASCII value of the character typed at the keyboard in the location pointed to by **buf**. The routine waits indefinitely for the next keystroke and only returns when a key is available. The effect on any buffered data in the standard input stream is host-defined.

Example

```
long int key;
long int numkeys;

numkeys = debuggetkey(&key);
```

See also

[*debugpollkey*](#)

debuggets

Read a string from the keyboard

Synopsis

```
#include <debug.h>

long int debuggets( void *buf,
                   const size_t size );
```

Arguments

void *buf	A pointer to the buffer where the string will be stored
const size_t size	The length of the buffer, that is, the maximum number of bytes to be read

Results

Returns the number of bytes read into the buffer. The value returned includes 1 byte for the null character which terminates the string. Other values are errors.

Errors

The function will fail to execute if another debug function is already executing (for example from another concurrent thread of execution) and a mutual exclusion function pointer has not been registered with the debug library. In this case the function returns **DEBUG_FN_REENTERED**. If the debugger is not connected to the target, the function will fail and return **DEBUG_NOT_CONNECTED**. The value returned for any other error is 0.

This function makes use of memory on the host, via the debugger or simulator, if there is no free memory available then this function may cause a fatal error in the debugger or simulator.

Description

A string of bytes, up to a maximum length of *size*, is read from the keyboard into the buffer pointed to by *buf*. Input is terminated by typing a newline character. The newline character does not form part of the string read in. The string has a null character appended to it by the function so space for this must be available in the buffer.

Example

```
char buf[100];
long int numbytes;
numbytes = debuggets(buf, 100);
```


debuginformsenabled

Determine informs status

Synopsis

```
#include <debug.h>

int debuginformsenabled( void );
```

Arguments

None

Results

Returns **1** if informs are enabled, **0** otherwise.

Errors

None

Description

Returns the informs status. An inform is an asynchronous communication from the target to the host. This communication results from a call to a function such as **debugmessage**, **debugread** and **printf**.

If informs are enabled the function returns **1** otherwise it returns **0**. Informs are disabled when the debugger is disconnected and when the **informs -disable** command has been issued to the host.

Example

```
if (debuginformsenabled()) {
    /* do something */
}
```

debugmessage

Write a string to the screen

Synopsis

```
#include <debug.h>

long int debugmessage( const char *msg );
```

Arguments

const char *msg A pointer to a null-terminated character string

Results

Returns 0 if the write succeeded. Other values are errors.

Errors

The function will fail to execute if another debug function is already executing (for example from another concurrent thread of execution) and a mutual exclusion function pointer has not been registered with the debug library. In this case the function returns **DEBUG_FN_REENTERED**. If the debugger is not connected to the target, the function will fail and return **DEBUG_NOT_CONNECTED**. The value returned for any other error is -1.

This function makes use of memory on the host, via the debugger or simulator, if there is no free memory available then this function may cause a fatal error in the debugger or simulator.

Description

The null-terminated string pointed to by **msg** is displayed on the screen.

Example

```
debugmessage("Hello World\n");
```

debugopen

Open a file

Synopsis

```
#include <debug.h>

long int debugopen( const char *filename,
                   const char *mode );
```

Arguments

const char *filename	The name of the file to be opened
const char *mode	The mode in which the file is to be opened. This argument follows the specification of the mode string accepted by the ANSI fopen function

Results

Returns the file descriptor on success. Negative values are errors.

Errors

The function will fail to execute if another debug function is already executing (for example from another concurrent thread of execution) and a mutual exclusion function pointer has not been registered with the debug library. In this case the function returns **DEBUG_FN_REENTERED**. If the debugger is not connected to the target, the function will fail and return **DEBUG_NOT_CONNECTED**. The value returned for any other error is **-1**.

This function makes use of memory on the host, via the debugger or simulator, if there is no free memory available then this function may cause a fatal error in the debugger or simulator.

Description

The file given by **filename** is opened on the host in the mode given by the **mode** string. The mode string follows the format used by the ANSI **fopen** function.

Example

```
char *teststring = "Hello World";
long int fd, numwritten;
if ((fd = debugopen("test.dat", "w"))>=0) {
    numwritten = debugwrite(fd, teststring, strlen(teststring));
    debugclose(fd);
}
```

See also

[debugclose](#), [debugread](#), [debugseek](#)

debugpollkey

Get a character from the keyboard

Synopsis

```
#include <debug.h>

long int debugpollkey( long int *buf );
```

Arguments

long int *buf A pointer to a location where the input key will be stored

Results

Returns **0** if no key was available, **1** if a key was available; other values are errors.

Errors

The function will fail to execute if another debug function is already executing (for example from another concurrent thread of execution) and a mutual exclusion function pointer has not been registered with the debug library. In this case the function returns **DEBUG_FN_REENTERED**. If the debugger is not connected to the target, the function will fail and return **DEBUG_NOT_CONNECTED**. The value returned for any other error is **-1**.

Description

debugpollkey gets a single character from the keyboard and stores it in the location pointed to by *buf*. If no keystroke is available the routine returns immediately with **0**. The effect on any buffered data in the standard input stream is host-defined.

Due to an operating system limitation, the **ESC** key cannot be polled on Windows® NT®, 2000 or XP; however, the control left square brace key combination (**^ [**) causes **debugpollkey** to register **ESC**.

Example

```
long int key;
long int numkeys;

numkeys = debugpollkey(&key);
```

See also

[*debuggetkey*](#)

debugread

Read bytes from a file

Synopsis

```
#include <debug.h>

long int debugread( const long int fd,
                   void *p,
                   const size_t size );
```

Arguments

const long int fd	The file descriptor of the file to read from
void *p	A pointer to the buffer where the bytes will be stored
const size_t size	The length of the buffer, that is the maximum number of bytes to be read

Results

Returns the number of bytes read into the buffer. Other values are errors.

Errors

The function will fail to execute if another debug function is already executing (for example from another concurrent thread of execution) and a mutual exclusion function pointer has not been registered with the debug library. In this case the function returns **DEBUG_FN_REENTERED**. If the debugger is not connected to the target, the function will fail and return **DEBUG_NOT_CONNECTED**. The value returned for any other error is **0**.

This function makes use of memory on the host, via the debugger or simulator, if there is no free memory available then this function may cause a fatal error in the debugger or simulator.

Description

A string of bytes, up to a maximum length of *size*, is read from the file given by the file descriptor *fd* into the buffer pointed to by *p*. Fewer than *size* bytes may be read if EOF is encountered while reading from the file. *fd* should have been returned by a previous call to **debugopen**.

Example

```
long int fd, numread; char buff[100];
fd = debugopen("test.dat", "r");
numread = debugread(fd, buff, 100);
debugclose(fd);
```

See also

[*debugclose*](#), [*debugopen*](#), [*debugseek*](#)

debugseek

Reposition file-pointer of a file

Synopsis

```
#include <debug.h>

long int debugseek( const long int fd,
                   const long int offset,
                   const long int whence );
```

Arguments

<code>const long int <i>fd</i></code>	The file descriptor of the file to seek
<code>const long int <i>offset</i></code>	The number of bytes to move
<code>const long int <i>whence</i></code>	Where to seek from

Results

Returns 0 on success. Other values are errors.

Errors

The function will fail to execute if another debug function is already executing, for example from another concurrent thread of execution and a mutual exclusion function pointer has not been registered with the debug library. In this case the function returns

DEBUG_FN_REENTERED. If the debugger is not connected to the target, the function will fail and return **DEBUG_NOT_CONNECTED**. The value returned for any other error is -1.

Description

The file-pointer of the file given by the file descriptor *fd*, which should have been returned by a previous call to **debugopen**, is repositioned:

If *whence* is:

- 0, the file-pointer is moved to *offset* bytes,
- 1, the file-pointer is moved to the current position plus *offset*,
- 2, the file-pointer is moved to the end of file plus *offset*.

debugsetmutexfn

Set mutual exclusion function of debug library

Synopsis

```
#include <debug.h>

typedef enum {
    debugmutexop_lock,
    debugmutexop_unlock,
    debugmutexop_loop
} debugmutexop;

typedef long int (*debugmutexfn)(debugmutexop, void*);
#pragma ST_nolink(debugmutexfn)
void debugsetmutexfn(debugmutexfn mfn, void* data);
```

Arguments

debugmutexfn mfn	Pointer to a mutual exclusion function
void* data	Pointer to user data

Description

The function pointer is registered with the debug library to provide mutual exclusion. Subsequently when a debug library function is called, the function pointer will be called with the **debugmutexop_lock** parameter and the user data pointer. While the debug library waits for the operation to complete, the function pointer is called repeatedly with the **debugmutexop_loop** parameter and the user data pointer. When the debug library function has completed, it calls the function pointer with the **debugmutexop_unlock** parameter and the user data pointer.

Note: The user debug mutex function is not passed a static link (gsb), so the user has to take steps to supply the gsb via the data parameter. The OS20 source file that implements debug concurrency support is supplied as part of this release, in the file:

src/os20/clibprot/debprot.c

debugtell

Return file-pointer position

Synopsis

```
#include <debug.h>

long int debugtell( const long int fd );
```

Arguments

const long int fd The file descriptor of the file to tell

Results

Returns the offset of the file-pointer relative to the beginning of the file. Negative values are errors.

Errors

The function will fail to execute if another debug function is already executing (for example from another concurrent thread of execution) and a mutual exclusion function pointer has not been registered with the debug library. In this case the function returns

DEBUG_FN_REENTERED. If the debugger is not connected to the target, the function will fail and return **DEBUG_NOT_CONNECTED**. The value returned for any other error is **-1**.

This function makes use of memory on the host, via the debugger or simulator. If there is no free memory available then this function may cause a fatal error in the debugger or simulator.

Description

The position of the file-pointer of the file given by the file descriptor **fd**, which should have been returned by a previous call to **debugopen**, is returned.

debugtime

Return elapsed time

Synopsis

```
#include <debug.h>

long int debugtime( void );
```

Arguments

None

Results

Returns the number of seconds since midnight, January 1, 1970. Negative values are errors.

Errors

The function will fail to execute if another debug function is already executing (for example from another concurrent thread of execution) and a mutual exclusion function pointer has not been registered with the debug library. In this case the function returns **DEBUG_FN_REENTERED**. If the debugger is not connected to the target, the function will fail and return **DEBUG_NOT_CONNECTED**.

This function makes use of memory on the host, via the debugger or simulator. If there is no free memory available then this function may cause a fatal error in the debugger or simulator.

Description

Returns the number of seconds since midnight, 1 January 1970.

debugwrite

Write bytes to a file

Synopsis

```
#include <debug.h>

long int debugwrite( const long int fd,
                    const void *p,
                    const size_t size );
```

Arguments

const long int fd The file descriptor of the file to write to

const void *p A pointer to the buffer where the bytes to be written are found

const size_t size The number of bytes to write

Results

Returns the number of bytes written to the file. Other values are errors.

Errors

The function will fail to execute if another debug function is already executing (for example from another concurrent thread of execution) and a mutual exclusion function pointer has not been registered with the debug library. In this case the function returns

DEBUG_FN_REENTERED. If the debugger is not connected to the target, the function will fail and return **DEBUG_NOT_CONNECTED**. The value returned for any other error is 0.

This function makes use of memory on the host, via the debugger or simulator. If there is no free memory available then this function may cause a fatal error in the debugger or simulator

Description

A string of *size* bytes is written from the buffer pointed to by *p*, to the file given by the file descriptor *fd*, which should have been returned by a previous call to **debugopen**.

Example

```
char *teststring = "Hello World";
long int fd, numwritten;
fd = debugopen("test.dat", "w");
numwritten = debugwrite(fd, teststring, strlen(teststring));
debugclose(fd);
```

See also

debugclose, debugopen, debugread



Device-independent I/O

17

17.1 Introduction

The ST20 device-independent I/O system is designed to provide a uniform interface to devices of different types. These devices typically fall into one of the following three categories:

- character based devices, such as serial ports,
- block devices, for example disk drives,
- file systems, which can be either on a local device such as a disk drive, or a remote host.

However, these differences are hidden behind a common set of application program interfaces (APIs). This makes it possible to change the device which provides a particular piece of functionality, without affecting the application code which uses it. For example, a file may be read from the host file system during development, and then transferred to a local disk for the final product, without having to change any of the code which accesses that file.

The ST20 run-time libraries provide three different APIs for accessing devices.

- **ANSI style I/O**

Examples include `printf()` and `fopen()`. This is a high level interface, which is provided by virtually all C implementations, and provides a rich set of functionality. In particular all file access can be buffered, to improve efficiency.

- **POSIX (or UNIX) style I/O**

Examples include `write()` and `open()`. This interface is lower level, providing a simple set of functions for accessing devices, in which virtually all operations are passed onto the device directly. Its main advantage is that it provides a standard interface to devices of all types.

- **Device specific functions**

Examples include `debugwrite()` and `debugopen()`. These functions are specific to a particular device, and applications using these APIs need to be modified if the device is

changed, or the application ported to a different system. Calling these functions directly is more efficient in terms of memory if it allows the ANSI and POSIX layers to be eliminated. Using device specific functions may cause the application to execute a little faster if ANSI buffering is not required. These APIs are layered on top of each other, as shown in [Figure 32](#).

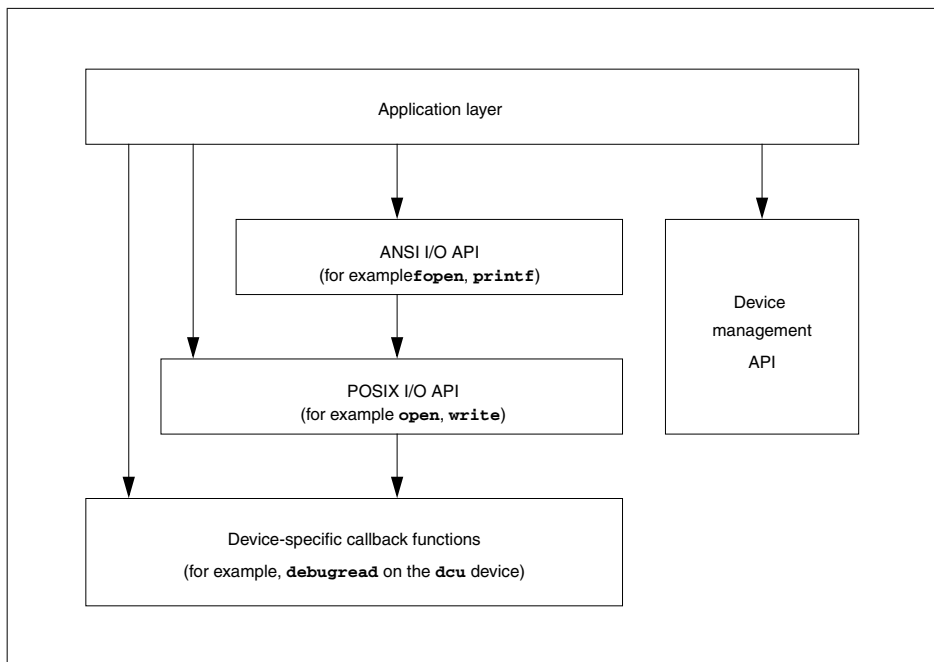


Figure 32: Device independent I/O layering

The device management API provides functions for registering and manipulating new devices, and is described in [Section 17.11](#).

Normally an application program only needs to use the ANSI or POSIX style interfaces to access a device.

The user may provide additional devices by writing a device driver. A device driver is a collection of functions which is called by the device-independent I/O functions, and which implements the specified operations on the device.

17.2 Filenames and devices

All devices in the system are accessed through named files, which are simple character strings. These filenames have two components:

- a device name, which specifies which device driver is called to operate on that device,
- a qualifier, which can be used by the device driver to identify a file on that device.

These two components are separated by a colon, for example:

- **disk:/a/b/x.c** specifies the file **/a/b/x.c** on the disk device.
- **uart:1** uses the UART device driver, and refers to the second device managed by that driver.

Omitting one or other (but not both) of the component of the filename is possible:

- **LED:** refers to a device which does not have any qualifiers (the colon must still be specified).
- **/fred/jim/john.txt** refers to a file on the default device (see [Section 17.4](#)).

17.3 Open file identification

Once a file has been opened, it needs to be identified to subsequent operations.

- Open files at the ANSI layer are identified by a **FILE** structure.
- Open files at the POSIX layer are identified by an integer value known as a **file descriptor**. This value starts at 0.
- Device drivers may also define their own identifiers.

These values are allocated by the appropriate file open functions, and returned to the user. They are passed as parameters to other functions from the same API.

In general, a single open file should only be accessed using one technique. For example, if a file is opened using POSIX **open()** then it should be read using POSIX **read()**. Mixing access techniques may cause data to be lost as it can confuse buffering systems.

Standard files

When the C run-time system starts up, three standard files are automatically opened. These can be used at either the ANSI or POSIX levels.

- At the ANSI level three **FILE** pointers exist: **stdin** (standard input), **stdout** (standard output) and **stderr** (standard error output).
- These correspond to the three POSIX level file handles 0, 1 and 2 respectively.

17.4 The default device

The C libraries provide a default device called **dcu**. By the time **main()** is called by the C startup code, this default device is already set up. All subsequent I/O operations are routed to the host via the **dcu** device unless a device qualifier is specified (as described in [Section 17.2](#)). For example, **fopen()** opens files on the disk of the host and **printf()** prints strings on the host console window. In this way, if the user has no interest in local device I/O, all operations are performed on the host, as was the case in releases prior to R1.9.3.

17.5 POSIX functions

The POSIX functions fall into a number of categories (the first two are provided by virtually all device drivers, the remainder are generally only implemented by file systems):

- basic file operations: **open()**, **close()**, **read()** and **write()**.
- device specific operations which are not covered by the generic API, handled through the **ioctl()** function,

The arguments to this function are interpreted by the device driver in whatever way it sees fit, and can be used to arbitrarily extend the API. For example, setting a baud rate only makes sense to a serial port, not a disk drive.

- file name related functions, such as **rename()** to rename an existing file or directory, **remove()** which deletes a file, **mkdir()** and **rmdir()** which create and delete a directory,
- status reporting functions, **stat()** and **fstat()**, both of which report the same information about a file or directory, but which operate on filename or a file handle respectively,
- the **lseek()** function which enables a file read/write pointer to be moved,
- the **fsync()** function which enables buffers to be flushed,
This is required when a device has buffered information before writing it to the final device.
- a set of functions which allow directories to be read in the same way as files:
opendir(), **readdir()** and **closedir()**.

All of these functions result in calls to the underlying implementation in the device driver. Not all device drivers provide all functions. An attempt to call a function which is not implemented by the device driver will fail, and set **errno** to **ENOENT** (in the case of **open()**) or **EBADF** (for all other functions).

17.6 Examples

There are three examples of device-independent I/O shipped with the toolset which can be found in the toolset installation directory **examples**:

- input/output to a buffer in the target memory (**examples/memio**),
- input/output to a UART (**examples/uartio**),
- input/output to a very simple file system contained within a file on the host's disk (**examples/minifs**).

17.7 Managing devices

A new device is defined by the user by providing a device driver. This consists of a set of callback functions which carry out fundamental operations on the new device, for example, opening the device or writing to the device. The device driver is then registered with the I/O system, specifying a unique name for the device. It is this name which is specified in the filename when a file is opened on that device.

The following sections describe how a device is registered and used from within an application program.

17.7.1 Registering a device

A new device is registered using the **device_register()** function call. This takes three arguments:

- a unique logical device name,
- a device ID number (must be zero, not currently used),
- a pointer to a **dev_op_t** structure containing a set of device function pointers.

This function calls **malloc()** to allocate space for a **dev_desc_t** structure. This space should be freed by the application if the device is unregistered (see [Section 17.7.9: Unregistering a device](#)).

The function **device_register_init()** may also be called to register a device. This function does not allocate space for a **dev_desc_t** structure; instead the caller passes the address of an allocated **dev_desc_t** structure as the first argument. The **dev_desc_t** structure whose address is passed must not be deleted until the device is unregistered (see [Section 17.7.9](#)).

The **dev_op_t** structure, used when registering a device with **device_register** or **device_register_init**, contains a set of pointers. The first entry is a pointer to a **file_operations_t** structure, or **NULL** if the device does not support file operations. The next five entries are pointers to callback functions which:

- open the device,
- close the device,
- write an array of bytes to the device,
- read an array of bytes from the device,
- perform device specific control operations.

The remaining three structure entries are reserved.

If a particular operation (for example, reading from a device) is not supported, set the corresponding callback function pointer to **NULL**.

The **file_operations_t** structure which may be referenced in the first element of the **dev_op_t** structure contains an entry which contains a version field. The version field must be set to 1. This is followed by entries which are pointers to callback functions which perform file-based operations on the device:

- remove a file,
- make a directory,
- remove a directory,
- open a directory,

- read the contents of a directory,
- close a directory,
- rename a directory,
- seek - move the position of the file pointer,
- flush buffers associated with a file,
- get information on a file,
- get information on an open file.

The remaining three structure entries are reserved.

If a particular operation (for example, removing a file) is not supported, set the corresponding function pointer to **NULL**. For example, a simple device which only supports open and read operations may be registered as follows:

```
#include <dev.h>
static dev_op_t buf_IO = {
    NULL,          /* no file operations table */
    my_device_open, /* open callback */
    NULL,          /* close callback */
    NULL,          /* write callback */
    my_device_read, /* read callback */
    NULL,          /* ioctl callback */
};

dev_desct * desc = device_register("mydevice", 0, &buf_IO);
```

Note: The header file **dev.h** is included; this is the only header file required when registering a new device.

17.7.2 Opening a device

The device may be opened with either the ANSI call, for example:

```
FILE *f;
f = fopen("mydevice:", "r");
```

or the POSIX call, for example:

```
int fd;
fd = open("mydevice:", O_RDONLY);
```

Each of these function calls executes a call to the open callback function **my_device_open** with the following arguments:

```
long int r = my_device_open("", O_RDONLY);
```

This callback function performs device-specific functionality. The open callback should return **-1** if it fails to open the device or any other long integer handle on success. The value of the handle returned on success is arbitrary and device-specific and it is passed to many of the callback functions registered in the **dev_op_t** and **file_operations_t** structures for operations on an open file. This allows a filesystem to be implemented on a device.

For example, the following code:

```
int fd, fd2;
fd = open("mydevice:/mydir/myfile", O_RDONLY);
fd2 = open("mydevice:/mydir/myfile_2", O_RDONLY);
```

calls:

```
m = my_device_open("/mydir/myfile", O_RDONLY);
```



```
n = my_device_open("/mydir/myfile_2", O_RDONLY);
```

which return respectively, non-negative long integer handles **m** and **n** on success. See [Section 17.7.3](#) for details about reading from these files.

Note: The value returned by the call to **open** is **not** the same value as the handle returned from the open callback. The integer value returned by **open** is internally generated by the I/O library.

17.7.3 Reading from a device

Data may be read from the opened device with either the ANSI call, for example:

```
int size = fread(buffer, x, y, f);
```

or the POSIX call, for example:

```
int size = read(fd, buffer, 256);
```

which call:

```
my_device_read(handle, buffer, 256);
```

where **handle** is the value returned from the open callback. The value returned by the read callback is returned by **read**.

Note: When the ANSI calls are used, buffering may occur in the ANSI libraries. As a result, the relevant device callback may not get called for each call to the ANSI function such as **fread()** or **fwrite()**.

In the example used in [Section 17.7.2](#), two files were opened on a device. By extending that example, a read operation is performed on each file:

```
int fd1, fd2, size1, size2;
char buffer1[256], buffer2[256];
fd1 = open("mydevice:/mydir/myfile", O_RDONLY);
fd2 = open("mydevice:/mydir/myfile_2", O_RDONLY);
size1 = read(fd1, buffer1, 256);
size2 = read(fd2, buffer2, 256);
```

The calls to **read** invoke calls to the read callback function:

```
my_device_read(handle1, buffer1, 256);
my_device_read(handle2, buffer2, 256);
```

where **handle1** and **handle2** are the respective handles returned by **my_device_open** after successfully opening **/mydir/myfile** and **/mydir/myfile2**.

17.7.4 Writing to a device

Writing to a device is very similar to reading; the write callback function is called by a call to **write**. The value returned by **write** is the value returned by the write callback.

17.7.5 Closing a device

The close callback for the device is called when **close()** is called with an open file descriptor as an argument or when **fclose()** is called with an open **FILE*** as an argument. The argument to the close callback is the handle returned by the open callback. The value returned by **close()** is the value returned by the close callback.

17.7.6 Device specific control

The function `ioctl()` performs device-specific control operations. It calls the `ioctl` callback with a variable argument list. This callback can be handled in a device-specific manner. The value returned by `ioctl()` is the value returned by the `ioctl` callback function.

17.7.7 File operations

File operations (such as renaming a file) may be performed on devices which support file operations. The callbacks in the `file_operations_t` structure are called to perform these operations. See [Table 49](#) for POSIX calls to `file_operations_t` entry mappings. Registering of a simple device which supports only file removal is illustrated in the following code fragment:

```
#include <dev.h>

static file_operations_t file_IO = {
    0x1,          /* Table version number, must be 1 */
    my_file_remove, /* remove callback */
    NULL,        /* mkdir callback */
    NULL,        /* rmdir callback */
    NULL,        /* opendir callback */
    NULL,        /* readdir callback */
    NULL,        /* closedir callback */
    NULL,        /* rename callback */
    NULL,        /* seek callback */
    NULL,        /* flush callback */
    NULL,        /* stat callback */
    NULL,        /* fstat callback */
};

static dev_op_t buf_IO = {
    &file_IO,
    my_device_open, /* open callback */
    NULL,          /* close callback */
    NULL,          /* write callback */
    my_device_read, /* read callback */
    NULL,          /* ioctl callback */
};

device_register("mydevice", 0, &buf_IO);

...

remove("mydevice:/a/c/x.c");
```

17.7.8 Error cases

The thread-safe global symbol `errno` is set when errors occur from within the I/O libraries. If a callback function fails ensure that `errno` is set to an appropriate value. For the default `dcu` device, `errno` is set to `EIO` if an underlying `debug...()` function fails.

17.7.9 Unregistering a device

A device which is no longer in use may be unregistered by calling `device_unregister()` or `device_unregister_desc()`. The former function takes the device name as an argument and the latter takes a `dev_desc_t` pointer. Both functions remove the device from the list of registered devices.

17.7.10 Concurrency support

The POSIX I/O code can be set to callback to user supplied mutex functions, which allows the internal data structures to be protected. These callbacks are registered using the function:

```
void* device_setmutexfn(DeviceMutexCallbackFn fn, void* data);
```

fn must have the prototype:

```
void DeviceMutexCallbackFn(_ST_mutexop, void*);
```

where `_ST_mutexop` is `_ST_mutexop_lock` to lock the mutex and `_ST_mutexop_unlock` to unlock it.

Note: This lock is always released before calling into any functions in a device driver. This means that the device driver must be responsible for protecting its own data structures from concurrent access by multiple threads.

When OS20 is used, the application does not have to manage the POSIX I/O mutex; a mutex function is provided and registered with `device_setmutexfn()` by the OS20 function `kernel_initialize()`. However, the application may override this default mutex function after `kernel_initialize()` has been called. The OS20 source file that implements concurrency support for device I/O is supplied in the file:

```
src/os20/clibprot/deviceprot.c
```

To provide alternative mutual exclusion, or if using an operating system other than OS20, concurrency support may be set up manually.

Note: If setting up concurrency support for POSIX I/O functions then concurrency support must also be provided for the debug support functions if they are used. See [Section 16.2: Concurrency support](#) for more details.

17.8 Redirecting streams

It is often desirable to redirect I/O streams such as `stdin`, `stdout` and `stderr` to a new device. By the time `main()` is called, these have already been attached to the `dcu` device. However, as no data has been buffered at this point, it is safe to change which device is associated with the file descriptor which a `FILE` stream uses. A call to `dup()` allocates the numerically lowest free file descriptor, so closing a file immediately prior to calling `dup()` causes its file descriptor to be reused. The following example illustrates this property for redirecting `stdout` to a new device.

Example:

```
int fd = open("asc0:", O_RDWR);
close(1); /* close the fd used by stdout */
assert( dup(fd) == 1 );
write(1, "Hello to asc0 stdout\n");
printf("Hello to asc0 on stdout\n");
close(fd);
```

In this example, file descriptor `1` is closed and a `dup()` is performed on the arbitrary file descriptor `fd`. This makes file descriptor `1` (the first unused file descriptor) have the same properties as file descriptor `fd`. The ANSI I/O handle `stdout` is associated with file descriptor `1` throughout the above operations, causing `printf` to use the new device.

Other functions which can be used to determine and control the relationship between **FILE** streams and file handles are:

- **fileno()** which returns the file handle used by the specified **FILE** stream.
- **fdopen()** which creates a new **FILE** stream using a file handle to an already open file.

17.9 Limitations

Although the ST20 implementation of the POSIX I/O library is based on the POSIX standard ISO/IEC 9945-1: 1990 (IEEE Std. 1003.1-1990), the current implementation has a number of limitations which it is necessary to be aware of when porting code from other systems. Most of these limitations are designed to reduce the memory or CPU overheads of the system, but may cause problems if a system is expecting strict POSIX compatibility.

- In most POSIX systems, file descriptors are local to a single process, however, the ST20 does not implement processes, so file descriptors are shared by all threads. Thus a thread can access a file which it did not open.
- Care should be taken when working with duplicated file handles. The ST20 run-time library does not count how many times an open file descriptor has been duplicated using the **dup()** function. This means that after a **dup()** call there are multiple file handles referring to the same open file, but only one of these file handles needs to be closed to close all of them.
- The concept of a current working directory is not implemented by the POSIX code, although individual device drivers may choose to.

17.10 POSIX API

This section lists the API functions provided at the POSIX level. The subset of the POSIX API supported is based on ISO/IEC 9945-1: 1990 (IEEE Std. 1003.1-1990) of the POSIX standard.

The implemented POSIX functions are listed in [Table 49](#). This table indicates the callback in the `dev_op_t` structure (specified when the device was registered) which is called by each POSIX function.

Function	Callback in <code>dev_op_t</code>	Description
<code>open</code>	<code>dev_open</code>	Open a file
<code>close</code>	<code>dev_close</code>	Close a file
<code>read</code>	<code>dev_read</code>	Read from a file
<code>write</code>	<code>dev_write</code>	Write to a file
<code>ioctl</code>	<code>dev_ioctl</code>	Device specific control
<code>dup</code>		Duplicate a file descriptor
<code>remove</code>	<code>fops->fs_remove</code>	Remove a file
<code>mkdir</code>	<code>fops->fs_mkdir</code>	Create a directory
<code>rmdir</code>	<code>fops->fs_rmdir</code>	Remove a directory
<code>opendir</code>	<code>fops->fs_opendir</code>	Open a directory
<code>readdir</code>	<code>fops->fs_readdir</code>	Read a directory
<code>closedir</code>	<code>fops->fs_closedir</code>	Close a directory
<code>rename</code>	<code>fops->fs_rename</code>	Rename a file
<code>lseek</code>	<code>fops->fs_seek</code>	Seek to a position in a file
<code>fsync</code>	<code>fops->fs_flush</code>	Flush buffers associated with a file
<code>stat</code>	<code>fops->fs_stat</code>	Return information about a file
<code>fstat</code>	<code>fops->fs_fstat</code>	Return information about an open file

Table 49: POSIX API

The POSIX functions are declared in the standard POSIX header files. Full descriptions of the functions listed in [Table 49](#) can be found in any POSIX reference book.

17.11 Device API

This section describes the API functions provided at the device management level.

[Table 50](#) lists functions in the device API, which support registration and management of devices.

Function	Description
<code>device_default</code>	Return current default device
<code>device_default_set</code>	Set default device
<code>device_getmutexdata</code>	Get mutual exclusion user data for POSIX layer
<code>device_getmutexfn</code>	Get mutual exclusion callback for POSIX layer
<code>device_register</code>	Allocate memory and register device
<code>device_register_init</code>	Register a device
<code>device_setmutexfn</code>	Set mutual exclusion callback for POSIX layer
<code>device_unregister</code>	Unregister a device - name lookup
<code>device_unregister_desc</code>	Unregister a device - descriptor look-up

Table 50: Device API

The device API functions are declared in the header file `dev.h`.

device_default

Return the default device

Synopsis

```
#include <dev.h>

dev_desc_t *device_default( void );
```

Arguments

None

Results

Returns a pointer to the device descriptor structure of the default device, or **NULL** if no default device has been set.

Errors

None

Description

`device_default()` returns a pointer to the device descriptor structure of the default device.

Example

```
dev_desc_t *dev;
dev = device_default();
```

See also

[*device_default_set*](#)

device_default_set

Set the default device

Synopsis

```
#include <dev.h>

dev_desc_t* device_default_set( dev_desc_t* device );
```

Arguments

dev_desc_t* device A pointer to the device descriptor

Results

Returns a pointer to the old default device.

Errors

None

Description

device_default_set() sets the default device to the specified device.

The default device is the device used by the functions **open()** and **fopen()** when a device descriptor component has not been supplied in the filename. See section [Section 17.2: Filenames and devices](#) for a detailed description of filename components.

```
static dev_op_t logIO = {
    NULL,
    logopen,      /* open */
    logclose,     /* close */
    logwrite,     /* write */
    logread,      /* read */
    NULL,         /* ioctl */
};

static dev_desc_t logDesc;

device_register_init(&logDesc, "mydevice", 0, &buf_IO);

device_default_set(&logIO);
```

See also

[device_default](#)

device_getmutexdata

Get mutual exclusion user data for the device-independent I/O layer

Synopsis

```
#include <dev.h>

void* device_getmutexdata( void );
```

Arguments

None

Results

A **void*** representing the user data.

Errors

None

Description

Returns a **void*** representing the user data.

device_getmutexfn

Get mutual exclusion callback for the device-independent I/O layer

Synopsis

```
#include <dev.h>

void* device_getmutexfn( void );
```

Arguments

None

Results

A pointer to the callback function.

Errors

None

Description

Returns a pointer to the callback function used to implement mutual exclusion for the device-independent I/O layer.

device_register

Register a device

Synopsis

```
#include <dev.h>

dev_desc_t* device_register( const char* name,
                             dev_t dev_id,
                             dev_op_t* ops );
```

Arguments

const char* name	The logical name of the device
dev_t dev_id	The device ID (must be 0, currently unused)
dev_op_t* ops	The device operations look-up table

Results

Returns a pointer to a **dev_desc_t** on success, or **NULL** if an error occurs.

Errors

Returns **NULL** if the arguments are invalid, if the device is already registered or if no memory could be allocated to register the device.

Description

device_register registers a device with the I/O system by validating the passed arguments. **name** and **ops** should be non-NULL pointers. **name** should be a unique device name and **dev_id** must have the value 0. This function calls **malloc** to allocate space for a **dev_desc_t** structure.

Example

```
#include <dev.h>
dev_op_t asc_ops = {
    NULL,
    asc_open,
    asc_close,
    asc_write,
    asc_read,
    asc_ioctl
};
int result = device_register("asc", 0, &asc_ops);
```

See also

[*device_register_init*](#), [*device_unregister*](#), [*device_unregister_desc*](#)

device_register_init

Register a device

Synopsis

```
#include <dev.h>

int device_register( dev_desc_t* desc,
                    const char* name,
                    dev_t dev_id,
                    dev_op_t* ops );
```

Arguments

dev_desc_t* desc	A pointer to a dev_desc_t structure
const char* name	The logical name of the device
dev_t dev_id	The device ID (must be 0, currently unused)
dev_op_t* ops	The device operations look-up table

Results

Returns 0 on success, or -1 if an error occurs.

Errors

Returns -1 if the arguments are invalid or if the device is already registered.

Description

device_register_init() registers a device to the I/O system, by validating the passed arguments. **name** and **ops** should be non-NULL pointers. **name** should be unique and **dev_id** must be 0.

Example

```
#include <dev.h>
dev_op_t asc_ops = {
    NULL,
    asc_open,
    asc_close,
    asc_write,
    asc_read,
    asc_ioctl
};
static dev_desc_t asc_dev;
int result;
result = device_register_init(&asc_dev, "asc", 0, &asc_ops);
```

See also

[device_register](#), [device_unregister](#), [device_unregister_desc](#)

device_setmutexfn

Set mutual exclusion callback for the device-independent I/O layer

Synopsis

```
#include <dev.h>

void* device_setmutexfn( DeviceMutexCallbackFn fn,
                        void* data);
```

Arguments

void* fn	Callback function pointer
void* data	Data passed to the callback function

Results

The value of the currently installed callback function pointer.

Errors

None

Description

Sets the mutual exclusion callback for the device-independent I/O (POSIX) layer. The callback is called when this layer needs protecting from concurrent accesses by multiple tasks. The callback function is called with the arguments (**_ST_mutexop op, void* data**). The callback function is called prior to a task accessing the POSIX layer with the **_ST_mutexop** argument set to **_ST_mutexop_lock**.

The callback function is called after the task has accessed the POSIX layer with the **_ST_mutexop** argument set to **_ST_mutexop_unlock**. The data argument is an arbitrary **void*** available to the user. This normally points to the semaphore used to implement the locking mechanism.

device_unregister

Unregister a device

Synopsis

```
#include <dev.h>

int device_unregister( char* name );
```

Arguments

char* name A pointer to the device name

Results

Returns 0 on success, or -1 if an error occurs.

Errors

Returns -1 if **ops** is **NULL** or **ops** is not a registered device look-up table.

Description

device_unregister() attempts to unregister a device previously registered using **device_register()** or **device_register_init()**. This involves searching the device list for the appropriate device.

device_unregister() checks the file descriptor table for files which are currently open and associated with the device being unregistered. Any such files are marked as having no associated device, thus causing further I/O operations on those files to fail.

Example

```
extern dev_op_t asc_ops;
static dev_desc_t* desc = device_register("asc", 0, &asc_ops);
...
device_unregister("asc");
```

See also

[*device_register*](#), [*device_register_init*](#), [*device_unregister_desc*](#)

device_unregister_desc

Unregister a device

Synopsis

```
#include <dev.h>

int device_unregister( dev_desc_t* desc );
```

Arguments

dev_desc_t* desc A pointer to a device descriptor

Results

Returns 0 on success, or -1 if an error occurs.

Errors

Returns -1 if **ops** is **NULL** or **desc** is not a registered device lookup table.

Description

device_unregister_desc() attempts to unregister a device previously registered using **device_register()** or **device_register_init()**. This involves searching the device list for the appropriate device.

device_unregister_desc() checks the file descriptor table for files which are currently open and associated with the device being unregistered. Any such files are marked as having no associated device, thus causing further I/O operations on those files to fail.

Example

```
extern dev_op_t asc_ops;
dev_desc_t* desc = device_register("asc", 0, &asc_ops);
...
device_unregister_desc(desc);
```

See also

device_register, device_register_init, device_unregister



Relocatable code support functions 18

18.1 Introduction

`rcu.lib` is provided to support relocatable code units (RCU)s. [Chapter 10: Building and running relocatable code](#) describes how to use RCUs.

Functions to initialize/deinitialize an RCU are provided in the header file `rcu.h`; see [Table 51](#). Functions to support programs importing and exporting functions are provided in the header file `dl.h`; see [Table 52](#). The functions are defined in [Section 18.3](#).

Function	Description
<code>rcu_init</code>	Initialize an RCU
<code>rcu_deinit</code>	De-initialize an RCU

Table 51: Functions defined in `rcu.h`

`rcu.h` uses the following data structures:

```
typedef struct {
    int magic_number;           /* Hex number denotes file is an RCU */
    int codesize;               /* Size of the code section in RCU */
    int entrypointoffset;       /* Entrypoint offset into the codesection */
    int datasize;               /* Size of the data section */
    int datalocationoffset;     /* The offset in the data section where
                                the data should be loaded */
    int bsssize;                /* Size of BSS section */
    int bsslocationoffset;     /* The offset into the data section to
                                place BSS */
    int staticlinkoffset;       /* The offset into the data section that
                                the static link should be assigned to */
    int relsize;                /* Size of relocation section */
    int conssize;               /* Size of constructor section */
};
```

```

int dessize;           /* Size of destructor section */
int stacksize;         /* Size of stack this RCU needs */
int heapsize;          /* Size of heap this RCU needs */
int dbgfilename;       /* Size of .dbg filename, including the
                        null terminator '\0' */
int slot0;             /* 4 words space for user defined data */
int slot1;
int slot2;
int slot3;
} rcu_header;

```

The above structure contains the RCU header. All sizes are given in bytes.

```

typedef struct {
    unsigned char *code;      /* Address of code section */
    int *destructors;         /* Address of destructor section */
    long dbgprogramid;        /* RCU program identity to pass to debugger */
    void (*entrypoint)();     /* Address of RCU entry point */
    int staticlink;           /* Value of RCU static link */
} rcu_data;

```

This structure contains the RCU data calculated by the function `rcu_init`; see [Section 18.2: Relocatable Code Unit \(RCU\) initialization](#).

Function	Description
<code>dl_get_exports</code>	Get list of exported functions
<code>dl_get_imports</code>	Get list of imported functions
<code>dl_match_imports_to_exports</code>	Match imported functions to exported functions
<code>dl_reset_imports_from_exports</code>	Reset import list

Table 52: Functions defined in `dl.h`

`dl.h` uses the following data structures:

```

typedef struct {
    char *symbol;           /* Function name */
    int staticlink;         /* Value of function's static link */
    void (*fp)();           /* Function pointer */
} dl_sym;

```

The above structure contains a single entry in the function list.

```

typedef struct {
    int size;               /* The number of entries in the function list */
    dl_sym *list;           /* Address of the function list */
} dl_sym_list;

```

This structure holds the function list.

18.2 Relocatable Code Unit (RCU) initialization

rcu_init

Initialize an RCU

Synopsis

```
#include <rcu.h>

void rcu_init( rcu_header *header,
               unsigned char *codesection,
               unsigned char *datasection,
               int *relocations,
               int *constructors,
               int *deconstructors,
               char *dbgfilename,
               rcu_data *data );
```

Arguments

rcu_header *header	Address of RCU header structure
unsigned char *codesection	Address of code section in RCU
unsigned char *datasection	Address of data section in RCU
int *relocations	Address of relocation table in RCU
int *constructors	Address of constructor table in RCU
int *deconstructors	Address of destructor table in RCU
char *dbgfilename	Address of debug information filename
rcu_data *data	Address of data structure in which rcu_init places the values it calculates

Results

None

Errors

None

Description

The **rcu_init** function performs the following operations:

- initializes the RCU data section,
- calls the constructors,
- provides information about the RCU for the debugger to read,
- calculates the entrypoint and static link for the RCU and places them in **rcu_data *data**.

Example

```
rcu_init(rh, codesection, datasection, relocations,  
constructors, destructors, dbgfilename, &rd);
```

where:

rcu_header *rh is the header of the RCU file.

unsigned char *codesection is the address of loaded code section.

unsigned char *datasection is the address of the data section.

int *relocations is the address of the relocation table.

int *constructors is the address of the constructor table.

int *destructors is the address of the destructor table.

char *dbgfilename is the address of the debug filename.

rcu_data rd is the structure where **rcu_init** places the entry point and static link.

See also

[*rcu_deinit*](#)

rcu_deinit

De-initialize an RCU

Synopsis

```
#include <rcu.h>

void rcu_deinit( rcu_header *header,
                 rcu_data *data );
```

Arguments

rcu_header *header	Address of RCU header structure
rcu_data *data	Address of data structure that contains the values calculated by rcu_init

Results

None

Errors

None

Description

The **rcu_deinit** function performs the following operations:

- calls the destructors,
- provides information for the debugger: that the RCU should be deleted from the debugger's symbolic information.

Example

```
rcu_deinit(rh, &rd);
```

This example would de-initialize the RCU shown in the example for **rcu_init**.

See also

[*rcu_init*](#)

18.3 Import/export support

dl_match_imports_to_exports

Match imported/exported functions

Synopsis

```
#include <dl.h>

void dl_match_imports_to_exports( dl_sym_list *imports,
                                dl_sym_list *exports );
```

Arguments

dl_sym_list *imports Pointer to a list of imported functions, obtained using function **dl_get_imports**.

dl_sym_list *exports Pointer to a list of exported functions, obtained using function **dl_get_exports**.

Results

None

Errors

None

Description

The **dl_match_imports_to_exports** function updates entries in the import list with data (the static link and a function pointer) from corresponding entries in the exports list. This allows an imported symbol to be called and updated with the corresponding symbol from another program. See [Section 10.3: Support for calling functions from other programs on page 154](#) for further details.

Example

If the program **lib1** requires the use of the symbol **fred** which is not defined, this is achieved using the command:

```
import fred
```

as input to **st20cc** when building **lib1**.

The loader program can export the symbol **fred** with the command:

```
export fred
```

When the program **lib1** is loaded it requires its imported symbol list to be updated with the exported symbols from the loader program. This can be done by calling the function **dl_match_imports_to_exports** with the **lib1** import list and the loader export list:

```
dl_match_imports_to_export(lib1ImportList, loaderExportList);
```

See also

[*dl_get_exports*](#), [*dl_get_imports*](#), [*dl_reset_imports_from_exports*](#)

dl_reset_imports_from_exports

Reset imported function list

Synopsis

```
#include <dl.h>

void dl_reset_imports_from_exports( dl_sym_list *imports,
                                   dl_sym_list *exports );
```

Arguments

<i>dl_sym_list *imports</i>	A pointer to a list of imported functions, obtained using the function <i>dl_get_imports</i>
<i>dl_sym_list *exports</i>	A pointer to a list of exported functions, obtained using the function <i>dl_get_exports</i>

Results

None

Errors

None

Description

The ***dl_reset_imports_from_exports*** function clears entries in the import and export lists that relate to programs that have been unloaded. Use this function whenever a program (RCU) is unloaded. This enables any further calls to the unloaded program to return correctly. See [Section 10.3: Support for calling functions from other programs on page 154](#) for further details.

See also

[*dl_get_exports*](#), [*dl_get_imports*](#), [*dl_match_imports_to_exports*](#)

dl_get_exports

Get list of exported functions

Synopsis

```
dl_sym_list *dl_get_exports( void );
```

Arguments

None

Results

Returns a pointer to a list of exported symbols. This may be **NULL** if the program does not export any symbols.

Errors

None

Description

dl_get_exports returns a pointer to a list of exported symbols. Symbols are exported using the **export** command; see [Chapter 21: Alphabetical list of commands](#).

See also

[dl_get_imports](#), [dl_match_imports_to_exports](#), [dl_reset_imports_from_exports](#)

dl_get_imports

Get list of imported functions

Synopsis

```
dl_sym_list *dl_get_imports( void );
```

Arguments

None

Results

Returns a pointer to a list of imported symbols. This may be **NULL** if the program does not import any symbols.

Errors

None

Description

dl_get_imports returns a pointer to a list of imported symbols. Symbols are imported using the **import** command; see [Chapter 21: Alphabetical list of commands](#).

See also

[dl_get_exports](#), [dl_match_imports_to_exports](#), [dl_reset_imports_from_exports](#)



ROM Restart functions

19

The ROM restart library allows an application to restart a ROM based system as if the processor had been reset. The application's data segments are re-initialized but no EMI or user pokes are performed.

The header file for the ROM restart functions is **restart.h**; see [Table 53](#). The functions are defined in [Section 19.1](#).

Function	Description
romrestart	Restarts a ROM based system.
romlaststart	Returns the type of the previous ROM start.

Table 53: Functions defined in restart.h

19.1 ROM restart function definitions

romrestart

Restart a ROM based system

Synopsis

```
#include <restart.h>

void romrestart( void );
```

Arguments

None

Results

None

Errors

None

Description

This function restarts a ROM based system. It re-initializes data in the **CONST** and **INIT** segments, zeroes data in the **BSS** segments and copies **MVTORAM** data from ROM to RAM. It then resets the user's workspace pointer before restarting the system. (See [Section 5.2: Memory map file on page 86](#) and [Section 6.2: Default placements on page 94](#) for more information about the different types of data segment).

romrestart does **not**:

- reprogram the EMI,
- execute the link pokes,
- execute any user pokes,
- call the functions **PrePokeLoopCallback** or **PostPokeLoopCallback**; see [Chapter 9: ROM systems on page 133](#).

Before calling **romrestart**, the following steps should be performed, as appropriate.

- The current process must be moved to high priority if at low.
- Peripherals must be closed down (as shown in the example below).
- If channels are in use they must be reset.
- Additionally, application specific hardware may need to be reset.

The following example, designed for an ST20TP3 device, uses OS-Link channels, of which two are reset.

Example

```

/* Shut down operating system: */

#define MINT 0x80000000
int regs[4] ;
int pr ;

/* check priority */
__asm { ldpr ; st pr ; } ;
if (pr==1)
{
    /* change from low to high priority */
    __asm {
        ldlabeldiff there-here ;
        ldpi ;
        here: ;
        stl -1 ;
        ldip 0 ;
        runp ;
        there: ;
    } ;
}
/* clear all channels */
__asm {
    ldc MINT ;/* OS Link channel */
    resetch ;
    ldc MINT+0x10 ;/* OS Link channel */
    resetch ;
} ;

/* clear interrupt controller */
*( (volatile int*) 0x200000c0 ) = 0 ;

/* remove timers from queue */
__asm { ldab 0, MINT ; swaptimer ; } ;
__asm { ldab 1, MINT ; swaptimer ; } ;

/* remove processes from queue */
__asm { ldabc 0, MINT, MINT ; swapqueue ; } ;
__asm { ldabc 1, MINT, MINT ; swapqueue ; } ;

/* clear shadow regs */
regs[1] = 0 ;
__asm { ldabc 2, 1, regs ; ldshadow ; } ;

/* disable all traps except BP handler */
__asm { ldab 0xfffe, 0 ; trapdis ; } ;
__asm { ldab 0xfffe, 1 ; trapdis ; } ;

/* Finished shutting down OS; now restart */

romrestart();

```

See also

[romlaststart](#)

romlaststart

Return the type of the previous ROM start

Synopsis

```
#include <restart.h>

int romlaststart( void );
```

Arguments

None

Results

Returns the type of the last start:

- **warm_t** if it was a warm start,
- **cold_t** if it was a cold start.

Errors

None

Description

This function returns the type of the last start. A warm start is performed with the **romrestart()** function. All other starts are cold.

Example

```
start_type start;

start = romlaststart();
if (start==cold_t)
    printf( "A cold start\n");
else if (start==warm_t)
    printf( "A warm start\n" );
```

See also

[*romrestart*](#)



Part 4

Command language





Command language programming 20

20.1 Introduction

This chapter describes the general features of the toolset command language and its programming constructs. Toolset commands and their meanings are described in [Chapter 21: Alphabetical list of commands](#).

When building code, the command language provides:

- specification of the target and code placement,
- specification of the contents of a ROM.

When running code, the command language provides facilities to:

- load and start the code,
- control debugging, tracing and profiling,
- define complex conditional events,
- display specific parts of structured variables,
- create of scripts that enable debugging scenarios to be quickly reached,
- generate customized debugging commands.

Some tools allow commands to be entered interactively at a prompt. Where a graphical interface is available with a **Command Console** window, commands may be typed directly into the **Command Console** window.

A command language script can be written and saved in a file, called a command file. A script may be used as a macro for customizing the command language or executed as a start-up routine. A command file can be written as ASCII text using a normal text editor and then executed on demand or included in another script using the **include** command.

include filename

Command filenames, by convention take the suffix **.cfg** and must not duplicate any of the default configuration filenames supplied in the **stdcfg** release directory.

A script may also be executed automatically on starting up any of the tools that support the command language, as described in the *ST20 Embedded Toolset User Manual, Introduction*.

A script may define procedures. Once a procedure has been defined it may be executed as a command. Procedures are described in [Section 20.9](#).

The debugger command language supports several programming constructs to enable users to create customized functionality. The syntax and semantics are similar to a subset of C.

20.2 Language format

The command language is not free format. New lines may be inserted only after a command, construct, an opening brace (`{`) or closing brace (`}`) or after the procedure name in a procedure definition. A set of commands can be sequenced by enclosing the commands in braces. The end of a command is either a new-line or a semicolon.

```
{
    command1
    command2
    command3
}

and

{ command1; command2; command3 }

are equivalent.
```

20.3 Comments

Comments are introduced with two hash signs (`##`). Anything following the `##` up to the end of the line is ignored by the command interpreter.

20.4 Operators

All “C-like” operators on variables are supported, including:

- relational operators: `==` (is equal to), `!=` (is not equal to), `>=`, `<`, `<=`,
- arithmetic operators: `+`, `-`, `*`, `/`, `%` (remainder), `>>`, `<<`,
- pre- and post-increment and decrement: `++`, `--`,
- logical operators: `&&` (and), `||` (or), `!` (not),
- bitwise operators: `&` (and), `|` (or), `^` (exclusive or), `~` (not),
- `+=`, `-=`, `>>=`, `<<=`, `&=`, `!=`, `^=`, `*=`, `\=` and `%=`.

Operands must be separated from the operator by spaces. Some operators may also be used with string variables.

- `+` String concatenation - simply works like `strcat()`.
- `-` Removes the second string from the first, if present; if the second string is not a substring in the first, the first is left unchanged.
- Relational operators: `==` (is equal to), `!=` (is not equal to), `>=`, `<`, `<=`.
- `<<`, `>>` These take a string and an integer, for example, `"hello world" << 5`, and return a specified number of characters from the head/tail of the string, for example `"hello world" << 5` returns `"hello"`, `"glove" >> 4` returns `"love"`. If more characters are asked for than are present, the whole string is returned.
- `/`, `%` These chop the tail or head of a string off respectively, given a substring at which to cut, for example, `"hello world" / "wor"` returns `"hello "`, `"chop.me" % "."` returns `"me"`. The substring is always part of the string removed, so is never in the resulting string. If the substring is empty, the string is returned unchanged, for example, `"foobar" / ""` returns `"foobar"`.
- `~`, `!` These flip and lower the case of alphabetic characters in strings respectively, for example, `~"hElLo WoRlD"` returns `"HeLlO wOrLd"`, and `!"Lower Me"` returns `"lower me"`.
- `*` This takes an input string and a regular expression string, and returns any match of the regular expression in the input, for example, `"hello world" * "e.*r"` returns `"ello wor"`. Operators supported are: `[^~] ^$ *?+ . () | \.`
- `&` This selects characters from one string based on non-space characters in another, for example, `"1234567890" & "x x x x x"` returns `"1 3 5 7 9 "`. Any leading spaces are trimmed off the result, so `"lament" & "eeee"` returns `"amen"`.
- `|` This copies characters from one string into spaces in another, for example, `"1 3 5 7 9" | " 2 4 6 8"` returns `"123456789"`, and `"one one" | "++++++"` returns `"one+one"`.
- Compound versions of all binary string operators also work, that is `+=`, `-=`, `>>=`, `<<=`, `/=`, `%=`, `*=` do the same as above but with assignment.

20.5 Command language objects

20.5.1 Literal integers

Literal integers may be written in several formats:

- decimal,
- hexadecimal, prefixed by **0x**, for example, **0x80000140**,
- hexadecimal offset from the most negative integer (**0x80000000**), prefixed by **0m**, for example, **0m140**, which means **0x80000140**,
- octal, prefixed by **0**,
- binary, prefixed by **0b**.

20.5.2 Arguments

The arguments of a command or procedure consist of all the text following the command or procedure name up to the next statement terminator. This may be a comma or a new-line or a right bracket (") or "}") corresponding to a left bracket **before** the function call.

This can produce surprising results for the programmer used to C syntax. For example:

```
> proc addtwo { $$= $1+2}
> x= addtwo(8) + 7
> write (x)
10
```

Here the "+ 7" is treated as an extra unused argument to **addtwo**. This can be avoided as follows:

```
> proc addtwo { $$= $1+2}
> x= (addtwo(8)) + 7
> write (x)
17
```

Arguments to commands and procedures are normally passed as strings. A string may be enclosed in quotation marks ("), and must be enclosed in quotation marks if it contains spaces. Inside quoted strings, the back slash character (\) is interpreted as the escape character, as in C quoted strings. Thus "\n" is a new line character, "\t" is a tab character, "\\" is the back slash character, and "\33" is the character with ASCII code octal 33.

If an argument is enclosed in brackets (), the command language attempts to evaluate it as an expression yielding an integer, before passing it as a argument. If the parameters are omitted, the expression is passed as a string.

For example:

```
> x = 3
> write x
x
> write (x)
3
```

The command language also accepts comma-separated lists of arguments within brackets, analogous to the normal C function calling syntax. In this case each argument is treated as if it were enclosed in its own set of brackets.

For example:

```
>write (4) (3) "string" (2) (1)
4 3 string 2 1
>write (4, 3, "string", 2, 1)
4 3 string 2 1
>
```

20.5.3 Filenames

Filenames may be given as arguments to some commands. A simple filename with no path may be given for files in the current directory for the tool. The current directory for the tool is defined using the `cd` command language command, and may be queried by the `pwd` command.

A full or relative pathname for the file may be given. Inside quoted strings, the back slash character (\) is interpreted as the C escape character. For this reason, MS-DOS/Windows-style pathnames must not be enclosed in quotes (").

20.5.4 Variables

The command language supports three kinds of variables - scalars, structures and arrays. A variable name consists of a sequence of alphanumeric characters starting with an alphabetic character.

A variable is created and its type decided when it is assigned. Local variables may be defined and initialized in a procedure definition, or in a "when" statement.

There are a number of special variables whose values have a particular meaning to **st20run**. They are pre-defined and set to 0. See [Section 20.7](#) for a full listing and description of these variables.

All non-special variables may be removed using the `remove` command.

Scalars

There are two types of scalar variables: strings and integers.

An integer variable can be assigned any expression yielding an integer, for example:

```
x = 4
y = 16 + x
```

String variables are assigned with a quoted string or an expression yielding a string. For example:

```
x = "hello world"
y = "hello" + "world"
```

When it is required to output '#' in a string then the second '#' must be preceded by a backslash '\'. For example:

```
write "one ## two"
```

This example outputs:

```
one ## two
```

If the backslash '\' is not used the '##' is parsed as a comment.

Scalars can change their type freely between string and integer, simply by assigning them an expression of the other type. For example:

```
> x = 1048576
> y = "foobar"
> x = y
> write (y)
foobar
```

To remove a scalar variable, simply use the command **remove name**, for example:

```
remove x
```

Structures

The command language supports structures of variables using C-like syntax. An element of a structure is denoted by the name of the structure followed by a point (.) followed by the name of the element, for example, **date.day**.

An element of a structure is defined simply by assigning to it, for example:

```
pokeable.value = 5
```

Element names can be any valid variable name.

Structure elements can be deleted using the command **remove full_element_name**. For example:

```
remove my.elem
```

Entire structures can be deleted using **remove structure_name**, for example:

```
remove my
```

A structure's elements and their values can be listed with the **write** command, using the syntax **write structure**.

Note: The order in which the elements are listed is undefined.

Example

```
> mystruct.member1 = "the quick"
> mystruct.member2 = "brown fox"
> write (mystruct)
member1=the quick
member2=brown fox
```

Arrays

The command language supports arrays of variables using C-like syntax. Any expression that evaluates to a string or integer can be used as a subscript, for example **var[i+1]**, **var["foo"]**. Space is allocated for an element of the array when that element is first assigned to.

Individual array elements can be removed using **remove element_name**. For example:

```
remove foo["bar"]
remove foo[1234]
```

The entire array can be deleted with the syntax **remove array_name[]**. The trailing square brackets are optional. For example:

```
remove entirearray
remove entirearray[]
```

An array's elements and their values can be listed just like those of a structure, using **write arrayname**. For example:

```
> myarray[0] = 123
> myarray["foo"] = 234
> write (myarray)
foo=234
0=123
```

Multi-dimensional arrays

Command language arrays can contain other arrays, resulting in a variable that functions just like a C multi-dimensional array. As with all variables, the elements are created when they are assigned to. For example:

```
foo[0][0] = 42
foo[0][1] = 24
```

It should be noted that a sub-array is just another element within the parent array.

This can be seen more easily if the **write** command is used on a multi-dimensional array:

```
> bar[0][0] = 10
> bar[0][1] = 20
> write (bar)
0=<array>
```

This means an array can contain a mixture of arrays and scalar variables. This allows code such as:

```
> foo[0] = 1
> foo[1][0] = 9991
> foo[1][1] = 9992
> foo[2] = 2
> write (foo)
0=1
1=<array>
2=2
```

```
> write (foo[1])
0=9991
1=9992
```

Array literals

An array literal is a set of values within square brackets, for example [7, 5, 3, 1], and can be used in any place that a normal array can be used. The position of a value determines its index, so element 1 in the array literal [5, 4, 3] is 4. For example:

```
> myarray = [6, 23, 41, 7]
> write (myarray[1])
23
```

Elements of an array literal may optionally be named by prefixing the value with a string expression and the "=>" operator. For example:

```
> myarray = ["foo" => 99, "bar" => 100]
> write (myarray)
foo=99
bar=100
```

Named and unnamed elements may be mixed in the same array literal, for example:

```
[1, 2, "foo" => 3, 4].
```

Array literals can also contain array literals, just as arrays can contain arrays. For example:

```
> myarray = [1, 2, 3, [4, 5, 6], 7]
> write (myarray)
0=1
1=2
2=3
3=<array>
4=7
```

20.5.5 Type conversion

Expressions are typed checked at run time. For example:

```
x = "hello world" + 3
```

results in an error. To convert to a string or an integer the built in commands **mkstr** and **mknum** are provided. For example:

```
n=3
s="3"
x = "5" + (mkstr (n))
y = 5 + (mknum (s))
```

would assign the variable **x** to be the string value "53" and **y** to the integer value 8.

20.6 Objects in the application program

There are several types of item in the application program that need to be specified as arguments to the commands, namely statements, programs, tasks, frames, symbols, variables and addresses. Each of these items and the syntax used to denote the item is defined in this section.

20.6.1 Specifying a statement

A statement reference is an identification of a unique statement in the source text. A statement reference consists of a file specification, a list of line numbers and, for C programs, an optional statement position within the line, all separated by spaces. A statement reference is enclosed by chevrons (<>).

```
<file_specification line_number_list [statement_position]>
```

The **line_number_list** is a list of one or more line numbers separated by commas (,). The first line number is a line number within the file given by **file_specification**. All but the last line number are the line numbers of **#include** statements, each giving the file to which the following line number refers. The **statement_position** is zero for the first statement on the line, one for the second statement and so on.

For example, in [Figure 33](#), **<app.c 13>** specifies line number 13 of the file **app.c**, which states **x[0] = 76;**. Similarly, **<app.c 16 2>** specifies the third statement (statement 2) of line 16, that is the statement **x=7**.

Line 12 of **app.c** is a **#include** statement referring to the file **icode.h**. If **icode.h** contains the code shown in [Figure 34](#). Then **<app.c 12**, means the file **icode.h**, so **<app.c 12,3 1>** specifies the second statement (statement 1) on line 3 of **icode.h**, that is the statement **x=45**.

20.6.2 Specifying a symbol

A symbol is a name of variable or a function. A symbol may be specified by a symbol reference, with the following syntax:

```
symbol_name [ : module [: function : occurrence ]]
```

The module, function and version parameters are only used when accessing static variables. They specify the module name, function name and occurrence number in that function respectively.

For example:

```
>break x                ## global variable that is visible in all matches
>break x:app.c          ## static variable that is visible throughout app.c
>break x:app.c:fn:0      ## static variable in function fn in app.c
                        ## see line 8 in Figure 33.
>break x:app.c:fn:1      ## another static variable in function fn in app.c
                        ## see line 11 in Figure 33.
```

When the debugger searches the symbol table for the specified symbol, if it finds more than one match, that is, the symbol entered is not unique, then the command generates an error. No processing takes place and a list of all matches found in the symbol table are displayed together with a version number for each one. The command may then be reentered, specifying either the complete symbol or the original string can be entered with the identifying version number.

Example

```
>addressof func          ## try to return the address for func
addressof error-incomplete symbol
    func\1 => func(char)
    func\2 => func(int,int,char*)
>addressof func\1        ## return the address for funcname(char)
```

Alternatively:

```
> addressof "func(char)" ## note use of "" to stop command line evaluation
```

In this example the **addressof** command generates an error and lists all the function declarations starting with "**func**". The example shows two alternative methods of specifying the required function to the **addressof** command.

```
7 void fn() {
8     int static x[40];
9     x[0] = 43;
10    {
11        static int x[20];
12        #include "incode.h"
13        x[0] = 76;
14        {
15            static int x;
16            x = 3; x = 5; x = 7; x = 8;
17        }
18    }
19    {
20        static int x;
21        x = 87;
22    }
23 }
24 }
```

Figure 33: Part of file app.c

```
1 {
2     int x;
3     x = 24; x=45; x=98;
4 }
```

Figure 34: File incode.h

20.6.3 Specifying an address

An address is an integer, and can be specified in any of the formats given in [Section 20.5.1](#).

20.6.4 Specifying a register

The **register** command enables:

- a symbolic name of a register to be associated with an address,
- defined registers/ register groups to be listed,
- a register to be added to a CPU register group.

A list of register names for the ST20-C1 is given in [Table 62: ST20-C1 core registers on page 420](#) and for the ST20-C2 in [Table 63](#). These are documented for the **register** command in [Chapter 21: Alphabetical list of commands](#).

20.7 Command language special variables and scope

Some commands can be applied globally or to a part of an application. It is necessary to define the command *scope*, that is the part of the application to which it applies. For this purpose, an application may be divided into address spaces, divided into programs, each of which may be divided into tasks, each of which may contain frames. Specifying a program, task or frame is described in [Section 20.6: Objects in the application program](#).

The default program identifier, task identifier and frame identifier are held in the command language given in [Table 54: Special command language variables](#). Some commands allow the default program, task or frame identifier to be overridden by an argument if required.

Where an application consists of a single program, the program identifier is always 0. Similarly, where a program consists of a single task, the task identifier is always 0. Since 0 is the initial value of any command language variable, where there is only a single program, task or frame, the corresponding identifier need not be explicitly specified.

Variable name	Meaning
exitval	The value returned by exit
programid	The default program identifier
spaceid	The default address space identifier
taskid	The default task identifier
reason	The value passed back by the trigger

Table 54: Special command language variables

[Table 55](#) lists the event identifiers available with the toolset.

Identifier	Meaning
exit	The event identifier of the breakpoint on the program exit
tracefull	The trace buffer is full
triggered	The event identifier of the triggered event
stopped	The eventid returned when a stop command or button press completes
userbreak	The eventid returned when a debugbreak completes

Table 55: Event identifiers

The special variables and event identifiers cannot be removed.



20.8 Command language constructs

20.8.1 Conditionals

A conditional command can be achieved using an **if** construct. The **if** construct can take an **else**.

```
if (condition) command [else command]
```

The condition is true if its value is non-zero. If a sequence of commands is used then the command in the **if** or **else** construct can start on a new line:

```
if (condition) {
    command
    command
} else {
    command
    command
}
```

20.8.2 Loops

Loops can be achieved using a **while** or a **for** construct.

```
while (condition) command
```

The **while** construct executes the *command* until the *condition* is zero.

```
for (command1; condition; command2) command3
```

This **for** loop executes *command3* until *condition* is zero. *command1* is executed once before the condition is tested for the first time and *command2* is executed before the condition is tested but after *command3* is executed each time around the loop.

20.9 Procedures

Procedures can be defined to enable new commands to be created. The syntax for defining a procedure is:

```
proc name local_variables_list { command_list }
```

This defines a procedure called *name* which is equivalent to *command_list*.

local_variables_list is a sequence of variable declarations that declare variables that are local to the procedure. A global variable of the same name as a local variable cannot be accessed locally. Variable declarations in the list are separated by semicolons (;) and each declaration consists of the variable name followed by "=" followed by an integer initial value. For example:

```
proc foobar
    x = 3; y = 2 {
    command1
    command2
}
```

declares **x** and **y** as local variables in the procedure **foobar**.

20.9.1 Procedure names

It is important to choose a unique name for user procedures, and not to use a name used by the toolset, for example, a command language command name or a procedure supplied with the toolset. Most procedure and variable names exported by the toolset use the prefix “_ST” and so this should be avoided. Use the command **clsymbol** to provide a list of reserved names which must not be used, for example:

```
clsymbol -procs
clsymbol -vars
```

20.9.2 Arguments and returned values

Arguments can be passed to a procedure, and are referenced using **\$1**, **\$2** and so on. If arguments are passed, they are always passed by value, that is a unique copy is made for the procedure to use, that exists only in the scope of the procedure.

The number of arguments given to the procedure is available in the pseudo-variable **\$#**. The list of arguments can be referenced using the pseudo-variable **\$***. An argument may be specified using the value of a variable; for example if the variable **i** had the value 4, the expression **\$(i)** would be equivalent to **\$4**.

When an argument is used in an operation that requires an integer (for example, **x=\$4**) the argument is considered to be an integer expression and is evaluated when it is referenced. It is an error to use an argument in an integer operation when the value of the argument is not an integer expression.

The type of an argument is evaluated when it is referenced. Run-time errors will occur if an argument turns out to be invalid in its context. For example, if a procedure is executed that contains the line:

```
x = $4 + 20
```

and **\$4** evaluates to a string, the message:

```
error-illegal operation on type(s): (string) + (int)
```

will be displayed, and execution of the procedure will stop.

Any kind of value can be returned from a procedure by assigning to the pseudo-variable **\$\$**.

20.9.3 Invoking procedures

A procedure is invoked by the name of the procedure followed by its actual arguments, as follows:

```
name actualArgument1 actualArgument2 ...
```

Procedures may be invoked recursively.

Using such calls, a built-in command, such as **break**, can be used in a command language program where a procedure would be used.

Each actual parameter to a procedure is a string. If a string with spaces or tabs is to be passed to a procedure then the string must be enclosed by double quotes ("). If a string is to be evaluated before being passed to a procedure then that string should be bracketed. For example, the value of the variable **k** may be passed to a procedure called **enter** as follows:

```
enter (k)
```

If, however, the string "**k**" is to be passed to **enter** then the command should be:

```
enter k
```

If the string "**" (k) "**" is to be passed to **enter** then the command should be:

```
enter " (k) "
```

Argument semantics are the same for invoking both procedures and commands.



Alphabetical list of commands

21

This chapter defines all the command language commands understood by the tools in this toolset. The commands are listed in alphabetical order.

A description of the structures and syntax of the command language is given in [Chapter 20: Command language programming on page 305](#). The appropriate commands for each tool are listed in functional groups in the reference chapter for the tool. The use of the debugging commands is described in the *ST20 Embedded Toolset User Manual*, chapter *st20run tool*.

#line

Define script line numbering

Command type

Miscellaneous

Tool environment

st20cc, **st20libr**, **st20run**, **st20sim**

Synopsis

```
#line linenumber [filename]
```

Arguments

linenumber Specifies the file line number

filename Specifies the filename

Description

The command **#line** is intended for use in scripts, where it may be used to define line numbering.

Example

```
#line 10  
##  
x = 345a
```

```
>Syntax error-x.in(11)
```


addhelp

Add help string to procedure

Command type

Miscellaneous

Tool environment

st20run

Synopsis

```
addhelp proc_name help_string
```

Arguments

proc_name Name of a command language procedure

help_string Help string to be associated with procedure

Description

The command **addhelp** associates the ***help_string*** with the given procedure name.

Example

```
>proc disable { events -disable $* }
>addhelp disable "disable events\n"
>help disable
  disable events
>
```

See also

[*clsymbol*](#), [*help*](#)

addressof

Return object address

Command type

Symbol handling

Tool environment

st20run, **st20cc**

Synopsis

addressof *string*

Arguments

- | | |
|----------------------|---|
| <i>string</i> | For st20cc , the string must be the name of a memory segment or a program section. For st20run the string can be either a statement reference, a C expression, a symbol, a program section, a memory segment or an address (see Section 20.6: Objects in the application program on page 312). |
| -q[uiet] | Return result for use in command language expression, but suppress display. This is the default behavior for st20cc . |

Returns

The address of the specified **expression** of the current program.

Debugger context

The **addressof** command accesses the debugger context for the default program. For C expression evaluation it can also use the task, frame and location defined by the **context** command.

Description

The command **addressof** returns the address of the specified **object** of the program. When evaluated by **st20run** the command also displays the address unless the **-q** option is supplied.

Example (st20run)

```
>addressof <hello.c 54>
0x80002050
>addressof "x[i+15]"
0x80006780
>display -i (addressof -q main)
0x80002050 ajw -181
```

Example (st20cc)

```
x = addressof DEBUG
```

See also*symbols*

alter

Alter task state

Command type

Task state

Tool environment

st20run

Synopsis

alter [*options*]

Options

- s[*tatement*]** *statement* Change stopped statement position to specified *statement*
- f[*rame*]** *frameid* Change stopped frame to specified *frameid*
- r[*egister*]** *name value* Change register *name* to given *value*

Debugger context

The **alter** command accesses the debugger context for the default task.

Description

The command **alter** changes the state of the task defined by the **context** command. Individual registers of the task can be changed with the **-r[*egister*]** option. The stopped statement position can be changed with the **-s[*tatement*]** option. The call stack can be changed using the **-f[*rame*]** option. With the current release, only stopped tasks can be changed.

Example

```
>where
  1 never at <main.c 5 0> main.lku
  0 main at <main.c 27 0> main.lku
>alter -frame 0                                ## ignore frame 1
>where
  0 main at <main.c 27 0> main.lku
>alter -statement <main.c 45>                 ## ignore code up to line 45
>alter -register breg 0xbbbbbbbb               ## set breg to know value
```

See also

task

bootdata

Specify where the boot data block should be placed

Command type

Code and data placement

Tool environment

st20cc

Synopsis

bootdata *segment_name*

Arguments

segment_name The name of the segment where the boot data block should be placed

Description

The **bootdata** command specifies where the boot data block should be placed. If a linked unit is being generated and no **bootdata** command is given then the boot data block is placed in the first RAM segment declared in the linker input. If a ROM is being generated and no **bootdata** command is given then the boot data block is placed in the first ROM segment declared in the linker input.

The boot data block is generated by the linker and contains the following information:

- heap size in bytes,
- pointer to base of heap,
- stack size in bytes,
- pointer to base of stack,
- pointer to the base of the main static data area,
- C++ specific data,
- poke data, as defined by the command language **poke** command.

The boot data block is used by the loader code and the run-time system.

Example

```
> bootdata segment
```

bootiptr

Determine the initial value of the instruction pointer (IPTR)

Command type

Boot

Tool environment

st20cc, **st20run**, **st20sim**

Synopsis

```
bootiptr iptr
```

Arguments

iptr The integer value of the initial instruction pointer

Description

The **bootiptr** command is used to determine the initial value of IPTR.

st20run and **st20sim** use this to define the starting IPTR value of the target.

st20cc uses this command to determine the address at which to locate the first instructions of the ROM loader code sequence. Behavior differs slightly between the ST20-C1 and ST20-C2 cores.

- For an ST20-C1 core, failure to use this command when creating a ROM image results in an error. The IPTR must be in a ROM segment, word aligned and be the base of a memory segment.
- For an ST20-C2 core the IPTR must specify a half-word aligned address two bytes below the top of a ROM segment.

Example

ST20-C2:

```
bootiptr 0x7ffffffe
memory ROMSEG 0x7fff0000 0x10000 ROM
```

ST20-C1:

```
bootiptr 0x70000000
memory ROMSEG 0x70000000 0x10000 ROM
```

break

Create and modify breakpoints

Command type

Event

Tool environment

st20run

Synopsis

break [*condition*] [*expression*] [*options*] [*action*] (at least one required)

Options

Condition

-b [<i>reakpoint</i>]	A code breakpoint
-h [<i>ardwarebreakpoint</i>]	A code breakpoint implemented with a diagnostic controller register
-a [<i>ccess</i>]	A data access breakpoint
-w [<i>rite</i>]	A data write breakpoint
-r [<i>ead</i>]	A data read breakpoint
-d [<i>atavalue</i>] <i>val</i>	A data watch breakpoint for value <i>val</i>
-triggerin	A trigger-in breakpoint

Expression

(see [Section 20.6: Objects in the application program on page 312](#))

<i>statement</i>	A statement reference
<i>C expression</i>	A C expression (should be quoted)
<i>symbol</i>	A symbol reference
<i>address</i>	A numeric address

Options

-m [<i>odify</i>] <i>eventid</i>	Modify an existing breakpoint rather than create a new one.
-s [<i>pace</i>] <i>spaceid</i>	Use given <i>spaceid</i> rather than that defined by the context command.
-p [<i>rogram</i>] <i>programid</i>	Use given <i>programid</i> rather than that defined by the context command.
-t [<i>ask</i>] <i>taskid</i>	Make the breakpoint task specific. Other tasks that hit the breakpoint will be restarted.

-f[<i>frame</i>] <i>frameid</i>	Make the breakpoint frame-specific. If the breakpoint is hit in another frame the task will be restarted. The defined task may be specified by the -t option or by the context command.
-c[<i>counter</i>] <i>start_value</i>	Decrement a counter on executing the breakpoint statement, and then hit the breakpoint if the counter is zero. The counter is initialized to the positive, non-zero integer <i>start_value</i> .
-o[<i>neshot</i>]	Remove the breakpoint after it is hit.
-disabled	Create the event in a disabled state.
-disableme	Disable this breakpoint when breakpoint is hit.
-scope <<i>statement</i>>	Use given statement as the scope to evaluate expression rather than the location defined by the context command.
-l[<i>length</i>] <i>len</i>	Set a range of <i>len</i> bytes over which the breakpoint can be triggered. Default for code breakpoints is one machine word, and for data breakpoints, the size of the data item.
-i[<i>invertedbreakrange</i>]	Apply to code and data watch breakpoints. Addresses accessed outside of the specified range cause a breakpoint.

Actions

-notrap	Do not call breakpoint traphandler when breakpoint is hit. Use with sequencing and trap handling options to avoid call to the traphandler ¹ .
-stop	Stop execution of the target application. This is default behavior.
-continue	Print a message when the event is matched then continues execution of the target application.
-traceon	Turn tracing on.
-traceoff	Turn tracing off.
-traceevent	When trace is configured, a breakpoint hit causes a trace record to be saved.
-triggeroutlevel	The trigger-out line is raised.
-triggeroutpulse	The trigger-out line is pulsed.
-enable <i>eventid</i>	Enable the event <i>eventid</i> .
-disable <i>eventid</i>	Disable the event <i>eventid</i> .

Returns

The event identifier of the breakpoint.

1. A **when** or **wait** command cannot be scheduled on a **-notrap** event.

Debugger context

The **break** command accesses the debugger context for the default address space, program, task, frame and location unless a **-space**, **-program**, **-task**, **-frame** or **-scope** option is supplied.

Description

The command **break** creates and modifies code, data and trigger-in breakpoints. For code and data breakpoint creation, an address, expression, statement or symbol is required as a further argument. If the breakpoint is specified as an address or a statement the default breakpoint type is code. If the breakpoint is specified as an expression or symbol that evaluates to a data section then a write data breakpoint is set by default.

If a data watch breakpoint is specified without a range, it is defaulted to a watch range (defined by **-length**) of either one word or **sizeof** variable.

The default behavior of the **-modify** option is to remove all options previously specified. Thus all options must be re-specified when modifying an event. The fundamental type of an event may not be modified. For example, it is not possible to convert a code breakpoint to a data breakpoint. Furthermore, the address, statement or symbol used to specify the condition of an event may not be changed.

Example

```
>break main                ## code breakpoint on function main for any task
>break -b mux -t 3         ## code breakpoint on function mux of task 3
>break <hello.c 43 2>
                          ## code breakpoint on statement 2 of line 43 of hello.c

>break deco -c 5 -o
                          ## stop and remove code breakpoint on 5th call of deco
>break <facs.c 12> -t 2 -f 3
                          ## code breakpoint on line 12 of facs.c on frame 3 of task 2
>break "buffer[12]"        ## data write breakpoint on buffer[12]
>break "*funcptr"          ## code breakpoint on *funcptr
>break 0x8003459           ## code breakpoint at address 0x8003459
>break -a 0x8003459        ## access breakpoint at address 0x8003459
>break -d 23 foo           ## data watch breakpoint on the variable foo
>break -w foo              ## data write breakpoint on the variable foo
>break -r -l 24 bar
                          ## data read breakpoint on the variable bar for a 24 byte range

>break the_start -traceon

## turns tracing on when the function the_start is called
```

See also

[events](#), [wait](#), [when](#)

cachecontrol

Control instruction and data caches

Command type

Execution

Tool environment

st20run

Synopsis

cachecontrol [*option*]

Options

-ddisable	Disable data cache
-denable	Enable data cache
-dflush	Flush data cache
-dinvalidate	Invalidate data cache
-idisable	Disable the instruction cache
-ienable	Enable the instruction cache
-iinvalidate	Invalidate the instruction cache
-lock	Lock the cache configuration registers

Description

The behavior of **cachecontrol** is dependent on the target chip being used.

The command **cachecontrol** controls the instruction and data caches. Only one option may be specified in each occurrence of the command.

Options are provided to enable, disable or invalidate either the data or the instruction cache or both. Invalidating marks every line as not containing valid data. The data cache can also be flushed, in this case every dirty line is forced to write back to memory. The cache configuration can also be locked; it can only be unlocked by a hardware reset.

If an option selected is not appropriate for the current target then that option has no effect.

See also

chip

cd

Change the current working directory

Command type

Miscellaneous

Tool environment

st20cc, **st20libr**, **st20run**, **st20sim**

Synopsis

cd *directory*

Arguments

directory Pathname of the new current directory

Returns

0 is returned on success, -1 is returned on error.

Description

The command **cd** changes the current working directory.

Example

```
>cd c:\example\fubar
```

See also

pwd

chip

Set target chip

Command type

Hardware configuration

Tool environment

st20run, **st20cc**, **st20sim**

Synopsis

chip *variant*

Arguments

variant The name of a chip variant.

Description

The **chip** command is used to declare core and chip specific information to the toolset. The core specific information includes:

- the processor type (ST20-C1/ST20-C2),
- details of the on-chip memory,
- the diagnostic controller unit (DCU2 or DCU3),
- the name of CPU registers.

The chip specific information includes:

- which procedure to call to perform cache control operations (see [cachecontrol on page 330](#)),
- which procedure to call to perform interrupt control operations (see [interrupts on page 387](#)),
- the name of the downloaded trap handler,
- the address and name of chip specific register groups such as MPEG.

The **chip** command removes the requirement for a **processor** command and the definition of memory segments that are internal to the processor (for example, internal memory and on-chip peripherals). It specifically does not define any external memory mapped devices, nor does it configure any memory interfaces for RAM, SDRAM or ROM.

The **chip** command declares the memory segments for the core related memory, such as memory segment INTERNAL for the on-chip SRAM. If external memory is defined using **memory** commands, it is important that these do not conflict with the use of the **chip** command. Be aware of the constraints placed on the use of the **memory** command which are documented in this chapter for the **memory** command.

[Table 56](#) lists the chip variants supported by the **chip** command.

Variant name	Description	Diagnostic controller unit
ST20C1SIM	ST20-C1 simulator	-
ST20C2SIM	ST20-C2 simulator	-
ST20DC1	ST20DC1 processor	DCU2
ST20DC2	ST20DC2 processor	DCU2
ST20GP6	ST20GP6 processor	DCU2
ST20MC2	ST20MC2 processor	DCU2
ST20TP3	ST20TP3 processor	DCU2
ST20TP4	ST20TP4 processor	DCU2
STi5500H5	STi5500 cut 1 processor	DCU2
STi5500	STi5500 cut 4 processor	DCU2
STi5505	STi5505 processor	DCU2
STi5508	STi5508 processor	DCU2
STi5510	STi5510 processor	DCU2
STi5512	STi5512 processor	DCU2
STi5514	STi5514 processor	DCU3
STi5516	STi5516 processor	DCU3
STi5517	STi5517 processor	DCU3
STi5518	STi5518 processor	DCU2
STi5519	STi5519 processor	DCU2
STi5580	STi5580 processor	DCU2
STV3500	STV3500 processor	DCU3
STV0396	STV0396 processor	DCU3

Table 56: Chip variants

The **chip** command is used by the tools for different purposes, as described in the sections below.

st20cc behavior

For **st20cc**, the **chip** command carries out the following operations:

- specifies the CPU core type,
- defines any internal memory segment of the chip, this is named **INTERNAL**,
- declares chip specific variables for the base addresses of register groups which may be used in the target program, for example:

```
extern const int EMIAddr;
```

The header files that declare the register group base addresses are provided. These files, called address header files, have names of the form **variantaddr.h** (for example, **ST20TP3addr.h**) and are supplied in the **include/chip** directory of the release. These files declare the base addresses of register groups as **extern** and the variables so declared may be used in the same way as any other variable. The address header files may be used by adding an appropriate **#include** directive to any C file. For example:

```
#include <chip/STi5512addr.h>
```

st20run behavior

For **st20run** the command **chip** specifies chip specific information including:

- the CPU core type,
- the way in which interrupts are enabled and disabled,
- the type of trap handler to use,
- the way in which the caches are flushed,
- the cache control and interrupt control procedures,
- definition of internal memory segments,
- the type of diagnostic controller,
- the symbolic definitions of the registers specific to that chip.

This means the register names, for example, **EMIConfigData0Bank0** may be used easily in the debugger from the **Registers** window or with the **registers** command in **st20run**.

Useful configuration files

All the files described here are to be found in the **stdcfg** release directory.

Configuration file	Description
cache.cfg	Procedures for handling the various cache models.
chip.cfg	All chip commands are routed through the procedure chip in this file. It also contains the processor command procedure and the basic core type procedures ST20C1CPU and ST20C2CPU .
commands.cfg	Basic (empty) versions of many command procedures which are overridden by specific tools.
commonchip.cfg	The register information which is common to many chips. This is used by many other configuration files.
interrupt.cfg	Interrupt control procedures.
stddefs.cfg	Procedures for initializing channel memory and very basic definitions of chips.
XXXaddr.cfg	Define the base address of register groups for a specific chip or group of chips. For example, STi5505addr.cfg defines addresses for the STi5505 chip . There are also more general files such as STi550xaddr.cfg defining data for several chips.
XXXchip.cfg	These define the memory configuration for the chip and actually define the registers. For example, STi5512chip.cfg defines memory and registers for the STi5512. There are also more general files such as STi550xchip.cfg containing procedures used by more specific configuration files.

Table 57: Useful configuration files

Example

chip STi5516

See also

define, interrupts, memory, processor, register, ST20 Embedded Toolset Delivery Manual, Section Referenced technology.

clerror

Create command language error

Command type

Miscellaneous

Tool environment

st20run

Synopsis

```
clerror errorstring [options]
```

Arguments

errorstring The error string to report

Options

-i[ignore] Ignore errors that occur while executing command language programs.

-s[top] When errors occur during command language execution, stop at the point of error (default behavior).

Description

The **clerror** command reports an error and causes the command language execution to terminate. The command can also be used to define the behavior of the debugger when an error occurs. The **-ignore** option enables the program to continue execution. The **-stop** option restores the default error behavior.

Example

```
clerror -ignore  
break <hello.c 49>  
write still going
```

This command language program always displays “still going” even if the break command fails for some reason.

```
if (val == 4) {  
    clerror "should never get here"  
}
```

This command language program displays “should never get here” and stops when the if statement evaluates (**val==4**) as true.

clinfo

Set command language information level

Command type

Miscellaneous

Tool environment

st20run, **st20cc**, **st20libr**, **st20sim**

Synopsis

clinfo level

Arguments

level The amount of information returned by the command language processor as it executes a command language program

Description

The command **clinfo** defines the amount of information returned by the command language processor. [Table 58](#) lists the levels supported.

Level	Information
0	none (default value)
1	on each file that is opened (included)
2	on each file that is opened (included) on each command language procedure called
3	on each file that is opened (included) on each command language procedure called on each source line that is executed

Table 58: Command language processor - information levels

Example

```
>clinfo 2
>include testproc.in
Info> Opening file "testproc.in"
Info> Calling procedure "square"
square 4 is 16
Info> Calling procedure "fubar"
(fubar 2 3) is 6
Info> Calling procedure "fubar"
```

clsymbol

List command language symbol type or symbols of a type

Command type

Miscellaneous

Tool environment

st20run, st20cc

Synopsis

```
clsymbol symbolname [options]
```

Arguments

symbolname The name of a symbol

Options

-all	Lists all command language symbols
-vars	Lists all command language variables
-procs	Lists all command language procedures
-commands	Lists all command language commands

Returns

When given a symbol name the command returns **0** if the symbol is not defined, **1** if the symbol is a variable, **2** if the symbol is a procedure and **3** if the symbol is a command.

Description

The **clsymbol** command can be used to test if a variable/procedure or command is available. It can also be used to list all symbols that are available or to produce lists of variables, procedure names and command names.

Example

```
if (!(clsymbol val)) {  
    val = 1  
}
```

This command language program tests if the variable **val** exists. If it does not exist then it creates the symbol and defines its value.

See also

[*remove*](#)

commandline

Define a customized command line option for **st20cc**

Command type

Tool configuration

Tool environment

st20cc

Synopsis

```
commandline option_name, args, help_type, help_string,  
{tool_and_opt}
```

Arguments

option_name	The name by which the option is to be known on the st20cc command line (no “-” is required).
args	Indicates whether arguments are required. args means that the option takes arguments, no_args means that no arguments are needed.
help_type	Indicates where the help information is to appear; must take one of the following values: help_all - help appears on the short help page. help_long_and_ext - help appears on the long help page.
help_string	The string to be displayed on the help page
tool_and_opt	Specifies what option should be sent to st20cc when option_name is given. The format of this option is a quoted string of the form: “ tool=option ”, where tool may be one of the following: edg_cpp is the C++ preprocessor compiler is the ANSI C compiler st20icc . linker is the toolset linker st20link . librarian is the toolset librarian st20libr . The option does not require a “-” which is added by st20cc . If two “-” are needed (as for many st20edg options) then one “-” must be given. tool_and_opt may be repeated as required.

Description

The command **commandline** enables a customized command line option to be defined for **st20cc** using options that are already supported by **st20edg**, **st20icc**, **st20link** and **st20libr**. **commandline** removes the need to use the **st20cc** options **-w0**, **-w1**, **-w3**, **-w6**.

Note: **commandline** may only be used in the command file **st20cc.cfg** in **\$ST20ROOT/stdcfg**, the directory of default definitions supplied with the toolset. If **commandline** is passed to **st20run** or **st20libr** it causes an error.

Example

```
commandline ("warn-unused", no_args, help_long_and_ext, "warn if unused  
functions are found", "compiler=UF", "linker=UF");
```

This command must be placed in the command file **st20cc.cfg** in **\$ST20ROOT/stdcfg**. When **st20cc** is invoked with the **-warn-unused** option, **st20cc** calls the compiler and linker with **-UF** options.

See also

The *ST20 Embedded Toolset User Manual*, chapter *st20cc compile/link tool*.

compare

Compare two address spaces

Command type

Memory dumping

Tool environment

st20run

Synopsis

```
compare spaceid_1 spaceid_2
```

Arguments

spaceid_1 The integer identifier of an address space

spaceid_2 The integer identifier of an address space

Description

The **compare** command compares two address spaces (memory and registers) and returns the addresses at which the two differ.

Example

```
>s1 = connect sti5516
>load prog.lku; b1=break wibble; run; wait(b1)
>s2 = connect sti5516_simm
>load prog.lku; b2=break wibble; run; wait(b2)
>compare s1 s2
address 0x40003453 different
address 0x40005679 different
```

This example loads and runs **prog.lku** up to a call to the function **wibble** both on silicon and a corresponding simulator. The **compare** command then compares the memory and registers of the silicon against the simulator.

See also

[connect](#), [copy](#), [core](#), [disconnect](#), [space](#)

connect

Connect to a target and execute its command language procedure

Command type

Target interface

Tool environment

st20run

Synopsis

```
connect target_name [-v]
```

Arguments

target_name The name of the target to be connected

Options

-v Generate progress information

Returns

The identifier of the address space of the connected target.

Debugger context

The address space context is changed to the identifier of the address space of the connected target.

Description

The command **connect** establishes a connection to **target_name** and executes the command language procedure associated with the target. The target must be a hardware processor or a simulator defined by a **target** command. Each connection creates an address space.

If there is more than one target with the specified name then **connect** tries each target until a connection can be made or there are no targets left.

The debugger is able to handle multiple connections.

Example

```
>target myboard tap "jei_soc myboard" my_connect_procedure
>s1 = connect myboard -v
```

See also

context, core, disconnect, reset, target

context

Define default values for debugger commands

Command type

Debugging

Tool environment

st20run

Synopsis

```
context -n[ew] -d[ele]te] [options]
```

Arguments

-n[ew] Push a new context on to context stack
-d[ele]te] Pop the top of the context stack

Options

-s[pace] *spaceid* Set context address space to *spaceid*
-p[rogram] *programid* Set context program to *programid*
-t[ask] *taskid* Set context task to *taskid*
-f[rame] *frameid* Set context frame to *frameid*
-l[ocation] *statement* Set context location to *statement*
-a[ddress] *addr* Set context address to *addr*

Description

The **context** command defines the default values for address space, program, task, frame, location and address that are used by other debugger commands. For example, the **display** and **print** commands use the address space and program context respectively. If the **context** command is used without any arguments or options, the current context is displayed for *spaceid*, *programid*, current *programid*, *taskid*, *frameid*, *location* and *address*.

The debugger context is organized as a stack which is manipulated using the **-n[ew]** and **-d[ele]te]** options. This stack can be used to execute a sequence of commands in a context while preserving the “global” context.

The command language variables *spaceid*, *programid* and *taskid* contain the values of the current address space, program and task context.

When a breakpoint occurs the context is updated to the address space, program and task that caused the breakpoint.

Example

```
>context -new -space 3
>display 0m0 15
... ## display memory on address space 3
>context -delete
>display 0m0 15
... ## display memory on previous address space
```

See also

[program](#), [space](#), [task](#)

copy

Copy address space

Command type

Memory dumping

Tool environment

st20run

Synopsis

copy *source_spaceid dest_spaceid*

Arguments

source_spaceid The *spaceid* of the source address space

dest_spaceid The *spaceid* of the destination address space

Description

The **copy** command copies the source address space memory and registers to the destination address space.

Example

```
>c1 = core core.in
>c2 = connect c1sim
>copy (c1, c2)      ## c1sim now has core.in memory and registers
>go
```

See also

[clsymbol](#), [compare](#), [connect](#), [core](#), [disconnect](#)

core

Create a hosted address space

Command type

Post Mortem debugging

Tool environment

st20run

Synopsis

core [*corefile*] [*options*]

Arguments

corefile The name of the core dump command language file

Options

-s [ave] Create a core dump command language file

-l [oad] Load a core dump command language file

Returns

The identifier of the hosted address space.

Debugger context

The *spaceid* from the debugger context specifies the address space the **-save** option applies to. The **-load** option updates the *spaceid* of the debugger context.

Description

The command **core** creates a hosted address space and sets the debugger context to the hosted address space.

If the **-l [oad]** option is provided, the command executes the corefile as a command language file (such a file should have been created with the **-s [ave]** option). This causes the hosted address space to have the appropriate memory and register values of the corefile. The debugger address space context is changed to the identifier of the address space of the hosted address space.

If the **-s [ave]** option is provided, the command saves the current address space memory state, processor registers and the commands to restore the state to the specified core file. The **-s [ave]** option creates two files:

- the corefile, which contains a command language script to restore the saved state to a hosted or target address spaced, including loading the **.hex** file,
- a **.hex** file, which contains the data dump.

Example

```
%st20run -t sti5516_simm
> ...                               ## debug session
>core -save coredump.in
>quit
%
% ...                               ## later
%
%st20run -d
>core -load coredump.in             ## restore the state
> ...                               ## debug sessio
%
```

This example saves the state of a simulation to a corefile and later on reloads that corefile into a hosted address space.

```
%st20run -t sti5516_simm
>include coredump.in               ## restore the state
>step                              ## continue debug session
```

This example reloads a saved corefile into a simulation and interactive debugging is carried out.

See also

[connect](#), [copy](#), [disconnect](#)

define

Define an integer constant

Command type

Symbol definition

Tool environment

st20cc

Synopsis

```
define symbol_name value
```

Arguments

<i>symbol_name</i>	The name of the symbol to be defined
<i>value</i>	The integer value of the symbol or a string containing a command language expression to be evaluated

Description

The **define** command defines a constant integer variable, ***symbol_name***, in the program. This can be used to avoid hard-coding constant values in the code of the program and to access segment and section information about the program. The value of the variable can be an integer or it can be the result of a command language expression which is supplied as a string.

Example

```
define DeviceAddress 0x20003004
define IFreeMemorySize "sizeof (INTERNAL)-sizeof (icode)"
define IFreeMemoryStart "sizeof (INTERNAL)+sizeof (icode)"
```

The above example defines three symbols corresponding to the address of a device and the start and size of unused internal memory. These variables can be accessed in the program as follows:

```
extern const int DeviceAccess;
extern const int IFreeMemorySize;
extern const int IFreeMemoryStart;
```

See also

[*addressof*](#), [*sizeused*](#)

directory

Add path to command language search space

Command type

File handling

Tool environment

st20cc, **st20libr**, **st20run**, **st20sim**

Synopsis

directory *pathname*

Arguments

pathname The pathname to be added to the search space

Description

The command **directory** adds **pathname** to the command language filename search space. The search path is used by the command language commands to find linked unit files and include files.

st20cc and **st20libr** use this command to extend the search path for command files, object files as well as for library files. For **st20cc**, **directory** has the same effect as the **st20cc** command line option **-L**.

This command cannot be used within a command procedure or within any other command language construct, for example an **if** or **while** loop.

See also

[*searchpath*](#)

disconnect

Close the current address space

Command type

Target interface

Tool environment

st20run

Synopsis

disconnect

Debugger context

The **disconnect** command accesses the debugger context for the default address space.

Description

The command **disconnect** closes the current address space. The address space can be removed with the *space* command.

Example

```
>connect miracle1
...
>disconnect
>connect miracle2
...
```

See also

connect, target

display

Return contents or register values of the target memory

Command type

Low level

Tool environment

st20run

Synopsis

```
display address [num_elements_after [num_elements_before]] [options]
display -r reg | all | cpu [options]
```

Arguments

address	The integer word address where the output will start. This is checked to be within a defined memory segment
num_elements_after	The maximum number of elements to be returned from locations at and after address , where an element is a word or as defined by the options. If not specified, then the number of elements after address defaults to 1
num_elements_before	The maximum number of elements to be returned from locations before address , where an element is a word or as defined by the options. If not specified, then the number of elements before address defaults to 0

Options

-device	Access device memory.
-r[egister] reg all cpu	Return the named register/register group, all the known registers or all the CPU registers. Possible CPU values of reg are listed in Table 62 and Table 63 on page 420 .
-r[egister] address	Return memory by executing CPU instructions.
-c[har]	Return bytes as 8-bit integers.
-s[hort]	Return half words as 16-bit integers.
-l[ong]	Return words as 32-bit integers.
-longlong	Return double words as 64-bit integers.
-f[loat]	Return words as IEEE 32-bit floating point numbers.
-double	Return double words as IEEE 64-bit floating point numbers.
-s[tatement]	Return the statements defined by the memory words treated as instruction addresses.

-a[scii]	Return bytes as ASCII characters.
-i[nstruction]	Return bytes as disassembled instructions.
-d[ecimal]	Return numbers in decimal base.
-u[nsigned]	Return numbers in unsigned decimal.
-o[ctal]	Return numbers in unsigned octal.
-b[inary]	Return numbers in binary.
-n[ullterminatedstring]	Return as a null terminated ASCII string.
-col num_columns	Specify the number of columns in the output.
-q[uiet]	Do not display the output; by default the output is displayed.
-h[ide]	Do not display symbolic information in disassembly listings; by default the information is displayed.
-noaddress -na	Return data only, not address(es).

Debugger context

The **display** command accesses the debugger context for the default address space.

Returns

The contents of the target memory as specified by options and arguments.

Description

The **display** command returns the contents of the target memory (starting from the specified **address**) or target register values. The default is to return words in hexadecimal. The output is also displayed unless the **-q** option is used. Only the memory in defined memory segments is displayed. Static **call** and **jump** instruction destinations are converted and displayed as program labels.

The address space to be used is identified by the command language variable **spaceid**.

If the **-p** option is supplied with the **register** command, then a command procedure is invoked to return the register value.

Example

```
>display (addressof -q sub1) 23 -instruction
## <foo2.c 10 0> in sub1, entry sub1
0x40000000 ajw -1
## <foo2.c 13 0> in sub1
0x40000002 ld1 3
0x40000003 ldc 0
0x40000004 gt
0x40000005 cj 0x4000000c ## <foo2.c 16 0> in sub1
## <foo2.c 14 0> in sub1
0x40000006 ld1 3
0x40000007 ld1 2
0x40000008 call 0x4000002c ## <foo3.c 7 0> in sub2, entry sub2
```


See also

peek, print, register

dump

Dump current address space to file

Command type

Low level

Tool environment

st20run

Synopsis

```
dump filename address length [options]
```

Arguments

<i>filename</i>	The name of file in which the memory will be dumped
<i>address</i>	The start address for the memory dump
<i>length</i>	The number of bytes to dump

Options

-d[evice]	Do a device memory access
-h[ex]	Dump file is hex format (default)
-s[record]	Dump file is Motorola s-record file
-b[inary]	Dump file is binary file

Debugger context

The **dump** command accesses the debugger context for the default address space.

Description

The **dump** command creates a file of the specified format (hex, binary or srecord). The file contains values that correspond to the memory of current address space, starting at the given address and ending at the given number of bytes.

Example

```
>connect sti5516
> ...
>dump ibuf.bin -binary (addressof -q ibuf) (sizeof -q ibuf)
```

This example saves the value of the global variable **ibuf** into the file **ibuf.bin**.

See also

fill

emi

Configure **stemi**

Command type

Low level

Tool environment

st20cc, **st20run**

Synopsis

emi [*options*]

Options

See [Table 10](#) to [Table 17](#) in [Chapter 2: Creating EMI configuration files on page 39](#).

Description

This command is briefly documented here for completeness, [Section 2.2.3: stemi command files on page 52](#) contains a fuller description with examples. The **emi** command is recognized by **stemi**, a tool which can be used to generate configuration values for ST devices which have an external memory interface.

emidelay

Cause a delay during EMI configuration

Command type

Low level, hardware configuration

Tool environment

st20cc

Synopsis

emidelay *time*

Arguments

time Minimum length of time to delay (in microseconds)

Description

The **emidelay** command is used to cause a delay during the EMI configuration. It causes a delay to be inserted between the previous target poke and the next target poke.

*Note: This command only has an effect when generating ROM images. When the EMI is being programmed by the debugger the time taken for the message to be transmitted between the host and the target and back is sufficiently large to make **emidelay** redundant.*

Example

emidelay 30

This command causes the ROM boot code to delay for a minimum of 30 μ s.

endorder

Define execution order for **ST_onshutdown** functions

Command type

System shutdown code

Tool environment

st20cc

Synopsis

endorder *symbol* [*priority*]

Arguments

symbol	Name of function attributed with the pragma ST_onshutdown .
priority	The integer priority of the function in the shutdown sequence. The default value is 0.

Description

The **endorder** command defines the order in which functions that have the **ST_onshutdown** attribute will be invoked.

Immediately after the system exits, it executes functions in the following order:

- functions that have not been explicitly referenced by an **endorder** command,
- the prioritized shutdown functions (the ones referenced in **endorder** commands) in order of priority, most negative first and most positive last.

To maintain backward compatibility, functions with equal priority run in reverse order of appearance of the **endorder** commands.

*Note: Calling **endorder** without a priority uses the default priority of 0. Having a priority of 0 is not the same as having no priority (through never calling **endorder**).*

There are predefined variables for the priority that allow the user to interleave shutdown code with the toolset's standard shutdown functions.

```
SHUTDOWN_END_PRIORITY = -10000
STDIO_END_PRIORITY    = -20000
```

These values ensure that those functions with default priority 0 are called after the standard ones. Be aware that at this point some of the usual tools have already been shutdown, in particular the debugger may have been disconnected.

Lower values than **STDIO_END_PRIORITY** for a function causes that function to be called before the I/O functions are closed. The actual values for the variables may be changed in future toolsets, so to achieve a lower value than **STDIO_END_PRIORITY** it would be advisable to use an expression like:

```
endorder symbol (STDIO_END_PRIORITY - 100)
```

Example

The following runs **endFn1** then **endFn2** after all toolset shutdown is complete:

```
endorder endFn2  
endorder endFn1
```

The following runs **endFn3** before any toolset shutdown is performed:

```
endorder endFn3 (STDIO_END_PRIORITY - 100)
```

See also

[*startorder*](#)

entry

Display definition statement for a variable or entry statement of a function

Command type

Symbolic debugging

Tool environment

st20run

Synopsis

entry *symbol*

Arguments

symbol A symbol or C expression whose statement reference is to be displayed. Statement references are described in [Section 20.6: Objects in the application program on page 312](#).

Debugger context

The **entry** command accesses the debugger context for the default program.

Description

The command **entry** displays the definition statement for a variable or the entry statement of a function of the current program. Statement and function references are described in [Section 20.6](#).

Example

```
>entry pickle
<hello.c 45>
>entry "x.fn"
<hello.c 3>
```

See also

[addressof](#), [statement](#)

entrypoint

Define main entry point of a linked program

Command type

Symbol handling

Tool environment

st20cc

Synopsis

entrypoint *symbol_name*

Arguments

symbol_name The symbol name of the main entry point

Description

The **entrypoint** command defines the main entry point of the program being linked, that is the point at which to start executing. This command is equivalent to the **st20cc** command line option **-e**.

Only one main entry point may be specified for a program. If there is more than one such symbol, **st20cc** warns that there is an ambiguity.

If no main entry is defined, an error is returned. For C programs the supplied linker command start-up file defines the system main entrypoint.

Example

```
> entrypoint startup
```


eval

Execute a command language procedure

Command type

Miscellaneous

Tool environment

st20cc, **st20libr**, **st20run**, **st20sim**

Synopsis

eval *procedure_name arguments*

Arguments

procedure_name The name of the command language procedure to be executed

arguments The arguments to be passed to the procedure that is invoked

Description

The **eval** command executes a named command language procedure. This command may be used to implement passing of procedure names as procedure parameters.

Example

```
>proc x { write $* }
>proc t fptr = $1 {
    for (i=0;i<3;i++) {
        eval(fptr, i)
    }
}
>t x
0
1
2
```

See also

[*parse*](#)

events

List, remove, enable or disable events

Command type

Event

Tool environment

st20run

Synopsis

events [*eventid*] [*options*]

Arguments

eventid The integer identifier of an event, as returned from the **break** command

Options

-s[atement] <i>statement</i>	Act on all events set on the statement with reference statement . Statement references are described in Section 20.6: Objects in the application program on page 312 .
-a[ll]	Act on all events
-p[rogram]	Act on all program specific events
-t[ask] <i>taskid</i>	Act on all events on the task with the specified identifier
-space <i>spaceid</i>	Act on all events on the address space with the specified identifier
-address	Set an event at a specific address
-e[nable]	Enable events
-d[isable]	Disable events
-r[emove]	Remove events

Description

The command **events** lists, removes, enables or disables events. The default action is to list events.

Example

```
>events                ## all events
breakpoint ...
>events -space 1       ## all events on address space 1
breakpoint ...
>events -t 3           ## all events on task 3
breakpoint ...
```

See also

break, forcewhen, wait, when

export

Specify the name of a function to be exported

Command type

Symbol handling

Tool environment

st20cc

Synopsis

```
export symbol_name [options]
```

Arguments

symbol_name The name of the function to be exported from the program

Options

-translate *name* Associates a `#pragma ST_translate` with ***name***

Description

The **export** command specifies the name of a function that is to be exported from the program. This command is designed to be used with the functions prototyped in **dl.lib** and implemented in **rcu.lib**. For more information see [Chapter 10: Building and running relocatable code](#) and [Chapter 18: Relocatable code support functions](#).

Example

```
export lib1fn
export lib2fn -translate "lib2fn%x"
```

See also

[import](#)

fappend

Append strings to file

Command type

Miscellaneous

Tool environment

st20cc, **st20libr**, **st20run**, **st20sim**

Synopsis

fappend *filename stringlist*

Arguments

filename	The file to append to
stringlist	A list of strings to write

Returns

No return value

Description

The command **fappend** writes a list of strings to the file **filename**. If the **fappend** command is the first file operation on the file then it is opened for appending. Subsequent **fappend** commands to **filename** append to the file and the file is only closed when an **fclose** command is called on the **filename**. Special characters such as the new line (**\n**) should be enclosed in double quotes (").

Example

```
> fappend afile "A string to a file\n"
```

See also

[*fclose*](#), [*fread*](#), [*fwrite*](#), [*write*](#)

fclose

Close a file

Command type

Miscellaneous

Tool environment

st20cc, **st20lib**, **st20run**, **st20sim**

Synopsis

fclose *filename*

Arguments

filename The file to close

Returns

0 is returned on success, -1 is returned on error.

Description

The command **fclose** closes the open file **filename**.

Example

See [fgets](#)

See also

[feof](#), [fgets](#), [fopen](#), [fputs](#), [rewind](#)

feof

Detect end of file

Command type

Miscellaneous

Tool environment

st20cc, **st20libr**, **st20run**, **st20sim**

Synopsis

feof *filename*

Arguments

filename The file previously opened for read, whose end-of-file state is to be tested

Returns

1 is returned if it is the end of the file, 0 is returned otherwise.

Description

The command **feof** tests whether a previous call to **fgets()** has encountered the end of file marker.

Example

See *fgets*

See also

fclose, *fgets*, *fopen*, *fputs*, *rewind*

fgets

Read line from file

Command type

Miscellaneous

Tool environment

st20cc, **st20libr**, **st20run**, **st20sim**

Synopsis

fgets *filename*

Arguments

filename The file previously opened to read

Returns

A line read from the file or an empty string if it is the end of file.

Description

The command **fgets** reads one line from a file that is open for read access. It returns the string read, without any terminating new line character. Successive uses of the command return successive lines from the file.

If an attempt is made to read past the end of the file, **fgets** returns an empty string. This could be confused with empty lines, so make a call to **feof()** to determine whether the end of the file has really been reached.

Example

The following code fragment reads a file into a command language array, then writes the file back, minus any lines containing the word “foobar”.

```
line_count = 0
infile = "myfile.in"

if ((fopen(infile, "r")) == -1) {
    write("could not open file", infile, "!")
    quit
}

while(!(feof(infile))) {
    lines[line_count] = fgets(infile)
    line_count++
}
fclose(infile)

fopen(infile, "w")
for(i=0; i < line_count; i++) {
    tmp = lines[i] * "foobar"
    if(tmp == "") {
```



```
        fputs(infile, lines[i] + "\n")
    }
}
```

```
fclose(infile)
```

The code uses an easy loop to put lines from a file into an array and opens and closes files with different modes.

Note: **fgets()** strips “\n” characters off the end of strings, so they have to be replaced for the **fputs()** call.

See also

[*fclose*](#), [*feof*](#), [*fopen*](#), [*fputs*](#), [*rewind*](#)

file

Specify input file

Command type

File handling

Tool environment

st20cc, **st20libr**

Synopsis

file *object_filename*

Arguments

object_filename The name of an input file

Description

The **file** command enables an input file to be specified. Any filename is acceptable.

When used as input to **st20libr**, the **file** command instructs the librarian to add the specified file to the resultant library. In this case, the file must either be a compiled object file or a library file.

Note: The **file** command cannot be used to specify source files.

Example

```
file hello.tco
file main.tco
file procs.lib
```

fill

Access debugger context for the default address space

Command type

Boot, low level

Tool environment

st20run

Synopsis

```
fill hex_filename [options]
```

Arguments

hex_filename The name of the input file to be loaded, which may be the filename of a file in the current directory or may be a full pathname.

Options

- a[ddress] addr** Load the data from the specified file starting at the address **addr**, and ignoring the addresses in the specified file.
- b[inary]** The file is a pure binary file. The **-address** option must be specified with this option.
- d[evice]** Do a device memory access.
- h[ex]** The file is a hexadecimal file. This is the default.
- s[record]** The file is a Motorola s-record file.

Debugger context

The **fill** command accesses the debugger context for the default address space.

Description

The **fill** command can be used to load a file in any of the formats supported by **st20cc**. By default, **fill** expects the file to be in hexadecimal format. If a binary file is specified, the **-address** option must be used to specify a load address. **fill** opens the specified file and writes its contents into the memory of the connected target. This can be used for a variety of purposes, such as putting test data into the target memory, loading a ROM image into the target memory or patching target application code.

Example

```
>fill testdata1.hex
>fill testdata1.hex -a 0x80004000
```

See also

dump, memset, modify, poke

fopen

Open file

Command type

Miscellaneous

Tool environment

st20cc, **st20lib**, **st20run**, **st20sim**

Synopsis

fopen *filename mode*

Arguments

filename	The file to open
mode	The mode of access required; possible values are: r for read access, w for write access, a for append access.

Returns

0 is returned on success, -1 is returned on error.

Description

The command **fopen** opens the file **filename** for reading, writing or appending.

Example

See *fgets*

See also

fclose, *feof*, *fgets*, *fputs*, *rewind*

forcewhen

Force all when clauses to be executed

Command type

Miscellaneous

Tool environment

st20run

Synopsis

forcewhen *event_id*

Arguments

event_id The event to force

Returns

Returns no value.

Description

The command **forcewhen** forces all **when** clauses associated with the supplied event to be executed. This is useful for testing scripts to ensure that **when** clauses are correct. If no **when** clauses are associated with the supplied event, nothing happens.

Example

To setup a **when** program and force it to execute:

```
> when (42) { write "when clause for event 42" }  
> forcewhen 42  
when clause for event 42
```

See also

events, wait, when

forward

Create forward reference

Command type

Symbol handling

Tool environment

st20cc

Synopsis

forward *symbol_name*

Arguments

symbol_name The name of the symbol to be referenced

Description

The **forward** command creates a forward reference to the given *symbol_name*. This allows names to be made known to the linker in advance, or forces linking of library modules that would otherwise be ignored. The purpose is to allow the inclusion of library initialization routines that might not otherwise be included.

Example

forward **fn_a**

The above example causes **fn_a** to be included in the link, whether it is needed or not.

fputs

Add string to file

Command type

Miscellaneous

Tool environment

st20cc, **st20libr**, **st20run**, **st20sim**

Synopsis

fputs *filename string*

Arguments

filename The file previously opened for writing or appending

string The string to write

Returns

No return value

Description

The command **fputs** adds the specified string to the named file. If the file was opened for writing, **fputs filename string** may overwrite existing text in that file. If the file was opened for appending, **fputs** appends the string at the end of the file. The string is written “as-is”, with no new line appended.

Example

See *fgets*

See also

fclose, *feof*, *fgets*, *fopen*, *rewind*

fread

Read file line to variable

Command type

Miscellaneous

Tool environment

st20cc, st20libr, st20run, st20sim

Synopsis

fread *filename* *variable*

Arguments

filename	The file to read
variable	The variable into which a value is read

Returns

1 is returned on success, 0 is returned on file read error or end of file.

Description

The command **fread** reads values from the file **filename** into a variable. If the **fread** command is the first file operation on the file then it is opened for reading. Subsequent **fread** commands to **filename** read the next line in the file; the file is only closed when an **fclose** command is called on the **filename**.

Example

```
ok=1
while (ok) {
    ok = fread datafile value
    if (ok) write (value)
}
```

See also

fappend, fclose, fwrite, write

fwrite

Write string(s) to file

Command type

Miscellaneous

Tool environment

st20cc, **st20libr**, **st20run**, **st20sim**

Synopsis

fwrite *filename stringlist*

Arguments

filename	The file to write to
stringlist	The list of strings to write

Returns

No return value

Description

The command **fwrite** writes a list of strings to the file **filename**. If the **fwrite** command is the first file operation on the file then it is opened for overwriting. Subsequent **fwrite** commands to **filename** append to the file; the file is only closed when an **fclose** command is called on the **filename**. Special characters such as newline (**\n**) should be enclosed in double quotes (").

Example

```
> fwrite afile "A string to a file\n"
```

See also

[*fappend*](#), [*fclose*](#), [*fread*](#), [*write*](#)

getenv

Get environmental variable value

Command type

Miscellaneous

Tool environment

st20cc, **st20libr**, **st20run**, **st20sim**

Synopsis

getenv *envname*

Arguments

envname Environment variable name

Returns

The value of an environment variable or an empty string if it does not exist.

Description

The command **getenv** returns the value of an environment variable.

Example

```
>write (getenv ST20ROOT)
\STM\ST20R1.8.1
```

go

Access the debugger context for the default address space

Command type

Boot, low level

Tool environment

st20run

Synopsis

go [*options*]

Options

-a [*ll*] All stopped targets continue execution

Debugger context

The **go** command accesses the debugger context for the default address space.

Description

The **go** command starts the target running in the currently defined register state. The register state is updated by the **modify** and **load** commands and when a breakpoint occurs.

Example

```
>connect sti5516_simm
>load prog1.lku
>load hello.lku
>go
```

This example loads two programs onto the connected address space. The **go** command starts executing at the entry point of the **hello.lku** program (because it was loaded last).

See also

stop

heap

Define the heap segment (and size)

Command type

Code and data placement

Tool environment

st20cc

Synopsis

heap *segment_name* [*length*]

Arguments

segment_name	The name of the segment where the heap is to be placed
length	The size of the heap in bytes, which must be a whole number of words, that is divisible by 4. It is also illegal to specify a size which is zero or negative (see below).

Description

The **heap** command is used to define the segment where the heap is placed, and optionally the size of the heap. The heap is the area of memory that the C library function **malloc** can use. The command takes the name of a memory segment where the heap is to be placed, plus an optional size.

The next available address within the specified segment is used as the start address, after all other sections have been placed.

The area allocated for heap also depends on whether the stack is allocated to the same memory segment. If the stack is allocated to the same memory segment as the heap then the following rules are used to determine the size of the stack and heap regions.

- If both the **heap** and **stack** commands specify **length** then they are each allocated an area of memory as specified.
- If either but only one of the **heap** or **stack** commands specify **length** then the command which specified **length** is allocated the specified area of memory, and the remaining unallocated memory is allocated to the other command.
- If both the **heap** and **stack** commands omit **length** then the unallocated memory is divided equally between them.

If the heap command is not specified then a zero sized heap is assumed.

Examples

heap segment

This example instructs **st20cc** to use the unallocated space at the end of **segment** for the heap.

heap mysegment 1024

This example directs **st20cc** to place the heap in **mysegment** and make it 1024 bytes long.

heap segment 1024

stack segment 512

This example directs **st20cc** to:

- place the heap in **segment** and make it 1024 bytes long,
- place the stack in **segment** and make it 512 bytes long.

heap segment 1024

stack segment

This example directs **st20cc** to:

- place the heap in **segment** and make it 1024 bytes long,
- place the stack in **segment** and to allocate it the remaining memory in the segment above the heap.

heap segment

stack segment

This example directs **st20cc** to divide the unallocated memory in **segment** equally between the stack and the heap.

See also

[*stack*](#)

help

Provide command help

Command type

Miscellaneous

Tool environment

st20run

Synopsis

help [*command*]

Arguments

command The name of a command which is recognized by **st20run**

Description

The **help** command returns detailed help on a particular command, for those commands which are recognized by **st20run**.

If no *command* argument is given, returns a help overview of the **st20run** commands.

Example

```
> help step
step -i -t -m -o
single step the task one source statement entering function call
-i single step a machine instruction
...
```

import

Specify imported function

Command type

Symbol handling

Tool environment

st20cc

Synopsis

```
import symbol_name [options]
```

Arguments

symbol_name The name of the function to be imported into the program

Options

-translate *name* Associates a **#pragma ST_translate** with *name*

Description

The **import** command specifies the name of a function that is to be used but that is not defined within a program. This command is designed to be used with the functions prototyped in **dl.lib** and implemented in **rcu.lib**. For more information see [Chapter 10: Building and running relocatable code](#) and [Chapter 18: Relocatable code support functions](#).

Example

```
import lib1fn
import lib2fn -translate "lib2fn%x"
```

See also

[export](#)

include

Insert command file

Command type

File handling

Tool environment

st20cc, **st20libr**, **st20run**, **st20sim**

Synopsis

include *file_name*

Arguments

file_name The name of the command file to be included. The file may be on the command language search path or a full pathname may be given.

Description

The **include** command is equivalent to inserting the contents of the command file **file_name** in place of the **include** command. The **include** command may be used in a command file or at a **st20run** command prompt. All the commands in the included command file are treated as though they appeared at the location of the **include** command and are interpreted accordingly.

If no directory is supplied, the current command language search path is searched.

This command must not be used within a command procedure or within any other command language construct, for example an **if** or **while** loop.

Example

```
include hardware.cfg  
include placement.cfg
```

See also

directory, *searchpath*

informs

Toggle debug I/O

Command type

Debug

Tool environment

st20run

Synopsis

informs [*options*]

Options

- | | |
|-----------------|----------------------------|
| -enable | Enable debug input/output |
| -disable | Disable debug input/output |

Description

The **informs** command enables the debug input/output to be turned on and off. When debug input/output is turned off, debug functions return the error code **DEBUG_NOT_CONNECTED** and standard input/output (**stdio**) functions return an error.

initstack

Specify top address of ROM loader stack

Command type

ROM file control

Tool environment

st20cc

Synopsis

initstack *address*

Arguments

address The address to be used as the top of the initial stack

Description

This command specifies the top address of an initial area of stack which is used by the ROM loader code when the application is booted. For the ST20-C2, seven words above this area are required by the bootstrap and so the **initstack** address must be seven words below the top of the available stack space.

The main stack specified using the **stack** command cannot be used initially, because it may be in external memory and the EMI is not guaranteed to be programmed until the ROM loader code has completed execution. Once the ROM loader code has completed, the stack is moved into the main stack as defined by the **stack** command. **initstack** has no effect if only a linked unit is produced.

If the **initstack** command is not given then the defaults are as follows:

- C2: address = Memstart (for example **0x80000140** for ST20MC1),
- C1: address = MostNeg (Most negative integer: **0x80000000**).

Example

```
initstack 0x80000200
```

See also

Chapter 9: ROM systems on page 133

interrupts

Specify interrupt behavior

Command type

Execution

Tool environment

st20run

Synopsis

interrupts [*options*]

Options

- | | |
|-------------------|-----------------------------------|
| -d[isable] | Disable CPU interrupts on restart |
| -e[nable] | Enable CPU interrupts on restart |
| -r[estore] | Restore CPU interrupt value |

Debugger context

The **interrupts** command accesses the debugger context for the default address space.

Description

Disable or enable or restore the existing behavior of interrupts on restarting after an event (breakpoint, step or stop). If no options are given then the default actions are listed.

See also

go, step, stepout

jtag

Manually control JTAG lines

Command type

Low level, hardware configuration

Tool environment

st20run

Synopsis

```
jtag [options]
```

Options

- on** Turn manual JTAG control on.
- off** Turn manual JTAG control off.
- set val** Set JTAG input to val and get value.
- get** Return value on JTAG lines.
- clock** Automatically clock TCK.

Description

The command **jtag** allows low level control of JTAG lines.

Example

```
jtag -on
lines = (jtag -get)    ## get value on jtag lines
lines = lines & ~(1<<2); lines = lines | (1<<2)    ## shift in value
jtag -set (lines)      ## set jtag lines
jtag -off
```

load

Load linked unit file into current address space

Command type

Boot

Tool environment

st20run

Synopsis

load *filename string*

Arguments

filename The name of the linked unit (**.lku**) file to be loaded. The file may be on the command language search path or a full pathname may be given.

string A string which is passed to **argv**.

Debugger context

The **load** command accesses the debugger context for the default address space.

Description

The **load** command loads the specified linked unit file into the current address space, and loads the debugging data, found in the associated (**.dbg**) file, into the debugger. The **load** command reads the file, places the code segments at their defined locations, and sets up the data block for the C run-time library.

If the linked unit was previously loaded then it deletes the debugging data. If the linked unit is reloaded then the debugging data and events are updated.

The **load** command updates the register state so that when the **go** command is used the application starts running.

Example

```
% st20run -d
>connect miracle1      ## connect to the target
>load hello.lku        ## load the application hello.lku
>break main            ## set break at start of program
>go                    ## run the application

>load hello.lku "one two three"
```

In this example, **argv[]** would take values as follows:

```
argv[0] == "hello.lku"
argv[1] == "one"
argv[2] == "two"
argv[3] == "three"
```

See also

go

log

Copy input commands and write to a file

Command type

Miscellaneous

Tool environment

st20run

Synopsis

log *filename* [*options*]

Arguments

filename The name of the file to which the log is to be copied. The file may be in the current directory or a pathname may be given

Options

-off Turn logging off

-on Turn logging on

-c[reate] Create a new log in **filename**. Otherwise, any log output is appended to **filename**

Description

The **log** command is used to copy input commands and writes to the named file.

Example

```
>log printrun -c -on
>print i
15
>log printrun -off
>quit
% cat printrun
15
%
```

memory

Set up memory regions

Command type

Hardware configuration

Tool environment

st20cc, **st20run**, **st20sim**

Synopsis

memory *segment_name* *lowest_address* *size* *type*

Arguments

segment_name	A unique name of up to 1024 characters for the memory segment. The name must not be a memory type (see below) or the name of program section. DEBUG memory segments must be named “ TRAPHANDLER ”.
lowest_address	The lowest address of the memory segment; this must be word-aligned
size	The size of the memory segment in bytes, expressed as a whole number of words
type	The type of memory, which may be CODE , CONST , DATA , DEBUG , DEVICE , PERIPHERAL , ROM , RAM , or RESERVED ; see Table 59 , below

Debugger context

The **memory** command accesses the debugger context for the default address space.

Description

The **memory** command is used to set up memory regions that can be referenced by subsequent commands. When used as input to **st20run**, **memory** declares a memory segment associated with the connected target (via the **connect** command).

For the **memory** command to work correctly, the following rules should be followed:

- A parent segment must be defined before any subsegment (see [Overlapping segments](#) below).
- Memory segments can be defined to exist within existing memory segments but the subsegment must be wholly contained within its parent (see [Overlapping segments](#) below).
- All the internal, chip specific, memory segments (except optional cache) are defined by a **chip** command, so should not be repeated.
- Only a memory segment of the same type can be defined within an existing memory segment. The exceptions to this are memory segments of the types **RESERVED** and **DEBUG** which may be combined with other memory types (see the examples at the end of this command definition).

Memory type	Description
CODE	This type of memory segment is intended for code; the linker generates a warning if it is used for anything else. When debugging code that is in a CODE segment the debugger uses hardware breakpoints. The simulator treats CODE segments as if they were ROM segments.
CONST	This type of memory segment is intended for const data; the linker generates a warning if it is used for anything else. CONST memory segments are read-only. The simulator treats CONST segments as if they were ROM segments.
DATA	This type of memory segment is intended for data (not code); the linker generates a warning if it is used for anything else. This memory type is assumed readable and writable by the application.
DEBUG	The user MUST NOT place any code or data in a DEBUG segment; the linker generates a warning if it is used for code or data. It is used by the debugger and or simulator for the breakpoint trap handler. This segment should be 1024 bytes long and may be placed in internal or external memory. DEBUG memory segments must be named " TRAPHANDLER ".
DEVICE	The DEVICE memory segment is provided to define the memory range of a peripheral, to which word sized accesses may be made via the DCU by the debugger. This type of memory segment is not intended for the user to place code or data in, however, in exceptional circumstances this may be done. Where code and data has been inappropriately placed, the linker returns warning messages. Accesses made by the debugger to this segment type bypass the data cache, if enabled. The simulator expects a device model to be associated with the segment.
PERIPHERAL	The PERIPHERAL memory segment defines a memory range for a peripheral and differs from DEVICE type in the following manner: The debugger uses a PERIPHERAL segment type either for making accesses which are not word sized (that is byte or short), or for accessing peripherals which are on the "peripheral bus" and cannot be "seen" via the DCU. The debugger uses the breakpoint trap handler to make accesses to this segment type so the target must be stopped when performing the access. The command display -r is used to make peripheral accesses. The register command may be used to define peripheral registers, within the PERIPHERAL segment, which may be accessed via the Registers window on the debugger GUI or via the register command. This type of memory segment is not intended for the user to place code or data in, however, in exceptional circumstances this may be done. Where code and data has been inappropriately placed, the linker returns warning messages.
RAM	This type is assumed to be readable and writable by the target program. Any code or data may be placed here. When debugging code that is in a RAM segment the debugger uses software breakpoints.
RESERVED	The RESERVED memory segment is provided to define a range of memory which is reserved. The linker generates a warning if either code or data is placed in this memory type.
ROM	This type is assumed to be read-only by the target program. Code may be executed from a ROM segment. Code and const data may be placed here for ROM images only. When debugging code that is in a ROM segment the debugger uses hardware breakpoints.

Table 59: Memory types

Overlapping segments

The memory map of a target chip is provided by memory statements in a command file. This memory map is used by **st20cc** and **st20run**. To provide finer control of section placement, a memory segment can be declared which is within another memory segment, this is called a subsegment, the enclosing segment is known as the parent segment.

A typical use for overlapping segments is to take a segment defined in a high level procedure, and divide it into smaller segments in application specific code.

For example, to define a cache segment within the internal memory segment created by the **chip** command, add the following **memory** command after the **chip** command.

```
memory DCACHE 0x80000800 0x800 RESERVED
```

This defines a subsegment which overlaps the final 2 Kbytes of the **INTERNAL** memory segment. As it is of type **RESERVED**, **st20cc** does not place any sections into this segment, and in effect reduces the size of the **INTERNAL** segment by 2 Kbytes.

A command procedure to do this called **dcache_internal()** is provided in the file **cache.cfg** in the directory **stdcfg** of the release. This can be used to remove any internal memory from the memory map, which is unavailable when the cache is enabled.

A subsegment must not exceed the top or bottom boundary of the parent segment that contains it.

A subsegment must have a memory type compatible with the parent segment. [Table 60](#) is a summary of what memory types a segment can be split into. It is arranged with the parent segment types in the columns and subsegment types in the rows. A ✓ in a box indicates a compatible type (for example, a RAM segment can have a RAM, CODE or DATA subsegment); no other combinations are permitted.

Subsegment type	Segment Type				
	RAM	ROM	CODE	DATA	CONST
RAM	✓				
ROM		✓			
CODE	✓	✓	✓		
DATA	✓			✓	
CONST		✓			✓

Table 60: Memory segment split types

All the other memory types can be a subsegment of any memory type, these are:

```
DEBUG  
RESERVED  
DEVICE  
PERIPHERAL
```

A subsegment is treated by **st20cc** and **st20run** in the same way as any other memory segment, so they can be used for example with the command language statements **place**, **addressof**, **sizeof**, **sizeused**.

Any command language references to the parent segment take into account that it may have been split by subsegments. The address of the parent segment is the first available part of the segment excluding any subsegments that are at the beginning of the segment. The parent segment size is what remains after the subsegment sizes have been deducted. Placement into the parent segment starts at the first available location in what remains of that segment after any subsegments have been removed. The placement continues through the parent segment, skipping any subsegments until the placement is complete or the end of the

parent segment is reached; see [Figure 35](#). If the parent segment is not large enough, the placement will fail.

To place individual functions in a specified order, divide the relevant memory segments into multiple subsegments in the configuration file and place the functions into these.

Examples

K = 1024

memory InternalRAM 0x80000000 (4 * K) RAM

This defines an area of memory given the name **InternalRAM** that starts at address 0x80000000 and is 4 Kbytes in length with the type RAM.

Note: In this example, the start of memory does not take into account, reserved locations.

K = 1024

memory SRAM (0x80000000 + 4 * K) (512 * K) RAM

This defines an area of memory given the name SRAM that starts at address 0x80000000 + 4 Kbytes and is 512 Kbytes in length with the type RAM.

M = (K * K)

memory DRAM 0xc0000000 (4096 * M) RAM

memory M_Control 0x20000000 0x00000004 DEVICE

memory BOOTROM 0x40000000 (256 * K) ROM

K = 1024

memory INTERNAL 0x80000000 (4*K) RAM

memory SYSTEM 0x80000000 0x140 RESERVED

This defines an area of internal memory in RAM, which is 4 Kbytes long, of which the first 0x140 bytes are reserved.

K = 1024

M = K * K

memory OUTER 0x40000000 (2*M) RAM

memory INNER (0x40000000+1024) 32 RA

This example has subsegment INNER defined within segment OUTER. The linker is able to use the first 1024 bytes of OUTER (if there is a section of code that fits there) and then uses the remainder of OUTER beyond INNER.

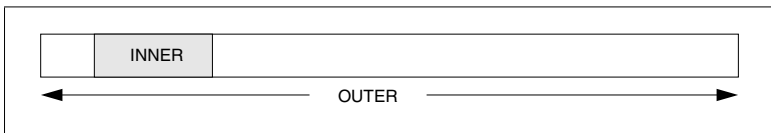


Figure 35: Example memory subsegments

See also

[addressof](#), [display](#), [sizeof](#), [sizeused](#), [register](#)

memset

Set memory range to a value

Command type

Low level

Tool environment

st20run

Synopsis

```
memset address value num_bytes [-d]
```

Arguments

<i>address</i>	The integer start address where the value is to be written
<i>value</i>	The 8-bit positive integer value to be written to each byte
<i>num_bytes</i>	The number of bytes to be set to <i>value</i>

Options

-d[evice]	Do a device memory access
------------------	---------------------------

Debugger context

The **memset** command accesses the debugger context for the default address space.

Description

The **memset** command sets ***num_bytes*** bytes of memory starting at ***address*** to the byte ***value***.

Example

```
>memset 0x80000140 0xff (1024 * 1024)
```

See also

fill, modify, poke

mkdir

Make directory

Command type

Miscellaneous

Tool environment

st20cc, **st20libr**, **st20run**, **st20sim**

Synopsis

mkdir *directory_list*

Arguments

directory_list List of directories to create

Returns

0 is returned on success, -1 is returned on error.

Description

The command **mkdir** creates the list of directories in the file store.

Example

```
>mkdir hello.1 hello.2
```

See also

rm, *rmdir*

mknum

Make into number

Command type

Miscellaneous

Tool environment

st20cc, **st20lib**, **st20run**, **st20sim**

Synopsis

mknum *arg*

Arguments

arg Can be a string or a number

Returns

mknum returns a number. If the argument is a string it converts it into a number, otherwise it returns the argument.

Description

This command and the command **mkstr** are used to convert from numbers to strings and vice-versa. They are used in command language programming because the command language does not do any implicit type conversion.

Example

```
>x = 3
>y = "1"
>z = 3+mknum(y)
>write z is (z)
z is 4
>z=y+mkstr(x)
>write z is (z)
z is 13
```

This example shows **mknum** and **mkstr** being used in the context of the addition operator. If these commands are not used then attempts to mix string and integer operands to this operator would result in a error.

See also

[*mkstr*](#)

mkstr

Make into string

Command type

Miscellaneous

Tool environment

st20cc, st20libr, st20run, st20sim

Synopsis

mkstr [*format*] *value*

Arguments

value A string or a number to be converted into a string, in the given format, or as a decimal if no format is given. If **value** is a string then it returns **value**.

Formats

-b	Binary
-c	ASCII character
-d	Signed decimal
-o	Octal
-u	Unsigned decimal
-x	Hexadecimal

Returns

mkstr returns a string. If the argument is a number it converts it into a string in the specified format (or in decimal if no format is specified). Otherwise it returns the argument.

Description

This command and the command **mknum** are used to convert from numbers to strings and vice-versa. They are used in command language programming because the command language does not do any implicit type conversion.

Example

```
>y = 10
>z = mkstr -x (y)
>write z is (z)
z is 0x0000000A
```

See also

mknum

modify

Change memory or register contents

Command type

Low level

Tool environment

st20run

Synopsis

```
modify address value [options]
```

```
modify value -r reg_name [options]
```

Arguments

address The integer address to be modified

value The integer value to be written to **address** or to the specified register

Debugger context

The **modules** command accesses the debugger context for the default address space.

Options

-r[egister] <i>reg_name</i>	Modify the register <i>reg_name</i> . Possible values of CPU registers are listed in Table 62 and Table 63 .
-r[egister] <i>address</i>	Modify the address executing CPU instructions
-d[evice]	Do a device memory access
-c[har]	Modify the byte at <i>address</i>
-s[hort]	Modify the half-word at <i>address</i>

Description

The **modify** command changes the contents of memory or a register of the current address space. If no options are given, then the address must be a word address and the **value** are written to that word. If the **-s** option is specified, then the **address** must be half-word aligned, and the **value** is written to that half-word. If the **-c** option is specified then the **value** is written to the byte at **address**. If the **-r** option is specified, the specified value is written to the named register or memory address by executing CPU instructions.

Example

```
>modify 0x8000400 0xfeedface    ## modify 0x80000400 to be 0xfeedface
>modify -r areg 0xaaaabbbb      ## modify aregister
>modify -r 0x2000 0xaaaabbbb    ## modify 0x2000 via CPU instructions
```

See also

fill, memset, poke

modules

Lists all current program modules

Command type

Symbolic debugging

Tool environment

st20run

Synopsis

modules [*statement_reference*] [*options*]

Arguments

statement_reference

List information on the module corresponding to the *statement_reference*

Options

-r[*awdebuginfo*] Return raw debug information for the module
-d[*ebuginfo*] Return debug information for the module

Debugger context

The **modules** command accesses the debugger context for the default program.

Description

The **modules** command lists all the modules of the current program. A module is a part of a compilation unit that is part of a section, hence multiple modules of the same name may be returned if a module contributes code to more than one code section. If an argument is supplied to the command the module details of the statement reference are displayed.

Example

```
>modules
hello.c
fubar.c
```

Lists all modules of the current program.

```
>modules <hello.c 2,4>
source: /toolset/st20/libs/stdio.h
...
command: st20icc ...
```

List details on the module corresponding to line 4 of the file included on line 2 of **hello.c**.

See also

[program](#), [symbols](#)

mv

Rename file

Command type

Miscellaneous

Tool environment

st20cc, st20libr, st20run, st20sim

Synopsis

```
mv old_filename new_filename
```

Arguments

old_filename Old filename

new_filename New filename

Returns

0 is returned on success, non-zero is returned on error.

Description

The command **mv** renames the ***old_filename*** to the ***new_filename***.

Example

```
>mv hello.tco hello2.tco
```

parse

Parse and execute program

Command type

Miscellaneous

Tool environment

st20run, **st20cc**, **st20link**, **st20sim**

Synopsis

parse *program*

Arguments

program The command language program to parse and execute

Returns

The return result of the executed command language program.

Description

The command **parse** takes a string argument that is parsed and executed as a command language program.

Example

```
>parse "x=15\n">write (x)15
```

See also

eval

peek

Display target memory address contents

Command type

Low level

Tool environment

st20run

Synopsis

peek *address* [*options*]

Arguments

address The byte aligned address to be read

Options

-q[uiet] Return the result without displaying

-d[evice] Do a device memory access

Debugger context

The **peek** command accesses the debugger context for the default address space.

Returns

The contents of the word of memory at **address** on the address space.

Description

The command **peek** displays the contents of the word of memory at **address** on the connected target. **address** is not checked against known memory segments. The **-device** option directs the debugger to perform a device memory access.

Example

```
>peek 0x80000140
0x80000120
>x = peek 0x80000140 -q
>peek (x)
0x80000000
```

See also

[*display*](#)

place

Place code or data section in a specified memory segment

Command type

Code and data placement

Tool environment

st20cc

Synopsis

```
place section_name segment_name [options]
```

Arguments

section_name A section to be placed in memory

segment_name The name of the segment where the section is to be placed

Options

-a[*alignment*] ***num_bytes*** Align the section being placed to be ***num_bytes*** aligned within the segment, that is, the section is placed at the next available address divisible by ***num_bytes***. ***num_bytes*** must be a multiple of four. By default, the section is word aligned.

-n[*oinit*] Do not initialize to zero during the boot process. This option is only valid when used in conjunction with a BSS section.

-s[*segmenttype*] Place the section in a segment of type ***segment_name*** as defined by the **memory** command.

Description

The **place** command is used to place a section in a specified segment. The section is loaded starting at the next available address in the memory segment. The section name can either have been specified in the source code using the **ST_section** pragma or can be a default section name provided by **st20cc**.

*Note: **st20cc** may also create certain additional sections as well as the default and user defined ones in order to implement some functionality.*

place defines the final location of code and data in an application. If code is to be loaded from ROM and executed in RAM then **place** defines its resting place in RAM. If it is necessary to specify the particular ROM segment from where the code is moved then the **store** command can be used.

section_name can also be a restricted regular expression. Legal regular expressions in this context:

- are a subset of standard UNIX regular expressions,
- must match the whole name of the actual section being placed,
- may use the following special characters only:

" () | [] ^ - . * ? + \".

To place a section name which actually contains any of these characters without doing a regular expression search, escape (backslash) the special characters (so for example "_" would become "_"). If backslashes are not used, **place** still places the section correctly, but may also place other sections that match the regular expression. Using regular expression characters in section names is therefore not recommended, as it may cause confusion.

The **-a[alignment]** option allows an alignment to be specified when placing a section into a memory segment. The **-segmenttype** option allows a section to be placed in any segment of a given type, as defined by the **memory** command. A subsequent **place** command can optionally place the section in a named segment and this overrides the previous placement. If the named segment is of a different type to that specified by the previous **place** command an error is generated.

The **-noinit** option allows the user to override the initialization of a BSS section to all-zero, which occurs at boot time. For example, it may be required to map this data to a peripheral which should not be written to at boot time.

Examples

```
place fast_ram DRAM
```

Places the section **fast_ram** at the next available slot in the DRAM memory segment.

```
place fred DRAM -a 64
```

Places the section called **fred** into the DRAM memory segment and ensures that the start of the section is aligned on a 64-byte boundary.

```
place fred_bss SEGNAME -noinit
```

Places **fred_bss** into the segment called **SEGNAME** and ensures that **fred_bss** is not initialized at boot time.

See also

[memory](#), [Chapter 6: Code and data placement on page 91](#)

poke

Write to target memory address

Command type

Low level, hardware configuration

Tool environment

st20cc, **st20run**, **st20sim**

Synopsis

```
poke address value [-d]
```

Arguments

address	The byte aligned address to be written to
value	The integer value to be written at address

Options

-d[evice]	Do a device memory access
------------------	---------------------------

Debugger context

The **poke** command accesses the debugger context for the default address space.

Description

When used with **st20run**, the command **poke** writes the **value** to the specified **address** on the connected target.

When used as input to **st20cc** during ROM image generation, the command **poke** causes the **address** and **value** to be stored in a table in the ROM image. The ROM loader code performs each poke in this table, that is writes each value to its associated address, before any code or data is moved to RAM. The loader executes the pokes in the order they are given in the configuration **.cfg** file. When used as input to **st20cc** during linked unit generation, this command has no effect.

This mechanism can be used to initialize the processor early in the boot cycle, for example it can be used to program the EMI registers. The **-device** option directs the debugger to perform a device memory access.

Example

The following example shows the **poke** command being used from **st20run**:

```
>poke 0x80000140 0xfeedface
>peek 0x80000140
0xfeedface
```

See also

fill, *memset*, *modify*

print

Interpret a C or C++ expression

Command type

Task state

Tool environment

st20run

Synopsis

```
print [taskid] "c_expr" [options]
```

Arguments

taskid	The integer identifier of the task which provides the context for c_expr
c_expr	The C expression which is to be evaluated. The expression should be written in quotation marks.

Options

-f[frame] frameid	Set the context to the frame with the specified frame identifier
-u[gly]	Do not “pretty print” structured variables
-n[ostruct]	Do not print structured variables
-x	Display in hexadecimal
-o[ctal]	Display in octal
-b[inary]	Display in binary
-a[scii]	Display as ASCII characters
-s[tring]	Interpret char* or char[] as a null terminated string
-e[lems] num_elements	Set the maximum number of array elements to num_elements
-s[cope] statement_ref	Set the scope to the code block which includes the statement specified by the reference statement_ref . Statement references are described in Section 20.6: Objects in the application program on page 312 .
-quiet	Return the result without displaying
-rtos	Display variable in an RTOS specific way. See Chapter 11: Using st20run with OS20 for a full description of this useful facility.
-v[irtual]	Display virtual function tables

Debugger context

The **print** command accesses the debugger context for the default address space, program, task, frame and location, unless a **taskid**, **-frame** or **-scope** option is supplied.

Returns

The value of **c_expr**.

Description

The command **print** interprets a C or C++ expression.

Slices of arrays are supported, as an extension of C syntax; for example, the array consisting of the elements **i** to **j** inclusive of the array **x** is written as **x[i..j]**. The following operations may be used in the expression argument of a **print** command:

- arithmetic: **+exp**, **-exp**, **exp+exp**, **exp-exp**, **exp*exp**, **exp/exp**, **exp%exp**
- bitwise: **~exp**, **exp&exp**, **exp|exp**, **exp<<exp**, **exp>>exp**
- logical: **!exp**, **exp&&exp**, **exp||exp**
- comparison: **exp>exp**, **exp<exp**, **exp>=exp**, **exp<=exp**, **exp==exp**, **exp!=exp**
- structures: **exp.field**, **exp[exp]**, **exp[exp..exp]**
- pointers: **&exp**, ***exp**, **exp->field**
- casts: **(type)exp**, **(type *)exp**, **(type[num])exp**
- miscellaneous: **exp=exp**, **(exp)**

The **print** command determines the scope of the expression using the value of the **-scope** option or the debugger context location. If a task is supplied or is defined in the debugger context an attempt is made to access variables as locals. If the access is not successful, the debugger tries to access variables as C++ member variables, if this is not successful then the debugger accesses variables as global variables.

When **print** lists the contents of a C++ class it shows the member variable and also the class to which it belongs.

The assignment operator is supported for integer and enumeration types.

Example 1

```
>print i
5
>print s->arr[0..5]           ## slice of an array
[3,4,5,6,7,8]
> print x -o                 ## in octal
010
>print x -s <hello.c 7>      ## x defined in scope of line 7
2
>x = print y -q              ## assign y to command language variable x
>write x is (x)
x is 6
```

Example 2

```
> print globalvar                                ## try to print globalvar
print error-incomplete symbol
  globalvar\1 => globalvarA
  globalvar\2 => globalvarB
  globalvar\3 => globalvarwithalongname
> print globalvar\3                              ## print globalvarwithalongname
42
```

C++ Examples

```
>print ::globalvar                                ## print global variable
>print *this                                       ## print class variables in member function
>print this->fruit::name                          ## print the name member of the fruit class
>print this->apple::name[3]                       ## print the 4th character of the name string
                                                ## in the class apple

>print a                                           ##print the instance of a which is of type class apple
{
  fruit::name
  fruit::colour
  apple::name
}
>print -v this->__vptr
delta: 0xffd8 call: 0x80000178 apple::identify(void)
delta: 0xffd8 call: 0x800001b0 fruit::getcolour(void)
```

The call field shows which function is actually called, and the delta field shows by how much the pointer will be adjusted. If the debugger is unable to identify which function will be called, this is indicated by a dash.

processor

Define processor core type

Command type

Hardware configuration

Tool environment

st20cc, **st20run**, **st20sim**

Synopsis

processor core

Arguments

core The processor core type. Valid values are **c1** and **st20c1** denoting the ST20-C1 or **c2** and **st20c2** denoting the ST20-C2.

Description

The **processor** command defines the processor core type (that is, the instruction set). Supported core types are ST20-C1 and ST20-C2.

When used as input to **st20cc**, it determines the processor core to be targeted and is equivalent to the processor type passed in via the command line options **-c1**, **-st20c1**, **-c2**, **-st20c2**. If the core is specified via the command line as well as the **processor** command and there is a discrepancy then an error is reported.

When used as input to **st20sim**, it determines the processor core type to simulate.

it is preferable to use the **chip** rather than **processor** command because **chip** implicitly specifies the core type as well as many more device specific operations.

It is mandatory to use one of these mechanisms to specify the processor core type.

Example

processor c1

See also

chip

profile

Control program profiling

Command type

Profiling

Tool environment

st20run

Synopsis

profile [*options*]

Options

- on** Turn on profiling data accumulation
- off** Set trigger to turn off profiling data accumulation
- prof mode** Set the profiling measuring *mode*. The *mode* may be **i[dl]** for idle profiling or **s[ample]** for flat profiling. The default is flat profiling.
- p[eriod] period** Set the sample period to *period* microseconds. The value of *period* must be not less than 1000 and not more than 10^6 . This option only applies to hardware targets. The default is 1000.
- o[utput] filename** Write the output to *filename*. The file may be on the command language search path or a full pathname may be given.
- f[lat]** Return flat profiling results
- idle** Return idle profiling results

Debugger context

The **spaceid** from the debugger context specifies the address space the command applies to.

Description

The command **profile** controls profiling for the program defined by the command language variable **programid**. Program identifiers are returned by the **program** command.

Profiling is performed in two stages. First the application is executed, accumulating profiling data. During this stage the accumulation can be switched on or off using the **profile** command with the **-on** or **-off** options. By default, the application starts execution with the data accumulation turned off. Second, after the execution has completed, the results are extracted and interpreted. If no options are given, **profile** extracts the accumulated results and returns the full results. The **-flat** and **-idle** options allow selected output. Output is to standard output unless the **-o** option is used.

Example

```
>connect st20          ## connect to target
>load hello.lku
>run                  ## application stops at the beginning of user code
>profile -on -p 40000  ## start flat profiling with period of 40 mS
... later
>profile -off          ## stop profiling
>profile -o test.pro   ## return all profiling results to test.pro
```

See also

[Chapter 7: Profiling on page 97.](#)

program

Display program information

Command type

Debugging model, task state

Tool environment

st20run

Synopsis

program [*programid*] [*options*]

Arguments

programid The integer identifier of the program

Options

- a[ll]** Display brief information on all programs
- n[ew] *dbg_filename*** Load debugging information into the debugger from the debug information file given by *dbg_filename*. The file may be on the command language search path or a full pathname may be given.
- c[ode] *address*** Specify code load address of the relocatable code unit (RCU)
- d[ata] *address*** Specify static data load address of the RCU
- b[ss] *address*** Specify BSS load address of RCU
- d[ele]te]** Delete the program
- u[pdate]** Reload debugging information into the debugger from the (possibly modified) debug file.
- current [*programid*]** Specify the current program as *programid* or the filename given in the **-dbg** option or the *key* in the **-uid** option.
- disable [*programid*]** Disable a program
- enable [*programid*]** Enable a program
- dbg *filename*** Specify a **.dbg** file for use with **-current**, **-enable** and **-disable** options
- uid *key*** Specify a unique ID for use with **-current**, **-enable** and **-disable** options
- detail [*programid*]** When listing programs, show whether a program is enabled or disabled and show its program *key*

Debugger context

The **programid** from the debugger context specifies the program the command applies to.

Returns

The program identifier

Description

The command **program** displays information about the program specified with the supplied **programid**, or the program specified in the debugger context if a **programid** is not supplied. The **-n[ew]** option loads symbolic debug information from the **.dbg** filename. If the **.dbg** file corresponds to a relocatable code unit (RCU) file then the **-code**, **-data** and **-bss** options should be supplied. The **-d[ele]te** option removes a program from the debugger. The **-update** option updates a program from its **.dbg** file. The **-a[ll]** option can be used to delete or update all programs the debugger knows about.

The options **-current**, **-disable** and **-enable** specify a **current** program, disable a program and enable a program respectively. These options may either take a **programid**, or may be used in conjunction with the **-dbg** or **-uid** options. The current program is searched first when looking for symbolic information. Disabled programs are not searched when looking for symbolic information.

The **-dbg** option is used to specify the filename of a **.dbg** file with the **-current**, **-disable** and **-enable** options. The **-uid** option is used to specify a unique key to the **-current**, **-disable** and **-enable** options. A 32-bit unique key is generated by the linker for every program and this key is placed in a global integer in the application program with the name **_ST_ProgramIdentifier**. The value specified with **-uid** is compared with the contents of the variable **_ST_ProgramIdentifier** for each program on the debugger's program list.

The **-detail** option may be used with a **programid** or with **-a[ll]** to display the enable/disable status of a program and to extract the value of **_ST_ProgramIdentifier**.

Note: If this symbol is not referenced by target code, it will not be present in the final linked output; linked units will not contain this symbol unless it is explicitly referenced by the application.

Example

```
>program -a
program 0 hello.lku
program 1 fubar.lku
>program -n hello.dbg
>program -d 1
```

This examples lists all programs, adds a new program and deletes program 1 from the debugger.

See also

[context](#),

[Chapter 9: ROM systems on page 133.](#)

pwd

Print working directory

Command type

Miscellaneous

Tool environment

st20cc, st20lib, st20run, st20sim

Synopsis

pwd

Returns

Current working directory.

Description

The command **pwd** returns the pathname of the current working directory.

Example

```
> pwd
c:\mikej\fubar
```

See also

cd

quit

Quit the debugger

Command type

Miscellaneous

Tool environment

st20run

Synopsis

quit *status*

Arguments

status Status code to return to the shell

Description

The command **quit** exits the debugger returning the status argument to the shell.

Example

```
>quit 42
%echo $status
42
```

register

Configure register addressing

Command type

Low level debug

Tool environment

st20run

Synopsis

```
register name -cpu
```

```
register name address options2
```

```
register [name] options3
```

Arguments

name Symbolic name of register

address Address of register

Option 1

-cpu CPU register group. A register ***name*** must be specified with this option.

Options 2

-c[har] Address is byte accessed

-s[hort] Address is half word accessed

-r[eadonly] Address cannot be written to

-w[riteonly] Address cannot be read from

-g[roup] *name* Put register into given group

-p[rocedure] *cmdproc* Use the command language procedure ***cmdproc*** to display a register

Options 3

-l[ist] List named register or register group

-d[etele] Delete named register or register group

-a[ll] List/delete all registers and groups

Description

The first form of the **register** command associates a symbolic name to an address. This symbolic name can be used with the **-r** option of the **display** and **modify** commands. By default the register is word accessed. The size of the register that is whether it is byte or half-word and the type of access can be specified optionally. Several registers can be put into a register group and the **display -r** command lists the values of all of the registers. If the **chip** command is used to specify the exact core type then it is not necessary to give any **register** definitions but the list and delete options are still useful inside the debugger.

If the **-p** option is supplied, then a user supplied command procedure is invoked to display the contents of the register when the **display** command is subsequently used. This is to allow for different register formats, such as big endian. The command procedure is invoked with the following parameter values:

\$1 The first parameter of the procedure: it has the value **1** if the command **display -q** is supplied and indicates that the register contents should not be displayed. Otherwise its value is **0** if the register value should be displayed.

\$2 The second parameter of the procedure: the register name.

\$3 The third parameter of the procedure: the register address.

The command procedure must return the register contents as its result in the return parameter **\$\$**.

The user can define their own procedures or use the set of command procedures supplied with the toolset in the file **\$ST20ROOT/stdcfg/commonchip.cfg**. These include the following subset:

Procedure name	Description
UnalignedRegister	Little endian, non-aligned
BigEndianRegister	Big endian, may be non-aligned
SerialRegister	Serial register where most significant byte is read first
SplitRegister	2-byte register that has most significant byte at address+2

Table 61: Register procedures

The command procedure supplied with the **register -p** command is only invoked when the **display** command is used, it is not invoked by the **modify** command.

The second form of the **register** command lists defined registers and register groups and can also be used to delete registers and register groups. The third form of the command adds a register to the CPU register group; see [Table 62](#) for a list of ST20-C1 registers and [Table 63](#) for ST20-C2 registers.

[Table 62](#) lists the possible values that may be given to specify a CPU register of the ST20-C1 core.

Register name	Register description
AREG	Evaluation stack AREG
BREG	Evaluation stack BREG
CREG	Evaluation stack CREG
WPTR	Workspace pointer
TDESC	Task descriptor
IPTR	Instruction pointer
STATUS	Status register

Table 62: ST20-C1 core registers

[Table 63](#) lists the possible values that may be given to specify a CPU register of the ST20-C2 core.

Register name	Register description
AREG	Evaluation stack AREG
BREG	Evaluation stack BREG
CREG	Evaluation stack CREG
WPTR	Workspace pointer
IPTR	Instruction pointer
PRIORITY	Priority register takes the values 1 or 0
ENABLES	Enables register
STATUS	Status register
SHAREG	Shadow evaluation stack AREG
SHBREG	Shadow evaluation stack BREG
SHCREG	Shadow evaluation stack CREG
SHWPTR	Shadow workspace descriptor
SHIPTR	Shadow instruction pointer
SHSTATUS	Shadow status register
FPTR0	High priority scheduling list front pointer
FPTR1	Low priority scheduling list front pointer
BPTR0	High priority scheduling list back pointer
BPTR1	Low priority scheduling list back pointer
TIMER0	High priority timer list front pointer
TIMER1	Low priority timer list front pointer
CLOCK0	High priority time
CLOCK1	Low priority time

Table 63: ST20-C2 core registers

Example

```
register LPTimerLS (addr+0x400) -group LowPowerController
register LPTimerMS (addr+0x404) -group LowPowerController
>display -r LPTimerLS
...
>display -r LPTimerMS
LPTimerLS = 0x67000000
LPTimerMS = 0x56743433
```

The following example, shows the use of the **register -p** option:

```
proc BigEndian4Register op=$1 name=$2 addr=$3 value=0 i=0 {
    ## Access unaligned big endian 4 byte registers using
    ## register -p option
    for (i=0;i<4;i++) {
        value = (value << 8) | (display -r (addr+i) -q -c)
    }
    if (op == 0) {
        write (name) = -x (value)
    }
    $$ = value
}
register TestBe4 0m0 -group Test -p BigEndian4Register

...
>poke 0m0 0x12345678
>display -r TestBe4
TestBe4 = 0x78563412
```

See also

[chip](#), [display](#), [modify](#)

remove

Remove symbol or when statement

Command type

Miscellaneous

Tool environment

st20cc, **st20libr**, **st20run**, **st20sim**

Synopsis

```
remove whenid
```

```
remove symbol
```

Arguments

whenid The integer identifier of a **when** statement, as returned by a **when** command

symbol The command language symbol to be removed

Description

The **remove** command deletes a specified variable or **when** statement from its event.

If no argument is given and the command is part of a **when** program, then the **when** trigger is removed when it finishes execution.

Examples

```
>taddr = addressof t
>break (taddr) -1 (sizeof t)
>remove taddr
```

This example uses a command language variable **taddr** as a temporary variable by removing it after it has been used.

```
>b = break munger
>wm = when (b) { write munger called ; go; remove }
>go
munger called
task 0 stopped at <munger.c 24>
```

This example invokes the **when** on the breakpoint, which returns a string, continues running and then removes the **when**. A subsequent hit of breakpoint will not invoke the **when**.

See also

[*break*](#), [*clsymbol*](#), [*events*](#), [*quit*](#), [*wait*](#), [*when*](#)

reset

Reset the target board

Command type

Target interface

Tool environment

st20run, **st20sim**

Synopsis

reset

Debugger context

The **spaceid** from the debugger context specifies the address space the command applies to.

Description

The **reset** command resets the target board. Following a **reset** command the CPU and all peripherals (including, for example, the watchdog timer, low power controller) are put into the state defined for them following reset. (Refer to the individual peripheral and device documentation for details of their state after reset).

reset can also be used to reset a simulator target.

See also

[*connect*](#), [*disconnect*](#)

restart

Restart the current address space

Command type

Execution

Tool environment

st20run

Synopsis

restart

Debugger context

The *spaceid* from the debugger context specifies the address space the command applies to.

Description

The **restart** command restarts the current address space. The address space is **reset**, the configuration procedure associated with the target is run, all programs that have been loaded are reloaded from the **.lk**, the symbolic debug information is updated from **.dbg** files, all breakpoints are recomputed and set, trace control is reset and the address space is started executing.

See also

[*reset*](#)

rewind

Rewind file pointer back to start of file

Command type

Miscellaneous

Tool environment

st20cc, **st20libr**, **st20run**, **st20sim**

Synopsis

rewind *filename*

Arguments

filename The file, previously opened for reading, that is to be rewound

Returns

No return value

Description

The command **rewind** moves the file pointer for *filename* back to the start of the file, so that it may be re-read.

See also

fclose, *feof*, *fgets*, *fopen*, *fputs*

rm

Remove list of files from the file store

Command type

Miscellaneous

Tool environment

st20cc, **st20libr**, **st20run**, **st20sim**

Synopsis

rm *file_list*

Arguments

file_list List of files to remove from the file store

Returns

0 is returned on success, -1 is returned on error.

Description

The command **rm** removes the list of files from the file store.

Example

```
>rm hello.tco hello.lku
```

See also

mkdir, *rmdir*

rmdir

Remove list of directories from the file store

Command type

Miscellaneous

Tool environment

st20cc, **st20libr**, **st20run**, **st20sim**

Synopsis

rmdir *directory_list*

Arguments

directory_list List of directories to remove from the file store

Returns

0 is returned on success, -1 is returned on error.

Description

The command **rmdir** removes the list of directories from the file store.

Example

```
>rmdir hello.1 hello.2
```

See also

mkdir, *rm*

romimage

Specify list of segments to be written to file

Command type

ROM image generation

Tool environment

st20cc

Synopsis

`romimage segmentlist romfilename [options]`

Arguments

- segmentlist** Optional string that contains a space separated list of memory segments
- romfilename** The name of the output file for the ROM image

Options

- slice slicetype** Specify type of ROM image slicing
- origin address** Specify alternative address for the start of the ROM image
- fill** Gaps in the ROM image are padded with a default fill value
- fillval value** Same as **-fill**, but the fill value is supplied
- off filetype** Specify type of ROM image file

Description

This command specifies the list of segments to be written to the file specified by **romfilename**. If a segment list is not supplied, then any ROM segments not associated with another **romimage** command are written to the specified file. The ROM file format type can be specified using the **-off** option. The default is to use the ROM output type specified by the **st20cc** command line option **-off**. If the ROM filename does not have a filename extension, then the extension added is dependent on the type of ROM output file produced; see [Table 64](#):

-off type	Output extension
-off hex	.hex
-off binary	.bin
-off srecord	.mot

Table 64: ROM file extensions

For example, given the following commands:

```
romimage misc -off hex
romimage seg1 seg1 -off srecord
romimage "seg2 seg3 seg4" segs.rom
```

The following files are generated:

- **misc.hex**, a hex format ROM file containing all the ROM segments not associated with another ROM image file,
- **seg1.mot**, a Motorola S-record format ROM file containing segment seg1,
- **segs.rom**, a default format ROM file containing segments seg2, seg3 and seg4.

If there are no **romimage** commands associated with a ROM segment, then the segment is written to a default format ROM file. The filename prefix is the segment name, and the filename extension is derived from the ROM format type

A specific segment and ROM filename may only be specified by one **romimage** command, that is the same filename can not be used by multiple **romimage** commands. A ROM segment must exist when specified in a **romimage** command. There may only be one **romimage** command with a default (missing) segment list.

If the **-origin** option is used, the start address of the ROM image is modified to the supplied address; this enables an address to be specified that the ROM blower understands, for example 0.

If the **-fill** or **-fillval** option is supplied then any gaps within a memory segment and between memory segments in the ROM image are set to an initial value. If these options are not supplied then the default behavior is to use the tool command line arguments **-filled** and **-filledval**. The exception to this is if the ROM type is binary, then the **-fill** option is implied so that gaps in the binary output are padded. If the **-fillval** option is supplied, then the given value is used as the fill value, otherwise a default fill value is used. If the tool command line argument **-filledval** is supplied, this is used as the default fill value, otherwise the value **0xff** is used. A warning is reported if a large gap between segments is filled.

The **-slice** option supports the use of multiple ROM devices to make up a single memory segment. This situation arises when ROM devices which are narrower than the EMI memory width are used, for example two 8-bit ROMs connected to a 16-bit wide memory bus. This option can be used to cause multiple output files to be generated for a single segment; this is termed ROM slicing. The **-slice** argument determines the type of slicing performed. When slicing, the **romfilename** forms the stem of the output filenames if multiple files are produced. Such filenames are numbered sequentially and given an extension indicating their type, for example **out1.hex**, **out2.hex**.

Slice type	EMI width	ROM configuration
2x8	16 bit	2 x 8 bit
4x8	32 bit	4 x 8 bit
2x16	32 bit	2 x 16 bit

Table 65: Slice types

*Note: Multiple files generated for a ROM segment by the **-slice** option have their origins set to 0 and the **-origin** option is ignored.*

Example

Examples are given of each of the slice types supported. Each example assumes that the file type selected is hexadecimal. The multiple output file examples indicate how the bytes of the first two words would be distributed between the ROM files.

16 bit wide EMI - 2 x 8 bit ROMs

In this example there are two 8-bit ROMs attached to a 16-bit memory bus. The bytes for the first two words are distributed as follows:

ROM0	ROM1
0	1
2	3
4	5
6	7

Table 66: 2 x 8-bit ROMs byte distribution example

Thus ROM0 contains all the even bytes and ROM1 contains the odd bytes. Two output files are required, one for each ROM device.

-slice 2x8

This command generates the files:

- **romfile0.hex** containing even bytes,
- **romfile1.hex** containing odd bytes.

32 bit wide EMI - 4 x 8 bit ROMs

In this example there are four 8 bit ROMs attached to a 32-bit memory bus. The bytes for the first two words are as follows:

ROM0	ROM1	ROM2	ROM3
0	1	2	3
4	5	6	7

Table 67: 4 x 8-bit ROMs byte distribution example

Four output files are therefore required, one for each ROM device.

Using the following command:

-slice 4x8

generates the files:

- **romfile0.hex**: first byte of word,
- **romfile1.hex**: second byte of word,
- **romfile2.hex**: third byte of word,
- **romfile3.hex**: fourth byte of word.

32 bit wide EMI - 2 x 16 bit ROMS

In this example there are two 16 bit ROMS attached to 32-bit memory. The bytes for the first two words are as follows:

ROM0	ROM1
0 and 1	2 and 3
4 and 5	6 and 7

Table 68: 2 x 16-bit ROMs byte distribution example

Two output files are required, one for each ROM devices. The following command is used:

```
-slice 2x16
```

This generates the files:

- **romfile0.hex**: first/second bytes of word,
- **romfile1.hex**: third/fourth bytes of word.

See also

[memory](#)

runtime

Return run-time system name or make current address space aware of run-time system tasks

Command type

Symbolic debugging

Tool environment

st20run

Synopsis

`runtime [rtos_name] [options]`

Arguments

`rtos_name` The name of the run-time system, as in [Table 69](#)

Options

- `-v[erbose]` Display information on run-time system
- `-a[ll]` List supported run-time systems

Debugger context

The *spaceid* from the debugger context specifies the address space to which the command applies.

Description

The command **runtime** with no arguments returns the name of the run-time system that is associated with the current address space. The `-a[ll]` option lists all supported run-time systems. When supplied with a run-time system name, the current address space is made aware of run-time system tasks, and can interpret variables in a run-time-specific way. [Table 69](#) defines the set of run-time systems that are currently supported.

Name	Description
c1rtl	Default for ST20-C1. The application is treated by the debugger as a single task.
c2rtl	Default for ST20-C2. The application is treated by the debugger as a single task.
c2os20	OS20 run-time on an st20c2 core
c1os20	OS20 run-time on an st20c1 core
c2psos	PSOS run-time on an st20c2 core
c2rtlmulti	ST20-C2 multi-tasking run-time system. The descriptor of the executing task is held in WDESC.

Table 69: Run-time systems

See also

[task](#)

searchpath

Add path to command language filename search space

Command type

File handling

Tool environment

st20run

Synopsis

searchpath [*pathname*] [*options*]

Arguments

pathname The pathname to be added to the search space

Options

-r [*eset*] Reset the search path to the default. This is the current directory and the **libs** subdirectory of the directory given by the value of the environment variable **ST20ROOT**.

-c [*lear*] Clear the search path completely

Description

The command **searchpath** adds **pathname** to the command language filename search space. The search path is used by the command language commands to find linked unit files and include files.

If the **pathname** argument is not specified then the current search path is returned.

Examples

searchpath pathname

Adds **pathname** to the file search path.

searchpath

Displays the search path.

searchpath -reset

Resets the search path to default values.

searchpath -clear

Clears the search path completely.

See also

[directory](#)

session

Save current debugger session to file

Command type

State saving

Tool environment

st20run

Synopsis

session filename

Arguments

filename The name of the session file to create

Description

The **session** command saves the current debugger session to a named file. This includes saving information on all target and hosted address spaces in use, all programs that have been loaded and all breakpoints and trace conditions that are set. The information is saved as command language program to enable a session to be easily recreated by including the session file.

Note: Information on target definitions and files that have been included is not saved.

Example

```
%st20run -d
>connect sti5516_simm
>load hello.lku
>break main
code breakpoint on main ...
>run
task 0 unknown stopped at <hello...
> ...                               ## debug session
>session debug.ses
%
%...                               ## later
%st20run -i debug.ses
code breakpoint on main ...
>run
task 0 unknown stopped at <hello...
> ...                               ## debug session
```

This example connects to a target, loads a program, sets a breakpoint and runs the target. A session file is created that is later included into an **st20run** invocation using the **-i** option to include the session file.

See also

include

sizeof

Return the size of an object in bytes

Command type

Symbolic debugging

Tool environment

st20run, **st20cc**

Synopsis

sizeof *string* [-q]

Arguments

- | | |
|----------------------|--|
| <i>string</i> | For st20cc , the string must be the name of a memory segment or a program section. For st20run , the string can be either a statement reference, a C expression, a symbol or a section name (see Section 20.6: Objects in the application program on page 312). |
| -q[uiet] | Return result for use in command language expression, but suppress display. This is the default behavior for st20cc . |

Returns

The sizeof of the specified *object* in bytes.

Debugger context

The **sizeof** command accesses the debugger context for the default program. For C expression evaluation, it can also use the task, frame and location defined by the **context** command.

Description

The command **sizeof** returns the size of the specified object in bytes.

Example (st20run)

```
>tracebuffer -size (( sizeof -quiet tracebuffer ) / 4 ) -location (
addressof -quiet tracebuffer )
>tracebuffer -print
tracebuffer -size 16384 -location 0x50010000 -traceoniptr -savetoiptr -
wraponfull -nonintrusive
>
```

Example (st20cc)

```
x = sizeof INTERNAL
```

See also

[addressof](#), [symbols](#)

sizeused

Return the number of bytes used in a memory segment

Command type

Linking

Tool environment

st20cc

Synopsis

sizeused *segmentname*

Arguments

segmentname The name of a memory segment

Returns

The number of bytes used in the segment.

Description

The command **sizeused** returns the number of bytes that have been used in the specified memory segment. **sizeused** may only be used in conjunction with the **define** command.

Example

```
define FreeMemAddr "addressof INTERNAL + sizeused INTERNAL"
define FreeMemSize "sizeused INTERNAL"
```

See also

[addressof](#), [define](#), [sizeof](#)

space

Return information about or delete an address space

Command type

Task state

Tool environment

st20run

Synopsis

space [*spaceid*] [*options*]

Arguments

spaceid The integer identifier of an address space

Options

- a[11]** Delete or return information about all address spaces.
- d[ele]te** Delete the address space or spaces. Otherwise by default, display information about them.
- deviceid** Return the *deviceid* of the target.

Debugger context

The **spaceid** from the debugger context specifies the address space the command applies to unless the **spaceid** is supplied as an argument to the command.

Description

The **space** command returns information about the specified address space. If the **-d[ele]te** option is supplied then the specified address space is deleted from the debugger. If the **-a[11]** option is supplied then the command applies to all address spaces the debugger knows about. If the **-deviceid** option is used, the command returns the *deviceid* of the target.

See also

[connect](#), [copy](#), [core](#), [disconnect](#), [space](#)

st20ccoptions

Specify **st20cc** options

Command type

Program Build

Tool environment

st20cc

Synopsis

st20ccoptions [*options*]

Options

Any **st20cc** command line option; see the *ST20 Embedded Toolset User Manual*, chapter *st20cc compile/link tool*.

If more than one option is specified, they must be enclosed by double quotes (") and separated by spaces.

Description

The **options** command treats its operand as an extension to the command line in the same way that the environment variable **ST20CCARG** is used. This allows command line options to be specified from the command file.

Example

Specifying a single **st20cc** option:

```
st20ccoptions -c
```

Specifying multiple **st20cc** options:

```
st20ccoptions "-c -c1 -p clproc"
```

st20toolsetversion

Return toolset version number

Command type

Miscellaneous

Tool environment

st20cc, **st20libr**, **st20run**, **st20sim**

Synopsis

```
st20toolsetversion [options]
st20toolsetversion version_string
```

Arguments

version_string A string in the form “*a*”, “*a.b*”, “*a.b.c*” or “*a.b.c.d*” where *a*, *b*, *c* and *d* are numeric values less than 127.

Options

-q[uiet] Return result for use in command language expression but suppress display. This option only applies when no version string is supplied.

Returns

st20toolsetversion returns a thirty-two bit integer representing either the current toolset version or the version described by **version_string**.

Description

Called with no arguments (or with **-q**) **st20toolsetversion** returns the current toolset version number as a single integer. Without **-q** it also displays the version.

Called with **version_string** as an argument, **st20toolsetversion** returns the version number associated with that toolset. This form never directly displays its output.

The **st20toolsetversion** command applies to ST20 toolset versions R1.9 and later releases. A procedure for identifying toolset versions R1.6.2 to R1.8.1 is provided in the **examples/scripts** directory supplied with the toolset.

Example

```
>st20toolsetversion
This is ST20 toolset version R1.9.1
>write -x (st20toolsetversion -q)
0x01090100
>if ((st20toolsetversion -q) > (st20toolsetversion "1.9"))
  {write true}
true
```

stack

Define the position of the initial stack

Command type

Code and data placement

Tool environment

st20cc

Synopsis

stack *segment_name* [*length*]

Arguments

segment_name	The name of the segment where the stack is to be placed.
length	The size in bytes of the space allocated for the main stack, which must be a whole number of words divisible by four. It is illegal to specify a size which is zero or negative.

Description

The **stack** command is used to define the position of the main stack (the stack, within which the application begins execution) and may define the stack's maximum size. It takes a segment name where the stack is to be placed and an optional size in bytes. It is mandatory to specify a stack.

If a memory segment is specified then the next available address within this segment is used as the start address, after all other functions have been placed.

The area allocated for stack also depends on whether the heap is allocated to the same memory segment. If the stack is allocated to the same memory segment as the heap then the following rules are used to determine the size of the stack and heap regions.

- If both the **heap** and **stack** commands specify **length** then they are each allocated an area of memory as specified.
- If either but only one of the **heap** or **stack** commands specify **length** then the command which specified **length** is allocated the specified area of memory, and the remaining unallocated memory is allocated to the other command.
- If both the **heap** and **stack** commands omit **length** then the unallocated memory is divided equally between them.

See the description of **heap** for examples of the heap and stack sharing the same memory segment.

Examples

stack segment

This form of the **stack** command instructs **st20cc** to use the last unallocated space in **segment** for the main stack.

stack segment 512

This example directs **st20cc** to allocate 512 bytes in the last unallocated space in **segment**.

See also

heap

startorder

Define execution order for **ST_onstartup** functions

Command type

System startup code

Tool environment

st20cc

Synopsis

```
startorder symbol [priority]
```

Arguments

symbol	Name of function attributed with the pragma ST_onstartup .
priority	The integer priority of the function in the startup sequence. The default value is 0.

Description

The **startorder** command defines the priority used to determine the order in which functions with the **ST_onstartup** attribute are invoked.

When the system boots, it executes functions in the following order:

- the bootstrap,
- the prioritized startup functions (the ones referenced by **startorder** commands) in order of priority, most positive first and most negative last,
- all startup functions that have not been explicitly referenced in **startorder** commands,
- **main**.

Startup functions with equal priority are called in the order of appearance of the **startorder** commands.

*Note: Calling **startorder** without a priority gives the default priority of 0. Having a priority of 0 is not the same as having no priority (through never calling **startorder**).*

There are predefined variables for the priority that allow the user to interleave startup code with the toolset's standard startup functions.

```
CLOCK_START_PRIORITY   = 50000
HEAP_START_PRIORITY    = 40000
STDIO_START_PRIORITY   = 30000
OS_START_PRIORITY      = 20000
```

These values ensure that those functions with default priority 0 are called after the standard ones. Using a priority higher than **CLOCK_START_PRIORITY** for a function causes that function to be called before the clock, heap or I/O functions are started. The actual values for the variables may be changed in future toolsets, so to achieve a higher priority than **CLOCK_START_PRIORITY** it would be advisable to use an expression like:

```
startorder symbol (CLOCK_START_PRIORITY + 100)
```

Examples

The following runs **startFn1** then **startFn2** after all toolset initialization is complete:

```
startorder startFn1  
startorder startFn2
```

The following runs **startFn3** before I/O is initialized but after heap startup:

```
startorder startFn3 (STDIO_START_PRIORITY + 100)
```

See also

[*endorder*](#)

statement

Display address source statement

Command type

Symbolic debugging

Tool environment

st20run

Synopsis

statement *address*

Arguments

address The integer address of the statement to be displayed

Debugger context

The *spaceid* from the debugger context specifies the address space the command applies to.

Description

The **statement** command displays the source statement that corresponds to the specified *address* of the defined address space.

Example

```
>statement 0x80002050
<hello.c 54> in main
```

See also

[*addressof*](#), [*entry*](#)

step

Step through program execution

Command type

Execution

Tool environment

st20run

Synopsis

step [*taskid*] [*options*]

Arguments

taskid The integer identifier of the task which is to step

Options

-o [*ver*] Step over function calls. If the statement is a function call, execute until that function returns.

-i [*ninstruction*] Execute one machine instruction

-t [*hrrminimal*] Step over functions with minimal debug information

-m [*inimal*] Step into functions with minimal debug information

-l [*ine*] Step a complete line of code

-h [*ardware*] Step the machine context rather than the task context

Returns

The identifier of the event that will be hit on step completion.

Debugger context

The **taskid** from the debugger context specifies the task the command applies to unless a **taskid** is supplied.

Description

The command **step** allows a task to execute one source statement entering function calls. Functions that have not been compiled with full debugging are stepped over unless the **-t** or **-m** options are supplied. The **-t** option causes the debugger to keep single stepping until a function compiled with full debugging is entered. The **-m** option single steps into a function compiled with minimal debugging. Functions that do not have any debug data associated with them are repeatedly single stepped an instruction at a time until a function with debug data is reached, unless the **-i** option is used. If the **-i** option is used a single instruction is executed. If the **-l** option is used then a complete line of source code is executed. The **-h** option provides machine level single stepping. For example, an interrupt would divert the stepping process from the task to the interrupt.

Single stepping is implemented by setting breakpoints on the current program flow, hence any task switching that occurs is not stepped into. If the task hits another breakpoint before the step breakpoint has completed, the task is stopped. When the task is continued, the original breakpoint set by the step command still exists.

The **step** command is not synchronous, so the next **step** command may start execution before the first step expression has completed. Step completion is signalled as a debugger event; the **step** command returns the identifier of the event that will be hit on step completion. The **wait** command can be used to effect synchronous single stepping.

Example

```
>wait(step)
stepped to <facts.c 18 3>
```

This example steps one source statement (to the third statement on line 18 of **facts.c**).

```
x=1; fn(x)      ## line 12
x=3             ## line 13

## at line 12:

>step; step      ## step goes to fn
>step; step -o    ## step goes to line 13
>step -l         ## step goes to line 13
>step -h         ## step could go to a different task
```

See also

[go](#), [interrupts](#), [stepout](#), [where](#)

stepout

Allow a task to execute until the current function completes

Command type

Execution

Tool environment

st20run

Synopsis

stepout [*taskid*] [*options*]

Arguments

taskid The integer identifier of the task which is to step out

Options

-f[*rame*] *frameid* Continue execution until the specified frame has returned

Returns

The identifier of the event that will be hit on stepout completion.

Debugger context

The **taskid** from the debugger context specifies the task the command applies to, unless a **taskid** is supplied.

Description

The **stepout** command allows the task to execute until the current function completes (returns). If the **-frame** option is specified, execution continues until the specified frame is the current function.

*Note: This command only applies to functions which are completed via return statements. Other ways of exiting a function, for example, by the ANSI C library functions **longjmp** and **exit**, are not supported.*

Example

```
>where
5 facs at <facs.c 19>
4 facs at <facs.c 18>
3 facs at <facs.c 18>
2 facs at <facs.c 12>
1 main at <main.c 13>
>stepout
stepped to <facs.c 18>
>wait(stepout -f 2)
stepped to <facs.c 12>
```

This example steps out one frame and then steps out to frame 2.

See also

go, interrupts, step, where

stop

Stop execution of an address space

Command type

Execution

Tool environment

st20run

Synopsis

stop [*options*]

Options

-a [*id*] Stop all address spaces

Debugger context

The *spaceid* from the debugger context specifies the address space the command applies to unless the **-a** [*id*] flag is supplied.

Description

The **stop** command stops execution of the address space and if a task was executing, shows details of that task. If the **-a** [*id*] flag is specified then all address spaces known to the debugger are stopped. When the stop completes, the event identifier **stopped** is set.

Example

```
>stop;wait(stopped)
task 2 main stopped at <hello.c 14>
```

See also

[*go*](#), [*interrupts*](#), [*step*](#), [*stepout*](#)

store

Store section into ROM segment

Command type

Code and data placement

Tool environment

st20cc

Synopsis

```
store section_name segment_name
```

Arguments

section_name A section to be stored in ROM for loading into RAM at run time

segment_name The ROM segment where *section_name* is held

Description

The **store** command instructs the ROM tool to store the named section into the defined ROM segment. It is not normally necessary to use this command, but it gives finer control over the construction of the ROM image where this is required.

This command is only used for sections which are stored in ROM and then loaded from ROM into RAM at run time. The **store** command can be used to define the location in ROM and the **place** command can be used to define where it will be loaded in RAM.

The **store** command is not necessary for those sections that are positioned in ROM using the **place** command, since these are already specified to be in the ROM segment. By default all other sections that are not placed or placed in RAM are concatenated into the unused parts of the ROM. The loader is responsible for moving code and data from ROM to RAM at run time.

Example

```
store ihandlers INTSR0M  
store other       OTHERROM
```

This example stores the code section **ihandlers** in the ROM segment **INTSR0M** and the section **other** in the ROM segment **OTHERROM**.

See also

[memory](#)

symbols

List program symbols

Command type

Symbolic debugging

Tool environment

st20run

Synopsis

```
symbols [partname] [options]
```

Arguments

[partname] The initial string of the symbol name

Options

-m[odule] module List only symbols in the source file **module**
-f[unctions] List only functions
-v[ariables] List only variable symbols

Debugger context

The **programid** from the debugger context specifies the program the command applies to.

Description

The **symbols** command lists the symbols of the program. A symbol is a name that the linker has had to resolve, for example functions, global and static variables Automatic variables and source lines are not considered as symbols.

The **-module** option restricts the list to symbols of the given module. The **-functions** and **-variables** options restrict the list to functions and variables only respectively.

If a string is specified for the symbol name, the **symbols** command matches that string to the start of any symbol before printing it out.

Example

```
>symbols -m main.lku
exit()
main()
strcmp()
strlen()
suprise
```

This lists all symbols defined in the module **main.lku** of the current program.

Note: Functions are indicated with () appended to the symbol.

See also

modules, program

sys

Execute host system command

Command type

Miscellaneous

Tool environment

st20run, **st20cc**, **st20libr**, **st20sim**

Synopsis

sys *command*

Arguments

command Host system command to be executed

Description

The **sys** command executes a host system command.

target

Define a simulator or hardware target

Command type

Hardware configuration

Tool environment

st20run

Synopsis

```
target [[name] type interface_arguments [peektime timeout]
[command]]
```

Arguments

name	The name of the target
type	The type of the target, which may be st20sim (for the simulator) or tap (for a test access port)
interface_arguments	A string defining the target. The content of the string depends on the type of target. See the <i>ST20 Embedded Toolset User Manual</i> , chapter <i>Interfacing to the target</i> for examples.
peektime timeout	set the timeout period for a peek from an HTI (ST Micro Connect) to target. This option is specified in seconds, with a default of 5 s, but is rarely used.

*Note: The environment variable **TMTIMEOUT** is more useful. This can be used to set the timeout for **peek** from host to target. It is also specified in seconds, with a default of 60 s. This variable can be used when connecting to a remote target.*

command	A command language program to parse and execute on connect
----------------	--

Description

The command **target** defines a target which may be a simulator or hardware. If no argument is given, then information is displayed about the targets that have been defined. If a single argument is given the corresponding target description is displayed.

Examples

```
%st20run -d
>target clsim st20sim "st20sim -q -m #80000 -r #2000 -c1" clproc
> proc clproc {
-   processor c1
-   memory RAM %0 (16 * 1024 * 1024) RAM
-   memory ROM 0x7ff00000 (1024 * 1024) ROM
-}
>
```

```
%st20run -d
>include targets.res
>target
target faxsim st20sim "st20sim -c1 -clk 33 -d ourfax"
...

>target st20 tap "jei-soc host1 peektime 30" st20boardhw

>set TMTIMEOUT = 30
...
```

See also

[*connect*](#)

task

Manage the debugger task list

Command type

Debugging model, task state

Tool environment

st20run

Synopsis

task [*taskid*] [*options*]

Arguments

taskid The integer identifier of a task

Options

-a[11] Operate on all address spaces

-d[delete] Remove the specified task from the list of tasks maintained by the run-time system

-u[pdate] Update the task list and display

Debugger context

The **spaceid** from the debugger context specifies the address space the task command operates on, unless the **-a[11]** flag is used.

Description

The **task** command manages the task list of the debugger. If a **taskid** is specified then the command lists details of the task, including its name, last known execution status, stack extent and register context. If no argument is given then the command lists the last known set of tasks of the current address space.

If the **-u[pdate]** flag is supplied, the task list of the current address space is updated from the run-time system.

*Note: The address space must be stopped for this option. If the **-d[delete]** option is used then the specified task is deleted from the task list. If the **-a[11]** flag is used the command applies to all address spaces known to the debugger.*

Examples

```
>task -update
task 0 main scheduled
task 1 smurfhandler breakpointed at <fubar.c 12>
task 2 eric timer wait
```

This command lists all the tasks the run-time system can find and gives details of the state of each task.


```
>task 1
task 1 smurfhandler stopped by breakpoint at <fubar.c 12>
in space 0 processor mcu
priority 15, stack 0x80004000 - 0x80004100
iptr = 0x80000413
wptr = 0x80004050
areg = 0xf
breg = 0x80004020
creg = 0x80000000
```

This command lists detailed information on a particular task.

```
>task 2 -delete
```

This command deletes task 2 from the debugger's task list.

```
>task -delete -all
```

This command deletes all tasks from the debugger's task list

See also

[*runtime*](#), [*space*](#)

timer

Manage timer events

Command type

Event

Tool environment

st20run

Synopsis

timer [*eventid*] [*options*]

Arguments

eventid The integer identifier of an event, as returned from the **timer -create** command

Options

-c[reate] *period* Create a timer event that occurs every *period* milliseconds

-o[nce] Restrict the timer to occur once

-all Act on all timers

-r[emove] Remove timers

-e[nable] Enable timers

-d[isable] Disable timers

Description

The command **timer** creates, lists, removes, enables or disables timer events. The default action is to list timers.

Note: The frequency at which timers can be triggered is host-dependant and is therefore not guaranteed.

Example

```
## print the value of x every 4 seconds
t1 = timer -c 4000
for (i=0;i<100;i++) {
    print x
    wait(t1)
}
```

See also

wait, when

trace

Control trace function

Command type

Trace

Tool environment

st20run

Synopsis

trace [*options*]

Options

-on	Switch tracing on.
-off	Switch tracing off.
-clear	Clear trace data on the host.
-filter <i>expr</i>	Set a new trace filter expression. expr is a string that contains a command language expression. Applies when trace is used in conjunction with the tracedump command.
-reset	Clear all settings. Deconfigure the trace hardware and clear trace data on the host.
-download	Force any trace data on the target to be downloaded to the host.
-nonintrusivedownload	Do not stall the CPU when downloading trace data. By default, the CPU is stalled whilst trace data is downloaded.
-print	Print trace data statistics.

Debugger context

The **spaceid** from the debugger context specifies the address space on which tracing is set.

Description

Used to manually control trace and to view trace acquisition statistics.

The **-print** report details statistics and status of the trace data held on the host. It also returns the enable state of the trace hardware and the current trace filter string. Filter update pending is set to **yes** if

- data has been uploaded to the host from the target, or
- the trace filter string has been updated, since the last execution of the **tracedump** command without the **tracedump** option **-inhibittablegeneration**.

The **-filter** option may be used to reduce the output from the **tracedump** command to only display records that match a selection criteria. For example, the **-filter** option can be

used to allow **tracedump** to only display records that result from execution of a specific function, or to only display records that did not occur in a specific task name.

The **-filter** option is followed by a string **expr**, that contains a command language expression. For each trace record (line) that **tracedump** may output, each field in that record is first assigned to a command language variable. **expr** is evaluated for that trace record, and this expression may use the trace field command language variable. If **expr** evaluates as the value 0, then **tracedump** does not output that record, otherwise it is a non-0 value and the record is output.

The name of the trace field command language variable is derived from the name of the **tracedump** field selection option by prefixing the option name with the symbol **_ST_trace_**.

For example, for the **tracedump** field selection option **-toiptrfunctionname** the command language variable **_ST_trace_toiptrfunctionname** contains the value of the **tracedump** field output.

The default value for **expr** is "1" which always evaluates as a non-0 value and so all records are displayed.

For example, to only display trace records between record index 4 and 14, do:

```
trace -filter "_ST_trace_recordid >=4 && _ST_trace_recordid <= 14"
```

For more information see the *Trace and Profile* chapter of the *ST20 Embedded Toolset User Manual*.

Example

```
trace -on          ## start tracing
trace -off         ## stop tracing
trace -print       ## Print trace data statistics
```

```
Number of target to host uploads : 0
Number of trace records : 0
Number of filtered trace records : 0 : Filter update pending : Yes
Number of function call sequence records : 0 : Filter update pending : Yes
Number of function change sequence records : 0 : Filter update pending : Yes
Number of task sequence records : 0 : Filter update pending : Yes
Filter string : 1
Tracing is enabled
```

See also

[tracebuffer](#), [tracedump](#)

tracebuffer

Configure the trace hardware

Command type

Trace

Tool environment

st20run

Synopsis

tracebuffer [*options*]

DCU2 options

-size <i>n</i>	Set the size of the trace buffer to <i>n</i> 32-bit words.
-location <i>address</i>	Set the location of the trace buffer to <i>address</i> .
-savefromiptr	Trace data - IPTR jump-from address.
-savetoiptr	Trace data - IPTR jump-to address.
-stoponfull	Download trace buffer when full and stop trace acquisition.
-downloadonfull	Download trace buffer when full then continue trace acquisition.
-wraponfull	Continue tracing on full, wrapping to buffer start.
-forcesave	If necessary, stall CPU to allow trace data write to trace buffer.
-nonintrusive	Drop trace records if bandwidth for save to trace buffer is unavailable.
-postmortem	Configure to read existing trace data from target.
-print	Print trace configuration.

DCU3 options

-size <i>n</i>	Set the size of the trace buffer to <i>n</i> 32-bit words.
-location <i>address</i>	Set the location of the trace buffer to <i>address</i> .
-traceoniptr	Store trace data on IPTR discontinuity.
-traceoncontext	Store trace data on context change.
-traceoncapture	Store trace data on DCU3 capture[0] event.
-traceonbreakpoint	Store trace data on DCU3 compare[0] code/data breakpoint.
-savefromiptr	Trace data - IPTR jump-from address.
-savetoiptr	Trace data - IPTR jump-to address.
-savecount	Trace data - CPU cycles, missed trace data writes count and records context-reason or capture-R/W flag.
-saveiptr	Trace data - current IPTR.
-savewptr	Trace data - current WPTR.
-savedata	Trace data - last data write value.
-saveaddress	Trace data - last data access address.
-stoponfull	Download trace buffer when full and stop trace acquisition.
-downloadonfull	Download trace buffer when full then continue trace acquisition.
-wraponfull	Continue tracing on trace buffer full condition, wrapping to buffer start.
-nonintrusivedownload	When downloading target trace data, don't stall the target.
-countcpustallcycles	Increment cycle counter when CPU stalled.
-ignorecpustallcycles	Cycle counter does not increment when CPU stalled.
-forcesave	If necessary, stall CPU to allow trace data write to trace buffer.
-nonintrusive	Drop trace records if bandwidth for save to trace buffer is unavailable.
-postmortem	Configure to read existing trace data from target.
-print	Print tracebuffer configuration. Returns tracebuffer command line.

Debugger context

The *spaceid* from the debugger context specifies the address space the trace buffer is allocated on.

Description

The **tracebuffer** command is used to configure the trace feature: location and size of trace buffer, the data to be recorded, what events cause a trace record to be recorded, what action to take when the trace buffer is full and sundry other configuration controls.

When configuring the trace feature with the **tracebuffer** command, the trace buffer size and location must be supplied. Given only a trace buffer size and location, the **tracebuffer** command configures the trace feature with the following default settings.

For example,

```
tracebuffer -size 256 -location 0x50000000
```

results in:

DCU2:

```
tracebuffer -size 256 -location 0x50000000 -traceoniptr -savefromiptr  
-savetoiptr -wraponfull -nonintrusive
```

DCU3:

```
tracebuffer -size 256 -location 0x50000000 -traceoniptr -savefromiptr  
-savetoiptr -wraponfull -nonintrusive
```

The default settings configure the trace feature to save From and To instruction pointers, to wrap round the trace buffer on the trace buffer full condition, to save trace records on discontinuities in instruction pointer (for example on **jump** or **call** instructions, or due to an exception, interrupt or trap handler) and to do this nonintrusively, that is, drop trace records if saving the trace record to the trace buffer would interfere with the performance of the application being traced.

The **-print** option can be used to interrogate the current trace configuration. The output is formatted as a **tracebuffer** command.

See also

trace, *tracedump*

tracedump

Dump trace data to file or screen

Command type

Trace

Tool environment

st20run

Synopsis

tracedump [*filename*] [*options*]

Generic options

filename	The file to which the trace is to be written. If this argument is not given then the trace is dumped to the output window.
-start n	Start at trace record <i>n</i> .
-count c	Dump <i>c</i> trace records.
-view viewname	where viewname is one of rawfiltered , functioncallsequence , functionchangessequence , asksequence .
-inhibittablegeneration	Inhibit the re-generation of filtered trace tables after a new filter spec has been set or a trace download has occurred.

Field selectors

-viewindex	Filtered record number. Used with -start to view subset of a trace table.
-downloadindex	Identities on which download trace record was acquired.
-recordid	Record ID number from unfiltered record set.

DCU2/ DCU3 options

-fromiptr	Include FROM instruction pointers (IPTR) in trace output.
-fromiptrfunctionname	Include FROM IPTR function name.
-fromiptr sourcedirectory	Include FROM IPTR source directory.
-fromiptr sourcefilename	Include FROM IPTR source filename.
-fromiptrlinenumber	Include FROM IPTR line number.
-fromiptrstatementposition	Include FROM IPTR statement position.
-fromiptr objectfilename	Include FROM IPTR object filename.

-fromiptrsource	Include FROM IPTR source reference.
-toiptr	Include TO IPTR in trace output.
-toiptrfunctionname	Include TO IPTR function name.
-toiptrsource	Include TO IPTR source directory.
-toiptrsourcefilename	Include TO IPTR source filename.
-toiptrlinenumber	Include TO IPTR line number.
-toiptrstatementposition	Include TO IPTR statement position.
-toiptrobjectfilename	Include TO IPTR object filename.
-toiptrsourcereference	Include TO IPTR source reference.

DCU3 options

-cyclesdelta	Include CPU cycles delta.
-cyclesaccumulator	Include CPU cycles accumulator.
-missedrecords	Include missed records (non-intrusive mode).
-missedrecordsaccumulator	Include missed records accumulator (non-intrusive mode).
-reason	Include reason flag (context change type, or R/W data).
-capturedataiptr	Include CAPTURE data IPTR.
-capturedataiptrfunctionname	Include CAPTURE data IPTR function name.
-capturedataiptrsource	Include CAPTURE data IPTR source directory.
-capturedataiptrsourcefilename	Include CAPTURE data IPTR source filename.
-capturedataiptrlinenumber	Include CAPTURE data IPTR line number.
-capturedataiptrstatementposition	Include CAPTURE data IPTR statement position.
-capturedataiptrobjectfilename	Include CAPTURE data IPTR object filename.
-capturedataiptrsourcereference	Include CAPTURE data IPTR source reference.
-capturedatawptr	Include CAPTURE data WPTR.
-capturedata taskname	Include CAPTURE data task name.
-capturedata taskindex	Include CAPTURE data task index.
-capturedata address	Include CAPTURE data address.
-capturedata data	Include CAPTURE data data.
-branchtype	Include branch type.

Debugger context

The *spaceid* from the debugger context defines the address space from which to dump the trace.

Description

The **tracedump** command is used to display trace data to the **st20run** console or to save the trace data to a file. The trace data recorded can be filtered to return only desired fields, and a range can be set to output only a subset of the records, see **trace -filter**. If no field filter options are specified then **tracedump** outputs all fields in the trace data record. If a trace data field requested by a filter option has not been recorded then that filter is ignored.

Trace data is output with a header record naming each field, followed by one trace data record per line. The fields are comma separated with strings surrounded by double quotes. This data format allows easy import into other applications for further data analysis.

Example

```
## Display all fields in the trace data record and all trace
## data records to the st20run console.
```

tracedump

```
## Display trace data records starting at record 20 for 20 records.
## Each record output will consist of fields, To instruction pointer, CPU
## cycles since previous trace record and the accumulated cycle count
## for the records displayed.
```

```
tracedump -start 20 -count 20 -toiptr -cyclesdelta -cyclesaccumulated
```

See also

[trace](#), [tracebuffer](#)

traphandler

Define or establish breakpoint trap handler

Command type

debug

Tool environment

st20run

Synopsis

```
traphandler -rcu name
```

```
traphandler -code addr -data addr
```

Arguments

- rcu *name* The filename of a relocatable code unit containing the trap handler code
- code *addr* The address of the trap handler's code
- data *addr* The address of the trap handler's data

Debugger context

The *spaceid* from the debugger context specifies the address space in which to update the trap handler.

Description

The first form of this command specifies the file from which the debugger will obtain the code of the trap handler when it loads the trap handler.

Note: This command must be issued immediately after the **chip** command.

The second form of the command establishes a breakpoint trap handler. This command can be used to restore an application-specific breakpoint trap handler before disconnecting from a target or alternatively to use a trap handler that has not been loaded by the debugger.

Examples

```
traphandler -rcu 5510.bin
```

The debugger will use the code in the file **5510.bin** for the trap handler.

```
traphandler (addressof mythcode) (addressof mythdata) disconnect
```

After disconnect, the target runs with a breakpoint trap handler code specified by the supplied symbols.

See also

[chip](#)

wait

Wait for event

Command type

Event

Tool environment

st20run

Synopsis

```
wait [eventid]
```

Arguments

eventid The integer identifier of the event, as returned by the **break** command or as defined in [Section 20.7](#)

Returns

The identifier of the event.

Description

The **wait** command waits for the specified event to be hit, it does not execute the next command until after the event has been hit. It returns the identifier of the event.

If no argument is given, wait for any event to be hit.

Examples

```
>b = break munger
>go
>wait (b)
>write munger called

>wait(step)
```

See also

[break](#), [events](#), [forcewhen](#), [timer](#), [when](#)

when

Execute a command language program on a specified event

Command type

Event

Tool environment

st20run

Synopsis

```
when (eventid) vars_list {commands}
```

Arguments

<i>eventid</i>	The integer identifier of the event as returned from the break command or as defined in Section 20.7 .
<i>vars_list</i>	A list of local command language variable declarations, used by the command language program that follows it. The variables in the list are separated by semicolons (;). Each declaration consists of an assignment, that is the variable name followed by = followed by an integer initial value (for example: var = 1).
<i>commands</i>	The command language program to be executed when the event <i>eventid</i> is hit.

Returns

The integer identifier of the ***eventid***.

Description

The **when** command sets an ***eventid*** to execute a command language program ***commands*** whenever a specified event is hit. The variables in ***vars_list*** are local to the program ***commands***, and their values are lost when the **when** command is completed. At the start of executing the ***commands*** program, the ***spaceid***, ***programid*** and ***taskid*** are defined in the context of the task that hit the event and the ***commands*** program is executed.

Examples

```
>b = break main
>wm = when (b) x = 0 {
  x++; write munger called (x) times;
  go
}
```

This example counts the number of times a breakpoint has occurred and continues execution from the breakpoint.

See also

[break](#), [events](#), [forcewhen](#), [timer](#), [wait](#)

where

Display the function call trace of a task

Command type

Task state

Tool environment

st20run

Synopsis

where [*taskid*] [*options*]

Arguments

taskid The integer identifier of the task which is to be traced

Options

-a[rgs] Include function argument values in the display

-r[egs] Include register values in the display

-n[umframes] n Display the most local **n** frames only

-f[unctionname] func Display the most local frames up to and including the last call to the function **func**

Debugger context

Uses the **taskid** from the debugger context if **taskid** is not supplied as argument.

Description

The **where** command generates and displays the function call trace of a task. By default it stops when it encounters a function called **main**, it has generated 1000 entries or locating to an instruction has failed. The **-n[umframes]** option limits the call trace to the specified number of elements. The **-f[unctionname]** option stops the call trace when it encounters a function that matches the specified function name. The **-a[rgs]** option displays arguments of each function call. Data structured arguments (arrays, structs, unions) are not displayed.

Example

```
>where -a
5 facs(n=1) at <facs.c 19>
4 facs(n=2) at <facs.c 18>
3 facs(n=3) at <facs.c 18>
2 nacs(n=3) at <facs.c 12>
1 main() at <main.c 13>
```

```
>where -a -n 2
5 facts(n=1) at <facts.c 19>
4 facts(n=2) at <facts.c 18>
>where -a -f nacs
5 facts(n=1) at <facts.c 19>
4 facts(n=2) at <facts.c 18>
3 facts(n=3) at <facts.c 18>
2 nacs(n=3) at <facts.c 12>
```

write

Write to the tool output

Command type

Miscellaneous

Tool environment

st20cc, **st20libr**, **st20run**, **st20sim**

Synopsis

```
write [-n] [format] string [[format] string]... [-- string_list]
```

Arguments

<i>string</i>	A string to be written in the preceding format, if any, or in decimal if no format is given
<i>string_list</i>	A list of strings to be written without formatting

Options

-n[ewline]	Suppress output of the newline character, which is normally sent at the end of the output.
-------------------	--

Formats

-b	Binary
-c	ASCII character
-d	Signed decimal
-o	Octal
-u	Unsigned decimal
-x	Hexadecimal
--	Do not interpret any format options after this option

Description

The write command sends a message to the tool output. The message may include expressions to be evaluated. Each expression is written in round brackets (). If an expression evaluates to an array, the names and values of the elements are listed.

Example

```
>write hello world
hello world
>x = 5
>write x is (x)
x is 5
>write -n x is (x)
x is 5
>write -n x is -x (x) in hex
x is 0x00000005 in hex
>myarray = [1,2,"foo"=>"bar",[3,4]];
write (myarray)
foo = bar
0 = 1
1 = 2
3 = <array>
>
```





Revision history

Version	Date	Comments
F	Nov 03	Supports the R2.0 Product Release of the ST20 Toolset. Assembly support macros (previously in Part 3 Libraries) merged into chapter 1 Assembly-level programming.
E	Aug 03	Moved to new template and reformatted. Updated for compliance with style guide. Revision history added. Technical corrections made. ROM chapter moved from <i>ST20 Embedded Toolset User Manual</i> . Updated for 2.0 Toolset.
D	Nov 01	This document contains all of the comments made on the previous revision.
C	Oct 01	Supports the R1.9 Product Release of the ST20 Toolset.
B	Aug 01	Supports the R1.9 Beta ST20 Embedded Toolset.
A	Apr 01	Draft, supports the ST20 toolset R1.9. Additional work required as a result of extended R1.9 functional requirements must be identified in DDTS.



Index

Symbols

- #indent** 167
- #line** command 320
- #pragma**
 - ST_device** 168, 191-193
 - ST_inline** 169
 - ST_interrupt_handler** 169, 198
 - ST_naked_function** 170, 198, 202
 - ST_nolink** 170, 195
 - ST_nosideeffects** 171
 - ST_off** 172
 - ST_on** 172
 - ST_onshutdown** 172
 - ST_onstartup** 172
 - ST_place_at_workspace_offset** 172
 - ST_section** 173
 - ST_translate** 67, 173
 - ST_trap_handler** 174, 198-200
- \$**
 - in assembly code 9, 30
- .ti** files 167
- <>** chevrons 312
- __asm** 4
 - syntax 8
 - use when optimizing 171
- __LOCAL_SIZE** 6, 170, 202-203
- __optasm** 4
 - syntax 8
- __ST_dinkum_setmutexfn** function 204
- __ST_generic_cpp_getmutexdata** function 204
- __ST_generic_cpp_getmutexfn** function 204
- __ST_generic_cpp_setmutexfn** function 204
- { }** braces
 - use in commands 306

A

- addhelp** command 321
- Address**
 - specifying an 314
- Address spaces** 437
 - accessing debugger context for 371
 - closing 350
 - comparing 341
 - copying 345
 - creating 346
 - default, accessing the debugger context 379
 - dumping to file 354
 - loading linked unit files 389
 - restarting 424
 - stopping execution of 449
- Addresses, of objects** 322
- addressof** command 322
- AJW_BEG** macro 37
- AJW_END** macro 37
- Aliasing** 197
- align** assembler directive 13-14
- align** pseudo instruction 5
- alter** command 324
- ANSI C language**
 - argument promotions 195
 - implementation 181-203
- Arguments**
 - ANSI C**
 - default promotions 195
 - to debug procedures 317
 - to main 187
- Arithmetic**
 - in command language 307
- Arrays**
 - implementation 183
- asctime** 222
- asmdebug.h** assembly debugging macros 37

Assembler 3-30

- directives 13-29
 - align bytes 14
 - change source file name 26
 - change source tool name 27
 - define debugger information record 19
 - define new section 24
 - define symbol as external 21
 - define symbol as global 22
 - define symbols with numeric value 20
 - encode next instruction in fixed byte size 25
 - insert byte block 15
 - insert byte sequence 17
 - insert comment 18
 - insert word block 16
 - Insert words 28
 - mark module language type 23
- inline code 4-9
 - syntax 8
- operands 10
- stand-alone code 10-30
 - syntax 29-30

atexit 222**atof** 237**B**

Binary lister 65-79

- command line 67

Bit fields

- implementation 189

blkb assembler directive 13, 15**blkw** assembler directive 13, 16

Boot data block 325

bootdata command 325

Booting

- boot data block placement 325
- instruction pointer (lptr) initialization 326

bootlptr command 326

Braces in command language 306

break command 327

Breakpoints

- creating and modifying 327
- trap handlers 112
 - defining 467

Bss data 86, 94

Building libraries 60

byte assembler directive 13, 17**byte** pseudo instruction 5**C**

C implementation 181-203

C++ language support

- EDG front end 207

cachecontrol command 330

Caches

- controlling 330

call pseudo instruction 5

Calling a debug procedure 317

Calling conventions 195

cd command 331

Change

- browser state 315

char

- implementation 182
- plain 189

Character

- constants
 - integer 188
 - multibyte 188
 - locale 188
- sets 188
 - execution 188
 - source 188

Chevrons (<>) 312

chip command 332**cj** pseudo instruction 5**clerror** command 336**clinfo** command 337**clsymbol** command 338

Code

- listing 70
- placement in memory 91-95

Command

- file, for simulator 113
- scope 315

Command language 305-318

<> 312

{ } 306

address reference 314

commands 319-473

constructs

else 316**for** 316**if** 316**proc** 316**while** 316

creating errors 336

listing symbol types 338

operators 307

- path, adding 349
 - procedures 316, 361
 - setting information level 337
 - statement reference 312
 - symbol reference 313
 - variables 309, 312
 - errno** 238
 - Command line conventions xvi
 - Command line options
 - st20cc**
 - assembler 12
 - st20libr** 60-61
 - st20list** 67
 - st20run**
 - profile 100
 - st20sim** 110
 - stemi** 42-52
 - commandline** command 339
 - Commands
 - executing host system command 453
 - comment** assembler directive 13, 18
 - Comments
 - in assembler code 13
 - Common variables 186
 - compare** command 341
 - Compatibility
 - earlier toolsets 141
 - Compiler
 - C++ 207
 - macro 177
 - predefines 178
 - selective loading of libraries 60
 - Concurrency support
 - debug** library 242, 263
 - device I/O 222
 - device i/o 275
 - stdio** library 222-232
 - connect** command 342
 - Constants
 - integer 188
 - defining 348
 - context** command 343
 - Conventions used in this manual xvi
 - Conversion
 - floating point 187
 - integer 187
 - copy** command 345
 - Core
 - C1 instruction set 31-32
 - C2 instruction set 33-37
 - core** command 346
 - ctime** 222
 - Cycle time definition file 124-129
 - CYCLES** pseudo-op 132
- ## D
- Data
 - listing all 76
 - placement in memory 91-95
 - types 181
 - Date and time
 - defaults 194
 - Date, default 194
 - debug** assembler directive 13, 19
 - debugbreak** library function 246
 - debugclose** library function 244, 247
 - debugcommand** library function 245, 248
 - debugconnected** library function 249
 - debugdisconnect** library function 250
 - debugexit** library function 245, 251
 - debugfilesize** library function 252
 - debugflush** library function 253
 - Debugger
 - commands, defining default values for 343
 - debuggetenv** library function 254
 - debuggetkey** library function 255
 - debuggets** library function 243, 256
 - Debugging
 - assembly source files 37
 - breakpoints, causing 246
 - concurrency support 242, 263
 - determining status 249
 - disconnecting 250
 - exiting 251
 - Files
 - closing on host 247
 - flushing operations 253
 - measuring 252
 - opening 259
 - pointer, position 264
 - reading bytes 261
 - repositioning pointer 262
 - writing bytes 266
 - getting host variable value 254
 - getting time 265
 - invoking a command 248
 - library, setting mutual exclusion function 263
 - reading characters from keyboard 255, 260
 - reading strings from keyboard 256
 - ROM systems 133-143

- strings, writing to screen 258
- support libraries 241-266
- toggleing debug I/O 385
- debuginformsenabled** library function 257
- debugmessage** library function 243, 258
- debugopen** library function 244, 259
- debugpollkey** library function 260
- debugread** library function 244, 261
- debugseek** library function 244, 262
- debugsetmutexfn** library function 263
- bugtell** library function 244, 264
- debugtime** library function 265
- debugwrite** library function 266
- Declarators
 - implementation 193
- def_code** 94
- define** command 348
- defsym** assembler directive 13, 20
- Device
 - default 279-280
 - memory access 168, 191
 - register 284
 - registering 283
 - unregistering 286-287
- device_default** library function 279
- device_default_set** library function 280
- device_getmutexdata** library function 281
- device_getmutexfn** library function 282
- device_register** library function 283
- device_register_init** library function 284
- device_setmutexfn** library function 285
- device_unregister** library function 286
- device_unregister_desc** library function 287
- Device-independent I/O layer
 - mutual exclusion
 - callback 282
 - data 281
 - setting mutual exclusion callback 285
- Directives in assembler 13-29
- Directories
 - adding path to search space 433
 - changing 331
 - creating 397
 - removing list of 427
 - show current 416
- directory** command 349
- disconnect** command 350
- Display

- object code 65
- reference 75
- string to screen 258
- display** command 351
- dl_get_exports** library function 297
- dl_get_imports** library function 298
- dl_match_imports_to_exports** library function 294
- dl_reset_imports_from_exports** library function 296
- double 182
- dump** command 354
- Dynamic code loading 145-157

E

- EDG C++ front end 207
- ehsetmutex** function 204
- else** construct 316
- EMI
 - configuration 39-58
 - command line 42
 - options 42-52
 - configuration, delay 356
- emi** command 52, 355
- emidelay** command 356
- endorder** command 357
- entry** command 359
- Entry point
 - RCU 150
- entrypoint** command 360
- enumeration 182
- Enumeration types
 - implementation 189
- Environment variables
 - ST20LIBRARG** 61
 - ST20LISTARG** 68
 - STEMIARG** 52
- errno** run-time library variable 238
- Error
 - at run time 238
- Errors
 - creating 336
- eval** command 361
- Events 362
 - executing programs on 469
 - on the simulator 129
 - waiting for 468
- events** command 362
- Examples

cycle time definition file 126
 debugging a ROM system 135
 inline assembler 7

Execution
 character set 188

EXIT_SERVER pseudo-op 131

export command 364

Exported names, listing 72

Exporting functions 154-155, 364
 getting list 297
 libraries 289-298

Expressions
 assembler 11
 in command language 307
 interpreting 408

extern assembler directive 13, 21

External references
 listing 79

F

False 316

fappend command 365

fclose command 366

feof command 367

fgets command 368

File
 access 244
 identification 78

file command 370

Files
 appending strings to 365
 closing 366
 detecting end 367
 inserting command files 384
 opening 370, 372
 reading 368, 376
 reading characters 261
 removing list of 426
 renaming 402
 repositioning pointer 262
 rewinding file pointer 425
 writing 377
 writing segments to 428
 writing strings to 375

fill command 371

Flat profiling 100-102

float 182

Floating point
 ANSI functions 237
 conversion 187

IEEE compliance 182, 187
 maths library 238
 truncation 187

fopen command 372

for construct 316

forcewhen command 373

forward command 374

fputs command 375

fread command 376

FUNC_BEG macro 37

FUNC_END macro 37

FUNC_RET macro 37

Functions

defining startup execution order 442
 entry statements 359
 exporting 364
 getting list of exported functions 297
 getting list of imported functions 298
 importing 383
 matching imported and exported 294
 onshutdown execution order 357
 resetting imported function list 296
fwrite command 377

G

getenv command 378

getstdiofiletablemutexfn library
 function 227

getstdiomutex library function 228

getstdiomutexfn library function 229

global assembler directive 13, 22

go command 379

H

Heap

mutual exclusion callback 235-236
 mutual exclusion user data 234

heap command 380

Heap segment
 defining 380

heap_getmutexdata library function 234

heap_getmutexfn library function 235

heap_setmutexfn library function 236

help command 382

Help, add string to procedure 321

Hexadecimal

listing 72

Host access library 243

I

Identifier
 implementation 188
 use to give scope 315

Idle profiling 102-103

IEEE
 floating point 182, 187

if command language construct 316

Implementation
 arrays 183
 characters 188
 declarators 193
 enumeration types 189
 floating point 182
 identifiers 188
 integers 189
 registers 189
 structures 183
 switch statement 193
 types 181
 unions 183
 volatile qualifier 190

import command 383

Importing functions 154-155, 383
 getting list 298
 libraries 289-298

include command 305, 384

Included files 312

inform mechanism 112

informs command 385

Informs status, determining 257

Initialization
 boot data block placement 325
 instruction pointer (Iptr) 326

initstack command 386

Inline assembler code 4-9

Instruction
 sets 31-37

int 182

Integer
 bitwise operations 189
 conversion 187
 remainder on division 189
 result of right shift 189

interrupt_install_sl library function 157

Interrupts
 on the simulator 129
 specifying behavior 387

interrupts command 387

J

j pseudo instruction 4

Jump tables 8

Jumps
 assembler 7

K

Keyboard access 243

L

Labels
 assembler 11
 in **__asm** 6

language assembler directive 13, 23

ld pseudo instruction 4

ldab pseudo instruction 4

ldabc pseudo instruction 4

ldlabeldiff pseudo instruction 5

Librarian 59-63
 command line 60
 options 61

Library 238-240
 ANSI C functions 220
 building 60
 debugging 241-266
 exploding 63
 extracting modules 62
 floating point 238
 import/export support 289-298
 index 59
 indirect files 59, 61
 listing index 75
 modifying 62
 Posix 267
 RCU support 289-293
 ROM restart 299
 selective loading of 60

Linked unit
 map files 81

Linker
 selective loading of libraries 60

Linking
 loading linked unit file into address space 389

load command 389

Locale 188

localtime 222

log command 391

Logging commands 391

Looping
 in command language 316

M

Macro
 definition 177
 predefined 178-179

main function
 meaning of arguments 187

Map
 memory 86
 stack depth analysis 81

Mathematics, floating point library 238

Memory
 displaying 351
 displaying section names 76
 for simulator 129
 map file 86
 placing code and data 91-95
 reading 404
 segments, measuring 436
 setting a range to a value 396
 setting up regions 392
 types 393
 writing 400, 407

memory command 392

memset command 396

mkdir command 397

mknum command 398

mkstr command 399

modify command 400

Module data
 listing 74

Modules
 listing 401

modules command 401

Multibyte characters
 shift states 188

Mutex 225

mv command 402

N

Numbers
 conversion from strings 398
 conversion to strings 399

O

Object code
 displaying 65

Objects
 addresses 322
 measuring 435

Operands
 assembler 10

Operators
 assembler 11

Options
 defining custom 339

OS20 Real-time kernel
 debug awareness 159-164
 debugging 143
 relocatable code 156-157

Output, writing 472

P

Parameters
 passing 195

parse command 403

Path, adding 349

peek command 404

Performance analysis 97-108

place command 405

Plain chars 189

poke command 407

POSIX independent I/O 267

PostPokeLoopCallback function 142

Pragmas
 compilation 168

PrePokeLoopCallback function 142

Preprocessor
 C++ 207
 use with assembler 12

print command 408

PRINT pseudo-op 131

printf 237

proc construct 316

Procedures
 command language 316
 load 305

processor command 411

Processor core type 411

profile command 412

Profiling 97-108, 412
 batch mode 108
 flat 100-102
 how it works 100, 102
 idle 102-103
 types of 98

Program
 key 140-141

program command 414

Programming
 with command language 305-318

Programs
 displaying information 414
 entry point 360
 parsing and executing 403

Pseudo assembler instructions 4

Pseudo-ops for simulator 131-132

pwd command 416

Q

quit command 417

R

RCU
 deinitializing 293
 initializing 291

RCU see Relocatable code

rcu_deinit library function 293

rcu_init library function 291

Reference
 to a statement 312
 to a symbol 313

References
 create forward 374

register command 418

Registers 189
 configuring addressing 418
 displaying 351
 writing 400

Relocatable code 145-157
 libraries 289-293
 OS20 156-157

remove command 422

reset command 423

restart command 424

rewind command 425

rm command 426

rmdir command 427

ROM
 bootstrap 134
 loader stack 386
 restart 299
 restarting system 300
 start type 302
 storing sections 450
 systems 133

romimage command 428

romlaststart library function 302

romrestart library function 300

Run time
 error behavior 238

runtime command 432

Run-time system 429, 432

S

Sampling 100
 period 101

Scalar types
 implementation 182

Scope
 of a command 315

Screen access 243

Scripts
 line numbering 320

searchpath command 433

section assembler directive 13, 24

Selective linking 60

session command 434

setstdiofiletablemutexfn library
 function 230

setstdiomutex library function 231

setstdiomutexfn library function 232

signal 222

Signed
 char 182
 int 182
 long 182

signed
 Short 182

SIGTERM 238

Simulator 109-132
 command file 113

size assembler directive 13, 25

sizeof command 435

sizeused command 436

snprintf 237

Source character set 188

sourcefile assembler directive 13, 26

space command 437

sprintf 237

sscanf 237

st pseudo instruction 4

ST_device 167-168, 191-193

ST_inline 169, 173

ST_interrupt_handler 169, 198

ST_naked_function 170, 198, 202

- ST_nolink** 170, 195
 - ST_nosideeffects** 171
 - ST_off** 172
 - ST_on** 172
 - ST_onshutdown** 172
 - ST_onstartup** 172
 - ST_place_at_workspace_offset** 172
 - ST_section** 173
 - ST_translate** 173
 - ST_trap_handler** 174, 198
 - st20coptions** command 438
 - st20libr** 59-63
 - command line 60
 - ST20LIBRARG** environment variable 61
 - st20list** 65-79
 - command line 67
 - ST20LISTARG** environment variable 68
 - st20run**
 - profiling 97-108
 - st20sim** 109-132
 - st20toolsetversion** command 439
 - stab** pseudo instruction 4
 - stabc** pseudo instruction 4
 - Stack
 - depth analysis 81
 - stack** command 440
 - Stand-alone assembler code 10-30
 - startorder** command 442
 - Start-up script 113
 - statement** command 444
 - Statement reference 312
 - Static
 - area for RCUs 148
 - data 194
 - link 150, 157, 194-195
 - Statistics
 - generation 122-129
 - tag 124, 131
 - STATTAG** 131
 - stdcfg** release directory 335
 - stdio library
 - file table mutual exclusion callback 227
 - file table mutual exclusion function 230
 - getting file table mutual exclusion callback 231-232
 - mutual exclusion object 228
 - per-file mutual exclusion callback 229
 - stdio** library
 - concurrency support 222-232
 - stemi** 39-58
 - command line 42
 - STEMIARG** environment variable 52
 - step** command 445
 - stepout** command 447
 - stop** command 449
 - STOP** pseudo-op 131
 - store** command 450
 - Strings
 - appending to files 365
 - conversion from numbers 399
 - conversion to numbers 398
 - strtod** 237
 - strtok** 222
 - Structures
 - implementation 183
 - Switch statement
 - implementation 193
 - Symbol reference 313
 - Symbols
 - listing 66, 87, 451
 - removing 422
 - symbols** command 451
 - Syntax
 - inline assembler 8
 - stand alone assembler code 29
 - sys** command 453
- ## T
- Target
 - connecting 342
 - defining 454
 - resetting 423
 - setting chip 332
 - target** command 454
 - Task
 - awareness 160
 - concurrent I/O support 222-232
 - state 161
 - task** command 456
 - task_create_sl** library function 157
 - task_init_sl** library function 157
 - Tasks
 - altering states 324
 - deferring end 447
 - displaying function call traces of 470
 - managing list of 456
 - stopping execution of 449

TCOFF

- listing files 78

Tentative variables 186**Text files**

- listing 78

Thread safety 203

- Debug functions 242

- Libraries 222

 - POSIX I/O 276

 - stdio** 223

- set_new_handler** function 203

Time, default 194**timer** command 458**Timers**

- managing events 458

toolname assembler directive 13, 27**Trace**

- tag 121, 131

trace command 459**tracebuffer** command 461**tracedump** command 464**TRACEOFF** 131**TRACEONx** 131**Tracing**

- configuring 461

- dumping data 464

- on the simulator 114-121

traphandler command 467**True** 316**Type**

- conversion 187

- implementation 181

U**Unions**

- implementation 183

unsigned char 182**unsigned int** 182**unsigned long** 182**unsigned short** 182**V****Variables**

- assembler built-in 6

- common 186

- definition statements 359

- getting environmental variables 378

- of command language 309, 312

 - errno** 238

 - tentative 186

Version

- toolset 439

volatile

- implementation 190

vsprintf 237**W****wait** command 468**When** clauses 373**when** command 469**where** command 470**while** construct 316**word**

- assembler directive 13, 28

- pseudo instruction 5

write command 472